كتاب مذاكرة عملي · بالعامية المصري

عالم الفرونت إند

الخريطة الكاملة للي حوالين JavaScript في المتصفح: من HTML و CSS بجد، لحد npm و Vite والـ Frameworks والنشر، قبل ما تبدأ React

مش كتاب JavaScript. ده الكتاب اللي بيشرحلك كل اسم هتسمعه حوالين اللغة: الـ Semantic HTML والـ Forms والـ Accessibility، والـ Cascade و Flexbox و Grid والـ Responsive، وليه الـ Frameworks اتعملت، و npm حتة حتة (package.json والـ Semver والـ Lockfile والـ Scripts والأمان)، والـ ES Modules و Vite والـ Build، وTailwind، وTypeScript، و CSR/SSR/SSG، و ESLint و Prettier و Lighthouse، والنشر على Vercel و Netlify و GitHub Pages. كل Flag وكل ملف وكل رمز متشرّح.

على الـ Mac · من الدوكيومنتيشن الرسمية والأوامر متجرّبة (أكتوبر 2026)



package.json · @container · display: grid · specificity · <main> · SSR · SSG · @import "tailwindcss" · vite build · npm ci · ^8.3.4 · LCP · INP · CLS

كتابك للمذاكرة

قبل React بخطوة

27 فصل · كل أمر اتشغّل فعلاً

+ Cheat Sheet وقاموس ولينكات الدوكيومنتيشن الرسمية

المحتويات

الكتاب ماشي معاك على 8 مراحل. الأول الصورة الكبيرة: الفرونت إند يعني إيه، ووصلنا لهنا إزاي، وخريطة بكل الأدوات. بعدين HTML و CSS على مستوى أعمق من الكورسات. وبعدها السؤال المهم: ليه الـ Frameworks اتعملت أصلاً. وفي النص npm بالتفصيل الممل، والـ Modules و Vite. وبعدين Tailwind و TypeScript وطرق الـ Rendering. وفي الآخر الجودة والنشر، ومشروع نهائي كامل، وخريطة طريق: إيه اللي لازم تعرفه قبل React وإيه اللي يستنى.

إزاي تستخدم الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **خلي بالك**: غلطة شائعة أو فخ
- **حاجة مشتركة**: فكرة بتتكرر في أكتر من أداة أو كتاب
- **تشريح**: كل حتة في الأمر أو الملف عليها شرح
- **تحديات الفصل**: سهل / متوسط / صعب، والحلول في الملحق
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **مثال عملي**: جرّبه بإيدك
- **لو حابب تتعمق**: تفاصيل زيادة تقدر تعدّيها أول مرة
- **مشكلة شائعة**: العَرَض والسبب والحل
- **ملخص + صح ولا غلط + أسئلة**: آخر كل فصل

01 الحكاية من الأول: الفرونت إند يعني إيه، ووصلنا لهنا إزاي؟ 9
1.1 مطعم فيه صالة ومطبخ 9
1.2 المرحلة 1 (1991): صفحات ثابتة، ورق على حيطة 10
1.3 المرحلة 2: السيرفر هو اللي يبني الصفحة 10
1.4 المرحلة 3 (2004–2006): AJAX و jQuery، الصفحة تتغير من جوه 11
1.5 المرحلة 4 (2010–2016): الـ SPA، التطبيق كله في المتصفح 11
1.6 المرحلة 5 (النهارده): الـ Meta-frameworks، رجوع ذكي للسيرفر 12
1.7 انت فين، والكتاب هياخدك لفين 13

02 خريطة الأدوات: كل اسم هتسمعه، وهو بيعمل إيه 15
2.1 ليه الأسامي كتير كده؟ 15
2.2 القاعدة الذهبية: المتصفح بيفهم 3 لغات بس 16
2.3 الطبقات واحدة واحدة 17
2.4 Library ولا Framework ولا Meta-framework؟ 17
2.5 package.json حقيقي: مين بيروح للمتصفح؟ 18
2.6 إيه اللي هتستخدمه فعلاً مع React؟ 19

03 الـ Semantic HTML: إنك تقول للمتصفح «ده إيه» مش «شكله إيه» 22
3.1 صفحتين شكلهم واحد 22
3.2 هيكل أي صفحة 24
3.3 الـ Landmarks: خريطة الصفحة 25
3.4 العناوين: الفهرس اللي محدش بيشوفه 25
3.5 اللينك ضد الزرار: أهم قرار 26
3.6 عناصر النص الصغيرة 26
3.7 الصور والجداول 27
3.8 تشوف الشجرة بنفسك في DevTools 27

04 الـ Forms: أقدم طريقة تبعت بيها بيانات، ولسه أذكى مما تتخيل 29
4.1 Form من غير ولا سطر JavaScript 29
4.2 الـ label: مش ديكور 31
4.3 أنواع الـ input: كل نوع بيديك حاجة ببلاش 31
4.4 الـ Validation المدمج: رسايل حقيقية من Chrome 32
4.5 autocomplete: صاحبك اللي محدش بيكلمه 33
4.6 الـ Form من JavaScript 33

05 الـ Accessibility: الموقع لكل الناس، مش للي زيك بس 36
5.1 مين بيستفيد؟ انت كمان 36

5.2 WCAG في سطرين 37
5.3 الكيبورد: أول اختبار وأسهل اختبار 37
5.4 التباين والألوان 39
5.5 ARIA: القاعدة الأولى إنك متستخدمهاش 39
5.6 الحركة والـ prefers-reduced-motion 40
5.7 Checklist يدوي في 5 دقايق 40

06 الـ Cascade والـ Specificity والـ Box model: ليه الـ CSS بتاعي مش شغال؟ 42
6.1 مين يكسب؟ 42
6.2 الـ Specificity: تحسبها في 10 ثواني 43
6.3 الـ inline و !important 44
6.4 :is() و :where(): نفس الشكل، Specificity مختلفة 44
6.5 الـ @layer: ترتيب الطبقات بإيدك 45
6.6 الوراثة: مين بيتورث؟ 45
6.7 الـ Box model: ليه 200 طلع 250؟ 46
6.8 display: النوع اللي بيحدد كل حاجة 47

07 Flexbox: ترتيب العناصر في خط واحد 49
7.1 المشكلة: توسيط حاجة في النص 49
7.2 الفكرة الوحيدة اللي لازم تفهمها: المحورين 49
7.3 على الأب (الـ Container) 50
7.4 على الولد (الـ Item): flex: 1 يعني إيه بالظبط؟ 52
7.5 start و end، مش left و right 52
7.6 أنماط هتكتبها كل يوم 53
7.7 تابة Flexbox في DevTools 54

08 CSS Grid: الصفحة كلها كشبكة 56
8.1 ليه Flexbox مش كفاية؟ 56
8.2 الأساسيات: ارسم الشبكة الأول 57
8.3 grid-template-areas: ترسم الصفحة بالكلام 58
8.4 repeat و minmax و auto-fill: السطر السحري 59
8.5 تحط عنصر في مكان معين 59
8.6 Grid ولا Flexbox؟ 60

09 الـ Responsive Design: صفحة واحدة لكل الشاشات 62
9.1 السطر اللي من غيره مفيش Responsive 62
9.2 Mobile-first: ابدأ بالصغير 63
9.3 الوحدات: إمتى تستخدم إيه 63
9.4 Media queries بالتفصيل 64
9.5 Container queries: الكومبوننت يقرا علبته 65
9.6 clamp(): خطوط بتكبر بنعومة 66
9.7 الصور اللي بتتأقلم 67
9.8 تجرّب: Device mode 67

10 CSS الحديثة: الحاجات اللي خلّت Sass مش ضرورية 69
10.1 الـ Custom properties: متغيرات بجد 69
10.2 الـ Nesting الأصلي 70
10.3 :has(): الأب يعرف ولاده 71
10.4 الـ Logical properties: هدية للعربي 71
10.5 min() و max() و clamp() 72
10.6 الألوان الحديثة 72
10.7 @supports و Baseline: ينفع أستخدمها؟ 73

11 ليه الـ Frameworks اتعملت أصلاً؟ مشكلة مزامنة الـ DOM 75
11.1 Bug صغير في عربية تسوّق 75
11.2 الحل الساذج: ارسم كل حاجة من الأول 76
11.3 UI = f(state): الفكرة اللي غيّرت كل حاجة 77
11.4 التلات أفكار اللي هتقابلها في React 78
11.5 إزاي بيحدّثوا الفرق بس؟ تلات مدارس 79
11.6 إمتى متحتاجش Framework خالص؟ 79

12 React و Vue و Angular و Svelte: نفس الفكرة بأربع لهجات 81
12.1 العداد بالأربعة 81

82 12.2 نفصّص الفروق
83 12.3 حجم اللي وصل للمتصفح
83 12.4 الفلسفة: Library ضد Framework كامل
84 12.5 إيه اللي بيتنقل معاك؟
13 86 Node و npm: المخزن اللي فيه ملايين الحتت الجاهزة
86 13.1 ليه محتاجين npm أصلاً؟
86 13.2 Node على الماك
87 13.3 npm = 3 حاجات بنفس الاسم
88 13.4 npm init: أول package.json
88 13.5 npm install: إيه اللي بيحصل بالظبط
89 13.6 جوه node_modules
91 13.7 Flags التسطيب اللي هتستخدمها
92 13.8 محلي ولا Global؟
93 13.9 npx: شغّل من غير ما تسطّب
94 13.10 ليه vite شغال في npm run ومش شغال في الـ Terminal؟
14 96 package.json مفتاح مفتاح
96 14.1 نوعين package.json
97 14.2 type: ليه import بيضرب Error؟
98 14.3 scripts بالتفصيل
100 14.4 أربع أنواع dependencies
101 14.5 package.json بتاع مكتبة
101 14.6 مفاتيح تانية هتقابلها
102 14.7 npm pkg: عدّل من غير ما تفتح الملف
15 104 الـ ^ والـ ~ والـ Lockfile: «بيشتغل عندي ومش عندك»
104 15.1 اللغز
104 15.2 SemVer: تلات أرقام وكل واحد ليه معنى
105 15.3 الـ Ranges: كل رمز يقبل إيه
106 15.4 dist-tags: latest مش دايماً الأحدث رقماً
107 15.5 الـ Lockfile: الحل
107 15.6 install ولا ci ولا update؟
109 15.7 npm outdated: مين محتاج تحديث
16 111 pnpm و Yarn و Bun، وأمان الـ Packages
111 16.1 ليه فيه أكتر من واحد؟
111 16.2 pnpm: نسخة واحدة على الجهاز كله
112 16.3 Bun و Yarn
113 16.4 انت بتثق في مين بالظبط؟
114 16.5 npm audit
115 16.6 npm 12: Install Scripts ممنوعة افتراضياً
116 16.7 عادات أمان يومية
17 118 الـ Modules وليه محتاجين Bundler
118 17.1 نظامين، وانت هتقابل الاتنين
118 17.2 Modules في المتصفح من غير أي أداة
120 17.3 ليه Bundler لو المتصفح بيفهم Modules؟
120 17.4 Tree-shaking و Code splitting بعينك
122 17.5 تاريخ سريع للأدوات
18 124 Vite من جوه: dev و build و preview
124 18.1 create-vite: كل Flag
125 18.2 كل ملف في القالب
126 18.3 npm run dev: بيحصل إيه
127 18.4 HMR: التجربة
128 18.5 npm run build
129 18.6 npm run preview
130 18.7 vite.config.js و Flags
19 132 متغيرات البيئة و .env: الإعدادات والأسرار
132 19.1 يعني إيه Environment variable؟
132 19.2 في Vite: .env و import.meta.env

19.3 مين بيكسب؟ 134
19.4 القاعدة الذهبية: الـ Front end مفيهوش أسرار 135
19.5 وفي Node: process.env 135

20 إيكوسيستم الـ CSS: Tailwind و CSS Modules و Sass والمكتبات الجاهزة 138
20.1 المشكلة 138
20.2 Tailwind CSS v4 138
20.3 CSS Modules 140
20.4 Sass 141
20.5 مكتبات Components جاهزة 142
20.6 تختار إيه؟ 142

21 TypeScript: نظرة من فوق 144
21.1 المشكلة 144
21.2 التجربة 144
21.3 مين بيشيك ومين بيشيل؟ 145
21.4 tsconfig.json سطر سطر 146
21.5 TypeScript في Node 24 مباشرة 147
21.6 TypeScript 7 147

22 CSR و SSR و SSG و ISR و Hydration: الصفحة بتتبني فين؟ 150
22.1 CSR: كل حاجة في المتصفح 150
22.2 SSR و SSG و ISR 151
22.3 Hydration: "ترطيب" الصفحة 152
22.4 Next.js: React + سيرفر + Router 152
22.5 Next.js ولا React + Vite؟ 156

23 Prettier و ESLint: مصحح ومنسّق أوتوماتيك 158
23.1 اتنين مختلفين 158
23.2 ESLint في المشروع النهائي 158
23.3 Prettier 161
23.4 في VS Code وفي الـ CI 162

24 Lighthouse و Core Web Vitals وفحص الـ Accessibility 164
24.1 Core Web Vitals: تلات أسئلة 164
24.2 Lab ولا Field؟ 165
24.3 نشغّل Lighthouse 165
24.4 الـ Accessibility: صفحة سيئة عن قصد 166
24.5 أشهر الإصلاحات 167

25 النشر: GitHub Pages و Netlify و Vercel 169
25.1 الـ Deploy بيعمل إيه؟ 169
25.2 GitHub Pages بـ GitHub Actions 170
25.3 Netlify 172
25.4 Vercel 172
25.5 Environment variables على الاستضافة 172
25.6 أشهر مشكلتين 173

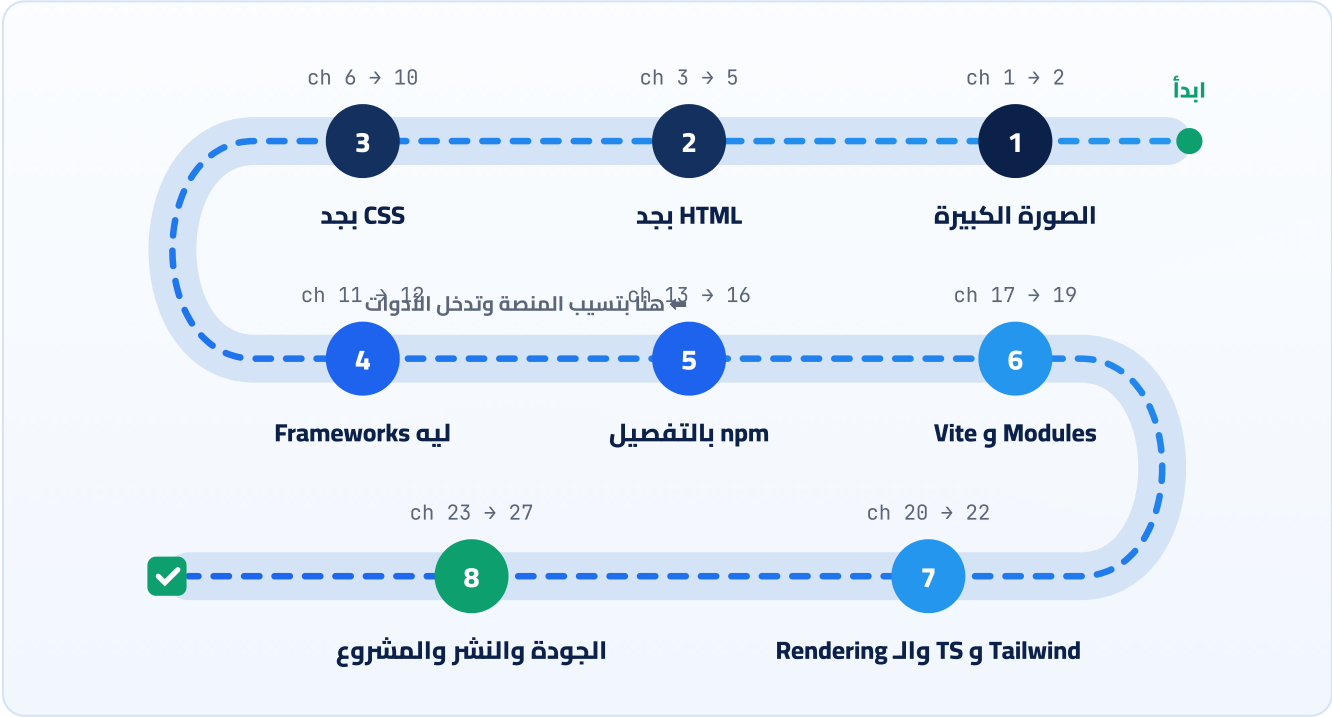
26 المشروع النهائي: صفحة روابط من الصفر للـ Deploy 175
26.1 · اعمل المشروع 176
26.2 · Tailwind 177
26.3 · الصفحة 178
26.4 · ESLint و Prettier 180
26.5 · package.json النهائي 180
26.6 · build و preview 181
26.7 · Git 182
26.8 · النشر (عليك) 182

27 الخريطة: تتعلم إيه وإمتى 184
27.1 الرحلة 184
27.2 إيه اللي محتاجه دلوقتي وإيه اللي يستنى 184
27.3 مشاريع صغيرة على الطريق 185
27.4 تتابع إزاي من غير ما تغرق 185

أ Cheat Sheet وجدول الـ Flags وجدول الرموز 187
أ.1 جدول الـ Flags 188
أ.2 جدول الرموز 189
ب قاموس المصطلحات 191
ج ملحق الروابط الرسمية 194
ج.1 MDN 194
ج.2 web.dev و Chrome 196
ج.3 W3C و WHATWG و WebAIM 197
ج.4 Node و npm ومديري الـ Packages 197
ج.5 Vite والأدوات حواليه 199
ج.6 React و Frameworks و TypeScript 200
ج.7 Next.js 201
ج.8 Tailwind و CSS 201
ج.9 ESLint و Prettier 202
ج.10 GitHub و Netlify و Vercel 202
ج.11 مراجع تانية 203
د المصادر ومنهجية التحقق 204
د.1 تاريخ اللقطة 204
د.2 النسخ اللي اتجربت عليها 204
د.3 إيه اللي اتجرب وإزاي 204
د.4 حاجات جديدة أو بتتغير 205
د.5 ملاحظات لقيناها وإحنا بنجرّب 205
ه الإجابات والحلول 206

خريطة الكتاب

بص على الطريق كله مرة واحدة. أول تلات مراحل عن **المنصة نفسها**: الويب و HTML و CSS، ودي حاجات هتفضل صح بعد عشر سنين. وآخر خمس مراحل عن **الأدوات** اللي الناس بنتها فوق المنصة: npm و Vite والـ Frameworks و Tailwind، ودي بتتغير كل سنة، فهنفهم الفكرة اللي وراها قبل الأمر. الأرقام اللي جوه الدوايز أرقام المراحل، وفوقها أرقام الفصول.



الطريق من "الفرونت إند يعني إيه" لحد ما تنشر مشروع Tailwind + Vite كامل وانت فاهم كل ملف فيه، وجاهز لكورس React.

1 · الصورة الكبيرة

فرونت وباك، وحكاية الويب من الصفحات الثابتة لـ jQuery للـ SPAs للـ Meta-frameworks، وخريطة بكل اسم هتقابله وهو بيعمل إيه.

2 · HTML بجد

الـ Semantic HTML والـ Landmarks، والـ Forms والـ Validation، وأساسيات الـ Accessibility: الكيبورد والـ Labels والـ alt والـ ARIA.

3 · CSS بجد

الـ Cascade والـ Specificity والـ Box model، و Flexbox و Grid، والـ Responsive بالـ Media و Container queries، و CSS الحديثة: المتغيرات والـ Nesting و :has() و @layer.

4 · Frameworks ليه

مشكلة مزامنة الـ DOM مع البيانات، والـ Components والـ State والـ UI الـ Declarative، ومقارنة React و Vue و Angular و Svelte بنفس المثال.

5 · npm بالتفصيل

Node و npm والـ Registry، و package.json مفتاح مفتاح، والـ Semver والـ Lockfile، والـ Scripts و npx، و pnpm/yarn/bun، والأمان وسلسلة التوريد.

6 · Modules و Vite

الـ ES Modules في المتصفح، وليه الـ Bundlers، و Vite من جوه: الـ Dev server والـ HMR والـ Build، وكل ملف في المشروع، ومتغيرات البيئة.

7 · Tailwind و TS والـ Rendering

Tailwind و CSS Modules و Sass والمكتبات الجاهزة وإمتى كل واحد، و TypeScript على مستوى الصورة، و CSR و SSR و SSG و ISR والـ Hydration ومكان Next.js.

8 · الجودة والنشر والمشروع

ESLint و Prettier، و Lighthouse والـ Core Web Vitals وفحص الـ Accessibility، والنشر على Vercel و Netlify و GitHub Pages، ومشروع نهائي، وخريطة طريق لـ React.

Best Practice

متقراش الكتاب وبس. افتح الـ Terminal والمحرر جنبك، واعمل كل معمل بإيدك على الماك. كل أمر في الكتاب اتشغّل فعلاً والـ Output اللي قدامك حقيقي. وخلي الـ **Cheat Sheet** و**قاموس المصطلحات** و**لينكات الدوكيومنتيشن** اللي في الآخر جنبك وانت بتذاكر.

الكتاب ده مبني على أربع كتب قبله

مش هنعيد اللي اتشرح: الـ Terminal والـ PATH و Homebrew في **كتاب الـ Terminal**، و Git و GitHub و GitHub Pages والـ Actions في **كتاب Git**، و HTTP والـ CORS والكاش و"المتصفح بيرسم إزاي" و DevTools في **كتاب الإنترنت و DevTools**، و fetch و JSON والـ APIs في **كتاب APIs و JSON**. هنفكّرك بسطر ونقولك الفصل. ولغة JavaScript نفسها في كورس Jonas، فالكتاب ده **مش** هيشرحها: هو بيشرح العالم اللي حواليها.

خلي بالك: أرقام النسخ صورة للحظة

HTML و CSS بيتغيروا ببطء ومن غير ما يكسروا القديم. لكن أدوات npm بتطلع نسخ كل أسبوع. كل رقم نسخة وكل Output في الكتاب متاخد من الـ Registry والدوكيومنتيشن الرسمية ومتجرّب يوم **23 أكتوبر 2026** (Node 24.22.1 و npm 11.19.1 و Vite 8.3.5)، وجنبه سطر **المصدر**. لو لقيت رقم أحدث عندك، ده طبيعي: الفكرة هي اللي تفرق.

01

المرحلة 1 · الصورة الكبيرة

الفصل الأول

الحكاية من الأول: الفرونت إند يعني إيه، ووصلنا لهنا إزاي؟

قبل ما تسمع كلمة Vite أو Next.js، لازم تعرف المشكلة اللي كل أداة اتعملت علشان تحلها. الفصل ده بيحكي حكاية الويب من أول صفحة ثابتة سنة 1991، لـ jQuery، للـ SPA، لحد الـ Meta-frameworks النهارده. وكل مرحلة هتشوف فيها المشكلة الأول وبعدين الحل، علشان لما تقابل أي أداة جديدة تسأل نفسك: دي بتحل أنهي مشكلة؟

بعد الفصل ده هتكون عارف

- الفرق بين الفرونت إند والباك إند، ومين بيشتغل فين
- الصفحات الثابتة، وبعدين الصفحات اللي السيرفر بيبنيها
- AJAX و jQuery: الصفحة تتغير من غير Reload
- الـ SPA: التطبيق كله في المتصفح، ومشاكله
- الـ Meta-frameworks: رجوع ذكي للسيرفر
- انت فين في الحكاية دي، والكتاب هياخدك لفين

1.1 مطعم فيه صالة ومطبخ

افتكر أي مطعم. فيه **صالة**: الترابيزات والمنيو والجرسون والديكور، الحاجات اللي الزبون بيشوفها ويلمسها. وفيه **مطبخ**: المخزن والطباخين والوصفات، حاجات الزبون مبيشوفهاش بس من غيرها مفيش أكل.

الويب نفس الحكاية بالظبط:

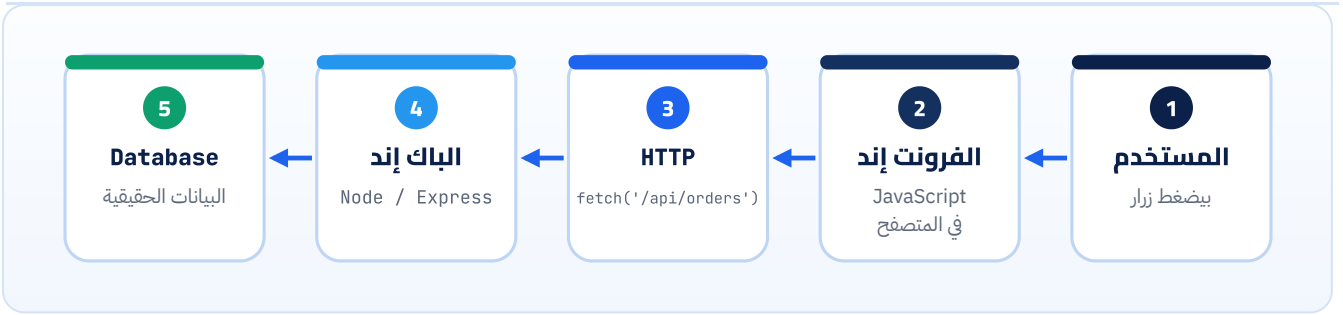
الفرونت إند (الصالة)

- كل حاجة بتشتغل **في المتصفح** عند المستخدم.
- لغاته الأساسية تلاتة بس: **HTML** (الهيكل)، و**CSS** (الشكل)، و**JavaScript** (الحركة والتفاعل).
- شغلته: الشكل، والتفاعل، والسرعة اللي المستخدم **بيحسها**، وإن أي حد يقدر يستخدمه (Accessibility).
- الكود بتاعه **مكشوف**: أي حد يفتح DevTools يشوفه.

الباك إند (المطبخ)

- بيشتغل **على سيرفر** بعيد عن المستخدم.
- لغات كتير: JavaScript (بـ Node)، و Python، و Go، و Java، و PHP...
- شغلته: البيانات (Database)، والصلاحيات، والدفع، والأسرار، وأي حاجة لازم تتحمى.
- الكود بتاعه **مخفي**: المستخدم بيشوف النتيجة بس (غالباً JSON، زي ما شفنا في كتاب APIs و JSON).

والاتنين بيتكلموا مع بعض بـ **HTTP**: المتصفح يطلب، والسيرفر يرد. ده كله اتشرح سطر سطر في **كتاب الإنترنت و DevTools** (فصل 7 و 8)، فهنا هنفترض إنك عارفه.



شكل 1.1 الصالة والمطبخ. المسار ده هتعيشه في كورس Maximilian من ناحية السيرفر، وفي كورس React من ناحية المتصفح. والكتاب ده عن النص الشمال.

> **وفين الـ Full-stack؟**
>
> الـ **Full-stack** = حد بيشتغل في الاتنين. وطريقك (JavaScript ← Node ← React) هو بالظبط طريق Full-stack بلغة واحدة. بس متخليش الكلمة تخدعك: حتى الـ Full-stack بيفكر في الصالة بعقلية، وفي المطبخ بعقلية تانية. والكتاب ده بيبني عقلية الصالة.

1.2 المرحلة 1 (1991): صفحات ثابتة، ورق على حيطة

أول موقع على الويب نزل سنة 1991 من CERN. كان عبارة عن **ملف HTML** على سيرفر: نص، وعناوين، ولينكات. مفيش ألوان تقريباً، ومفيش تفاعل. تطلب الصفحة، السيرفر يبعتلك الملف زي ما هو، خلاص.

index.html · ويب التسعينات

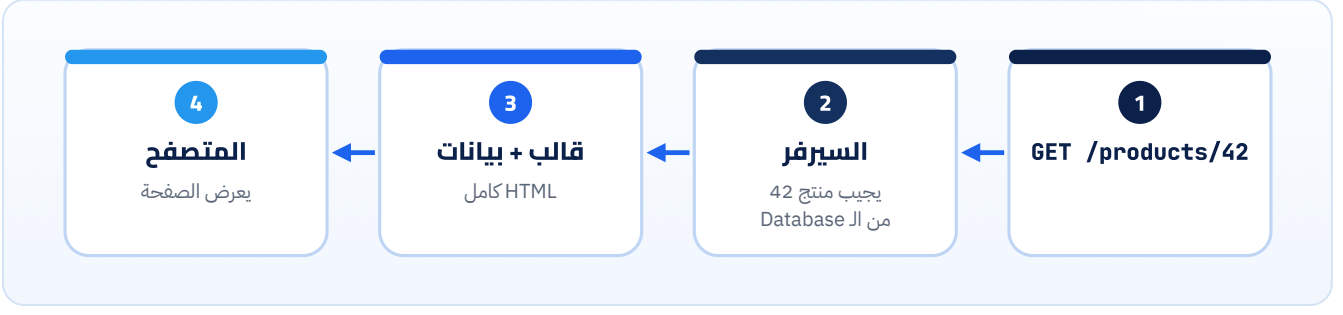
```
<h1>World Wide Web</h1>
<p>The WorldWideWeb (W3) is a wide-area hypermedia information retrieval initiative.</p>
<a href="people.html">People</a>
```

وبعدين جات الإضافات بسرعة: **JavaScript** سنة 1995 في Netscape (اتكتبت أول نسخة منها في أيام معدودة)، و**CSS** سنة 1996 علشان تفصل الشكل عن المحتوى بدل ما يبقى كل لون جوه الـ HTML.

المشكلة: كل صفحة ملف منفصل. عايز تعرض 1000 منتج؟ اكتب 1000 ملف بإيدك.

1.3 المرحلة 2: السيرفر هو اللي يبني الصفحة

الحل كان إن السيرفر **يبني الـ HTML وقت الطلب**: ياخد قالب واحد، ويملاه من الـ Database. ده اللي عملته لغات زي PHP و Perl وبعدين Ruby on Rails و Django. ولسه بيتعمل لحد النهارده.



شكل 1.2 Server-rendered: كل ضغطة لينك = طلب جديد = صفحة HTML جديدة كاملة. والفكرة دي هترجعلنا في فصل 22 باسم SSR.

المشكلة: كل حاجة صغيرة محتاجة صفحة جديدة. ضغطت "لايك"؟ الصفحة كلها تختفي وترجع. كتبت حرف في البحث؟ مفيش اقتراحات. التجربة كانت "ورق"، مش "تطبيق".

1.4 المرحلة 3 (2004–2006): AJAX و jQuery، الصفحة تتغير من جوه

سنة 2004 و 2005 ظهر **Gmail** و**Google Maps**، والناس اتصدمت: الإيميلات بتظهر، والخريطة بتتسحب، **من غير ما الصفحة تعمل Reload**. السر كان إن JavaScript بتبعت طلب HTTP في الخلفية، وتغيّر **حتة** من الصفحة بس بالرد. الفكرة دي اتسمت **AJAX** سنة 2005. والنهارده بنعملها بـ `fetch` (كتاب APIs و JSON فصل 10).

بس كان فيه مشكلة تانية: كل متصفح (Internet Explorer و Firefox و Safari) كان بيتصرف بطريقة مختلفة. نفس الكود يشتغل هنا ويبوظ هناك. فسنة 2006 نزلت **jQuery**: مكتبة بتديك طريقة واحدة قصيرة تشتغل في كل المتصفحات.

jQuery زمان

```javascript
$('#like').on('click', function () {
  $.post('/api/like', { id: 42 },
function (res) {
    $('#count').text(res.likes)
  })
})
```

نفس الكلام في JS النهارده

```javascript
const like =
document.querySelector('#like')
const count =
document.querySelector('#count')
like.addEventListener('click', async ()
=> {
  const res = await fetch('/api/like',
{
    method: 'POST',
  })
  const data = await res.json()
  count.textContent = data.likes
})
```

المتصفحات اتعلمت من jQuery: `querySelector` و `fetch` و `classList` كلها بقت جوه اللغة. فـ jQuery لسه موجودة (آخر نسخة على npm وقت الكتابة 4.0.0، ومواقع قديمة كتير شغالة بيها)، بس مشروع جديد مش محتاجها.

المشكلة الجديدة: الصفحات بقت تطبيقات كبيرة، والكود بقى كوم `$('#x').text(...)` في كل حتة. مين غيّر العنصر ده؟ وليه الرقم اللي فوق مش زي اللي تحت؟ دي مشكلة **مزامنة الـ DOM مع البيانات**، وهي بطلة فصل 11.

1.5 المرحلة 4 (2010–2016): الـ SPA، التطبيق كله في المتصفح

الخطوة اللي بعدها كانت جريئة: ليه السيرفر يبني HTML خالص؟ خلّيه يبعت **صفحة واحدة فاضية + ملف JavaScript كبير**، والـ JavaScript هي اللي تبني كل الشاشات، وتكلم السيرفر بـ JSON بس. ده الـ **SPA** (Single-Page Application). وعلشان تكتب تطبيق بالحجم ده، ظهرت الـ **Frameworks**: AngularJS و Backbone سنة 2010، و**React** سنة 2013 من Facebook، و Vue سنة 2014، و Angular الجديدة 2016، و Svelte 2016.

index.html · شكل SPA نموذجي

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

الـ HTML ده حقيقي: هو اللي `create-vite` بيعمله لمشروع React (هنشرحه في فصل 18). لاحظ إن الـ `body` فاضي تقريباً. كل اللي هتشوفه على الشاشة JavaScript هي اللي بتعمله.

وفي نفس الوقت، سنة 2009 ظهر **Node.js** (JavaScript برة المتصفح)، وسنة 2010 ظهر **npm**. والاتنين دول غيّروا شغل الفرونت إند نفسه: بقى عندك أدوات مكتوبة بـ JavaScript تشتغل على جهازك علشان **تجهّز** الكود قبل ما يروح للمتصفح. ودي مرحلة 5 و 6 من الكتاب.

> ⚠ **مشاكل الـ SPA اللي خلّت الناس ترجع تفكر**
>
> **أول تحميل بطيء:** المستخدم بيستنى JavaScript كلها تنزل وتشتغل قبل ما يشوف أي حاجة. و**محركات البحث والمعاينات** (لما تبعت لينك في واتساب) ممكن تشوف صفحة فاضية. و**الموبايلات الضعيفة** بتتعب من الـ JavaScript الكتير. المشاكل دي هي سبب المرحلة الجاية.

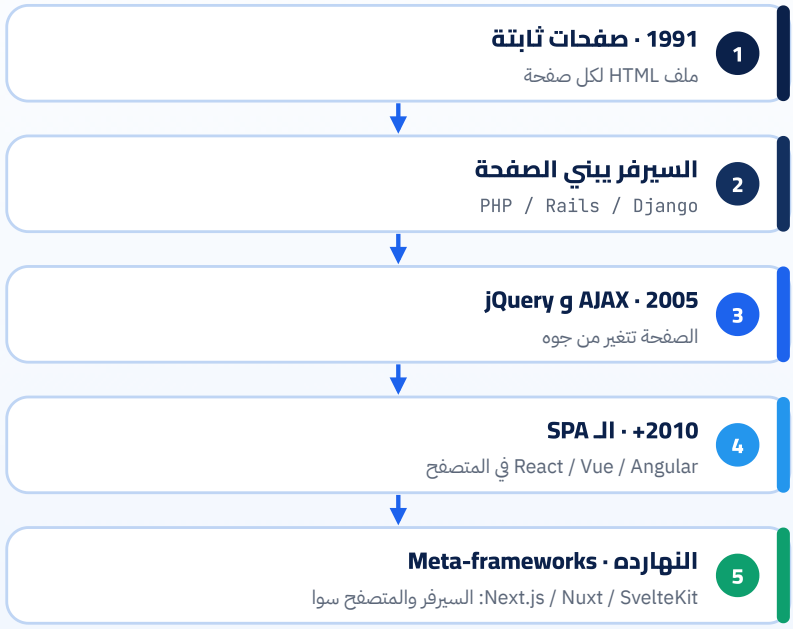
1.6 المرحلة 5 (النهاردة): الـ Meta-frameworks، رجوع ذكي للسيرفر

الحل ماكانش إننا نرمي React ونرجع لـ PHP. الحل كان إننا ناخد الأحسن من الاتنين: نكتب الـ Components بـ React (أو Vue أو Svelte)، بس نخلّي **السيرفر يبني أول HTML** علشان المستخدم يشوف حاجة فوراً، وبعدين JavaScript "تصحّي" الصفحة وتخليها تفاعلية (ده اسمه **Hydration**). والأدوات اللي بتعمل ده اسمها **Meta-frameworks**: Framework فوق Framework.

الـ Meta-framework	مبني على	تسمع عنه فين
Next.js	React	الأشهر في عالم React، ومن شركة Vercel (فصل 22)
(Framework mode) React Router	React	Remix اندمج فيه من React Router 7
Nuxt	Vue	المقابل لـ Next في عالم Vue
SvelteKit	Svelte	الطريقة الرسمية تبني تطبيق Svelte
Astro	أي Framework أو من غير	مواقع محتوى (مدونات ودوكيومنتيشن) بأقل JavaScript

جدول 1.1 أشهر الـ Meta-frameworks (للمعرفة، مش للمذاكرة دلوقتي)

ودي الحكاية كلها في صورة واحدة:



شكل 1.3 كل مرحلة حلّت مشكلة اللي قبلها، وجابت مشكلة جديدة. ولاحظ إن آخر مرحلة رجعت جزئياً لتاني مرحلة، بس بأدوات أذكى.

> **السؤال الذهبي لأي أداة جديدة**
>
> كل شهر هتسمع عن أداة جديدة. متجريش وراها. اسأل نفسك: **دي بتحل أنهي مشكلة؟ وأنا عندي المشكلة دي؟** لو مش عارف المشكلة، يبقى لسه مش محتاج الأداة. والكتاب ده كله مبني على السؤال ده.

1.7 انت فين، والكتاب هياخدك لفين

انت دلوقتي في نص كورس Jonas بتتعلم **اللغة**. وبعدين Node مع Maximilian، وبعدين React مع Jonas. وأول ما تفتح كورس React، أول درس هيقولك: "اكتب `npm create vite@latest`". وفجأة هيظهرلك 15 ملف و 100 Package في `node_modules`، وكلمات زي "dev server" و"build" و"JSX" و"ESLint". الكتاب ده موجود علشان اللحظة دي: لما توصلها، تكون عارف كل ملف بيعمل إيه وليه.

الموضوع	فين
لغة JavaScript: المتغيرات والـ Functions والـ Arrays والـ Objects والـ async	كورس Jonas (مش الكتاب ده)
HTML و CSS على مستوى أعمق	الكتاب ده، فصول 3 لـ 10
ليه React موجودة أصلاً	الكتاب ده، فصول 11 و 12
npm و package.json و Vite والـ Build	الكتاب ده، فصول 13 لـ 19 (وهتحتاجها في Node كمان)
Hooks و JSX و State بالتفصيل	كورس React
السيرفر و Express والـ Database	كورس Maximilian

جدول 1.2 هتتعلم إيه فين

المصدر MDN · How the web works · MDN · SPA (Single-page application) · MDN · AJAX · CERN · The birth of the web

ملخص الفصل

- الفرونت إند = اللي بيشتغل في المتصفح (HTML و CSS و JS). الباك إند = اللي على السيرفر. وبيتكلموا بـ HTTP.
- الويب عدّى بخمس مراحل: صفحات ثابتة، سيرفر يبني الصفحة، AJAX و jQuery، الـ SPA، الـ Meta-frameworks.
- كل مرحلة حلّت مشكلة اللي قبلها: كتابة الصفحات بإيدك، الـ Reload الكتير، المتصفحات المختلفة، الكود اللي بقى كوم، أول تحميل البطيء.
- Node و npm خلّوا JavaScript تشتغل على جهازك علشان تجهّز كود المتصفح، ودي أساس كل الأدوات الحديثة.
- قبل أي أداة: اسأل بتحل أنهي مشكلة.

صح ولا غلط؟

1. كود الفرونت إند ممكن يخبي سر (زي API key) لأن المستخدم مش بيشوفه. صح / غلط
2. الـ SPA بيبعت HTML جديد كامل من السيرفر مع كل ضغطة لينك. صح / غلط
3. Next.js مبني على React. صح / غلط

أسئلة مراجعة

1. اشرح الفرق بين الفرونت والباك بمثال غير المطعم.
2. إيه المشكلة اللي AJAX حلّها؟ وإيه المشكلة اللي jQuery حلّها؟ (هما مش نفس المشكلة)
3. اذكر مشكلتين في الـ SPA خلّوا الناس تعمل Meta-frameworks.
4. ليه Node.js مهم للفرونت إند، رغم إنه مش بيشتغل في المتصفح؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح `https://info.cern.ch/hypertext/WWW/TheProject.html` في Safari أو Chrome، واعمل View Source. عدّ كام Tag مختلف مستخدم.
2. متوسط افتح Gmail أو أي موقع كبير، وافتح تابة Network (كتاب الإنترنت فصل 18)، وفلتر على **Fetch/XHR**. اضغط على أي حاجة في الصفحة وشوف الطلبات اللي بتطلع من غير Reload: ده AJAX النهارده.
3. صعب اختار موقع بتستخدمه، وحاول تخمّن: ده Server-rendered ولا SPA ولا Meta-framework؟ دليلك: View Source (فيه محتوى حقيقي ولا `div` فاضي؟) وتابة Network. اكتب الأدلة اللي لقيتها.

02

المرحلة 1 · الصورة الكبيرة

الفصل التاني

خريطة الأدوات: كل اسم هتسمعه، وهو بيعمل إيه

Node و npm و Vite و React و Next.js و Tailwind و TypeScript و ESLint و Prettier و Vercel... لو حاسس إن الأسامي دي كلها شبه بعض ومش عارف مين بيعمل إيه، الفصل ده علشانك. هنحط كل أداة في طبقة، ونعرف السؤال اللي كل واحدة بتجاوبه، والأهم: مين فيهم بيوصل للمتصفح فعلاً، ومين بيشتغل على جهازك بس.

بعد الفصل ده هتكون عارف

- ☑ الطبقات: المنصة، واللغة، والأدوات، والمكتبات، والاستضافة
- ☑ الفرق بين Library و Framework و Meta-framework
- ☑ القاعدة الذهبية: المتصفح بيفهم HTML و CSS و JS بس
- ☑ أدوات وقت التطوير ضد كود وقت التشغيل
- ☑ تقرا package.json حقيقي وتعرف كل سطر تبع أنهي طبقة
- ☑ إيه اللي هتستخدمه مع React فعلاً

2.1 ليه الأسامي كتير كده؟

افتح أي إعلان وظيفة فرونت إند، هتلاقي: "React, TypeScript, Next.js, Tailwind, Vite, Jest, ESLint, Git, CI/CD". ده بيخوّف. بس الحقيقة إن الأسامي دي مش 9 حاجات من نفس النوع. دي **طبقات** مختلفة، كل طبقة بتجاوب سؤال واحد. ولما تعرف السؤال، الاسم بيبطّل يخوّف.



شكل 2.1 ست طبقات. كل حاجة هتسمع عنها تبع طبقة منهم. والكتاب ماشي من فوق لتحت تقريباً.

2.2 القاعدة الذهبية: المتصفح بيفهم 3 لغات بس

ده أهم سطر في الفصل: **المتصفح مش بيفهم غير HTML و CSS و JavaScript** (ومعاهم صور وخطوط وحاجات زي كده). مش بيفهم TypeScript، ولا JSX، ولا Sass، ولا كلاسات Tailwind كـ"لغة"، ولا `import React from 'react'` لوحدها.

طب إزاي كل ده بيشتغل؟ فيه أدوات بتشتغل **على جهازك** قبل ما الكود يروح للمتصفح، وبتحوّل كل حاجة للتلات لغات دول. ده اسمه **Build**:



شكل 2.2 كل الأدوات الحديثة بتعيش في الخطوة التانية. والمستخدم مش بيشوفها خالص: هو بيستلم الناتج بس. هنشوف ده بعينينا في فصل 18.

يعني Node بيشتغل في المتصفح؟

لأ. ودي أكتر حاجة بتلخبط. **Node** هو برنامج على الماك بتاعك بيشغّل JavaScript **برة** المتصفح. في الفرونت إند بنستخدمه علشان نشغّل **الأدوات** (Vite و ESLint...)، مش علشان نشغّل الموقع نفسه. وفي كورس Maximilian هتستخدمه لحاجة تانية خالص: تكتب السيرفر بيه.

2.3 الطبقات واحدة واحدة

الأداة	الطبقة	السؤال اللي بتجاوبه	فين في الكتاب
JS / CSS / HTML	المنصة	إيه اللي على الشاشة، وشكله، وبيتحرك إزاي؟	فصول 3 لـ 10
Web APIs (DOM و fetch و localStorage)	المنصة	إزاي JS تكلم المتصفح؟	كورس Jonas وكتاب APIs
TypeScript	لغة فوق اللغة	إزاي أكتشف الأخطاء قبل ما أشغّل؟	فصل 21
JSX	لغة فوق اللغة	إزاي أكتب شكل الـ UI جوه JS؟	فصل 12 وكورس React
Node.js	أدوات	إيه اللي هيشغّل أدوات JS على جهازي؟	فصل 13
npm (أو pnpm/yarn/bun)	أدوات	إزاي أنزّل مكتبات وأدوات وأتحكم في نسخها؟	فصول 13 لـ 16
Vite	أدوات	إزاي أشوف تعديلي فوراً وأنا بشتغل، وأطلّع نسخة سريعة للنشر؟	فصول 17 لـ 19
Prettier / ESLint	أدوات	الكود فيه غلطة محتملة؟ شكله موحّد؟	فصل 23
Angular / Svelte / Vue / React	Framework/Library	إزاي أبني UI كبير من غير ما الـ DOM يبوظ مني؟	فصول 11 و 12
Sass / CSS Modules / Tailwind	ستايل	إزاي أنظّم CSS في مشروع كبير؟	فصل 20
Next.js	Meta-framework	إزاي React يشتغل على السيرفر كمان، ومعاه Routing جاهز؟	فصل 22
Lighthouse	جودة	الموقع سريع ومتاح للكل؟	فصل 24
GitHub Pages / Netlify / Vercel	استضافة	أحط الموقع فين علشان الناس تفتحه؟	فصل 25

جدول 2.1 كل أداة بتجاوب على سؤال

2.4 Library ولا Framework ولا Meta-framework؟

الكلمات دي بتتقال كأنها نفس الحاجة، بس فيه فرق مهم اسمه **مين بينادي مين**:

Library (مكتبة)

انت اللي بتناديها لما تحتاجها. زي علبة عدة: تمسك المفك وقت ما تحب. مثال: `date-fns` للتواريخ. وReact بتوصف نفسها على موقعها إنها **Library** لا UI، لأنها بتعمل حاجة واحدة (الـ Components) وتسيبلك الباقي.

Framework (إطار)

هو اللي بينادي كودك. انت بتملا أماكن هو محددها، وهو بيقرر إمتى يشغّلها. زي بيت مبني وانت بتفرشه. Angular أوضح مثال: Routing و Forms و HTTP كلهم جوه. ودي الفكرة اللي اسمها **Inversion of control**.

والـ **Meta-framework** = Framework مبني فوق Library أو Framework، وبيزوّد الحاجات اللي ناقصاه علشان يبقى تطبيق كامل: Routing (صفحات بعناوين)، وتشغيل على السيرفر، وطرق Rendering (فصل 22). فـ Next.js = React + كل ده.

Best Practice

في الكلام اليومي الناس بتقول "React framework" ومحدش هيصححلك. المهم تفهم الفرق العملي: React لوحدها مش هتقولك تعمل الصفحات إزاي، ولا تجيب البيانات منين. ودي حاجات بتختارلها أدوات (أو Meta-framework بيختارها عنك).

2.5 package.json حقيقي: مين بيروح للمتصفح؟

ده الـ `package.json` اللي `create-vite` عمله لما طلبنا قالب React مع ESLint (نسخة 9.3.1 من الأداة، يوم كتابة الكتاب). متقلقش من التفاصيل، فصل 14 هيشرحه مفتاح مفتاح. بص بس على القسمين:

react-app/package.json

```
{
  "name": "react-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.3.1",
    "react-dom": "^19.3.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "@vitejs/plugin-react": "^6.1.2",
    "eslint": "^10.13.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.7",
```

```
    "globals": "^17.13.0",
    "vite": "^8.3.4"
  }
}
```

كل سطر تبع أنهي طبقة

1 **dependencies**: الكود ده **هيتبعت للمتصفح** جوه الـ Build. `react` = فكرة الـ Components والـ State، و `react-dom` = اللي بيحوّلها لعناصر DOM حقيقية في الصفحة.	`"react", "react-dom"`
2 **devDependencies**: أداة على جهازك. الـ Dev server والـ Build. المستخدم عمره ما هيستلم كود Vite.	`"vite"`
3 إضافة لـ Vite بتعلّمه يحوّل JSX لـ JS عادي، ويعمل Fast Refresh. أداة برضه.	`"@vitejs/plugin-react"`
4 كلهم أدوات فحص الكود (فصل 23). بيشتغلوا على جهازك وبس.	`"eslint", "@eslint/js", "eslint-plugin-…", "globals"`
5 أوصاف الأنواع لـ TypeScript (فصل 21). حتى لو مشروعك JS، المحرر بيستخدمها علشان يكمّلك الكود ويشرحلك الـ Functions.	`"@types/react", "@types/react-dom"`

يعني من 11 Package، اتنين بس هيوصلوا للمستخدم. والباقي "عدة الورشة". وده بيفسر رقم مخضّ هتشوفه: `node_modules` في المشروع ده طلع **90 ميجا** و**106** فولدر، بينما الـ JavaScript اللي طلعت من الـ Build للمتصفح كانت **222 كيلوبايت** بس (69 كيلو بعد الضغط). قسنا الاتنين فعلاً:

Terminal

```
user@MacBook ~ % du -sh node_modules
 90M    node_modules
user@MacBook ~ % npm run build
…
dist/assets/index-DykytF2W.css    4.10 kB │ gzip:  1.47 kB
dist/assets/index-DSXtc1GT.js   222.44 kB │ gzip: 69.28 kB
✓ built in 170ms
```

متقلقش من حجم node_modules

الـ 90 ميجا دي على جهازك بس، ومش بترفعها على Git (فيه سطر `node_modules` في `.gitignore`، كتاب Git فصل 5)، ومش بتروح للمستخدم. اللي يهمك هو حجم `dist/`. وده اللي هنقيسه ونحسّنه في فصل 24.

2.6 إيه اللي هتستخدمه فعلاً مع React؟

من كل الخريطة دي، دي التوليفة اللي غالباً هتقابلها في كورس React وفي أول شغل:

1. **Node + npm**: علشان تنزّل وتشغّل كل حاجة.
2. **Vite**: علشان تعمل المشروع وتشتغل عليه (`npm create vite@latest`).
3. **React**: الـ UI.
4. **CSS** عادي أو **Tailwind** أو **CSS Modules**: للشكل.
5. **ESLint** (جاي جاهز مع القالب) و**Prettier** في المحرر.
6. **GitHub + Git**، و**Vercel** أو **Netlify** للنشر.
7. وبعدها: **TypeScript**، و**Next.js** لما تحتاج سيرفر. دول مش لازم من أول يوم (فصل 27).

المصدر Node.js · Introduction · Vite · Getting Started · React · react.dev · MDN · Introduction to client-side frameworks

ملخص الفصل

- الأدوات في 6 طبقات: المنصة، ولغات فوقها، وأدوات على جهازك، ومكتبات/Frameworks، و Meta-frameworks، واستضافة.
- المتصفح بيفهم HTML و CSS و JS بس. أي حاجة تانية لازم تتحول بالـ Build.
- Node بيشغّل الأدوات على جهازك، مش الموقع في المتصفح.
- Library: انت بتناديها. Framework: هو بينادي كودك. Meta-framework: Framework فوق Library بيزوّد Routing وسيرفر.
- `dependencies` بتروح للمتصفح غالباً، و`devDependencies` عدة الورشة.
- حجم `node_modules` مش مهم. حجم `dist/` هو اللي مهم.

صح ولا غلط؟

1. المتصفح يقدر يشغّل ملف TypeScript مباشرة. صح / غلط
2. Vite بيتبعت للمستخدم مع الموقع. صح / غلط
3. React بتوصف نفسها كـ Library للـ UI. صح / غلط

أسئلة مراجعة

1. ارسم الطبقات الست من الذاكرة، وحط أداتين في كل طبقة.
2. إيه الفرق العملي بين Library و Framework؟ (فكّر في: مين بينادي مين)
3. ليه `node_modules` ممكن يبقى 90 ميجا والموقع نفسه 70 كيلو؟
4. Next.js بيحل مشاكل إيه React لوحدها مبتحلهاش؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل هات إعلان وظيفة Frontend حقيقي، وصنّف كل أداة فيه حسب الطبقات الست.
2. متوسط افتح `https://github.com/vitejs/vite/tree/main/packages/create-vite`، وادخل فولدر `template-react`، وقارن الـ `package.json` بتاعه باللي في الفصل. إيه اللي اتغير؟
3. صعب افتح موقع React مشهور، واعمل View Source ودوّر على أسماء ملفات الـ JS. تقدر تخمّن من الأسماء الأداة اللي عملت الـ Build؟ (تلميح: `/_next/` معناها إيه؟)

03

المرحلة 2 · HTML بجد

الفصل التالت

الـ Semantic HTML: إنك تقول للمتصفح «ده إيه» مش «شكله إيه»

في الكورسات بتتعلم HTML في ساعة وتجري على CSS و JavaScript. بس الـ HTML هو الأساس اللي قارئ الشاشة ومحرك البحث ووضع القراءة والكيبورد بيعتمدوا عليه. الفصل ده بيوريك الفرق بعينك: نفس الصفحة مرة بـ div وبس، ومرة بالعناصر الصح، وهنطبع شجرة الـ Accessibility الحقيقية اللي Chrome بيبنيها للاتنين.

بعد الفصل ده هتكون عارف

- يعني إيه Semantic، وليه div و span مش كفاية
- هيكل الصفحة: doctype و lang و dir و head
- الـ Landmarks: header و nav و main و aside و footer
- section ضد article ضد div
- العناوين h1 لـ h6 كفهرس للصفحة
- اللينك ضد الزرار: أهم قرار في HTML
- الصور والـ alt، والجداول للبيانات بس
- تشوف شجرة الـ Accessibility في DevTools

3.1 صفحتين شكلهم واحد

تخيّل صفحتين بنفس الـ CSS بالظبط، شكلهم على الشاشة متطابق. الأولى مكتوبة كلها `div` و `span` (الناس بتسميها **Div soup**، شوربة div). والتانية بالعناصر اللي ليها معنى:

soup.html

index.html

```
<div class="header">
  <div class="logo">Shop</div>
  <div class="nav">
    <span onclick="go('/')">Home</span>
    <span
onclick="go('/cart')">Cart</span>
  </div>
</div>
<div class="content">
  <div class="title">Shoes</div>
  <div class="btn"
onclick="buy()">Buy</div>
  <img src="x.png">
</div>
```

semantic.html

index.html

```
<header>
  <a href="/">Shop</a>
  <nav aria-label="Main">
    <a href="/">Home</a>
    <a href="/cart">Cart</a>
  </nav>
</header>
<main>
  <h1>Shoes</h1>
  <button type="button">Buy</button>
  <img src="x.png" alt="Red running
shoes">
</main>
```

المتصفح، غير إنه بيرسم الصفحة، بيبني شجرة تانية اسمها **Accessibility tree**: نسخة مبسّطة فيها "كل عنصر ده **إيه** واسمه إيه". دي اللي قارئ الشاشة (VoiceOver على الماك) بيقراها، والأدوات الآلية بتفحصها. طبعنا الشجرة الحقيقية من Chrome للصفحتين:

شجرة soup.html

Accessibility tree

```
RootWebArea "soup"
  StaticText "Shop"
  StaticText "Home"
  StaticText " "
  StaticText "Cart"
  StaticText "Shoes"
  StaticText "Buy"
```

شجرة semantic.html

Accessibility tree

```
RootWebArea "semantic"
  banner
    link "Shop"
    navigation "Main"
      link "Home"
      link "Cart"
  main
    heading "Shoes" level=1
    button "Buy"
    image "Red running shoes"
```

شوف الفرق: في الأولى كله **نص**. مفيش لينك ولا زرار ولا عنوان ولا صورة. حد بيستخدم قارئ شاشة هيسمع "Shop Home Cart Shoes Buy" كأنها جملة، ومش هيعرف يضغط على حاجة. وفي التانية كل حاجة ليها **دور** (Role). وجرّبنا كمان زرار Tab (اللي بيستخدمه أي حد بالكيبورد):

Tab order (اتسجّل فعلاً)

```
soup.html     : BODY → BODY → BODY …   (nothing gets focus)
semantic.html : Shop → Home → Cart → Buy
```

يعني إيه Semantic؟

Semantic = "ليه معنى". العنصر الـ Semantic بيقول **ده إيه** (عنوان، لينك، قايمة تنقّل)، مش **شكله إيه**. والشكل ده شغلة CSS. والمكسب إنك بتاخد حاجات كتير ببلاش: الـ Focus بالكيبورد، والـ Enter والـ Space على الزرار، وفهم قارئ الشاشة، ومحركات البحث، و Reader mode في Safari.

3.2 هيكل أي صفحة

index.html

```html
<!doctype html>
<html lang="ar" dir="rtl">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My Links</title>
    <meta name="description" content="Links to my work" />
  </head>
  <body>
    …
  </body>
</html>
```

`<!doctype html>`	مش Tag، ده إعلان: "اشتغل بالـ Standards mode الحديث". من غيره المتصفح بيدخل **Quirks mode** ويقلّد متصفحات التسعينات (والـ Box model بيتغير!).
`lang="ar"`	لغة الصفحة. قارئ الشاشة بيختار الصوت والنطق بيها، والمتصفح بيستخدمها في الترجمة والخطوط والتقسيم. Lighthouse بيعتبر غيابها مشكلة Accessibility (هنشوفها في فصل 24).
`dir="rtl"`	اتجاه الكتابة من اليمين لليسار. مهم جداً لينا: بيقلب ترتيب الـ Flexbox والـ Grid والنص. ومع الـ Logical properties (فصل 10) الـ CSS بيتقلب لوحده.
`meta charset`	الترميز. `UTF-8` دايماً، وإلا العربي يطلع رموز غريبة. ولازم يبقى في أول 1024 بايت من الملف.
`meta viewport`	من غيره الموبايل بيعرض الصفحة كأنها شاشة كمبيوتر ومصغّرة. ده أساس الـ Responsive (فصل 9).
`title`	اسم التابة، والعنوان في نتيجة جوجل، وأول حاجة قارئ الشاشة بيقولها.
`meta description`	الوصف اللي ممكن يظهر تحت العنوان في نتايج البحث. Lighthouse بيخصم نقط في SEO لو مش موجود (حصل معانا فعلاً في فصل 24).

3.3 الـ Landmarks: خريطة الصفحة

الـ **Landmarks** هي المناطق الكبيرة في الصفحة. قارئ الشاشة بيقدر يقفز بينهم مباشرة (زي ما انت بتبص بعينك على الهيدر أو المحتوى). وكل عنصر منهم بياخد Role في الشجرة:

العنصر	الـ Role	استخدامه
`<header>`	banner	الهيدر بتاع الصفحة: اللوجو والتنقّل. (لو جوه `article` بيبقى هيدر المقال بس، مش banner)
`<nav>`	navigation	مجموعة لينكات تنقّل رئيسية. لو فيه أكتر من واحد، ميّزهم بـ `aria-label`.
`<main>`	main	المحتوى الأساسي. **واحد بس** ظاهر في الصفحة.
`<aside>`	complementary	حاجة جانبية مرتبطة: Sidebar، "مقالات ذات صلة".
`<footer>`	contentinfo	الفوتر: حقوق ولينكات ثانوية.
`<section>` / `<form>`	region / form	بيبقوا Landmark بس لو ليهم اسم (`aria-label` أو عنوان مربوط).

جدول 3.1 العناصر الـ Landmark والـ Role بتاعها

section ولا article ولا div؟



شكل 3.1 سؤالين وخلاص. والـ div مش وحش: هو الاختيار الصح لما مفيش معنى، زي علبة بتعملها Flexbox.

3.4 العناوين: الفهرس اللي محدش بيشوفه

العناوين `h1` لـ `h6` مش "أحجام خط". دي **فهرس الصفحة**. وأكتر طريقة بيتنقل بيها مستخدمي قارئ الشاشة هي إنهم يقفزوا من عنوان لعنوان. فالقواعد:

1. `h1` واحد للصفحة، بيقول الصفحة عن إيه.
2. متنطّش مستويات: بعد `h2` يا `h2` يا `h3`، مش `h4`.
3. عايز خط كبير لحاجة مش عنوان؟ ده CSS، مش `h2`.
4. عايز عنوان بخط صغير؟ استخدم المستوى الصح وصغّره بالـ CSS.

جرّب: فهرس أي صفحة في ثانية

افتح أي صفحة، وافتح Console (كتاب الإنترنت فصل 20) واكتب:

```
[...document.querySelectorAll('h1,h2,h3')].map(h => h.tagName + ' ' + h.textContent.trim())
```

. هتشوف الفهرس اللي قارئ الشاشة شايفه. لو طلع ملخبط، الصفحة ملخبطة لناس كتير.

3.5 اللينك ضد الزرار: أهم قرار

أشهر غلطة في الفرونت إند كله: `<div onclick>` . والقاعدة بسيطة جداً:

<a href>: بيوديك مكان

- بيغيّر الـ URL (صفحة تانية، أو `#section`).
- تقدر تفتحه في تابة جديدة بـ ⌘+Click، وتنسخ عنوانه.
- بيشتغل بـ Enter.
- من غير `href` مبيبقاش لينك ولا بياخد Focus.

<button>: بيعمل حاجة

- بيعمل Action في نفس الصفحة: يفتح Menu، يضيف للسلة، يبعت Form.
- بيشتغل بـ Enter وSpace لوحده.
- بياخد Focus لوحده.
- جوه `form` الـ `type` الافتراضي `submit` (اتأكدنا: `button.type` رجّعت `"submit"`)، فاكتب `type="button"` لو مش عايزه يبعت.

ليه div onclick وحش؟

علشان تخلّي `div` يتصرف زي زرار لازم تزوّد: `role="button"` و `tabindex="0"` و Listener لـ Enter وSpace وستايل للـ Focus. ده كله الـ `button` بيديهولك ببلاش. والمشكلة هتتكرر معاك في React: `<div onClick={…}>` شغال بالماوس بس، و ESLint فيه Plugin مخصوص بيمسكها (`eslint-plugin-jsx-a11y`).

3.6 عناصر النص الصغيرة

العنصر	معناه	متستخدموش لـ
`<strong>`	مهم أو خطير أو عاجل	تتقيل الخط وبس (استخدم CSS أو `<b>`)
`<b>`	لفت نظر من غير أهمية زيادة (كلمة مفتاحية، اسم منتج)	تحذير مهم
`<em>`	تأكيد بيغيّر معنى الجملة	المايل للزينة
`<time datetime="2026-10-23">`	تاريخ أو وقت تقدر الأجهزة تفهمه	—
`<kbd>` / `<code>`	كود / زرار كيبورد	—
`<ol>` / `<ul>`	قايمة (قارئ الشاشة بيقول "قايمة فيها 5 عناصر")	مسافات وإزاحة

العنصر	معناه	متستخدموش لـ
` `	سطر جديد جوه نفس الفقرة (عنوان بوستة، شعر)	مسافات بين العناصر (دي `margin`)

جدول 3.2 عناصر بيتلخبط فيها الناس

3.7 الصور والجداول

تشريح الأمر

```
<figure>
<img src="chart.png" alt="Sales doubled from March to June" width="600" height="400">
<figcaption>Sales 2026</figcaption>
</figure>
```

1. `alt="…"` النص البديل: بيتقري بدل الصورة، وبيظهر لو الصورة مَحمّلتش. اكتب **المعلومة** اللي الصورة بتوصلها، مش "صورة لـ..". وفي التجربة فوق، الصورة اللي من غير `alt` في soup.html **اختفت من الشجرة خالص**.
2. `alt=""` (فاضي، مش غايب) = "صورة للزينة، تجاهلها". ده مختلف تماماً عن إنك متكتبش `alt`.
3. `width / height` بيخلّوا المتصفح يحجز مكان الصورة قبل ما تنزل، فالصفحة متتنططش. ده بيأثر على مقياس اسمه **CLS** (فصل 24).
4. `figure / figcaption` صورة (أو كود أو رسمة) مع تعليق مربوط بيها.

والجداول (`<table>`) للـ **بيانات** اللي ليها صفوف وأعمدة بس، زي جدول أسعار. أيام زمان كانوا بيعملوا بيها تصميم الصفحة كله، ودي حاجة انتهت مع Flexbox و Grid. والجدول الصح فيه `<caption>` (عنوانه) و`<th>` للعناوين، و`scope="col"` أو `scope="row"` لو الجدول معقد.

3.8 تشوف الشجرة بنفسك في DevTools

1. افتح الصفحة في Chrome، واعمل Inspect على أي عنصر (كتاب الإنترنت فصل 17).
2. في تابة **Elements**، جنب Styles، افتح تابة **Accessibility**.
3. هتلاقي الـ **Role** والـ **Name** للعنصر، وشجرة الـ Accessibility كلها لو فعّلت "Enable full-page accessibility tree".
4. وعلى الماك تقدر تجرّب بنفسك: ⌘+F5 بيشغّل ويقفل **VoiceOver**. جرّب صفحة من الاتنين.

VoiceOver على الماك

أول مرة هتحس إنه مزعج، وده طبيعي. شغّله دقيقتين بس على موقعك، وامشي بـ Tab. هتفهم في الدقيقتين دول أكتر من أي شرح. ولو المفتاح ماشتغلش، فعّله من System Settings ← Accessibility ← VoiceOver.

المصدر MDN · Semantics · MDN · HTML elements reference · MDN · HTML: A good basis for accessibility · Chrome DevTools · Accessibility features reference

ملخص الفصل

- الـ Semantic HTML بيقول العنصر إيه، والـ CSS بيقول شكله إيه.
- المتصفح بيبني Accessibility tree. الـ div soup بيطلع فيها نص وبس.
- `lang` و`dir` و`charset` و`viewport` و`title` في كل صفحة.
- Landmarks: `header` و`nav` و`main` (واحد) و`aside` و`footer`.
- العناوين فهرس: `h1` واحد ومتنطّش مستويات.
- بيوديك مكان = `a href`. بيعمل حاجة = `button`. ومفيش `div onclick`.
- `alt` بالمعلومة، أو `alt=""` للزينة، و`width` / `height` للصور.

صح ولا غلط؟

1. `<h3>` مناسب لأي نص عايزه أكبر شوية من الفقرة. صح / غلط
2. `alt=""` زي إنك متكتبش `alt` خالص. صح / غلط
3. `<button>` جوه `<form>` من غير `type` هيبعت الـ Form لما تضغطه. صح / غلط

أسئلة مراجعة

1. إيه الـ Accessibility tree؟ ومين بيستخدمها؟
2. إمتى تستخدم `article` وإمتى `section` وإمتى `div`؟
3. ليه `<div onclick>` مش زي `<button>`؟ اذكر 3 فروق.
4. إيه اللي بيحصل لو نسيت `<!doctype html>`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — حوّل soup.html لـ semantic.html بإيدك من غير ما تبص، وبعدين قارن.
2. متوسط — اعمل صفحة "بروفايل" فيها header و nav و main و article لكل مشروع و aside و footer، وافتح شجرة الـ Accessibility في DevTools واتأكد إن كل Landmark ظاهر.
3. صعب — اختار موقع حقيقي بتستخدمه، وامشي فيه بـ Tab بس من غير ماوس، وبعدين بـ VoiceOver. اكتب 5 مشاكل لقيتها وإزاي كنت هتصلحها بالـ HTML.

04

المرحلة 2 · HTML بجد

الفصل الرابع

الـ Forms: أقدم طريقة تبعت بيها بيانات، ولسه أذكى مما تتخيل

أغلب الناس بتكتب Form بـ input وزرار وبعدين تكتب 50 سطر JavaScript علشان تشيك إن الإيميل صح. بس المتصفح بيعرف يعمل ده لوحده من غير ولا سطر. الفصل ده بيشرح الـ Form من غير JavaScript الأول: إيه اللي بيتبعت وليه، وكل Attribute بيعمل إيه، ورسايل الـ Validation الحقيقية اللي Chrome بيطلعها، وبعدين إزاي تاخد الـ Form بـ JavaScript لما تحتاج.

بعد الفصل ده هتكون عارف

- الـ Form بيبعت إيه وفين: action و method و name
- label مربوط بـ input، وليه placeholder مش label
- أنواع الـ input والكيبورد اللي بيظهر على الموبايل
- الـ Validation المدمج: required و minlength و pattern و min/max
- autocomplete وليه بيفرق مع الـ Password managers
- fieldset و legend للـ Radio
- FormData و preventDefault: الـ Form من JavaScript
- الـ Validation في المتصفح للراحة، والسيرفر للأمان

4.1 Form من غير ولا سطر JavaScript

ده Form تسجيل حقيقي، مفيهوش أي JavaScript:

forms/index.html

```
<form action="/thanks" method="get">
  <label for="name">Name</label>
  <input id="name" name="name" required minlength="3" autocomplete="name">

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required autocomplete="email">

  <label for="age">Age</label>
  <input id="age" name="age" type="number" min="13" max="120">

  <label for="code">Code</label>
  <input id="code" name="code" pattern="[A-Z]{3}-[0-9]{3}" title="Like ABC-123">

  <fieldset>
    <legend>Plan</legend>
    <label><input type="radio" name="plan" value="free" checked> Free</label>
    <label><input type="radio" name="plan" value="pro"> Pro</label>
  </fieldset>
```

```
  <label><input type="checkbox" name="news" value="yes"> Newsletter</label>
  <input type="text" placeholder="no name attr">
  <button>Sign up</button>
</form>
```

ملينا الـ Form ده بـ Chrome (بـ Puppeteer، يعني متصفح حقيقي بنتحكم فيه بالكود)، وضغطنا Sign up. ده الـ URL اللي اتفتح بعدها بالحرف:

URL after submit

```
http://localhost:8097/thanks?name=Sara+Ali&email=sara%40example.com&age=25&code=ABC-123&plan=pro&news=yes
```

الـ URL ده جه منين	
1 من `action` : العنوان اللي البيانات بتروحله. لو مكتبتهوش: نفس الصفحة.	`/thanks`
2 `method="get"` بيحط البيانات في الـ Query string (كتاب الإنترنت فصل 6). المسافة بقت `+` والـ `@` بقت `%40` : ده الـ URL encoding.	`?name=Sara+Ali&…`
3 المفتاح هو الـ `name` **Attribute**، مش الـ `id` . الـ `id` للـ label والـ CSS والـ JS، والـ `name` للبيانات اللي بتتبعت.	`name= email= age= code=`
4 الـ Radio بيبعت `value` بتاع الواحد المختار بس. وكلهم نفس الـ `name` علشان يبقوا مجموعة واحدة (تختار واحد بس).	`plan=pro`
5 الـ Checkbox بيتبعت **بس لو متعلّم**. لو مش متعلّم، المفتاح مش هيظهر خالص (مش `news=no`).	`news=yes`
6 الـ input اللي ملوش `name` **متبعتش خالص**. ودي غلطة شائعة جداً: "كتبت في الخانة والسيرفر مش شايفها".	`(no placeholder field)`

GET ولا POST؟

`get` للبحث والفلترة (الـ URL ينفع يتحفظ ويتبعت). و`post` لأي حاجة بتغيّر بيانات أو فيها أسرار: التسجيل وتسجيل الدخول والدفع، لأن البيانات بتروح في الـ Body مش في الـ URL (فالـ Password ميتحفظش في الـ History). ده نفس كلام الـ Methods في كتاب APIs و JSON فصل 15.

4.2 الـ label: مش ديكور

تشريح الأمر

```
<label for="email">Email</label>
<input id="email" name="email" type="email">
```

1 `for="email"` بيربط الـ label بالـ input اللي الـ `id` بتاعه `email`. الربط ده بيعمل حاجتين: قارئ الشاشة بيقول "Email, edit text" لما توصل للخانة، والضغط على كلمة Email نفسها بيحط الـ Focus في الخانة (هدف أكبر للصباع على الموبايل).

2 `id="email"` لازم يبقى فريد في الصفحة كلها.

والطريقة التانية إنك تلف الـ input جوه الـ label: `<label><input type="checkbox"> Newsletter</label>`. ودي مريحة للـ Checkbox والـ Radio.

> **الـ placeholder مش label**
>
> الـ `placeholder` بيختفي أول ما تكتب، فتنسى الخانة دي كانت إيه. ولونه فاتح فصعب يتقري. استخدمه كـ **مثال** بس (`placeholder="sara@example.com"`)، وخلي الـ label موجود دايماً.

4.3 أنواع الـ input: كل نوع بيديك حاجة ببلاش

النوع	بيديك إيه ببلاش
`email`	Validation للشكل، وكيبورد فيه `@` على الموبايل
`tel`	كيبورد أرقام التليفون (من غير Validation: أشكال الأرقام كتير)
`url`	Validation إنه URL، وكيبورد فيه `/` و `.com`
`number`	أسهم زيادة ونقص، و `min` / `max` / `step`. (لأرقام زي الكود البريدي أو الكارت استخدم `text` + `inputmode="numeric"`)
`password`	بيخبي الحروف، والـ Password manager بيعرفه
`time` / `date`	Date picker من المتصفح نفسه
`checkbox` / `radio`	اختيار متعدد / اختيار واحد من مجموعة
`search`	زرار مسح، وبعض الكيبوردات بتكتب "Search" بدل Enter
`file`	اختيار ملفات، و `accept="image/*"` و `multiple`

جدول 4.1 أهم الأنواع

وفيه كمان `<select>` (قايمة اختيار) و `<textarea>` (نص طويل). والاتنين بياخدوا `name` و `required` زي أي input.

4.4 الـ Validation المدمج: رسايل حقيقية من Chrome

دي الحتة اللي ناس كتير متعرفهاش. المتصفح بيشيك لوحده، وبيطلع رسالة، وبيمنع الإرسال. سألنا Chrome عن كل خانة (الخاصية `validationMessage`) مرتين: والـ Form فاضي، وبعد ما كتبنا قيم غلط:

Chrome · empty form

```
name:  valid=false msg="Please fill out this field."
email: valid=false msg="Please fill out this field."
age:   valid=true  msg=""
code:  valid=true  msg=""
```

Chrome · after typing: Sa / sara@ / 10 / abc-12

```
name:  valid=false msg="Please lengthen this text to 3 characters or more (you are currently
using 2 characters)."
email: valid=false msg="Please enter a part following '@'. 'sara@' is incomplete."
age:   valid=false msg="Value must be greater than or equal to 13."
code:  valid=false msg="Please match the requested format."
```

`required`	الخانة لازم تتملي. لاحظ إن `age` و `code` كانوا `valid=true` وهما فاضيين: من غير `required`، الفاضي مقبول، والقواعد التانية بتتشيك بس لما تكتب حاجة.
`minlength / maxlength`	أقل وأكتر عدد حروف. و `maxlength` بيمنعك تكتب أكتر أصلاً.
`type="email"`	شيك شكل الإيميل. الرسالة نفسها بتقولك إيه الناقص (`'sara@' is incomplete`).
`min / max / step`	للأرقام والتواريخ.
`pattern="[A-Z]{3}-[0-9]{3}"`	Regular expression لازم القيمة **كلها** تطابقه (كأن فيه `^…$` لوحده). هنا: 3 حروف Capital، شرطة، 3 أرقام. و `abc-12` فشلت في الاتنين.
`title="Like ABC-123"`	مع `pattern`، المتصفحات بتعرضه كتلميح يوضح الشكل المطلوب (وبعضها بيضيفه لفقاعة الخطأ). بس مش جزء من `validationMessage` في Chrome زي ما شفنا، فمتعتمدش عليه لوحده واكتب الشكل المطلوب جنب الخانة.
`novalidate`	على الـ `form`: "اقفل الـ Validation بتاعك". بتستخدمه لو هتعمل رسايلك بنفسك بـ JS.

وتقدر تلوّن الخانات بالـ CSS حسب حالتها: `input:invalid` و `input:valid`. بس `:invalid` بتشتغل من أول ما الصفحة تفتح (الخانة الفاضية الـ required حمرا قبل ما تلمسها!). علشان كده فيه `:user-invalid`: بتشتغل بس بعد ما المستخدم يتعامل مع الخانة.

style.css

```
input:user-invalid {
  border-color: crimson;
}
```

4.5 autocomplete: صاحبك اللي محدش بيكلمه

الـ `autocomplete` بيقول للمتصفح والـ Password manager (زي Passwords app على الماك) الخانة دي فيها إيه بالظبط، فيملاها صح:

القيمة	للخانة
`name` / `email` / `tel`	الاسم والإيميل والتليفون
`username`	اسم المستخدم في تسجيل الدخول
`current-password`	الباسورد في تسجيل الدخول
`new-password`	الباسورد في التسجيل أو التغيير (المتصفح بيقترح باسورد قوي)
`one-time-code`	كود الـ SMS، والـ iPhone والماك بيقترحوه من الرسايل
`street-address` / `postal-code`	العنوان

جدول 4.2 أشهر القيم

4.6 الـ Form من JavaScript

في التطبيقات الحديثة غالباً مش عايز الصفحة تتقلب. عايز تاخد البيانات وتبعتها بـ `fetch`. والطريقة الصح إنك تفضل مستخدم الـ `form` (علشان الـ Validation والـ Enter والـ Accessibility) وتمسك حدث `submit`:

JavaScript

```
const form = document.querySelector('form')

form.addEventListener('submit', async (event) => {
  event.preventDefault()
  const data = Object.fromEntries(new FormData(form))
  const res = await fetch('/api/signup', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  })
  if (!res.ok) showError(await res.json())
})
```

`'submit'`	مش `'click'` على الزرار. الـ `submit` بيتنده بالزرار **وبـ Enter**، **وبعد** ما الـ Validation المدمج ينجح. لو فيه خانة غلط، الحدث مش هيتنده أصلاً.
`event.preventDefault()`	"متعملش السلوك الافتراضي" (اللي هو تقليب الصفحة لـ `/thanks`).
`new FormData(form)`	بيجمع كل الخانات اللي ليها `name`، بنفس قواعد الإرسال (الـ Checkbox بس لو متعلّم). طبعناه فعلاً: `[["code","ABC-123"],["plan","pro"],["news","yes"]]`.
`Object.fromEntries(…)`	بيحوّل الأزواج دي لـ Object عادي، فتعمله `JSON.stringify`. لاحظ إن كل القيم **Strings**، حتى `"25"`.
`fetch + res.ok`	بالظبط زي كتاب APIs و JSON فصل 10.

حاجة مشتركة: نفس الكلام في React

في React هتكتب `<form onSubmit={handleSubmit}>` وجوه الـ Function `event.preventDefault()`. نفس الحدث بالظبط ونفس الـ Attributes (`required` و`type="email"` …). اللي هيتغير إن الخانات ممكن تبقى "Controlled" (القيمة في State). فكل اللي في الفصل ده هيفضل صح.

الـ Validation في المتصفح مش حماية

أي حد يقدر يفتح DevTools ويشيل `required`، أو يبعت الطلب بـ curl من غير Form خالص (كتاب APIs فصل 8). فالـ Validation في المتصفح علشان **راحة المستخدم** (يعرف غلطته فوراً). أما **الأمان** فلازم السيرفر يشيك تاني على كل حاجة. ودي هتعملها في كورس Maximilian.

المصدر MDN · Your first form · MDN · Client-side form validation · MDN · autocomplete · MDN · FormData · MDN · :user-invalid

ملخص الفصل

- `action` فين، و`method` إزاي (`get` في الـ URL، و`post` في الـ Body)، و`name` هو المفتاح.
- الخانة من غير `name` متتبعتش، والـ Checkbox بيتبعت بس لو متعلّم.
- كل input ليه `label` مربوط بـ `for` / `id`، والـ placeholder مثال مش label.
- الـ `type` الصح بيديك Validation وكيبورد موبايل مناسب.
- `required` و`minlength` و`pattern` و`min` / `max` بيشتغلوا من غير JS.
- من JS: اسمع `submit`، و`preventDefault()`، و`new FormData(form)`.
- الـ Validation في المتصفح للراحة، والسيرفر للأمان.

صح ولا غلط؟

1. الـ input اللي ليه `id` بس من غير `name` بيتبعت عادي. صح / غلط
2. خانة `pattern` من غير `required` وفاضية بتعتبر valid. صح / غلط
3. طالما عملت `required` في الـ HTML، السيرفر مش محتاج يشيك. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `id` و `name` في الـ input؟
2. ليه بنسمع `submit` على الـ form بدل `click` على الزرار؟
3. إيه الفرق بين `:invalid` و `:user-invalid`؟
4. إمتى تستخدم `new-password` وإمتى `current-password`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اكتب Form تسجيل دخول فيه username و password بالـ `autocomplete` الصح، وجرّب Safari وهو بيقترحلك.
2. متوسط اعمل Form بحث بـ `method="get"` فيه خانة و `select` للترتيب، وشوف الـ URL بعد الإرسال. وبعدين حوّله يبعت بـ `fetch` من غير ما الصفحة تتقلب.
3. صعب اعمل خانتين Password و Confirm، واستخدم `setCustomValidity('Passwords do not match')` على التانية في حدث `input` علشان المتصفح نفسه يمنع الإرسال ويعرض رسالتك.

05

المرحلة 2 · HTML بجد

الفصل الخامس

الـ Accessibility: الموقع لكل الناس، مش للي زيك بس

انت بتستخدم ماوس وشاشة واضحة وعينيك سليمة. بس فيه ناس بتستخدم الكيبورد بس، أو قارئ شاشة، أو مكبّر، أو إيدها مكسورة النهارده، أو قاعدة في الشمس. الـ Accessibility (a11y) مش ميزة إضافية: هي إن الموقع يشتغل. والخبر الحلو إن 80% منها بيجي من HTML صح وكام قاعدة CSS. الفصل ده هو الكام قاعدة دول.

بعد الفصل ده هتكون عارف

- مين بيستفيد: إعاقة دائمة ومؤقتة وظرفية
- WCAG في سطرين: المبادئ الأربعة ومستوى AA
- الكيبورد: Tab والـ Focus الظاهر و skip link
- التباين: 4.5:1 للنص العادي
- متعتمدش على اللون لوحده
- ARIA: القاعدة الأولى إنك متستخدمهاش
- aria-label و aria-expanded و aria-live
- prefers-reduced-motion
- Checklist يدوي في 5 دقايق

5.1 مين بيستفيد؟ انت كمان

كلمة **a11y** اختصار لـ Accessibility (حرف a، و11 حرف، وحرف y). وناس كتير فاكرة إنها "للمكفوفين بس". بس بص على الجدول ده:

الاحتياج	دائم	مؤقت	ظرفي
من غير ماوس	مشكلة حركية	إيد مكسورة	شغّال بالكيبورد بسرعة (Power user)
صعوبة في الرؤية	ضعف نظر أو عمى	بعد عملية في العين	موبايل في الشمس
من غير صوت	صمم	التهاب في الودن	في مواصلات من غير سماعة (Captions)
تركيز أقل	اضطراب انتباه	تعبان ومنمتش	بتستخدم الموقع وانت ماشي

جدول 5.1 نفس الاحتياج بتلات أشكال

يعني كل حاجة بتعملها للـ Accessibility بتحسّن الموقع لكل الناس. وفي دول كتير ليها قوانين بتلزم المواقع بيها، وعقود الشركات الكبيرة بتطلبها.

5.2 WCAG في سطرين

WCAG (Web Content Accessibility Guidelines) هي المواصفة الرسمية من W3C، وآخر نسخة نهائية هي **2.2** (أكتوبر 2023). مبنية على 4 مبادئ اختصارها **POUR**:

Perceivable · ممكن يتدرك

المعلومة توصل بأكتر من حاسة: `alt` للصور، Captions للفيديو، تباين كفاية.

Operable · ممكن يتشغّل

كل حاجة تشتغل بالكيبورد، ووقت كفاية، ومفيش حاجة بتومض بشكل خطير.

Understandable · مفهوم

لغة الصفحة متحددة، ورسايل أخطاء واضحة، والتصرف متوقع.

Robust · متين

كود صح تقدر الأدوات المساعدة تفهمه: HTML سليم وأدوار واضحة.

وكل قاعدة ليها مستوى: **A** (الحد الأدنى)، و**AA** (ده اللي الشركات والقوانين بتطلبه عادة)، و**AAA** (الأعلى). خلي هدفك **AA**.

5.3 الكيبورد: أول اختبار وأسهل اختبار

افصل الماوس واستخدم الموقع بـ `Tab` (لقدّام) و`⇧+Tab` (لورا) و`Enter` و`Space` والأسهم. لازم تقدر توصل لكل حاجة وتشغّلها، وتشوف انت فين.

Safari على الماك مش بيعمل Tab على اللينكات افتراضياً

في Safari، الـ Tab بيروح للخانات والزراير بس. علشان يروح للينكات كمان: Safari ← Settings ← Advanced ← فعّل "Press Tab to highlight each item on a webpage"، أو استخدم `⌥+Tab`. وفي macOS نفسه فيه إعداد Keyboard navigation في Keyboard ← System Settings. فلو بتختبر، اختبر في Chrome كمان.

الـ Focus لازم يبان

لما توصل لعنصر بالـ Tab، المتصفح بيرسم حواليه إطار (Focus ring). وأشهر جريمة في CSS:

style.css

```css
/* لا تعمل كده أبداً */
*:focus {
  outline: none;
}
```

كده اللي بيستخدم الكيبورد مش هيعرف هو فين. ولو الإطار مش عاجبك في الشكل، غيّره، متشيلوش. واستخدم `:focus-visible` : بيظهر مع الكيبورد بس، مش لما تضغط بالماوس:

style.css

```
button:focus-visible {
  outline: 3px solid #1d63ed;
  outline-offset: 2px;
}
```

ده نفس اللي قالب Vite عامله في `style.css` بتاعه للزرار `.counter` (هنشوفه في فصل 18).

الترتيب والـ tabindex

ترتيب الـ HTML	الـ Tab بيمشي بترتيب العناصر في الـ HTML، مش بترتيبها على الشاشة. فلو قلبت الترتيب بالـ CSS (`order` في Flexbox أو Grid)، الكيبورد هيتنطط. خلي ترتيب الـ HTML منطقي.
`tabindex="0"`	"خلّي العنصر ده ياخد Focus في ترتيبه الطبيعي". نادراً ما تحتاجه لو استخدمت العناصر الصح.
`tabindex="-1"`	"ميتوصلّوش بالـ Tab، بس أقدر أحط عليه Focus بالكود" (`el.focus()`)، زي رسالة خطأ أو Modal.
`tabindex="5"`	أي رقم موجب: **متستخدموش**. بيكسر الترتيب الطبيعي للصفحة كلها.

الـ Skip link

تخيّل كل صفحة فيها 30 لينك في الهيدر، ومستخدم الكيبورد لازم يعدّي عليهم كل مرة. الحل: أول لينك في الصفحة "اقفز للمحتوى"، مخفي لحد ما ياخد Focus:

index.html

```
<a class="skip" href="#main">Skip to content</a>
…
<main id="main">…</main>
```

style.css

```
.skip {
  position: absolute;
  inset-inline-start: 1rem;
  top: -100px;
}
.skip:focus {
  top: 1rem;
}
```

5.4 التباين والألوان

Contrast ratio = الفرق في الإضاءة بين لون النص ولون الخلفية، من 1:1 (نفس اللون) لـ 21:1 (أسود على أبيض). ومستوى AA بيطلب:

النوع	أقل تباين
نص عادي	4.5:1
نص كبير (حوالي 24px، أو 18.66px تقيل)	3:1
أيقونات وحدود الخانات (حاجات مش نص)	3:1

جدول 5.2 حدود التباين في WCAG (مستوى AA)

وفي فصل 24، Lighthouse مسك فعلاً نص رمادي `#bbb` على أبيض في صفحة تجربة، وقال: "Background and foreground colors do not have a sufficient contrast ratio". وفي DevTools، لما تعمل Inspect على نص وتضغط على مربع اللون في تابة Styles، هتلاقي الـ Contrast ratio مكتوب وجنبه علامة صح أو غلط.

> **متعتمدش على اللون لوحده**
>
> "الخانات الحمرا فيها أخطاء" مش هتوصل لحد عنده عمى ألوان (وده تقريباً 1 من كل 12 راجل). زوّد أيقونة أو كلمة: "⚠ Email is required". ونفس الكلام في الرسوم البيانية واللينكات جوه النص (خلي ليها Underline).

5.5 ARIA: القاعدة الأولى إنك متستخدمهاش

ARIA = Attributes بتبدأ بـ `role` و`aria-`، بتغيّر اللي الـ Accessibility tree بتقوله عن العنصر. ودي **مبتغيرش السلوك**: `role="button"` مش بتخلي الـ div يشتغل بالكيبورد. هي بتقول "أنا زرار" وبس، وانت عليك تعمل الباقي.

> **القاعدة الأولى لـ ARIA (من W3C نفسها)**
>
> لو فيه عنصر HTML بيعمل اللي انت عايزه بالسلوك والمعنى، **استخدمه** بدل ما تاخد عنصر تاني وتلبّسه ARIA. يعني `<button>` مش `<div role="button">`. وتقرير WebAIM Million (فحص آلي لمليون صفحة رئيسية، فبراير 2026) لقى إن الصفحات اللي فيها ARIA فيها في المتوسط **59.1** خطأ مكتشف مقابل **42** للصفحات اللي مفيهاش. التقرير نفسه بيقول إن ده مش معناه إن ARIA هي السبب (الصفحات دي أعقد كمان)، بس الدرس واضح: ARIA الغلط أوحش من مفيش.

بس فيه حالات ARIA فيها ضرورية، ودي أشهرها:

`aria-label="Close"`	اسم لعنصر مفيهوش نص: زرار عليه أيقونة ✕ بس. من غيره قارئ الشاشة هيقول "button" وخلاص. (وفي فصل 24 Lighthouse قال بالظبط: "Buttons do not have an accessible name").
`aria-expanded="false"`	على زرار بيفتح ويقفل حاجة (Menu، Accordion). بتغيّرها لـ `"true"` بالـ JS لما يفتح، فقارئ الشاشة يقول "collapsed" أو "expanded".

`aria-controls="menu-1"`	بيربط الزرار بالعنصر اللي بيتحكم فيه.
`aria-live="polite"`	على علبة الرسايل: لما محتواها يتغير، قارئ الشاشة يقراه من غير ما المستخدم يروحله. مفيد لـ "اتضاف للسلة" أو أخطاء الـ Form.
`aria-hidden="true"`	"تجاهل ده" لحاجة للزينة، زي أيقونة جنب كلمة. قالب Vite بيستخدمها على أيقونات الـ SVG بتاعته فعلاً.
`aria-describedby="hint-1"`	بيربط الخانة بنص توضيحي أو رسالة خطأ، فبيتقري بعد اسمها.

صح زرار Menu

```
<button type="button" aria-expanded="false" aria-controls="main-menu">
  <svg aria-hidden="true">…</svg>
  Menu
</button>
<ul id="main-menu" hidden>…</ul>
```

5.6 الحركة والـ prefers-reduced-motion

فيه ناس الحركة الكتير (Parallax، تكبير، Scroll animations) بتعملهم دوخة فعلاً. والماك فيه إعداد: System Settings ← Accessibility ← Display ← Reduce motion. والـ CSS يقدر يقراه:

style.css

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

ونفس الفكرة `prefers-color-scheme: dark` للوضع الليلي (قالب Vite بيستخدمه، وهنشوفه في فصل 9).

5.7 Checklist يدوي في 5 دقايق

1. **Tab** في الصفحة كلها: كل حاجة بتتوصل؟ الـ Focus باين؟ الترتيب منطقي؟
2. **Zoom 200%** (⌘++ كذا مرة): مفيش حاجة اتقطعت أو اتغطت؟
3. **العناوين**: `h1` واحد ومفيش قفز؟ (سطر الـ Console من فصل 3)
4. **الصور**: كل صورة ليها `alt` مناسب أو `alt=""`؟
5. **الـ Forms**: كل خانة ليها label؟ رسايل الخطأ مكتوبة مش لون بس؟
6. **التباين**: DevTools على النصوص الفاتحة.

7. **VoiceOver** (⌘+F5) دقيقة واحدة.

8. وفي الآخر أداة آلية (Lighthouse أو axe، فصل 24). بس خلي بالك: الأدوات الآلية بتمسك **جزء** بس من المشاكل. في التجربة اللي هنعملها، Lighthouse مامسكش الـ `div onclick` خالص.

المصدر

W3C · Using ARIA (the first rule) · W3C · WCAG 2.2 · W3C WAI · Introduction to Web Accessibility
WebAIM · The WebAIM Million · web.dev · Learn Accessibility · MDN · Accessibility

ملخص الفصل

- الـ a11y لكل الناس: إعاقة دائمة أو مؤقتة أو ظرف.
- WCAG 2.2، والمبادئ POUR، وهدفك مستوى AA.
- كل حاجة بالكيبورد، والـ Focus ظاهر (`:focus-visible`)، ومفيش `outline: none` من غير بديل.
- ترتيب الـ HTML = ترتيب الـ Tab. ومفيش `tabindex` موجب.
- تباين 4.5:1 للنص العادي و 3:1 للكبير، ومتعتمدش على اللون لوحده.
- ARIA آخر حل: العنصر الصح الأول. وأهمها `aria-label` و`aria-expanded` و`aria-live`.
- احترم `prefers-reduced-motion`.

صح ولا غلط؟

1. `role="button"` على div بتخليه يشتغل بالـ Enter والـ Space لوحده. صح / غلط
2. `tabindex="-1"` بيخلي العنصر يتوصل بالـ Tab. صح / غلط
3. أداة آلية زي Lighthouse بتمسك كل مشاكل الـ Accessibility. صح / غلط

أسئلة مراجعة

1. اذكر مثال لإعاقة ظرفية بيستفيد صاحبها من الـ Captions.
2. ليه `:focus-visible` أحسن من إنك تشيل الـ outline؟
3. إيه القاعدة الأولى لـ ARIA؟ وليه ARIA الغلط أوحش من مفيش؟
4. إمتى تستخدم `aria-live`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — زوّد Skip link لصفحة البروفايل بتاعتك من فصل 3، واختبره بالـ Tab.
2. متوسط — اعمل زرار Menu بيفتح ويقفل قايمة، وغيّر `aria-expanded` و`hidden` بالـ JS، واسمعه بـ VoiceOver.
3. صعب — اعمل Form فيه رسايل خطأ تحت كل خانة مربوطة بـ `aria-describedby`، وعلبة `aria-live` بتقول عدد الأخطاء بعد الإرسال. واختبره بالكيبورد و VoiceOver بس.

06

المرحلة 3 · CSS بجد

الفصل السادس

الـ Cascade والـ Specificity والـ Box model: ليه الـ CSS بتاعي مش شغال؟

أشهر جملة في الفرونت إند: «أنا كاتب color: red والزرار لسه أزرق!». والحل اللي الناس بتلجأ له: !important في كل حتة، لحد ما الملف يبقى حرب. الفصل ده بيشرح القواعد اللي المتصفح بيستخدمها علشان يقرر مين يكسب، بالترتيب، وبنتيجة حقيقية من Chrome. وبعدين الـ Box model: ليه العنصر اللي عرضه 200px طلع 250px.

بعد الفصل ده هتكون عارف

- الـ Cascade: المتصفح بيختار قيمة واحدة إزاي من كذا قاعدة
- الترتيب: الأهمية والمصدر، والـ Layers، والـ Specificity، والترتيب في الملف
- تحسب الـ Specificity كـ (Elements ,Classes ,IDs)
- الـ inline و !important ومتى (نادراً) تستخدمهم
- :where() و :is() وتأثيرهم على الـ Specificity
- الوراثة (Inheritance): مين بيتورث ومين لأ
- الـ Box model: content و padding و border و margin
- box-sizing: border-box وليه كل مشروع بيبدأ بيه
- تقرا تابة Styles و Computed في DevTools

6.1 مين يكسب؟

عندنا زرار واحد: `<button id="buy" class="btn">` جوه `<div class="card">`. وكتبنا له 3 قواعد بتقول ألوان مختلفة:

style.css

```
button      { color: gray; }
.card .btn  { color: blue; }
.btn        { color: red; }
```

اللون النهائي إيه؟ ناس كتير هتقول **أحمر**، علشان هو الأخير في الملف. بس Chrome قال (سألناه بـ `getComputedStyle`):

Console · Chrome DevTools

```
> getComputedStyle(document.querySelector('#buy')).color
'rgb(0, 0, 255)'
```

أزرق. ليه؟ علشان الترتيب في الملف هو **آخر** معيار المتصفح بيبص عليه، مش أوله. والعملية كلها اسمها **الـ Cascade** (الشلال): كل القيم اللي بتحاول تتحط على نفس الخاصية لنفس العنصر بتعدّي على مراحل، وكل مرحلة بتصفّي، لحد ما يفضل واحد.



شكل 6.1 الشلال مبسّط بالترتيب اللي يهمك. أول مرحلة فيها فايز واضح بتنهي الموضوع، واللي بعدها ميتبصّش عليه. ده تبسيط للمواصفة، اللي فيها كمان الـ Shadow DOM والـ @scope.

6.2 الـ Specificity: تحسبها في 10 ثواني

الـ **Specificity** (الدقة) هي "قد إيه الـ Selector محدد". والمتصفح بيحسبها كـ **3 أرقام** (A, B, C):

العمود	بيعد إيه	أمثلة
A	IDs	`#buy`
B	Classes و Attributes و Pseudo-classes	`.btn` و `[type="email"]` و `:hover` و `:focus-visible`
C	Elements و Pseudo-elements	`button` و `p` و `::before`
صفر	الـ Universal والـ Combinators	`*` و `>` و `+` و `~` والمسافة

جدول 6.1 عدّ ده في الـ Selector

والمقارنة زي الأرقام بالظبط: العمود الشمال الأول. **الـ ID واحد يكسب أي عدد من الـ Classes**. يعني (1,0,0) أكبر من (0,15,0).

الـ Selector	(A, B, C)	النتيجة
`button`	(0, 0, 1)	خسر
`.btn`	(0, 1, 0)	خسر رغم إنه الأخير
`.card .btn`	(0, 2, 0)	**كسب**: أزرق
`#buy.btn`	(0, 1, 1)	كان هيكسب أي حاجة فوق

الـ Selector	(A, B, C)	النتيجة
`nav ul li a:hover`	(0, 1, 4)	أقل من `.card .btn` رغم إنه طويل

جدول 6.2 نحسب قواعد المثال

DevTools بيحسبها لك

في Chrome، في تابة Styles، حط الماوس على أي Selector: هيظهرلك "Specificity: (0,2,0)". والقواعد اللي خسرت بتلاقيها **مشطوبة** بخط. ودي أسرع طريقة تجاوب "ليه الـ CSS بتاعي مش شغال": شوف هو مشطوب ولا مش موجود أصلاً (يبقى الـ Selector غلط أو الملف مش متحمّل).

6.3 الـ inline و !important

جرّبنا زرار تاني عليه `style="color: black"` مباشرة:

Console · Chrome DevTools

```
> getComputedStyle(document.querySelector('.card:nth-child(2) .btn')).color
'rgb(0, 0, 0)'
```

الـ **inline style** كسب كل الـ Selectors، حتى اللي فيها IDs، لأنه مرحلة قبل الـ Specificity. ودي مشكلته: مش هتعرف تغيّره من ملف CSS غير بـ `!important`. وده بيفسّر حاجة هتشوفها في React: `style={{ color: 'black' }}` بيطلع inline، فاستخدمه للقيم اللي بتتحسب بالـ JS بس (زي مكان عنصر بيتسحب).

!important: سلاح نووي

`color: red !important` بتنقل القاعدة لمرحلة 1، فبتكسب كل اللي مش `!important`. ولو اتنين `!important` اتخانقوا، نرجع للـ Specificity بينهم. والنتيجة الطبيعية: كل واحد يحط `!important` علشان يكسب اللي قبله. استخدامها المقبول: Utility classes لازم تكسب دايماً، أو تتغلب على CSS من مكتبة مش بتاعتك، أو قاعدة الـ reduced-motion من فصل 5.

6.4 :is() و :where(): نفس الشكل، Specificity مختلفة

تشريح الأمر

```
:is(header, footer) a { color: inherit; }
:where(header, footer) a { color: inherit; }
```

1. `:is(…)` "أي واحد من دول". بتختصر `header a, footer a`. والـ Specificity بتاعتها = **أدق** Selector جواها. هنا (0,0,1) + `a` = (0,0,2).
2. `:where(…)` نفس المعنى بالظبط، بس الـ Specificity بتاعتها **دايماً صفر**. فالقاعدة دي (0,0,1) بس. مفيدة جداً لـ CSS أساسي (Reset أو مكتبة) عايزه يتغيّر بسهولة.

6.5 الـ @layer: ترتيب الطبقات بإيدك

الـ **Cascade layers** (من 2022 في كل المتصفحات) حلّت حرب الـ Specificity من جذورها: بتقسّم الـ CSS لطبقات، وترتيب الطبقات **أهم من** الـ Specificity. جرّبنا ده على نفس الزرار:

style.css

```
@layer base, components;

@layer components { .btn { color: green; } }
@layer base       { #buy { color: orange; } }

button     { color: gray; }
.card .btn { color: blue; }
.btn       { color: red; }
```

اللون طلع **أزرق** برضه (`rgb(0, 0, 255)`). رغم إن `#buy` فيه ID! والسبب:

`@layer base, components;`	بيعلن الترتيب: `base` الأول، وبعدها `components`. والطبقة **الأحدث تكسب**. فـ `.btn` الأخضر في components بيكسب `#buy` البرتقالي في base، **رغم** الـ ID.
القواعد اللي برة أي `@layer`	بتتعامل كأنها في طبقة أخيرة مخفية، فبتكسب **كل** الطبقات. علشان كده الأزرق كسب الاتنين.

وده هيقابلك أول ما تستخدم Tailwind 4: الـ CSS بتاعه متقسّم في `@layer theme, base, components, utilities`. وده أول سطر حقيقي في `node_modules/tailwindcss/index.css`، وهنرجعله في فصل 20.

6.6 الوراثة: مين بيتورث؟

كتبنا `p { color: navy; }` بس، وجوه الـ p فيه `span` و `a`:

getComputedStyle (Chrome)

```
p color:                rgb(0, 0, 128)
span color (inherits):  rgb(0, 0, 128)
a color (UA style):     rgb(0, 0, 238)
```

`span` ورث navy	خصايص النص (`color` و `font-*` و `line-height` و `text-align`) **بتتورث** من الأب. علشان كده بتحط الخط على `body` مرة واحدة.
`a` مورثش	لأن المتصفح نفسه عنده CSS افتراضي (**User-agent stylesheet**) بيدي اللينكات أزرق (`rgb(0, 0, 238)`). والقيمة الموروثة أضعف من **أي** قاعدة بتستهدف العنصر نفسه، حتى قاعدة المتصفح. والحل: `a { color: inherit; }`.

مبتتورثش	خصايص الصندوق: `margin` و `padding` و `border` و `width` و `background` و `display`. (تخيّل لو كل ابن ورث border أبوه!)
`inherit / initial / unset`	كلمات بتجبر الوراثة، أو ترجّع القيمة الافتراضية من المواصفة، أو "اتورث لو بيتورث، وإلا initial".

6.7 الـ Box model: ليه 200 طلع 250؟

كل عنصر في الصفحة **صندوق** من 4 طبقات من جوه لبرة:



شكل 6.2 الأربع طبقات. في تابة Computed في DevTools هتلاقي رسمة الصندوق ده بالأرقام الحقيقية لأي عنصر.

وجرّبنا عنصرين بنفس الـ CSS بالظبط: `width: 200px; padding: 20px; border: 5px solid; margin: 10px;`، والفرق الوحيد إن التاني عليه `box-sizing: border-box`:

offsetWidth (Chrome)

```
#b1 offsetWidth: 250    (content-box, the default)
#b2 offsetWidth: 200    (border-box)
```

`content-box` (الافتراضي)	الـ `width` = عرض المحتوى بس. والـ padding والـ border بيتزادوا عليه: 200 + 2×20 + 2×5 = **250**. فلو عملت عمودين كل واحد `width: 50%` وفيهم padding، مش هيكفّوا جنب بعض.
`border-box`	الـ `width` = لحد الـ border. والمتصفح بيصغّر المحتوى علشان يكفّي: **200** زي ما كتبت. والـ margin برة الحساب في الحالتين.

علشان كده تقريباً كل مشروع (و Tailwind نفسه، شفناها في الـ CSS بتاعه: `*,:after,:before,::backdrop{box-sizing:border-box}`) بيبدأ بالسطر ده:

style.css

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

لو حابب تتعمق: الـ margin collapse — تقدر تعدّيه في أول قراية

لو فقرتين فوق بعض، الأولى `margin-bottom: 20px` والتانية `margin-top: 30px`، المسافة بينهم **30** مش 50. الـ Margins الرأسية بين العناصر العادية بتتداخل وياخد الأكبر. ده مبيحصلش جوه Flexbox ولا Grid، وده سبب من أسباب إن الناس بقت تستخدم `gap` (الفصلين الجايين).

6.8 display: النوع اللي بيحدد كل حاجة

القيمة	السلوك	أمثلة افتراضية
`block`	بياخد السطر كله، والـ width/height شغالين	`div` و `p` و `h1` و `section`
`inline`	جوه السطر مع النص، والـ width/height **مش** شغالين، والـ margin الرأسي مش بيزق	`span` و `a` و `strong`
`inline-block`	جوه السطر، بس الـ width/height شغالين	`button` تقريباً
`flex` / `grid`	العنصر نفسه block، وولاده بيترتبوا بنظام تاني	فصل 7 و 8
`none`	مش موجود خالص، ولا حتى لقارئ الشاشة	—

جدول 6.3 أهم قيم display

المصدر

MDN · Introduction to the CSS cascade · MDN · Specificity · MDN · @layer · MDN · Inheritance · web.dev · Learn CSS: The cascade · MDN · The box model

ملخص الفصل

- الـ Cascade بالترتيب: الأهمية والمصدر، ثم الـ inline، ثم الـ Layers، ثم الـ Specificity، ثم الترتيب في الملف.
- الـ Specificity = (IDs, Classes, Elements)، والشمال يكسب. ID واحد بيكسب أي عدد Classes.
- `:where()` صفر، و`:is()` = أدق واحد جواها.
- الـ Layers أهم من الـ Specificity، واللي برة أي Layer بيكسبهم كلهم.
- خصايص النص بتتورث، وخصايص الصندوق لأ. والقيمة الموروثة أضعف من أي قاعدة مباشرة.
- `box-sizing: border-box` بيخلي الـ width يشمل الـ padding والـ border.
- DevTools: القاعدة المشطوبة خسرت، والـ Specificity بتظهر مع الماوس.

صح ولا غلط؟

1. `.a.b.c.d` (0,4,0) بيكسب `#x` (1,0,0). صح / غلط
2. قاعدة برة أي `@layer` بتكسب قاعدة جوه Layer حتى لو أقل Specificity. صح / غلط
3. الـ `margin` بيتورث من الأب. صح / غلط

؟ أسئلة مراجعة

1. رتّب مراحل الـ Cascade الخمسة من الذاكرة.
2. احسب Specificity: `ul#menu li.active > a:hover` .
3. ليه اللينك جوه `p` لونه أزرق رغم إن الـ p لونه navy؟
4. عنصر `width: 300px; padding: 10px; border: 2px` . عرضه كام في content-box وكام في border-box؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح أي موقع، واعمل Inspect على زرار، ولاقي في Styles قاعدة مشطوبة، واعرف مين كسبها وليه.
2. متوسط اعمل ملف فيه 3 Layers (`reset, base, components`) واثبت بالتجربة إن `.btn` في components بيكسب `#id` في base.
3. صعب اكتب Selector واحد بـ `:where()` يعمل Reset للـ margin لكل العناوين والفقرات، واثبت إن أي Class بسيطة تقدر تغيّره من غير `!important` .

07

المرحلة 3 · CSS بجد

الفصل السابع

Flexbox: ترتيب العناصر في خط واحد

قبل 2012 تقريباً، علشان تحط عنصر في نص الصفحة بالظبط كنت محتاج حيل غريبة: float و clear و tables و margin سالب. Flexbox حلّت ده في سطرين. الفصل ده بيشرح الفكرة الأساسية (المحورين)، وكل خاصية على الأب وعلى الولاد، ومعنى flex: 1 بالظبط، وليه flex-start في الصفحات العربي بيروح يمين، ومعاه صورة حقيقية لكل قيمة.

بعد الفصل ده هتكون عارف

- المشكلة اللي Flexbox حلّها
- المحورين: Main axis و Cross axis، و flex-direction
- على الأب: justify-content و align-items و gap و flex-wrap
- على الولد: flex-grow و flex-shrink و flex-basis، ومعنى flex: 1
- start و end مش left و right: Flexbox في RTL
- أنماط جاهزة: Navbar و توسيط و Footer لازق تحت
- تابة Flexbox في DevTools

7.1 المشكلة: توسيط حاجة في النص

كان فيه نكتة مشهورة بين مطوري الويب: "أصعب حاجة في CSS إنك تحط عنصر في النص بالظبط، أفقي ورأسي". وده كان حقيقي. دلوقتي:

style.css

```
.center {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

تلات سطور. والسر إن Flexbox بيتعامل مع العناصر كـ**صف** (أو عمود) بيوزّع المساحة بينهم بذكاء. وبيشتغل على مستوى **بُعد واحد**: صف واحد أو عمود واحد. (البعدين مع بعض ده Grid، الفصل الجاي.)

7.2 الفكرة الوحيدة اللي لازم تفهمها: المحورين

أول ما تكتب `display: flex` على عنصر، بيبقى **Flex container**، وولاده المباشرين بس بيبقوا **Flex items**. وفيه محورين:

Main axis · المحور الرئيسي

الاتجاه اللي العناصر بتترص فيه. بيتحدد بـ `flex-direction` : الافتراضي `row` (أفقي). والتحكم فيه بـ `justify-content` .

Cross axis · المحور العرضي

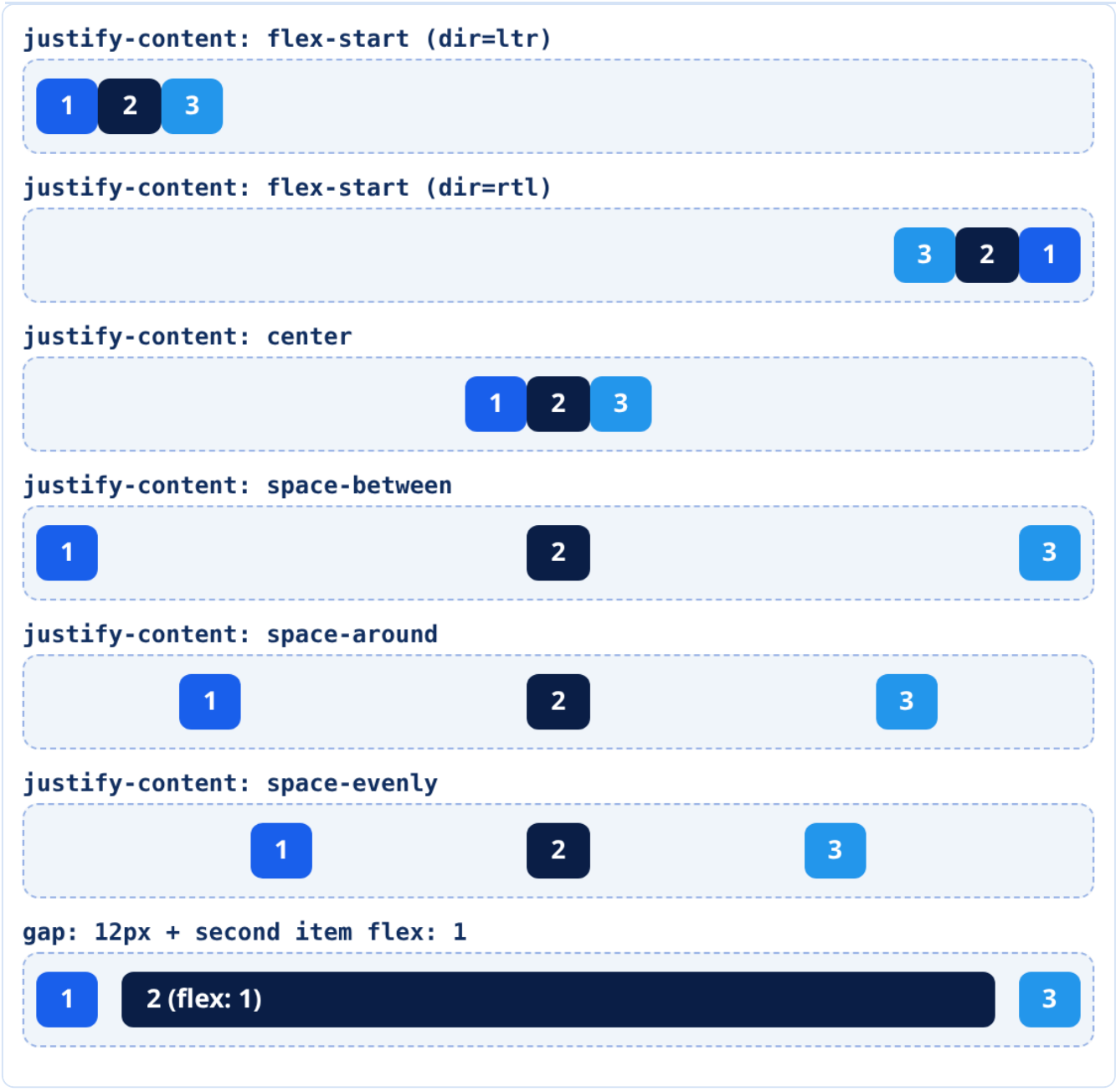
العمودي على الرئيسي. لو الصف أفقي، العرضي رأسي. والتحكم فيه بـ `align-items` .

الفخ الأشهر

لو غيّرت `flex-direction: column` ، المحاور **بتتبدّل**: `justify-content` بقى رأسي و `align-items` بقى أفقي. ودي سبب 90% من "ليه justify-content مش شغال". افتكر: **justify = الرئيسي، align = العرضي**، أياً كان الاتجاه.

7.3 على الأب (الـ Container)

دي كل قيم `justify-content` متصوّرة من Chrome فعلاً (نفس الـ 3 عناصر في كل صف):



شكل 7.1 لقطة حقيقية من Chrome. لاحظ الصف التاني: نفس `flex-start` بالظبط، بس `dir="rtl"` خلّاه يبدأ من اليمين، وخلّى 1 على اليمين. والصف الأخير: `flex: 1` خلّى العنصر التاني ياكل كل المساحة الفاضية.

`display: flex`	بيحوّل العنصر لـ Flex container. والولاد المباشرين بس بيتأثروا (الأحفاد لأ).
`flex-direction`	`row` (الافتراضي) أو `column` أو `row-reverse` أو `column-reverse`. والـ reverse بيقلب الشكل بس، مش ترتيب الـ Tab (فصل 5).
`justify-content`	التوزيع على المحور الرئيسي: `flex-start` و `center` و `flex-end` و `space-between` (المسافة بينهم بس) و `space-around` (نص مسافة على الأطراف) و `space-evenly` (كل المسافات متساوية).

`align-items`	على المحور العرضي: `stretch` (الافتراضي: كلهم بطول أطول واحد) و `flex-start` و `center` و `flex-end` و `baseline` (على خط النص).
`gap: 12px`	مسافة **بين** العناصر بس، مش على الأطراف. أضف بكتير من `margin` على كل عنصر. و `gap: 8px 16px` = صفوف ثم أعمدة.
`flex-wrap: wrap`	الافتراضي `nowrap`: كله في سطر واحد حتى لو هيتزنق. و `wrap` بيسمح بالنزول لسطر جديد. (ولما يبقى فيه أكتر من سطر، `align-content` بيوزّع السطور.)

7.4 على الولد (الـ Item): flex: 1 يعني إيه بالظبط؟

كل عنصر عنده 3 أرقام بتقرر حجمه:

`flex-basis`	الحجم **المبدئي** قبل توزيع المساحة. الافتراضي `auto` (يعني حسب الـ width أو المحتوى).
`flex-grow`	لو فيه مساحة **فاضلة**، ياخد منها كام نصيب. الافتراضي `0` (مياخدش). لو عنصر `1` وعنصر `2`، التاني ياخد ضعف الأول من الفاضل.
`flex-shrink`	لو المساحة **مش مكفّية**، يصغر بنسبة كام. الافتراضي `1` (كلهم يصغروا). و `0` = "متصغرنيش" (مفيد للوجو أو صورة).
`flex: 1`	اختصار لـ `flex: 1 1 0%`: يكبر، ويصغر، ويبدأ من صفر. يعني "قسّموا المساحة بالتساوي بغض النظر عن المحتوى". وده اللي حصل للعنصر التاني في الصورة.
`flex: none`	`0 0 auto`: حجمك ثابت، متكبرش ومتصغرش.
`align-self`	زي `align-items` بس لعنصر واحد يخالف الباقيين.
`order`	يغيّر مكان العنصر في الشكل. **استخدمه بحذر:** الكيبورد وقارئ الشاشة لسه ماشيين بترتيب الـ HTML.

> **حاجة مشتركة: margin-inline-start: auto**
>
> عايز اللوجو في جنب والـ Links في الجنب التاني؟ بدل `space-between`، حط على أول لينك `margin-inline-start: auto`. الـ `auto` margin في Flexbox بياكل كل المساحة الفاضية في الاتجاه ده. والكلمة `inline-start` (مش `left`) بتخليها تشتغل صح في العربي والإنجليزي (فصل 10).

7.5 start و end، مش left و right

شفت في الصورة: `flex-start` في `dir="rtl"` راح **يمين**. علشان Flexbox مبيفكرش في "شمال ويمين"، بيفكر في "**بداية ونهاية**" حسب اتجاه اللغة. وده هدية لينا: لو كتبت صح، الصفحة بتتقلب لوحدها لما تغيّر `dir`. ونفس الفكرة في Grid، وفي الـ Logical properties (فصل 10)، وفي Tailwind (`ms-4` و`me-4` بدل `ml-4` و`mr-4`).

7.6 أنماط هتكتبها كل يوم

Navbar

style.css

```css
.nav {
  display: flex;
  align-items: center;
  gap: 1rem;
}
.nav .logo {
  margin-inline-end: auto; /* يزق الباقي للناحية التانية */
}
```

كروت بتلف لسطر جديد

style.css

```css
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
.cards > * {
  flex: 1 1 220px; /* ابدأ من 220 وكبّر لو فيه مكان */
}
```

الفوتر لازق تحت حتى لو الصفحة قصيرة

style.css

```css
body {
  min-height: 100svh;
  display: flex;
  flex-direction: column;
}
main {
  flex: 1;
}
```

الـ `100svh` = 100% من ارتفاع الشاشة "الصغير" على الموبايل (لما شريط العنوان ظاهر). قالب Vite نفسه بيستخدمها: `min-height: 100svh` على `#app`.

Flexbox في قالب Vite الحقيقي

ده جزء من `style.css` اللي `create-vite` عمله لينا (هنشرح القالب كله في فصل 18):

vanilla-app/src/style.css

```css
#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;
}
```

`flex-direction: column`	العناصر فوق بعض. فالمحور الرئيسي بقى رأسي.
`place-content: center`	اختصار لـ `align-content` + `justify-content` مع بعض.
`place-items: center`	اختصار لـ `align-items` + `justify-items`. (والـ `justify-items` ملهاش تأثير في Flexbox، بس مفيدة لو العنصر اتحوّل لـ Grid.)
`flex-grow: 1`	الـ `#center` نفسه ولد لـ `#app` (اللي هو `display: flex` كمان)، فبياخد كل المساحة الرأسية الفاضية. Flexbox جوه Flexbox: عادي جداً.

7.7 تابة Flexbox في DevTools

في Chrome، أي عنصر `display: flex` بيظهر جنبه في تابة Elements علامة صغيرة مكتوب عليها **flex**. اضغطها، هيرسملك المحاور والمسافات على الصفحة. وفي تابة Styles، جنب `display: flex` فيه أيقونة بتفتح **Flexbox editor**: زراير تغيّر بيها `justify-content` و`align-items` وتشوف النتيجة فوراً. أسرع طريقة تتعلم بيها.

المصدر web.dev · Learn CSS: Flexbox · MDN · Basic concepts of flexbox · MDN · Flexbox · Chrome DevTools · Inspect and debug CSS flexbox

ملخص الفصل

- Flexbox لبعد واحد: صف أو عمود.
- `justify-content` للمحور الرئيسي، و`align-items` للعرضي، وبيتبدّلوا مع `column`.
- `gap` للمسافات بين العناصر، و`flex-wrap: wrap` للنزول لسطر جديد.
- `flex: 1` = `1 1 0%`: قسّموا المساحة بالتساوي.
- `flex-shrink: 0` علشان عنصر ميصغرش.
- `start` و`end` بيتقلبوا مع `dir="rtl"`. فكّر "بداية ونهاية" مش "شمال ويمين".
- DevTools فيه Flexbox editor بيرسم المحاور.

صح ولا غلط؟

1. مع `flex-direction: column` ، الـ `justify-content: center` بيوسّط رأسياً. صح / غلط
2. `display: flex` بيأثر على كل الأحفاد مش الولاد بس. صح / غلط
3. في صفحة `dir="rtl"` ، `justify-content: flex-start` بيحط العناصر يمين. صح / غلط

؟ أسئلة مراجعة

1. إيه الفرق بين `space-between` و `space-evenly` ؟
2. `flex: 1` اختصار لإيه؟ وإيه الفرق بينه وبين `flex: auto` ؟
3. ليه `order` ممكن يعمل مشكلة Accessibility؟
4. إزاي تخلي الفوتر لازق تحت في صفحة قصيرة؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل Navbar: لوجو على البداية و3 لينكات على النهاية، وجرّبه بـ `dir="rtl"` و `dir="ltr"` من غير ما تغيّر ولا سطر CSS.
2. متوسط اعمل 6 كروت بـ `flex: 1 1 220px` و `flex-wrap` ، وصغّر الشاشة وشوف بيلفّوا إزاي. إيه اللي بيحصل لآخر سطر لو فيه كارت واحد؟ (ده سبب إنك ممكن تفضّل Grid)
3. صعب العب Flexbox Froggy لحد آخر مستوى، واكتب في ورقة كل خاصية جديدة قابلتها.

08

المرحلة 3 · CSS بجد

الفصل التامن

CSS Grid: الصفحة كلها كشبكة

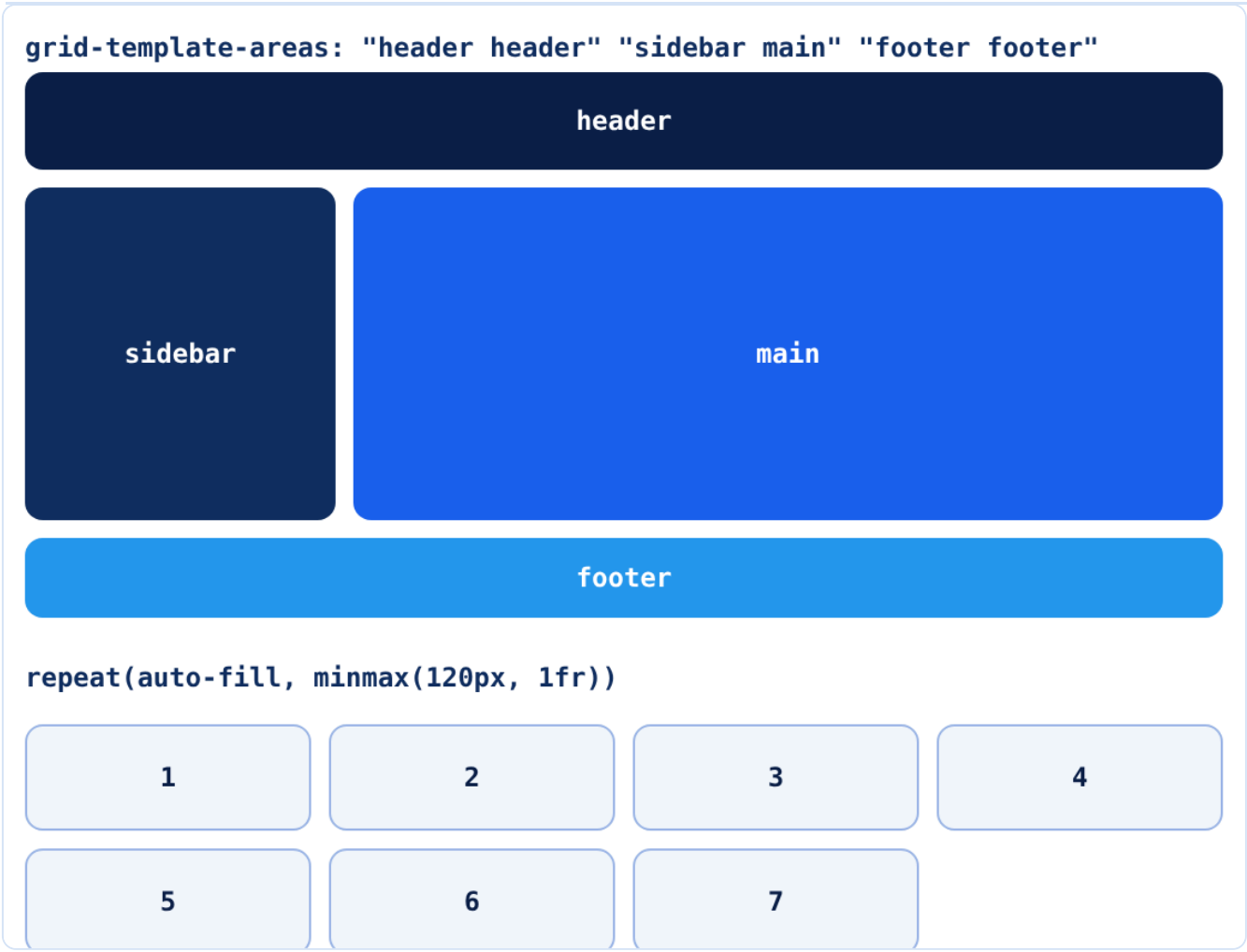
Flexbox ممتاز لصف واحد. بس لما تحتاج صفوف وأعمدة مع بعض، زي هيكل صفحة فيها هيدر وسايدبار ومحتوى وفوتر، أو كروت كلها نفس العرض بالظبط، Flexbox بيبتدي يتعبك. Grid بتخليك ترسم الشبكة الأول، وبعدين تحط العناصر فيها. الفصل ده بيشرح fr و repeat و minmax و auto-fill و grid-template-areas، ومعاه صورة حقيقية، وإمتى تختار Grid وإمتى Flexbox.

بعد الفصل ده هتكون عارف

- الفرق بين Grid (بعدين) و Flexbox (بعد واحد)
- grid-template-columns و rows و gap
- الوحدة fr، و repeat()، و minmax()
- auto-fill و auto-fit: كروت Responsive من غير Media query
- grid-template-areas: ترسم الصفحة بالكلام
- تحط عنصر في مكان معين: grid-column و span
- الـ subgrid في سطر
- Grid ولا Flexbox؟ قاعدة بسيطة

8.1 ليه Flexbox مش كفاية؟

افتكر تحدي الفصل اللي فات: 6 كروت بـ `flex: 1 1 220px`. لو آخر سطر فيه كارت واحد، هيتمط ياخد السطر كله، وشكله يبقى غريب جنب الكروت اللي فوقه. ليه؟ لأن كل سطر في Flexbox **مستقل** عن اللي فوقه. مفيش "أعمدة". Grid بتحل ده لأنها بتفكر في **الصفوف والأعمدة مع بعض**.



شكل 8.1 لقطة حقيقية من Chrome. فوق: هيكل صفحة بـ `grid-template-areas`. تحت: 7 كروت بـ `repeat(auto-fill, minmax(120px, 1fr))`، والكارت الأخير فضل بنفس عرض اللي فوقه بالظبط.

8.2 الأساسيات: ارسم الشبكة الأول

style.css

```css
.page {
  display: grid;
  grid-template-columns: 140px 1fr;
  grid-template-rows: 44px 150px 36px;
  gap: 8px;
}
```

`display: grid`	العنصر بقى Grid container، وولاده المباشرين Grid items بيتحطوا في الخانات بالترتيب أوتوماتيك.
`grid-template-columns: 140px 1fr`	عمودين: الأول 140px ثابت، والتاني **1fr**.
`1fr`	**fraction**: نصيب من المساحة الفاضلة بعد الحاجات الثابتة. `1fr 2fr` = التاني ضعف الأول. ودي وحدة Grid بس.

`grid-template-rows`	نفس الفكرة للصفوف. ولو مكتبتهاش، الصفوف بتتعمل لوحدها بطول المحتوى (اسمها **Implicit grid**).
`gap: 8px`	زي Flexbox: مسافة بين الخانات. وتقدر تفصل: `row-gap` و `column-gap`.

8.3 grid-template-areas: ترسم الصفحة بالكلام

دي أجمل حاجة في Grid. بتكتب شكل الصفحة كأنك بترسمه بالحروف:

style.css

```css
.page {
  display: grid;
  grid-template-columns: 140px 1fr;
  grid-template-areas:
    "header  header"
    "sidebar main"
    "footer  footer";
}
.h { grid-area: header; }
.s { grid-area: sidebar; }
.m { grid-area: main; }
.f { grid-area: footer; }
```

`"header header"`	كل String = صف، وكل كلمة = خانة. نفس الاسم مرتين جنب بعض = الخانة دي ممتدة على العمودين.
`grid-area: header`	على الولد: "حطني في المنطقة اللي اسمها header". ترتيب العناصر في الـ HTML مش مهم للشكل.
`.`	نقطة بدل اسم = خانة فاضية.

والحلو إن الـ Responsive بقى سهل جداً: على الموبايل غيّر الرسمة بس:

style.css

```css
@media (max-width: 600px) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas: "header" "main" "sidebar" "footer";
  }
}
```

> ⚠ **بس افتكر الكيبورد**
>
> زي `order` في Flexbox: لو حطيت الـ sidebar قبل الـ main في الشكل وبعده في الـ HTML، الـ Tab هيمشي بترتيب الـ HTML. خلّي ترتيب الـ HTML هو الترتيب المنطقي للقراية، والـ Grid يغيّر الشكل بس.

8.4 repeat و minmax و auto-fill: السطر السحري

style.css

```css
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));
  gap: 8px;
}
```

السطر ده بيعمل كروت Responsive **من غير ولا Media query**. نفصّصه من جوه لبرة:

تشريح الأمر

```
repeat(auto-fill, minmax(120px, 1fr))
       ③          ①
```

1. `minmax(120px, 1fr)` كل عمود عرضه **مش أقل من** 120px، و**مش أكتر من** 1fr (نصيب متساوي من الفاضل).
2. `repeat(N, …)` كرّر العمود ده N مرة. `repeat(3, 1fr)` = `1fr 1fr 1fr`.
3. `auto-fill` بدل رقم ثابت: "حط أكبر عدد أعمدة يكفّي". في صورتنا العرض كان كفاية لـ 4، فطلعوا 4 أعمدة، والكارت 7 نزل تحت بنفس العرض. صغّر الشاشة: يبقوا 3، ثم 2، ثم 1، لوحدهم.
4. `auto-fit` زي `auto-fill`، بس لو العناصر قليلة (مثلاً 2 بس في شاشة واسعة)، بيلم الأعمدة الفاضية فالعنصرين يتمطوا ياخدوا العرض كله. و`auto-fill` بيسيب الأعمدة الفاضية موجودة.

8.5 تحط عنصر في مكان معين

الخطوط بين الأعمدة ليها أرقام بتبدأ من 1. وتقدر تقول للعنصر يبدأ وينتهي فين:

style.css

```css
.featured {
  grid-column: 1 / -1;   /* من أول خط لآخر خط: العرض كله */
}
.wide {
  grid-column: span 2;   /* خد عمودين من مكانك */
}
.tall {
  grid-row: span 2;
}
```

`1 / -1`	"من الخط 1 للخط الأخير". الرقم السالب بيعد من الآخر. وفي `dir="rtl"` الخط 1 على **اليمين** (نفس فكرة start و end).
`span 2`	"امتد على خانتين" من غير ما تحدد بالظبط فين.

لو حابب تتعمق: subgrid

تقدر تعدّيه في أول قراية

لو جوه كل كارت عنوان وصورة وزرار، وعايز العناوين كلها على نفس الخط في كل الكروت حتى لو طولها مختلف: خلي الكارت نفسه `display: grid; grid-template-rows: subgrid; grid-row: span 3;` . كده صفوف الكارت بتتبع صفوف الـ Grid الكبيرة. والـ subgrid بقى مدعوم في كل المتصفحات الرئيسية من 2023.

8.6 Grid ولا Flexbox؟

السؤال	Flexbox	Grid
كام بعد؟	واحد: صف **أو** عمود	اتنين: صفوف **و** أعمدة
مين بيقرر الحجم؟	المحتوى (Content-out)	الشبكة (Layout-in)
أمثلة	Navbar، زراير جنب بعض، توسيط، عناصر جوه كارت	هيكل الصفحة، Gallery، Dashboard، كروت بنفس العرض

جدول 8.1 قاعدة بسيطة

وفي الحقيقة هتستخدم الاتنين مع بعض طول الوقت: Grid للهيكل الكبير، وجوه كل خانة Flexbox. وما تقلقش من "أنهي أصح": لو الشكل طلع صح والكود مقروء، يبقى صح.

DevTools: تابة Grid

زي Flexbox: جنب أي عنصر `display: grid` في Elements فيه علامة **grid**. اضغطها، وهيرسم الخطوط بأرقامها وأسماء الـ Areas على الصفحة. ومن تابة **Layout** تقدر تشغّل الرسم ده لكل الـ Grids في الصفحة مرة واحدة.

المصدر MDN · CSS grid layout · MDN · grid-template-areas · MDN · repeat() · MDN · Subgrid · web.dev · Learn CSS: Grid · Chrome DevTools · Inspect CSS grid

ملخص الفصل

- Grid لبعدين، و Flexbox لبعد واحد، والاتنين مع بعض عادي.
- `grid-template-columns` و `rows` بيرسموا الشبكة، و `gap` للمسافات.
- `fr` نصيب من المساحة الفاضلة.
- `repeat(auto-fill, minmax(Xpx, 1fr))` = كروت Responsive من غير Media queries.
- `grid-template-areas` بترسم الصفحة بالكلام، و `grid-area` على الولد.
- `grid-column: 1 / -1` للعرض كله، و `span 2` لخانتين.
- ترتيب الـ HTML هو ترتيب القراية والكيبورد، مهما غيّرت الشكل.

صح ولا غلط؟

1. الوحدة `fr` بتشتغل في Flexbox كمان. صح / غلط
2. `repeat(auto-fill, minmax(200px, 1fr))` محتاجة Media query علشان تبقى Responsive. صح / غلط
3. `grid-column: 1 / -1` بيخلي العنصر ياخد عرض الشبكة كلها. صح / غلط

أسئلة مراجعة

1. ليه آخر كارت في Flexbox بيتمط، وفي Grid لأ؟
2. اشرح `minmax(120px, 1fr)` بكلامك.
3. إيه الفرق بين `auto-fill` و `auto-fit`؟
4. اكتب `grid-template-areas` لصفحة فيها هيدر، ومحتوى في النص، وسايدبار على الجنبين، وفوتر.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل Gallery صور بـ `repeat(auto-fill, minmax(150px, 1fr))` وغيّر عرض الشاشة.
2. متوسط اعمل هيكل الصفحة بـ `grid-template-areas`، وعلى الشاشات الصغيرة خليه عمود واحد بـ Media query واحدة.
3. صعب اعمل Dashboard فيه 4 كروت، واحد منهم `grid-column: span 2` و `grid-row: span 2`، والباقي يترصوا حواليه أوتوماتيك. جرّب `grid-auto-flow: dense` وشوف الفرق.

09

المرحلة 3 · CSS بجد

الفصل التاسع

الـ Responsive Design: صفحة واحدة لكل الشاشات

زمان الشركات كانت بتعمل موقعين: www للكمبيوتر و m. للموبايل. النهارده صفحة واحدة بتتأقلم مع أي عرض: من موبايل صغير لشاشة 32 بوصة. الفصل ده بيبدأ من سطر الـ viewport اللي من غيره مفيش حاجة هتشتغل، والوحدات (rem و vw و svh)، والـ Mobile-first، والـ Media queries بالصيغة الجديدة، وبعدين النجم الجديد: الـ Container queries، والصور اللي بتتأقلم.

بعد الفصل ده هتكون عارف

- ☑ meta viewport وليه من غيره الموبايل بيصغّر الصفحة
- ☑ Mobile-first: ابدأ بالصغير وزوّد
- ☑ الوحدات: px و rem و em و % و vw و svh و ch
- ☑ Media queries: min-width وصيغة النطاقات الجديدة
- ☑ prefers-color-scheme و hover و pointer
- ☑ Container queries: الكومبوننت يقرا عرض علبته مش الشاشة
- ☑ clamp() للخطوط اللي بتكبر وتصغر بنعومة
- ☑ الصور: srcset و sizes و picture و loading=lazy
- ☑ تجرّب بـ Device mode في DevTools

9.1 السطر اللي من غيره مفيش Responsive

index.html

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

لما الـ iPhone نزل سنة 2007، كل المواقع كانت معمولة لشاشات كمبيوتر. فـ Safari على الموبايل عمل حيلة: يعرض الصفحة كأن عرض الشاشة **px980**، ويصغّرها علشان تكفّي. فالمستخدم يعمل Zoom بصوابعه. والحيلة دي لسه شغالة لحد النهارده لأي صفحة مفيهاش السطر ده. فحتى لو كتبت أحلى Media queries، الموبايل هيقول "أنا عرضي 980" ومش هتشتغل.

`width=device-width`	"اعتبر عرض الصفحة = عرض الجهاز الحقيقي" (بالـ CSS pixels، مثلاً 390 في iPhone حديث، مش عدد البيكسلات الفعلي في الشاشة).
`initial-scale=1.0`	ابدأ من غير Zoom. ومتزودش `maximum-scale=1` أو `user-scalable=no`: دول بيمنعوا الناس اللي نظرها ضعيف من التكبير (مشكلة Accessibility).

والسطر ده موجود جاهز في كل قوالب Vite (شفناه في `index.html`).

9.2 Mobile-first: ابدأ بالصغير

فيه طريقتين تكتب بيهم Responsive:

Desktop-first (القديمة)

تكتب للشاشة الكبيرة، وبعدين `max-width` تلغي وتصغّر. النتيجة: الموبايل بينزّل CSS كتير وبيلغيه.

Mobile-first (المنصوح بيها)

تكتب للموبايل الأول (عمود واحد، أبسط حاجة)، وبعدين `min-width` تزوّد للشاشات الأكبر. الكود أبسط، لأن الشكل الأبسط هو الافتراضي.

style.css

```css
/* الموبايل: الافتراضي */
.layout {
  display: grid;
  gap: 1rem;
}

/* من 768px وأكبر: عمودين */
@media (width >= 768px) {
  .layout {
    grid-template-columns: 240px 1fr;
  }
}
```

و Tailwind مبني على نفس الفكرة: `md:grid-cols-2` معناها "من الـ md وأكبر"، والكلاس من غير بادئة للموبايل (فصل 20).

9.3 الوحدات: إمتى تستخدم إيه

الوحدة	معناها	استخدمها لـ
`px`	CSS pixel ثابت	الحدود والظلال والحاجات الصغيرة الثابتة
`rem`	مضاعف حجم خط الـ `html` (افتراضياً 16px)	**الخطوط والمسافات**. لو المستخدم كبّر خط المتصفح من الإعدادات، كله يكبر معاه
`em`	مضاعف حجم خط العنصر نفسه	حاجة لازم تتناسب مع النص اللي جنبها (padding زرار، أيقونة)
`%`	نسبة من الأب	العروض جوه حاوية
`vw` / `vh`	1% من عرض / ارتفاع الشاشة	أقسام بملء الشاشة. بس `100vh` على الموبايل بيتخطى الشاشة لما شريط العنوان يظهر
`svh` / `lvh` / `dvh`	ارتفاع الشاشة الصغير / الكبير / الحالي (مع ظهور واختفاء شريط المتصفح)	`min-height: 100svh` بدل `100vh` (زي قالب Vite)
`ch`	عرض حرف "0" في الخط الحالي	عرض الفقرات: `max-width: 65ch` سطر مريح للقراية

جدول 9.1 الوحدات اللي هتستخدمها فعلاً

Best Practice

القاعدة العملية: `rem` للخطوط والمسافات، و`px` للحدود، و`%` / `fr` للعروض، و`svh` للارتفاعات بملء الشاشة. ومتحطش `font-size` بالـ `px` على الـ `html` : ده بيلغي إعداد حجم الخط اللي المستخدم اختاره.

9.4 Media queries بالتفصيل

تشريح الأمر

```
@media screen and (width ≥ 768px) and (width < 1024px) { … }
```

1. `@media` "القواعد اللي جوه تتطبق بس لو الشرط صح".
2. `screen` نوع الجهاز (Media type): `screen` أو `print` (للطباعة) أو `all` . ولو مكتبتهوش يبقى `all` .
3. `(width ≥ 768px)` صيغة النطاقات الجديدة (Range syntax)، مدعومة في كل المتصفحات الحديثة من 2023. أوضح من القديمة `(min-width: 768px)` ، والاتنين نفس المعنى. هتشوف القديمة كتير في الكود اللي موجود.
4. `and` الشرطين لازم يتحققوا. والـ `,` بين Queries = "أو".

ومش بس العرض. فيه Media features بتقرا تفضيلات المستخدم وجهازه:

الـ Feature	بتسأل عن إيه
`(prefers-color-scheme: dark)`	المستخدم مفعّل Dark mode في الماك أو الموبايل
`(prefers-reduced-motion: reduce)`	عايز حركة أقل (فصل 5)
`(hover: hover)`	الجهاز عنده ماوس يقدر يعمل Hover (التاتش لأ)
`(pointer: coarse)`	المؤشر "خشن" زي الصباع: كبّر الزرايز
`(orientation: landscape)`	الشاشة بالعرض

جدول 9.2 Media features مفيدة

وقالب Vite نفسه بيستخدم `prefers-color-scheme` . ده جزء حقيقي من `style.css` بتاعه:

vanilla-app/src/style.css

```
:root {
  --text: #6b6375;
  --bg: #fff;
  color-scheme: light dark;
}

@media (prefers-color-scheme: dark) {
  :root {
    --text: #9ca3af;
    --bg: #16171d;
  }
}
```

هو بيغيّر **المتغيرات** بس (`--text` و `--bg`)، وكل حاجة بتستخدمها بتتغير لوحدها. ودي قوة الـ Custom properties (الفصل الجاي). والـ `color-scheme: light dark` بيقول للمتصفح إن الصفحة تدعم الاتنين، فيلوّن الـ Scrollbars والخانات صح.

9.5 Container queries: الكومبوننت يقرا علبته

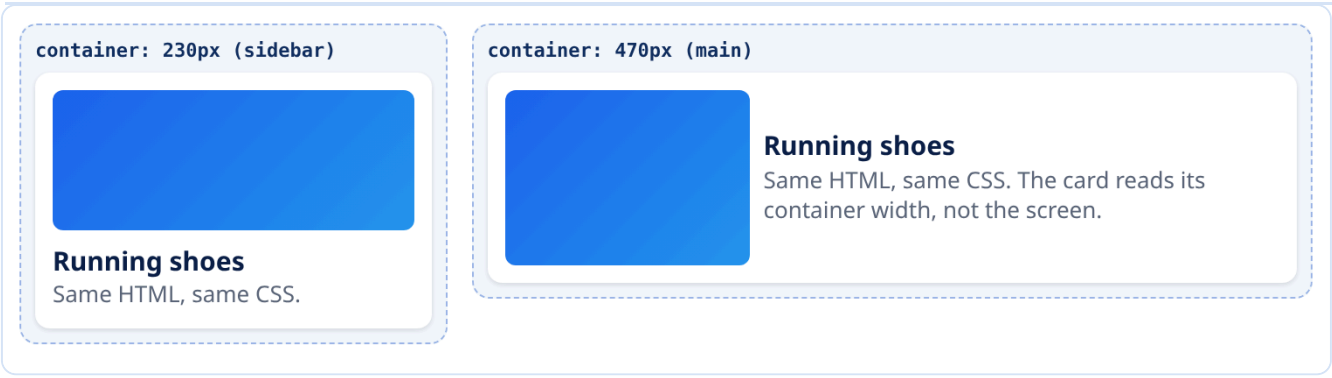
المشكلة: عندك كارت منتج. في الصفحة الرئيسية بيتحط في عمود عريض، وفي الـ Sidebar في عمود ضيق. والشاشة **نفس العرض** في الحالتين. فالـ Media query مش هتفرق بينهم. الكارت محتاج يسأل: "**العلبة اللي أنا فيها** عرضها كام؟"

style.css

```
.slot {
  container-type: inline-size;
}

.card {
  display: grid;
  gap: 8px;
}

@container (width >= 400px) {
  .card {
    grid-template-columns: 140px 1fr;
  }
}
```



شكل 9.1 لقطة حقيقية من Chrome: نفس الـ HTML ونفس الـ CSS بالظبط، والشاشة نفسها. الكارت في العلبة الضيقة (230px) عمودي، وفي العريضة (470px) أفقي.

`container-type: inline-size`	على **الأب**: "أنا علبة، وولادي يقدروا يسألوا عن عرضي". `inline-size` = العرض في الكتابة الأفقية.
`@container (width >= 400px)`	زي `@media` بالظبط، بس بيسأل عن أقرب علبة فوقه مش عن الشاشة.
`container-name: sidebar`	اختياري: اسم للعلبة، فتقول `@container sidebar (width >= 300px)` لو فيه علب جوه علب.

ليه ده مهم مع React؟

في React هتفكر في **Components** بتتحط في أي مكان. والـ Container queries بتخلي الكومبوننت مسؤول عن شكله في أي علبة، من غير ما يعرف الصفحة كلها. Media queries للهيكل الكبير، و Container queries للكومبوننتات.

9.6 clamp(): خطوط بتكبر بنعومة

تشريح الأمر

```
h1 { font-size: clamp(1.75rem, 1rem + 3vw, 3.5rem); }
```

1. `clamp(MIN, PREFERRED, MAX)` — "خد القيمة المفضّلة، بس متقلّش عن MIN ومتزيدش عن MAX".
2. `1rem + 3vw` — جزء ثابت + جزء بيكبر مع عرض الشاشة. فالعنوان بيكبر بنعومة بدل ما يقفز عند كل Breakpoint. والـ `rem` فيها مهم: من غيره (`3vw` لوحدها) الـ Zoom مش هيكبّر الخط.

9.7 الصور اللي بتتأقلم

index.html

```
<img
  src="hero-800.jpg"
  srcset="hero-400.jpg 400w, hero-800.jpg 800w, hero-1600.jpg 1600w"
  sizes="(width >= 768px) 50vw, 100vw"
  alt="Team at work"
  width="800" height="450"
  loading="lazy">
```

`srcset="… 400w, …"`	قايمة نسخ من نفس الصورة بعروضها الحقيقية (`w` = بيكسل عرض). المتصفح بيختار الأنسب حسب الشاشة وكثافة البيكسلات (شاشة Retina في الماك محتاجة ضعف).
`sizes="…"`	بتقول للمتصفح الصورة هتتعرض بعرض قد إيه: نص الشاشة على الكبير، والشاشة كلها على الصغير. من غيره المتصفح بيفترض `100vw`.
`src`	احتياطي لو المتصفح مش فاهم srcset.
`loading="lazy"`	متنزّلش الصورة غير لما تقرّب تظهر. **متحطهاش** على الصورة الكبيرة اللي فوق في أول الصفحة: دي محتاجة تنزل فوراً (بتأثر على LCP، فصل 24).
`<picture>`	لما عايز **صورة مختلفة** مش بس حجم مختلف (قص مختلف للموبايل، أو صيغ حديثة زي AVIF مع احتياطي: `ce type="image/avif" srcset="…"><img …></picture>`.

9.8 تجرّب: Device mode

1. في Chrome DevTools اضغط أيقونة الموبايل والتابلت (أو ⌘+⇧+M).
2. اختار جهاز من فوق، أو "Responsive" واسحب الحافة بإيدك وشوف الـ Breakpoints.
3. من قايمة النقط التلاتة: "Show media queries" بترسم شريط بكل الـ Media queries اللي في الصفحة، تضغط على أي واحدة تروحلها.
4. وتقدر تجرّب `prefers-color-scheme` و`prefers-reduced-motion` من تابة **Rendering** (من Command menu: ⌘+⇧+P واكتب rendering).
5. ولما تخلص، جرّب على موبايلك الحقيقي: `npm run dev -- --host` (فصل 18) بيخلي Vite يظهر على الشبكة المحلية.

المصدر MDN · Container queries · MDN · Using media queries · MDN · Viewport meta element · MDN · Responsive design · web.dev · Learn Responsive Design · MDN · Responsive images

ملخص الفصل

- من غير `meta viewport` الموبايل بيتظاهر إن عرضه 980px.
- Mobile-first: الافتراضي للصغير، و`min-width` (أو `width >=`) للأكبر.
- `rem` للخطوط والمسافات، و`svh` للارتفاعات، و`ch` لعرض الفقرات.
- Media features: `prefers-color-scheme` و`prefers-reduced-motion` و`hover` و`pointer`.
- Container queries: `container-type: inline-size` على الأب، و`@container` للولد.
- `clamp(min, preferred, max)` لخطوط ناعمة، ومعاها `rem`.
- الصور: `srcset` و`sizes` و`width` / `height`، و`lazy` للي تحت بس.

صح ولا غلط؟

1. من غير `meta viewport`، Media queries بتشتغل عادي على الموبايل. صح / غلط
2. `@container` بيسأل عن عرض الشاشة. صح / غلط
3. تحط `loading="lazy"` على أول صورة كبيرة في الصفحة علشان تبقى أسرع. صح / غلط

أسئلة مراجعة

1. إيه المشكلة اللي `width=device-width` بيحلها؟ ومنين جت؟
2. ليه `rem` أحسن من `px` للخطوط؟
3. اكتب Media query بالصيغة القديمة والجديدة لـ "من 640 لأقل من 1024".
4. إمتى Container query أنسب من Media query؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — شيل سطر الـ viewport من أي صفحة عندك وافتحها بـ Device mode، وبعدين رجّعه. لاحظ الفرق.
2. متوسط — اعمل صفحة Mobile-first: عمود واحد، وعمودين من `768px`، وتلاتة من `1200px`، ومعاها Dark mode بالمتغيرات زي قالب Vite.
3. صعب — اعمل كومبوننت كارت بـ Container queries وحطه في 3 أماكن بعروض مختلفة في نفس الصفحة، واتأكد إن كل نسخة بتاخد الشكل المناسب لعلبتها.

10

المرحلة 3 · CSS بجد

الفصل العاشر

CSS الحديثة: الحاجات اللي خلّت Sass مش ضرورية

لو اتعلمت CSS من فيديوهات قديمة، فاتك كتير. في آخر كام سنة CSS بقى فيها متغيرات حقيقية بتتغير وقت التشغيل، و Nesting زي Sass، و :has() اللي الناس كانت بتسميه «الـ Parent selector المستحيل»، و Logical properties بتقلب الصفحة للعربي لوحدها، ودوال زي clamp و color-mix. الفصل ده جولة على أهمهم، ومعاها إزاي تعرف ميزة معينة ينفع تستخدمها ولا لسه (Baseline).

بعد الفصل ده هتكون عارف

- [x] الـ Custom properties: var() والـ fallback والـ scope وتغييرها بالـ JS
- [x] الـ Nesting الأصلي في CSS و &
- [x] :has(): تختار الأب حسب ولاده
- [x] الـ Logical properties: inline و block بدل left و right
- [x] min() و max() و clamp()
- [x] الألوان الحديثة: oklch() و color-mix()
- [x] @supports للميزات الجديدة
- [x] Baseline: تعرف الميزة مدعومة ولا لأ

10.1 الـ Custom properties: متغيرات بجد

شفت في قالب Vite: `--text` و `--bg` اتعرّفوا مرة، واتغيروا في Dark mode، وكل حاجة اتغيرت معاهم. ده بالظبط اللي الـ **Custom properties** (الناس بتقول عليها CSS variables) بتعمله:

style.css

```css
:root {
  --brand: #1d63ed;
  --space: 1rem;
}

.btn {
  background: var(--brand);
  padding: var(--space) calc(var(--space) * 2);
}

.danger {
  --brand: crimson;/* نفس الزرار، لون تاني جوه .danger بس */
}
```

```
--brand: #1d63ed
```

التعريف. الاسم لازم يبدأ بـ `--`، وحساس للـ Capital والـ Small (`--Brand` غير `--brand`).

`:root`	= عنصر `html`. التعريف هنا يبقى متاح للصفحة كلها، لأن المتغيرات **بتتورث** زي `color`.
`var(--brand)`	الاستخدام. و `var(--brand, blue)` = "لو مش متعرّف استخدم "blue" (الـ Fallback).
`calc(var(--space) * 2)`	تقدر تحسب بيها. و `calc` بتخلط وحدات كمان: `calc(100% - 2rem)`.
`.danger { --brand: crimson }`	المتغير ليه **Scope**: إعادة تعريفه على عنصر بتأثر عليه وعلى ولاده بس. ودي حاجة متغيرات Sass مبتعملهاش.

الفرق الجوهري عن متغيرات Sass

متغيرات Sass (`$brand`) بتتحسب **وقت الـ Build** وبتختفي: المتصفح بيستلم `#1d63ed` مكتوبة. أما `--brand` فبتوصل للمتصفح **حية**: تتغير بالـ Media queries، وبالـ Classes، وبالـ JavaScript: `document.documentElement.style.setProperty('--brand', 'green')`. علشان كده Tailwind 4 بنى نظام الألوان كله عليها (`@theme` في فصل 20).

10.2 الـ Nesting الأصلي

لسنين كان أهم سبب الناس تستخدم Sass هو إنها تكتب القواعد جوه بعض. دلوقتي CSS نفسها بتعمل كده (في كل المتصفحات الرئيسية من 2023). وقالب Vite بيستخدمها فعلاً:

vanilla-app/src/style.css

```css
.counter {
  color: var(--accent);
  border: 2px solid transparent;
  transition: border-color 0.3s;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

h1 {
  font-size: 56px;
  @media (max-width: 1024px) {
    font-size: 36px;
  }
}
```

`&:hover`	الـ `&` = "الـ Selector الأب". فدي = `.counter:hover`.
`.hero { .base { … } }`	من غير `&` = حفيد: `.hero .base`. (القالب بيعمل كده بالظبط في `.hero`).
`@media` جوه القاعدة	الـ Media query تتكتب جوه العنصر نفسه. فكل حاجة عن `h1` في مكان واحد.

> **خلي بالك**
>
> متعششش أكتر من 2 أو 3 مستويات. الـ Nesting العميق بيعمل Selectors طويلة بـ Specificity عالية (فصل 6)، وصعب تتغيّر بعدين.

10.3 :has(): الأب يعرف ولاده

لعشرين سنة تقريباً، CSS كانت بتختار من فوق لتحت بس: "الـ `img` اللي جوه `.card`". ومكانش فيه طريقة تقول "الـ `.card` **اللي فيه** `img`". `:has()` عملت ده:

style.css

```css
/* الكارت اللي فيه صورة: عمودين */
.card:has(img) {
  grid-template-columns: 120px 1fr;
}

/* الـ label اللي بعده خانة غلط */
label:has(+ input:user-invalid) {
  color: crimson;
}

/* الفورم كله لو فيه أي خانة غلط */
form:has(:user-invalid) button[type="submit"] {
  opacity: 0.5;
}
```

حاجات كانت محتاجة JavaScript (تشيك وتزوّد Class) بقت سطر CSS. وخلي بالك إن الـ Specificity بتاعة `:has(…)` = أدق حاجة جواها، زي `:is()`.

10.4 الـ Logical properties: هدية للعربي

لو كتبت `margin-left: 1rem` لأيقونة جنب كلمة، وبعدين قلبت الصفحة `dir="rtl"`، الأيقونة هتبقى ملزوقة في الناحية الغلط. الـ **Logical properties** بتتكلم بـ "بداية ونهاية" زي Flexbox:

القديمة (Physical)	الحديثة (Logical)	في الإنجليزي / في العربي
`margin-left`	`margin-inline-start`	شمال / يمين
`padding-right`	`padding-inline-end`	يمين / شمال
`margin-right` + `margin-left`	`margin-inline`	الجنبين
`margin-bottom` + `margin-top`	`margin-block`	فوق وتحت
`height` / `width`	`block-size` / `inline-size`	العرض / الارتفاع
`left: 0; right: 0`	`inset-inline: 0`	—
`text-align: left`	`text-align: start`	شمال / يمين
`border-left`	`border-inline-start`	شمال / يمين

جدول 10.1 Logical ← Physical

وقالب Vite بيستخدمها فعلاً: `border-inline: 1px solid var(--border)` على `#app`، و `inset-inline: 0` في `.hero`. والقاعدة: لو بتكتب لموقع ممكن يبقى عربي في أي يوم، اكتب Logical من الأول.

10.5 min() و max() و clamp()

`width: min(100%, 60rem)`	"خد الأصغر": 100% من الأب، بس متعدّيش 60rem. بديل سطر واحد لـ `width: 100%; max-width: 60rem`.
`padding: max(1rem, 3vw)`	"خد الأكبر": مسافة بتكبر مع الشاشة، بس مش أقل من 1rem.
`clamp(a, b, c)`	الاتنين مع بعض (فصل 9).

10.6 الألوان الحديثة

style.css

```css
:root {
  --brand: oklch(55% 0.2 260);
  --brand-hover: oklch(from var(--brand) calc(l - 10%) c h);
  --brand-soft: color-mix(in oklch, var(--brand) 15%, white);
}
```

`oklch(L C H)`	**L** = الإضاءة (0% أسود لـ 100% أبيض)، **C** = التشبّع (Chroma)، **H** = درجة اللون (0 لـ 360). ميزتها إن الـ L "صادقة" للعين: لونين بنفس الـ L باينين بنفس الإضاءة، عكس `hsl()`. و Tailwind 4 كاتب ألوانه الافتراضية كلها بـ `oklch`.

`oklch(from var(--brand) …)`	**Relative color syntax**: خد لون موجود وعدّل جزء منه. هنا: نفس اللون أغمق 10% لـ Hover. (ميزة أحدث من الباقي، اتأكد من الـ Baseline تحت.)
`color-mix(in oklch, A 15%, B)`	اخلط لونين بنسبة. هنا: 15% من اللون الأساسي على أبيض = خلفية فاتحة من نفس العيلة.

10.7 @supports و Baseline: ينفع أستخدمها؟

كل الميزات دي جديدة نسبياً. فإزاي تعرف إنها هتشتغل عند المستخدمين؟

Baseline

مبادرة من المتصفحات الكبيرة (Chrome و Edge و Firefox و Safari) بتدي لكل ميزة حالة: **Newly available** = اشتغلت في الأربعة. و**Widely available** = عدّى عليها 30 شهر من ساعتها، فآمنة لأغلب الناس. هتلاقي علامة Baseline في أول كل صفحة على MDN.

@supports

شرط في الـ CSS نفسه: "طبّق ده بس لو المتصفح فاهمه". مثال: `@supports (container-type: inline-size) { … }`. والمتصفح القديم هيتجاهل البلوك كله ويستخدم الشكل الأساسي.

الـ Progressive enhancement

اكتب شكل أساسي يشتغل في كل حتة، وبعدين زوّد الحاجات الجديدة كتحسين. لو المتصفح مش فاهم `:has()`، القاعدة كلها بتتجاهل، والصفحة لسه شغالة بس من غير الحتة الحلوة دي. CSS مصممة إنها متقعش لما متفهمش حاجة: بتتجاهلها وتكمّل.

المصدر

MDN · :has() · MDN · Using CSS nesting · MDN · Using CSS custom properties
web.dev · Baseline · MDN · color-mix() · MDN · oklch() · MDN · Logical properties and values

ملخص الفصل

- الـ Custom properties حية في المتصفح: بتتورث، وليها Scope، وبتتغير بالـ Media والـ JS.
- الـ Nesting بقى في CSS نفسها، و`&` = الأب. متعششش كتير.
- `:has()` بتختار العنصر حسب اللي جواه أو اللي بعده.
- الـ Logical properties (`inline-start` و`block`) بتقلب مع `dir` لوحدها.
- `min()` و`max()` و`clamp()` بيوفّروا Media queries.
- `oklch()` و`color-mix()` لألوان متناسقة.
- Baseline يقولك الميزة آمنة ولا لأ، و`@supports` للتحسين التدريجي.

صح ولا غلط؟

1. متغيرات CSS (`--x`) بتختفي بعد الـ Build زي متغيرات Sass. صح / غلط
2. `margin-inline-start` في صفحة `dir="rtl"` = `margin-right` . صح / غلط
3. لو المتصفح مش فاهم خاصية CSS، الصفحة كلها بتبوظ. صح / غلط

أسئلة مراجعة

1. إزاي تغيّر متغير CSS من JavaScript؟ وليه ده مستحيل مع Sass؟
2. اكتب Selector بـ `:has()` يختار أي `li` فيه لينك.
3. حوّل للـ Logical: `padding-left: 1rem; border-right: 1px solid; text-align: left;`
4. إيه الفرق بين Newly Baseline و Widely available؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل زرار بلونه الأساسي في `--brand` ، و3 نسخ بتغيّر المتغير بس: `.danger` و `.success` و `.muted` .
2. متوسط اعمل زرار Dark/Light بالـ JS يغيّر Attribute على الـ `html` (`data-theme="dark"`)، والـ CSS يغيّر المتغيرات بس جوه `[data-theme="dark"]` .
3. صعب اعمل Form يقفل زرار الإرسال بالـ CSS بس (`:has()` و `:user-invalid`)، من غير ولا سطر JS، واتأكد من Baseline لكل Selector استخدمته على MDN.

11

المرحلة 4 · ليه Frameworks

الفصل الحادي عشر

ليه الـ Frameworks اتعملت أصلاً؟ مشكلة مزامنة الـ DOM

في كورس Jonas هتبني مشاريع بـ querySelector و textContent و classList. وده ممتاز علشان تفهم. بس لما المشروع يكبر، هتلاقي نفسك بتطارد Bugs من نوع «الرقم فوق مش زي الرقم تحت». الفصل ده بيوريك المشكلة دي بكود حقيقي، وبيجرّب الحل «الساذج» وبيوريك بيقع فين بنتيجة حقيقية من Chrome، وبعدين بيشرح الأفكار التلاتة اللي كل Framework حديث مبني عليها: الـ Components والـ State والـ UI الـ Declarative.

بعد الفصل ده هتكون عارف

- المشكلة: نفس المعلومة ظاهرة في كذا مكان في الصفحة
- الكود الـ Imperative: انت بتقول للمتصفح كل خطوة
- الحل الساذج: ارسم كل حاجة من الأول، وليه بيبوظ
- UI = f(state): الفكرة اللي غيّرت كل حاجة
- الـ State والـ Derived state
- الـ Components والـ Props
- إزاي الـ Frameworks بتحدّث الـ DOM بكفاءة: Virtual DOM والـ Signals والـ Compiler
- إمتى متحتاجش Framework خالص

11.1 Bug صغير في عربية تسوّق

تخيّل صفحة متجر فيها 3 أماكن بتعرض حاجة عن السلة: رقم صغير على أيقونة السلة فوق، وقايمة المنتجات في السايدبار، والإجمالي تحت. والكود اللي كتبته بالطريقة اللي بتتعلمها دلوقتي:

cart.js · Imperative

```
const cart = []

function addToCart(product) {
  cart.push(product)

  // 1) الرقم اللي فوق
  document.querySelector('#badge').textContent = cart.length

  // 2) القايمة
  const li = document.createElement('li')
  li.textContent = product.name
  document.querySelector('#cart-list').append(li)

  // 3) الإجمالي
  const total = cart.reduce((sum, p) => sum + p.price, 0)
  document.querySelector('#total').textContent = total
}
```

```
function removeFromCart(id) {
  const i = cart.findIndex((p) => p.id === id)
  cart.splice(i, 1)
  document.querySelector('#badge').textContent = cart.length
  document.querySelector(`[data-id="${id}"]`).remove()
  // نسيت أحدّث الإجمالي!
}
```

شايف الـ Bug؟ في `removeFromCart` نسينا الإجمالي. فالمستخدم يشيل منتج، والرقم اللي فوق ينقص، والقايمة تتغير، والإجمالي **لسه زي ما هو**. ومحدش هياخد باله غير عميل زعلان.

وده مش لأنك مبرمج وحش. ده لأن الطريقة نفسها بتطلب منك تفتكر **كل مكان** في الصفحة بيعرض نفس المعلومة، **في كل Function** بتغيّرها. 3 أماكن × 5 Functions (إضافة، حذف، تغيير كمية، كوبون، تفريغ) = 15 حتة لازم تفتكرها. وزوّد مكان رابع (رسالة "الشحن مجاني فوق 500")، ارجع عدّل الـ 5 Functions.

> **الاسم الرسمي للمشكلة**
>
> دي اسمها **مزامنة الـ UI مع الـ State** (أو مزامنة الـ DOM). عندك **بيانات** (السلة) و**شكل** بيعرضها (الـ DOM)، ولازم يفضلوا متطابقين دايماً. والكود اللي فوق اسمه **Imperative** (أمري): انت بتقول للمتصفح "اعمل كذا وبعدين كذا" خطوة بخطوة.

11.2 الحل الساذج: ارسم كل حاجة من الأول

فكرة ذكية: بدل ما نحدّث كل حتة، نعمل Function واحدة اسمها `render()` ترسم **الصفحة كلها** من البيانات، وننديها بعد أي تغيير. مستحيل ننسى حاجة! جرّبنا ده فعلاً:

naive.html

```
const state = { items: ['Milk'], draft: '' }

function render() {
  document.querySelector('#app').innerHTML = `
    <h1>Cart (${state.items.length})</h1>
    <input id="draft" value="${state.draft}">
    <ul>${state.items.map((i) => `<li>${i}</li>`).join('')}</ul>`

  document.querySelector('#draft').addEventListener('input', (e) => {
    state.draft = e.target.value
    render()
  })
}
render()
```

وفتحناه في Chrome (بـ Puppeteer)، وضغطنا على الخانة، وكتبنا كلمة `Bread` حرف حرف:

```
Chrome · the real result
focused before typing: draft
state.draft: "B"
input value: "B"
focused after typing: "BODY"
```

كتبنا 5 حروف، ووصل **حرف واحد**! ليه؟ أول حرف نده `render()`، و`innerHTML` **مسح** الخانة القديمة وعمل واحدة جديدة مكانها. الجديدة شكلها نفس القديمة بالظبط، بس مش هي. فالـ Focus ضاع (راح للـ `BODY`)، والحروف الباقية راحت في الهوا. وكمان لو كان فيه Scroll أو نص متعلّم أو Animation شغالة، كلها هتضيع. وده غير إن رسم الصفحة كلها مع كل حرف بطيء لو الصفحة كبيرة.

> **وفيه مشكلة أمان كمان**
>
> لو `state.draft` فيه `<img src=x onerror=alert(1)>` (حد كتبه في الخانة أو جه من API)، الـ `innerHTML` هيشغّله كـ HTML حقيقي. دي ثغرة اسمها **XSS**. والـ Frameworks بتحميك منها افتراضياً: React بتعامل أي نص كنص.

يعني الفكرة صح (ارسم من البيانات)، بس التنفيذ غلط (امسح وارسم). محتاجين حاجة تاخد "الشكل اللي المفروض يبقى" وتقارنه بـ "الشكل الموجود"، وتغيّر **الفرق بس**: الرقم في الـ `h1` يتغير، والخانة **تفضل هي نفسها**. ده بالظبط اللي الـ Frameworks بتعمله.

11.3 الـ UI = f(state): الفكرة اللي غيّرت كل حاجة

الـ Frameworks الحديثة كلها مبنية على معادلة واحدة: **الشكل دالة في البيانات**. انت بتوصف "لو البيانات كذا، الشكل يبقى كذا"، والـ Framework يتولى الباقي.



شكل 11.1 انت مسؤول عن أول خطوتين بس. تغيّر البيانات، والشكل يتبعها لوحده، في كل مكان بيعرضها.

Imperative (أمري)

زي ما تركب تاكسي وتقول للسواق: "يمين، وبعدين تاني شمال، اقف عند الإشارة، ادخل الشارع اللي بعد الجامع". لو غلطت في خطوة، هتتوه.

Declarative (وصفي)

زي ما تطلب Uber وتحط العنوان. انت قلت **عايز توصل فين**، والتطبيق بيقرر **إزاي**. والـ Frameworks كده: انت بتقول الشكل، وهي تقرر الخطوات.

ونفس السلة بأسلوب Declarative (ده شكل React تقريباً، متقلقش من التفاصيل، ده للفكرة):

Cart.jsx · Declarative

```
function Cart({ items }) {
  const total = items.reduce((sum, p) => sum + p.price, 0)
  return (
    <>
      <span className="badge">{items.length}</span>
      <ul>
        {items.map((p) => <li key={p.id}>{p.name}</li>)}
      </ul>
      <p>Total: {total}</p>
    </>
  )
}
```

مفيش `querySelector` ولا `textContent` ولا `append`. مفيش "نسيت أحدّث الإجمالي"، لأن الإجمالي **بيتحسب** من `items` كل مرة. تضيف ولا تحذف ولا تفرّغ: نفس الوصف.

11.4 التلات أفكار اللي هتقابلها في React

1 · الـ State: البيانات اللي بتتغير

الـ **State** = أي معلومة بتتغير مع الوقت والشكل لازم يتبعها: محتوى السلة، الخانة فيها إيه، القايمة مفتوحة ولا مقفولة، البيانات اللي جت من API. وفيه نوع مهم اسمه **Derived state**: حاجة **بتتحسب** من State تانية، زي `total` و `items.length`. والقاعدة الذهبية: **متخزّنش حاجة تقدر تحسبها**. لو خزّنت `total` في متغير لوحده، رجعت لنفس مشكلة "نسيت أحدّثه".

2 · الـ Components: قطع ليجو

الـ **Component** = حتة UI ليها اسم، بتاخد بيانات (اسمها **Props**) وبترجّع شكل. زي Function: نفس المدخلات = نفس الشكل.



شكل 11.2 الصفحة شجرة Components. ProductCard اتكتب مرة واتستخدم 20 مرة بـ Props مختلفة. وده بيشبه الـ Landmarks والـ Semantic HTML اللي اتعلمتها: كل حتة ليها اسم ودور.

3 · الـ Declarative UI

زي ما شفنا: بتوصف **إيه** مش **إزاي**. وده بيغيّر طريقة تفكيرك: بدل "لما يضغط على الزرار، افتح المينيو"، بتفكر "لو `isOpen` بـ true، المينيو ظاهر". والزرار بيغيّر `isOpen` وبس.

11.5 إزاي بيحدّثوا الفرق بس؟ تلات مدارس

كلهم بيحلوا نفس المشكلة (يغيّروا الفرق بس، والخانة تفضل هي نفسها)، بس بطرق مختلفة:

المدرسة	الفكرة	مين
Virtual DOM	كل ما الـ State تتغير، الـ Component يتنفّذ تاني ويطلّع "وصف" للشكل كـ Objects في الـ Memory. الـ Framework يقارن الوصف الجديد بالقديم (**Diffing**)، ويغيّر في الـ DOM الحقيقي الفرق بس.	React (وفيه React Compiler بيقلل الشغل اللي ملوش لازمة)
Reactivity / Signals	كل قيمة في الـ State "عارفة" مين بيستخدمها. لما تتغير، بتحدّث الأماكن دي بس، من غير ما تقارن حاجة.	Vue (`ref`)، و Angular (`signal`)، و Svelte 5 (`$state`)، و Solid
Compiler	أداة الـ Build بتقرا الكومبوننت وتكتب كود JS أمري جاهز بيغيّر الأماكن الصح بالظبط. يعني الـ Framework نفسه بيختفي تقريباً من الناتج.	Svelte أشهر مثال

جدول 11.1 تلات طرق لنفس الهدف

وانت كمستخدم React مش محتاج تفهم الـ Diffing بالتفصيل دلوقتي. المهم تفهم: **انت بتغيّر الـ State، والـ Framework بيغيّر الـ DOM**. ولو لقيت نفسك بتكتب `document.querySelector` جوه React، غالباً فيه حاجة غلط.

11.6 إمتى متحتاجش Framework خالص؟

مش كل موقع محتاج React. صفحة هبوط، مدونة، صفحة بروفايل، موقع شركة صغيرة، كلها HTML و CSS وشوية JS بتكفّيها جداً، وهتبقى أسرع وأبسط. الـ Framework بيستاهل لما يبقى عندك **State كتير بتتغير** وبتظهر في **أماكن كتير**: Dashboard، تطبيق شات، متجر فيه سلة وفلاتر، أي "تطبيق" مش "صفحة". والمشروع النهائي في الكتاب ده (فصل 26) معمول من غير Framework عن قصد.

المصدر

React · Thinking in React
React · Reacting to Input with State
MDN · Introduction to client-side frameworks
MDN · Cross-site scripting (XSS)

ملخص الفصل

- المشكلة: نفس البيانات ظاهرة في كذا مكان، والكود الـ Imperative لازم يفتكر يحدّثهم كلهم.
- الحل الساذج (`innerHTML` لكل حاجة) بيضيّع الـ Focus والكتابة (جرّبناها: 5 حروف وصل منهم 1)، وبطيء، وفيه خطر XSS.
- الفكرة الحديثة: **UI = f(state)**. توصف الشكل، والـ Framework يغيّر الفرق بس.
- الـ State بيانات بتتغير، والـ Derived state بتتحسب ومتتخزنش.
- الـ Components قطع ليها اسم بتاخد Props.
- تلات مدارس: Virtual DOM (React)، و Signals (Vue و Angular و Svelte)، و Compiler (Svelte).
- موقع بسيط مش محتاج Framework.

صح ولا غلط؟

1. `innerHTML` بيحافظ على الـ Focus في الخانة لو شكلها متغيرش. صح / غلط
2. الإجمالي في السلة مثال على Derived state. صح / غلط
3. كل موقع محترم لازم يتعمل بـ Framework. صح / غلط

؟ أسئلة مراجعة

1. اشرح الفرق بين Imperative و Declarative بمثال من حياتك.
2. ليه الكتابة في `naive.html` وصلت حرف واحد بس؟
3. ليه "متخزّنش حاجة تقدر تحسبها"؟
4. إيه الفرق بين طريقة Virtual DOM وطريقة Signals؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `naive.html` بنفسك (فايل HTML عادي)، وجرّب تكتب في الخانة، وشوف المشكلة بعينك.
2. متوسط صلّح `naive.html` من غير Framework: خلّي `render()` متلمسش الخانة، وتحدّث الـ `h1` والـ `ul` بس. لاحظ الكود بقى أعقد قد إيه.
3. صعب اكتب عربية التسوّق بالـ Imperative بـ 5 Functions (إضافة، حذف، كمية، كوبون، تفريغ) و4 أماكن عرض. وبعدين اكتبها تاني بـ `state` واحد و `render` لكل جزء لوحده. قارن عدد السطور وعدد الأماكن اللي ممكن تنسى فيها.

12

المرحلة 4 · ليه Frameworks

الفصل التاني عشر

React و Vue و Angular و Svelte: نفس الفكرة بأربع لهجات

انت رايح React، وده اختيار ممتاز. بس هتسمع الناس بتتخانق على Vue و Svelte و Angular، وهتشوف وظايف بتطلبهم. الفصل ده بيكتب نفس العداد بالأربعة، وكلهم اتعملوا Build فعلاً بنسخ أكتوبر 2026، وبيقارن الأفكار مش الـ Syntax: الـ State فين، والـ Template شكله إيه، وبيحدّثوا الـ DOM إزاي، وحجم الناتج كام. والهدف إنك تشوف إن اللي هتتعلمه في React بيتنقل لأي حاجة تانية.

بعد الفصل ده هتكون عارف

- نفس الكومبوننت بالأربعة، واتعمله Build فعلاً
- فين الـ State: useState و ref و signal و $state
- شكل الـ Template: JSX ضد HTML بإضافات
- حجم الـ JavaScript اللي طلع للمتصفح لكل واحد (قسناه)
- مقارنة على مستوى الفلسفة: Library ضد Framework كامل
- إيه اللي بيتنقل معاك من Framework للتاني
- ليه React اختيار منطقي ليك

12.1 العداد بالأربعة

أبسط Component ممكن: زرار مكتوب عليه العدد، وكل ضغطة تزوّده واحد. فيه **State** (العدد)، و**Event** (الضغطة)، و**عرض** (الرقم على الزرار). عملنا كل واحد في مشروع جديد (React و Vue و Svelte بـ `create-vite`، و Angular بـ `@angular/cli` 22)، وعملنالهم `build` فعلاً:

React 19.3

App.jsx

```jsx
import { useState } from 'react'

export default function App() {
  const [count, setCount] = useState(0)
  return (
    <button onClick={() =>
setCount(count + 1)}>
      Count: {count}
    </button>
  )
}
```

Vue 3.5

App.vue

```vue
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

<template>
  <button @click="count++">Count: {{
count }}</button>
</template>
```

Svelte 5.57

App.svelte

```svelte
<script>
  let count = $state(0)
</script>

<button onclick={() => count++}>Count:
{count}</button>
```

Angular 22.2

app.ts

```ts
import { Component, signal } from
'@angular/core';

@Component({
  selector: 'app-root',
  template: `<button
(click)="increment()">
    Count: {{ count() }}</button>`,
})
export class App {
  count = signal(0);
  increment() {
    this.count.update((c) => c + 1);
  }
}
```

12.2 نفصّص الفروق

الفكرة	React	Vue	Svelte 5	Angular
الـ State	`useState(0)` يرجّع القيمة + Function تغيّرها	`ref(0)` : Object جواه `.value` بيفكّه (والـ Template لوحده)	`$state(0)` : متغير عادي، بتغيّره بـ `count++`	`signal(0)` : تقراه كـ `count()` Function، وتغيّره بـ `update` / `set`
الـ Template	**JSX**: HTML جوه JavaScript	HTML بإضافات زي `@click` و `v-if` و `{{ }}`	HTML بإضافات زي `{ }` و `{#if}`	HTML بإضافات زي `(click)` و `@if` و `{{ }}`
الحدث	`onClick={…}` بحرف C كبير	`@click="…"`	`onclick={…}` زي HTML	`(click)="…"`
شكل الملف	`.jsx` : JavaScript	`.vue` : ملف واحد فيه script و template و style	`.svelte` : نفس فكرة Vue	`.ts` فيه Class، وممكن ملف HTML منفصل
تحديث الـ DOM	Virtual DOM + Diffing	Reactivity (Signals) + Virtual DOM خفيف	Signals + Compiler (Runes)	Change + Signals detection

جدول 12.1 نفس الأفكار، أسماء مختلفة

الـ $ في Svelte مش jQuery

الـ `$state` في Svelte 5 اسمها **Rune**: إشارة للـ Compiler إن المتغير ده State. هو مش Function حقيقية بتتنده وقت التشغيل زي `useState` . الـ Compiler بيشوفها وقت الـ Build ويكتب الكود اللي يحدّث الـ DOM.

12.3 حجم اللي وصل للمتصفح

نفس العداد بالظبط، من غير CSS، والأرقام من الـ Build الحقيقي (`vite build` للتلاتة الأولانيين و`ng build` لـ Angular). وزوّدنا نفس العداد بـ JavaScript عادي من غير أي Framework للمقارنة:

الـ Framework	الحجم الخام	بعد الضغط (gzip)
JavaScript عادي	0.83 kB	0.47 kB
Svelte 5.57	25.30 kB	10.15 kB
Vue 3.5	62.39 kB	24.59 kB
Angular 22.2	97.14 kB	29.32 kB (تقدير Angular CLI)
React 19.3	219.61 kB	68.64 kB

جدول 12.2 حجم الـ JavaScript لعداد واحد (أكتوبر 2026)

متفهمش الجدول ده غلط

الرقم ده هو "الثمن الثابت" اللي بتدفعه علشان تجيب الـ Framework نفسه. ومش معناه إن Svelte أحسن من React بـ 6 مرات. لما التطبيق يكبر، كود التطبيق نفسه بيبقى أكبر بكتير من الـ Framework، والفرق النسبي بيقل. والحجم معيار واحد من كذا معيار: الـ Ecosystem والوظايف والمكتبات والمجتمع بيفرقوا أكتر. بس الجدول بيوضح ليه صفحة بسيطة متحتاجش React (فصل 11)، وليه الـ Meta-frameworks بتحاول تقلل الـ JS اللي بيتبعت (فصل 22).

12.4 الفلسفة: Library ضد Framework كامل

React: صغيرة ومعاها Ecosystem

بتعمل الـ Components والـ State وبس. الـ Routing؟ اختار مكتبة. البيانات؟ اختار مكتبة. الـ Forms؟ اختار. ده بيديك حرية، وبيديك قرارات كتير. وعلشان كده Next.js بقى مشهور: بيختارلك. ومن Meta (فيسبوك سابقاً)، وأكبر Ecosystem وأكتر وظايف.

Angular: كل حاجة في العلبة

Routing و Forms و HTTP و Testing، كلهم رسميين ومن Google. ومبني على TypeScript من الأول. بيتكلم بلغة "الشركات الكبيرة": قواعد واضحة، وكل المشاريع شبه بعض. منحنى التعلم أعلى في الأول.

Vue: النص الحلو

أسهل بداية لحد جاي من HTML. فيه Router و State رسميين (Vue Router و Pinia)، بس مش إجباريين. مشهور جداً في آسيا وأوروبا وفي مشاريع Laravel (PHP). ومؤسسه Evan You هو كمان اللي عمل Vite.

Svelte: الـ Compiler

أقل كود تكتبه وأصغر ناتج. الـ Syntax أقرب لـ HTML و JS العاديين. المجتمع أصغر، والوظايف أقل، بس الناس اللي بتستخدمه بتحبه جداً. و SvelteKit هو الـ Meta-framework بتاعه.

12.5 إيه اللي بيتنقل معاك؟

الخبر الحلو: لو اتعلمت React صح، هتفهم الباقيين في أيام. لأن الأفكار نفسها:

1. **Components** بتاخد Props وترجّع UI.
2. **State** بتتغير، والـ UI يتبعها لوحده.
3. **Derived state** بتتحسب (`useMemo` في React، و`computed` في Vue و Angular، و`$derived` في Svelte).
4. **Effects** للحاجات اللي برة الـ UI: طلب API أو Timer (`useEffect` و`watch` و`$effect` و`effect`).
5. **Lists** بـ Key لكل عنصر، و**Conditional rendering**.
6. نفس **الأدوات** تحتهم: npm و Vite و ESLint و TypeScript. ودي بالظبط اللي جاية في الكتاب.

ليه React اختيار منطقي ليك دلوقتي

أكبر سوق وظايف، وأكبر عدد مكتبات ودروس وإجابات على أي مشكلة، و Next.js لما تحتاج سيرفر، و React Native لو حبيت تعمل تطبيقات موبايل بنفس المعرفة. ومش هتخسر حاجة: الأفكار هتتنقل معاك لأي Framework تاني. متضيعش وقت في "أنهي أحسن". اختار واحد واتعمق.

المصدر React · Quick Start · Vue · Introduction · Svelte · Overview · Svelte · $state · Angular · Overview · Angular · Signals

ملخص الفصل

- الأربعة بيعملوا نفس الحاجة: Declarative UI + State + Components.
- الـ State: `useState` و`ref` و`$state` و`signal`.
- React بتكتب الـ UI بـ JSX جوه JS، والباقيين HTML بإضافات.
- الحجم الثابت لعداد واحد: من 10 كيلو (Svelte) لـ 69 كيلو (React) بعد الضغط، بس ده معيار واحد.
- React مكتبة ومعاها Ecosystem، و Angular كل حاجة جوه، و Vue في النص، و Svelte Compiler.
- الأفكار بتتنقل: لو فهمت React كويس، الباقيين سهلين.

صح ولا غلط؟

1. في Svelte 5 الـ `$state` Function بتتنده وقت التشغيل زي `useState`. صح / غلط
2. Angular فيه Routing و Forms رسميين جوه الـ Framework. صح / غلط
3. React أكبر في الحجم الثابت، فمعناه إن أي تطبيق React أبطأ من أي تطبيق Svelte. صح / غلط

؟ أسئلة مراجعة

1. اكتب مكان الـ State في الأربعة من الذاكرة.
2. إيه الفرق بين JSX والـ Templates في Vue و Svelte؟
3. ليه حجم الـ Framework مش المعيار الوحيد؟
4. اذكر 3 أفكار هتتنقل معاك من React لأي Framework تاني.

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل مشروع Vue بـ `npm create vite@latest my-vue -- --template vue`، وحط فيه العداد، وشغّله بـ `npm run dev`.
2. متوسط اعمل نفس الحاجة لـ Svelte و React، وقارن الـ `package.json` بتاع التلاتة: إيه في `dependencies` وإيه في `devDependencies`؟
3. صعب زوّد على العداد: زرار Reset، ورسالة "Big number!" لما العدد يعدّي 10 (Conditional rendering)، في اتنين Frameworks على الأقل. اكتب الفرق اللي حسيته.

13

المرحلة 5 · npm بالتفصيل

الفصل التالت عشر

Node و npm: المخزن اللي فيه ملايين الحتت الجاهزة

أول أمر في أي كورس React هو npm create، وبعده npm install، وفجأة عندك فولدر اسمه node_modules فيه مية Package. الفصل ده بيشرح من الصفر: يعني إيه Package، و npm بيتكون من إيه، وإيه اللي بيحصل بالظبط لما تكتب npm install، وإيه node_modules و bin.، والفرق بين التسطيب المحلي والـ Global، و npx بيعمل إيه. وكل Output في الفصل حقيقي من Node 24.22.1 و npm 11.19.1.

بعد الفصل ده هتكون عارف

- Node على الماك: تسطّبه إزاي وتتأكد منه
- يعني إيه Package، و npm = موقع + Registry + أداة
- npm init -y بيعمل إيه (والـ type commonjs الجديد)
- npm install خطوة بخطوة: Resolve و Download و Lockfile و node_modules
- جوه node_modules: Hoisting و bin. والـ Packages الخاصة بالماك
- D- و E- و uninstall و npm ls
- محلي ولا Global: ليه الأفضل محلي
- npx و npm create و npm exec، والـ -- بيعمل إيه

انت استخدمت npm قبل كده

لو مشيت بالترتيب، فإنت كتبت `npm install` وشغّلت `node file.mjs` في كتاب APIs و JSON، وشفت `node_modules` في كتاب Git، وعرفت npm في دقيقة في كتاب الـ Terminal (فصل 11). الفصل ده والفصول اللي بعده بيشرحوا اللي كان بيحصل ورا الأوامر دي بالتفصيل.

13.1 ليه محتاجين npm أصلاً؟

تخيّل عايز تعرض "منذ 3 دقايق" جنب كل تعليق. ممكن تكتبها بنفسك: ثواني ودقايق وساعات وأيام وشهور، والجمع والمفرد، والمناطق الزمنية... أو تستخدم كود حد كتبه واختبره ملايين الناس قبلك. زمان كنت هتروح موقعه، تنزّل ملف `.js`، وتحطه في فولدر، وتفتكر تحدّثه كل فترة بإيدك. ولو هو نفسه بيعتمد على مكتبة تانية، تنزّلها هي كمان. ومع 20 مكتبة؟ كارثة.

npm حل المشكلة دي: سطر واحد بينزّل المكتبة وكل اللي بتعتمد عليه، وبيكتب النسخة، وأي حد في الفريق يقدر يجيب **نفس** الحاجة بالظبط بأمر واحد.

13.2 Node على الماك

Node.js = برنامج بيشغّل JavaScript برة المتصفح (فصل 2). و**npm** بييجي معاه جوه نفس التسطيبة. فيه كذا طريقة تسطّبه على الماك، وكلهم على `nodejs.org/en/download`:

الطريقة	الأمر أو المكان	مناسبة لـ
Installer رسمي	ملف `.pkg` من nodejs.org	أسهل حاجة، نسخة واحدة
Homebrew	`brew install node` (كتاب الـ Terminal فصل 11)	لو كل برامجك من brew
مدير نسخ	`nvm` أو `fnm`	لو هتحتاج أكتر من نسخة (مشروع قديم على Node 20 ومشروع جديد على 24)

جدول 13.1 طرق تسطيب Node على الماك

أياً كانت الطريقة، اختار نسخة **LTS** (Long-Term Support). وقت الكتابة: **Node 24.22.1 LTS** (والـ Current هي 26.12.1، دي للي عايز أحدث المميزات). والـ Node 24 جاي معاه **npm 11.19.1**. اتأكد:

```
Terminal
user@MacBook ~ % node -v
v24.22.1
user@MacBook ~ % npm -v
11.19.1
user@MacBook ~ % npx --version
11.19.1
```

الـ LTS يعني إيه؟

Node بيطلع نسخة كبيرة كل 6 شهور. النسخ الزوجية (20 و 22 و 24 و 26) بتبقى **LTS**: بتاخد تحديثات أمان وإصلاحات لحوالي 3 سنين. والفردية (25) بتعيش شهور قليلة. فللشغل الحقيقي: زوجية LTS. و `engines` في `package.json` بتاع Vite 8 بيطلب `^20.19.0 || >=22.12.0`، يعني أي LTS حديث هيشتغل.

13.3 npm = 3 حاجات بنفس الاسم

الـ Registry

سيرفر ضخم (`registry.npmjs.org`) فيه كل الـ Packages وكل نسخها. ده "المخزن". وسألنا npm عنه: `npm config get registry` رجّع `https://registry.npmjs.org/`.

الـ CLI

الأمر `npm` اللي على جهازك. بيكلّم الـ Registry، وبينزّل، وبيكتب الملفات. ده اللي هنشرحه.

الموقع

`npmjs.com`: تدوّر على Package، وتقرا الـ README، وتشوف عدد التنزيلات وآخر تحديث والـ License.

والـ Package نفسه؟

أي فولدر فيه ملف `package.json`. بس كده. مشروعك نفسه Package، و `vite` Package، و `react` Package.

13.4 npm init: أول package.json

Terminal

```
user@MacBook ~ % mkdir npm-play && cd npm-play
user@MacBook ~ % npm init -y
Wrote to /…/npm-play/package.json:

{
  "name": "npm-play",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

`npm init`	بيعمل `package.json` بأسئلة (الاسم؟ النسخة؟ ...).
`-y` / `--yes`	"قول آه لكل الأسئلة" وخد الافتراضي. الاسم بياخده من اسم الفولدر.
`"type": "commonjs"`	جديد في npm 11: بيكتبها صريحة. معناها إن ملفات `.js` هنا بتستخدم `require()` القديم مش `import`. وقوالب Vite بتكتب `"type": "module"`. هنشرح الفرق في فصل 14 و 17.

وكل مفتاح في الملف ده هنشرحه في الفصل الجاي. دلوقتي خلينا نسطّب حاجة.

13.5 npm install: إيه اللي بيحصل بالظبط

Terminal

```
user@MacBook ~ % npm install lodash@4.17.20

added 1 package, and audited 2 packages in 390ms

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

(النسخة القديمة دي اخترناها عن قصد علشان فيها ثغرات معروفة، وهنستخدمها في فصل 15 و 16. فمتسطّبهاش في مشروع حقيقي.)



شكل 13.1 ست خطوات ورا أمر واحد. والجملة «audited 2 packages» معناها مشروعك + lodash.

وده اللي اتغير في المشروع:

package.json (الجزء الجديد)

```
"dependencies": {
  "lodash": "^4.17.20"
}
```

لاحظ: طلبنا `4.17.20` بالظبط، و npm كتب `^4.17.20`. العلامة `^` دي معناها "4.17.20 أو أحدث، بشرط يفضل x.4". ودي أهم حاجة في npm كله، وليها فصل لوحدها (فصل 15). ومعاها اتعمل ملف `package-lock.json` وفولدر `node_modules`.

13.6 جوه node_modules

سطّبنا قالب Vite الـ Vanilla (اللي فيه Package واحد بس في `devDependencies`: `vite`)، وبصينا جوه:

Terminal

```
user@MacBook ~ % npm install
added 15 packages, and audited 16 packages in 3s
user@MacBook ~ % ls node_modules
@oxc-project  fdir         lightningcss-linux-x64-gnu  picocolors  rolldown        vite
@rolldown     lightningcss nanoid                      picomatch   source-map-js
detect-libc   postcss      tinyglobby
user@MacBook ~ % ls node_modules/.bin
nanoid  rolldown  vite
user@MacBook ~ % du -sh node_modules
 34M    node_modules
```

`15 packages`	انت طلبت واحد (`vite`)، بس `vite` نفسه محتاج `rolldown` و `lightningcss` و `postcss` ...، ودول محتاجين حاجات. دي اسمها **Transitive dependencies** (اعتماديات الاعتماديات).
كلهم في نفس المستوى	npm بيحطهم كلهم جنب بعض في `node_modules` مباشرة بدل ما كل واحد جوه اللي محتاجه. ده اسمه **Hoisting** (التسطيح). لو اتنين محتاجين نسختين مختلفتين من نفس الـ Package، واحدة بتتحط فوق والتانية جوه الـ Package اللي محتاجها.
`@oxc-project` و `@rolldown`	الـ `@` = **Scope**: Namespace لمؤسسة أو شخص. `@rolldown/binding-…` كله تبع فريق Rolldown، ومحدش غيرهم يقدر ينشر تحت الاسم ده.
`lightningcss-linux-x64-gnu`	Package **مخصوص لنظام التشغيل**. احنا جرّبنا على Linux، فنزل نسخة Linux. على ماك M1/M2/M3/M4 هينزل `lightningcss-darwin-arm64` بدلها (الـ Registry بيقول عليه `os = 'darwin'` و `cpu = 'arm64'`). وده سبب إنك **متنسخش** `node_modules` من جهاز لجهاز: نسخة الماك مش هتشتغل على Linux والعكس.
`node_modules/.bin`	فولدر مخفي فيه الأوامر اللي الـ Packages بتوفّرها. `vite` هنا ده اللي بيشتغل لما تكتب `npm run dev`. (الـ Package بيعلن عنها في مفتاح `bin` : بيقول `vite` : `"vite": "bin/vite.js"`).
`34M`	علشان Package واحد. ومشروع React بـ ESLint طلع M90 (فصل 2). عادي، ومش بيترفع على Git.

وتقدر تشوف الشجرة بـ `npm ls` :

Terminal

```
user@MacBook ~ % npm ls
vanilla-app@0.0.0 /…/vanilla-app
└── vite@8.3.5
user@MacBook ~ % npm ls --all
vanilla-app@0.0.0 /…/vanilla-app
```

```
└─┬ vite@8.3.5
  ├── UNMET OPTIONAL DEPENDENCY @types/node@^20.19.0 || >=22.12.0
  ├── UNMET OPTIONAL DEPENDENCY fsevents@~2.3.3
  ├── UNMET OPTIONAL DEPENDENCY less@^4.0.0
  …
  ├─┬ lightningcss@1.33.0
  │ ├── detect-libc@2.1.2
  │ ├── UNMET OPTIONAL DEPENDENCY lightningcss-darwin-arm64@1.33.0
  │ ├── UNMET OPTIONAL DEPENDENCY lightningcss-darwin-x64@1.33.0
  │ ├── lightningcss-linux-x64-gnu@1.33.0
  …
  ├─┬ postcss@8.5.31
  │ ├── nanoid@3.3.20
  │ ├── picocolors@1.1.1
  │ └── source-map-js@1.2.2
  ├─┬ rolldown@1.2.15
  …
```

`npm ls`	الـ Packages اللي **انت** طلبتها بس (المستوى الأول).
`--all`	الشجرة كاملة.
`UNMET OPTIONAL DEPENDENCY`	مش Error. ده Package **اختياري** متسطبش، لسببين: يا إما لنظام تاني (نسخة الماك `darwin-arm64` على جهاز Linux، و `fsevents` اللي بيراقب الملفات على الماك بس)، يا إما حاجة Vite بيستخدمها **لو** انت سطبتها (زي `less` لو بتكتب Less). على الماك بتاعك هتلاقي نسخة الماك متسطبة ونسخة Linux هي اللي UNMET.
`npm ls vite` / `npm explain lodash`	"مين جاب الـ Package ده؟". `npm explain lodash` رد: `lodash@"^4.17.20" from the root project`. مفيد جداً لما `npm audit` يقولك على ثغرة في Package انت عمرك ما سطبته.

13.7 Flags التسطيب اللي هتستخدمها

الأمر	بيعمل إيه
`npm install` (أو `npm i`)	من غير اسم: سطّب كل اللي في `package.json`. أول أمر بعد ما تعمل `git clone` لأي مشروع.
`npm i react`	سطّب وزوّد في `dependencies` (الكود اللي هيروح للمتصفح).
`npm i -D eslint`	`-D` = `--save-dev`: في `devDependencies` (أدوات).
`npm i -E prettier`	`-E` = `--save-exact`: اكتب `3.9.10` من غير `^`. Prettier نفسه بيطلب كده في الدوكيومنتيشن، لأن كل نسخة ممكن تغيّر شكل التنسيق.
`npm i ms@~2.0.0`	نسخة أو Range معين بعد `@`. و `@latest` = أحدث نسخة.
`npm uninstall ms`	امسح من `node_modules` ومن `package.json` والـ Lockfile. (اختصاراته: `un` و `rm` و `remove`).

الأمر	بيعمل إيه
`npm ci`	تسطيب "نظيف" من الـ Lockfile بالحرف (فصل 15).

جدول 13.2 npm install وأصحابه

جرّبنا `-D -E` وبعدها `~`، ودي النتيجة في `package.json`:

```json
"dependencies": {
  "lodash": "^4.17.20",
  "ms": "~2.0.0"
},
"devDependencies": {
  "is-number": "7.0.0"
}
```

13.8 محلي ولا Global؟

كل اللي فات كان **محلي** (Local): جوه فولدر المشروع. وفيه نوع تاني: **Global** بـ `-g`، بيتسطب في مكان واحد على الجهاز كله، والأمر بيبقى متاح في أي فولدر في الـ Terminal. علشان منلخبطش الجهاز، جرّبناه في فولدر تجريبي بـ `--prefix`:

```
user@MacBook ~ % npm install -g cowsay --prefix /tmp/gdemo
user@MacBook ~ % ls /tmp/gdemo/bin
cowsay  cowthink
user@MacBook ~ % npm ls -g --depth=0 --prefix /tmp/gdemo
/tmp/gdemo/lib
└── cowsay@1.6.0
```

ومن غير `--prefix`، المكان بيتحدد بـ `npm prefix -g`. على جهاز التجربة طلع `/home/box/.local`. وعلى الماك، لو Node من Homebrew غالباً هيطلع `/opt/homebrew` (على أجهزة Apple Silicon)، فالـ Packages في `/opt/homebrew/lib/node_modules` والأوامر في `/opt/homebrew/bin` اللي في الـ PATH بتاعك (كتاب الـ Terminal). جرّب الأمر عندك.

> **القاعدة: محلي دايماً، تقريباً**
>
> لو سطبت Vite **Global**، كل مشاريعك هتستخدم نفس النسخة. ومشروع قديم محتاج Vite 6 ومشروع جديد محتاج Vite 8؟ مشكلة. وزميلك اللي عمل `git clone` مش هيلاقي Vite خالص. فالأدوات اللي مشروعك محتاجها (Vite و ESLint و Prettier و TypeScript) تتسطب **محلي** بـ `-D`، فتتكتب في `package.json` وكل الفريق ياخد نفس النسخة. الـ Global بس لأدوات **انت** بتستخدمها على الجهاز كله مش تبع مشروع، زي CLI بتاع Vercel أو Netlify (فصل 25). وحتى دول تقدر تشغّلهم بـ `npx` من غير تسطيب.

مشكلة 1 **EACCES: permission denied مع npm install -g**

العَرَض: `npm install -g …` بيفشل بـ `EACCES`.

السبب: الـ prefix بتاع npm في فولدر محتاج صلاحيات Admin (بيحصل غالباً مع تسطيبة Node الرسمية من `.pkg`).

الحل: **متستخدمش** `sudo npm`، ده بيلخبط صلاحيات الملفات وممكن يشغّل Scripts من Packages كـ root. الحل الرسمي من npm: استخدم مدير نسخ زي nvm أو fnm (بيسطّب في فولدر بتاعك)، أو غيّر الـ prefix لفولدر في الـ Home بتاعك.

13.9 npx: شغّل من غير ما تسطّب

Terminal

```
user@MacBook ~ % npx -y cowsay "npx works"
 ___________
< npx works >
 -----------
        \   ^__^
         \  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

`npx cowsay`	"شغّل الأمر `cowsay`". لو موجود في `node_modules/.bin` بتاع المشروع، يشغّله. لو مش موجود، ينزّله في كاش مؤقت (مش في مشروعك) ويشغّله.
`-y` / `--yes`	من غيرها، npx هيسألك "Need to install the following packages... Ok to proceed?" قبل ما ينزّل حاجة جديدة. خليها في دماغك: npx ممكن ينزّل كود من الإنترنت ويشغّله، فاتأكد من الاسم (فصل 16).
`npx = npm exec`	نفس الحاجة بالظبط، و`npx` اختصار أقصر.

npx create = npm create-...

أول أمر في كورس React: `npm create vite@latest`. ده مش أمر سحري. `npm help init` بيقول بالحرف:

npm help init

```
npm init <package-spec> (same as `npx create-<package-spec>`)
npm init <@scope> (same as `npx <@scope>/create`)
```

و`npm create` اسم تاني لـ `npm init`. يعني:

تشريح الأمر

```
npm create vite@latest my-app -- --template react
```

1. `npm create vite@latest` = `npx create-vite@latest` : نزّل Package اسمه `create-vite` (أحدث نسخة) وشغّله. ده Package عادي على npm، وقت الكتابة نسخته 9.3.1.
2. `my-app` بيتبعت لـ `create-vite` : اسم الفولدر.
3. `--` "اللي بعدي مش لـ npm، ابعته زي ما هو للأمر اللي بيتشغّل". من غيرها npm ممكن يفتكر `--template` Flag ليه هو.
4. `--template react` Flag لـ `create-vite` : القالب. وشفنا كل القوالب بـ `npx create-vite@latest --help` (فصل 18).

ونفس الفكرة: `npm init @eslint/config@latest` = `npx @eslint/create-config@latest` (فصل 23).

13.10 ليه vite شغال في npm run ومش شغال في الـ Terminal؟

جرّبنا الأمر بتلات طرق في نفس المشروع:

```
Terminal

user@MacBook ~ % vite --version
bash: vite: command not found
user@MacBook ~ % npx vite --version
vite/8.3.5 linux-x64 node-v24.22.1
user@MacBook ~ % npm run v
> npm-play@1.0.0 v
> vite --version

vite/8.3.5 linux-x64 node-v24.22.1
```

الـ Terminal بيدوّر على الأوامر في الـ **PATH** بس (كتاب الـ Terminal)، و`node_modules/.bin` مش فيه. فـ `vite` لوحدها "command not found" (على الماك بـ zsh: `zsh: command not found: vite`). أما `npx` و`npm run` فبيزوّدوا `node_modules/.bin` للـ PATH مؤقتاً قبل ما يشغّلوا الأمر. علشان كده الـ `scripts` في `package.json` بتكتب `"dev": "vite"` عادي.

المصدر

npm Docs · About npm
npm Docs · npm install
npm Docs · npm exec (npx)
npm Docs · npm init / create
npm Docs · Folders (prefix, node_modules, .bin)
npm Docs · Resolving EACCES
Node.js · Download
Node.js · Releases (LTS)

ملخص الفصل

- Node بيشغّل JS برة المتصفح، و npm بييجي معاه. اختار LTS (وقت الكتابة 24.22.1 مع npm 11.19.1).
- npm = Registry + CLI + موقع. والـ Package = فولدر فيه `package.json`.
- `npm install` : Resolve ثم Fetch ثم Verify ثم Extract ثم Save ثم Audit.
- `node_modules` فيه الـ Transitive dependencies متسطّحة، و`.bin` فيه الأوامر، وفيه Packages مخصوصة لكل نظام.
- `-D` للأدوات، و`-E` لنسخة بالظبط، و`npm ls` و`npm explain` للشجرة.
- محلي دايماً تقريباً، والـ Global لأدوات الجهاز بس، ومن غير `sudo`.
- `npx` بيشغّل من `.bin` أو ينزّل مؤقتاً، و`npm create x` = `npx create-x`، و`--` بيعدّي الـ Flags للأمر.

صح ولا غلط؟

1. `npm i -D vite` بيكتب Vite في `dependencies`. صح / غلط
2. ينفع تنسخ `node_modules` من الماك لسيرفر Linux وتشتغل عادي. صح / غلط
3. `npm create vite@latest` بيشغّل Package اسمه `create-vite`. صح / غلط

أسئلة مراجعة

1. ليه طلبت Package واحد واتسطب 15؟ وإيه اسم الظاهرة دي؟
2. إيه الفرق بين `npm i lodash@4.17.20` واللي اتكتب في `package.json`؟
3. ليه الأفضل تسطّب Vite محلي مش Global؟
4. اشرح وظيفة الـ `--` في `npm create vite@latest my-app -- --template react`.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `node -v` و`npm -v` و`npm prefix -g` و`npm config get registry` على الماك بتاعك، واكتب النتايج.
2. متوسط اعمل مشروع بـ `npm init -y`، وسطّب `vite` بـ `-D`، وبعدين `ls node_modules` ودوّر على الـ Package الخاص بالماك بتاعك (`darwin-arm64` ولا `darwin-x64`؟).
3. صعب اعمل فولدر جديد، واكتب فيه `package.json` بإيدك (من غير `npm init`) فيه اسم ونسخة بس. سطّب حاجة، وامسح `node_modules` و`package-lock.json`، وشغّل `npm install`. اتأكد إن كل حاجة رجعت. إيه اللي كان ممكن يختلف لو الـ Lockfile اتمسح؟ (الفصل الجاي فيه الإجابة)

14

المرحلة 5 · npm بالتفصيل

الفصل الرابع عشر

package.json مفتاح مفتاح

package.json هو «بطاقة الهوية» بتاعة أي مشروع JavaScript: اسمه، ونوعه، وأوامره، والحاجات اللي محتاجها. أول ملف تفتحه في أي مشروع جديد، ومنه تعرف المشروع بيتشغّل إزاي. الفصل ده بيشرحه مفتاح مفتاح، ويركّز على scripts (pre و post وتمرير الـ Arguments) و type (ليه import بيضرب Error) والأنواع الأربعة لـ dependencies و overrides.

بعد الفصل ده هتكون عارف

- تقرا package.json أي مشروع وتعرف هو إيه وبيشتغل إزاي
- name و version و private و description
- type: module ولا commonjs، والـ Error اللي بيطلع لو غلط
- scripts: npm run و start و test و pre/post و --
- dependencies و devDependencies و peerDependencies و optionalDependencies
- engines و bin و exports (لما تبص على package.json بتاع مكتبة)
- overrides: تجبر نسخة جوه الشجرة
- npm pkg get/set: تعدّل من الـ Terminal

14.1 نوعين package.json

هتقابل package.json بشكلين: بتاع **تطبيق** (مشروعك، مش هيتنشر على npm) وبتاع **مكتبة** (Package متنشر، زي vite). الأول بسيط. ده اللي عمله `create-vite` للقالب الـ Vanilla:

Dockerfile

1 اسم المشروع. حروف صغيرة ومن غير مسافات. في التطبيق مش مهم أوي، في المكتبة هو اللي الناس بتكتبه في `npm i`.	`{` `  "name": "vanilla-app",`
2 رقم النسخة بنظام SemVer (الفصل الجاي). 0.0.0 لأنه لسه بادئ.	`  "version": "0.0.0",`
3 "ممنوع يتنشر". لو كتبت `npm publish` بالغلط، npm هيرفض. أي تطبيق لازم يبقى فيه ده.	`  "private": true,`
4 ملفات `.js` هنا ES Modules (`import` / `export`). تحت بالتفصيل.	`  "type": "module",`

5 الأوامر بتاعة المشروع: `npm run dev` و `npm run build` و `npm run preview`. فصل 18 بيشرح كل واحد بيعمل إيه.	`"scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },`
6 الأدوات. لاحظ إن القالب كاتب `^8.3.4`، والمتسطب فعلاً طلع 8.3.5 (الـ `^` سمحت بكده، فصل 15).	`"devDependencies": { "vite": "^8.3.4" }`

ومشروع React فيه نفس الشكل، بس فيه `dependencies` (react و react-dom) و`devDependencies` أكتر (فصل 2). ولو شفت أي مشروع على GitHub، أول حاجة تعملها: افتح package.json واقرا الـ scripts، تعرف بيشتغل إزاي.

14.2 type: ليه import بيضرب Error؟

JavaScript فيها نظامين Modules (فصل 17 بالتفصيل): القديم بتاع Node (**CommonJS**: `require` و`module.exports`) والرسمي (**ES Modules**: `import` و`export`). و Node بيعرف الملف من أنهي نوع من `type`. جرّبنا نفس الملف بتلات قيم:

a.js

```
import { sep } from "node:path"; console.log("sep is", sep)
```

"type": "commonjs"

```
Warning: Failed to load the ES module: /tmp/ty/a.js. Make sure to set "type": "module" in the
nearest package.json file or use the .mjs extension.
SyntaxError: Cannot use import statement outside a module
```

Terminal

```
"type": "module" #
user@MacBook ~ % node a.js
sep is /
```

خالص type من غير

```
[MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///tmp/ty/a.js is not specified
and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to /tmp/ty/package.json.
> sep is /
```

⚠ الفخ ده هيقابلك في كورس Node

`npm init -y` في npm 11 بيكتب `"type": "commonjs"`. فلو عملت مشروع Node جديد وكتبت `import express from "express"`، هيضرب `Cannot use import statement outside a module`. الحل: غيّر لـ `"module"` (أو استخدم `require` زي ما الكورس بيعمل). وكل قوالب Vite بتكتب `"module"`. ونفس القاعدة بالامتداد: `.mjs` دايماً ES Module، و`.cjs` دايماً CommonJS، مهما كان الـ type.

14.3 scripts بالتفصيل

في مشروع التجارب زوّدنا شوية scripts:

package.json (npm-play)

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "hello": "echo Hello",
  "prehello": "echo before",
  "posthello": "echo after",
  "greet": "node args.js",
  "v": "vite --version"
}
```

`npm run` من غير اسم بيعرض الكل:

Terminal

```
user@MacBook ~ % npm run
Lifecycle scripts included in npm-play@1.0.0:
  test
    echo "Error: no test specified" && exit 1
available via `npm run`:
  hello
    echo Hello
  prehello
    echo before
…
```

pre و post

Terminal

```
user@MacBook ~ % npm run hello

> npm-play@1.0.0 prehello
> echo before

before
```

```
> npm-play@1.0.0 hello
> echo Hello

Hello

> npm-play@1.0.0 posthello
> echo after

after
```

أي script اسمه `preX` بيشتغل تلقائياً قبل `X`، و`postX` بعده. وكل script بيطبع سطرين `>` : اسم المشروع@النسخة واسم الـ script، والأمر الحقيقي اللي بيتنفّذ. ولو أي خطوة فشلت (Exit code مش صفر، كتاب الـ Terminal)، الباقي بيقف.

تمرير Arguments: الـ -- تاني

args.js

```
console.log(process.argv.slice(2))
```

Terminal

```
user@MacBook ~ % npm run greet -- --name=Sara x

> npm-play@1.0.0 greet
> node args.js --name=Sara x

[ '--name=Sara', 'x' ]
user@MacBook ~ % npm run greet --name=Sara x

> npm-play@1.0.0 greet
> node args.js x

[ 'x' ]
```

من غير `--`، npm افتكر `--name=Sara` إعداد ليه هو وأكله، ووصل `x` بس. ودي فايدتها العملية: `npm run dev -- --host` علشان تفتح Vite من موبايلك على نفس الـ Wi-Fi (فصل 18)، و`npm run lint -- --fix` .

أسماء ليها اختصار

`npm test / npm t`	= `npm run test`. والقالب الافتراضي `exit 1` عن قصد: "لسه مفيش Tests". جرّبناه: طبع `Error: no test specified` ورجع exit 1.
`npm start`	= `npm run start`. ولو مفيش script اسمه start، npm بيشغّل `node server.js` لوحده لو الملف موجود (جرّبناه وطبع `> node server.js`). هتشوف `start` كتير في كورس Node.
`npm run dev / build`	مالهمش اختصار، لازم `run`.

الـ script عارف هو مين

تقدر تعدّيه في أول قراية

جرّبنا :script لأي بيئة متغيرات بيحط npm في `console.log(process.env.npm_package_name, process.env.npm_lifecycle_event)` اسمه script

`who` ، وطبع `ty who` . مفيدة لو script واحد عايز يتصرف حسب اسمه.

14.4 أربع أنواع dependencies

المفتاح	إمتى	مثال حقيقي
`dependencies`	كود هيشتغل مع التطبيق (يدخل الـ Bundle أو يشتغل على السيرفر)	`react` و `react-dom` ، و `express` في كورس Node
`devDependencies`	أدوات وقت التطوير والبناء بس	`vite` و `eslint` و `prettier` و `typescript`
`peerDependencies`	(في المكتبات) "أنا محتاج X، بس **انت** اللي تسطّبه"	`@tailwindcss/vite` بيقول `vite: '^5.2.0 || ^6 || ^7 || ^8'` ، و `react-dom` بيقول `react: '^19.3.1'`
`optionalDependencies`	(في المكتبات) "لو فشل تسطيبه، كمّل عادي"	`lightningcss` فيه 11 نسخة لأنظمة مختلفة، واحدة بس اللي بتتسطب

جدول 14.1 مين بيروح فين

الـ **peer** مهم تفهمه: `react-dom` لازم يشتغل مع **نفس** نسخة `react` اللي في مشروعك، مش نسخة خاصة بيه. فبدل ما يسطّب واحدة لنفسه، بيعلن "أنا محتاج react 19.3.1 أو أحدث في 19". ولو مشروعك فيه React 18، npm هيضرب Error اسمه `ERESOLVE` .

مشكلة 1 **npm error ERESOLVE unable to resolve dependency tree**

العَرَض: `npm install` بيفشل، وفي الرسالة `peer` و `Could not resolve dependency` .

السبب: Package عايز peer بنسخة غير اللي عندك (مثلاً مكتبة قديمة عايزة React 18 ومشروعك React 19).

الحل: الأول دوّر على نسخة أحدث من المكتبة تدعم نسختك. والـ `--legacy-peer-deps` اللي هتلاقيه في Stack Overflow بيقول لـ npm "تجاهل الـ peers"، وده ممكن يشتغل بس ممكن كمان يطلع Bugs غريبة وقت التشغيل. استخدمه وانت فاهم.

ومتقلقش أوي من dev ولا لا

في تطبيق Vite، الـ Bundler بياخد اللي انت عملته `import` بس، بغض النظر هو في أنهي قسم. فلو حطيت `react` في devDependencies بالغلط، الموقع هيشتغل برضه. الفرق بيبان في سيرفرات Node (اللي ممكن تسطّب بـ `--omit=dev`) وفي المكتبات المنشورة. فخليك منظم: اللي بيشتغل مع التطبيق في dependencies، والأدوات في devDependencies.

14.5 package.json بتاع مكتبة

افتح `node_modules/vite/package.json`، أو اسأل الـ Registry:

```
Terminal
user@MacBook ~ % npm view vite engines type
engines = { node: '^20.19.0 || >=22.12.0' }
type = 'module'
user@MacBook ~ % npm view vite bin license
bin = { vite: 'bin/vite.js' }
license = 'MIT'
```

`engines`	نسخ Node اللي المكتبة بتشتغل عليها. لو نسختك أقدم، npm بيطلع تحذير `EBADENGINE`. لو شفته، حدّث Node.
`bin`	"لما حد يسطبني، اعمل أمر اسمه `vite` يشغّل `bin/vite.js`". ده اللي بيتحط في `node_modules/.bin` (الفصل اللي فات).
`exports / main`	لما تكتب `import x from "vite"`، الملف اللي بيتفتح. `main` القديم ملف واحد، و`exports` الأحدث بيحدد كل مسار مسموح (Vite عنده `"."` و`"./client"` و...) ويمنع الباقي.
`files / repository / license`	إيه اللي يتنشر، ورابط الكود على GitHub، والـ License (MIT في vite). شوف الـ License قبل ما تستخدم أي مكتبة في شغل.

14.6 مفاتيح تانية هتقابلها

`packageManager`	زي `"pnpm@10.30.3"`: "المشروع ده بيتسطب بـ pnpm النسخة دي". لقيناه جوه Packages في `node_modules` نفسها. (فصل 16)
`browserslist`	المتصفحات اللي عايز تدعمها (أدوات زي Autoprefixer بتقراها).
`overrides`	تجبر نسخة معينة لـ Package جوه الشجرة حتى لو مش انت اللي طالبه. تحت.
`allowScripts`	جديد في npm 12: مين مسموحله يشغّل Scripts وقت التسطيب (فصل 16).

overrides: لما الثغرة في Package انت ماطلبتوش

`debug@2.6.9` بيطلب `ms` بالظبط `2.0.0`. نفترض إن 2.0.0 فيها مشكلة وصاحب debug مش هيحدّث. تقدر تجبرها:

```
Terminal
user@MacBook ~ % npm ls --all
ov@1.0.0 /tmp/ov
└┬ debug@2.6.9
  └── ms@2.0.0
user@MacBook ~ % npm pkg set overrides.ms=2.1.3
user@MacBook ~ % npm install
user@MacBook ~ % npm ls --all
ov@1.0.0 /tmp/ov
└┬ debug@2.6.9
  └── ms@2.1.3 overridden
```

كلمة `overridden` بتفكّرك إنك انت اللي غيرت. واعرف إنك بكده بتشغّل debug على نسخة صاحبه ماجرّبهاش، فاختبر كويس.

14.7 npm pkg: عدّل من غير ما تفتح الملف

```
Terminal
user@MacBook ~ % npm pkg get name version type
{
  "name": "npm-play",
  "version": "1.0.0",
  "type": "commonjs"
}
user@MacBook ~ % npm pkg set type=module
user@MacBook ~ % npm pkg set scripts.lint="eslint ."
```

مفيد في الشرح والـ Scripts، ومفيش خوف من فاصلة ناقصة تبوّظ الـ JSON. وده حصل كتير: `package.json` JSON صارم، فممنوع Comments وممنوع فاصلة بعد آخر عنصر (كتاب الـ APIs فصل 3).

المصدر npm Docs · package.json · npm Docs · scripts · npm Docs · npm run · npm Docs · npm pkg

Node.js Docs · Modules: Packages (type, exports)

ملخص الفصل

- package.json = بطاقة هوية المشروع. اقرا الـ scripts الأول.
- `private: true` لأي تطبيق، و`type: "module"` علشان `import` يشتغل في Node.
- `preX` / `postX` تلقائي، و`--` علشان تعدّي Flags للأمر، و`npm start` / `npm test` من غير run.
- dependencies للتطبيق، و devDependencies للأدوات، و peer = "سطّبه انت"، و optional = "لو فشل كمّل".
- `engines` و`bin` و`exports` في المكتبات، و`overrides` تجبر نسخة في الشجرة.

صح ولا غلط؟

1. لو مشروعك فيه `"type": "commonjs"` ، `import` في ملف `.js` هيشتغل عادي. صح / غلط
2. `npm run build --minify` بيبعت `--minify` للأمر. صح / غلط
3. `react-dom` بيعلن `react` كـ peerDependency. صح / غلط

أسئلة مراجعة

1. ليه أي تطبيق لازم يبقى فيه `"private": true` ؟
2. عندك scripts فيها `prebuild` و `build` . إيه اللي هيحصل لما تكتب `npm run build` ؟ ولو `prebuild` فشل؟
3. إيه الفرق بين dependency و peerDependency؟
4. إمتى تستخدم `overrides` ؟ وإيه خطورتها؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح package.json بتاع أي مشروع من كورس Jonas، وصنّف كل Package: تطبيق ولا أداة؟
2. متوسط اعمل script اسمه `clean` بيمسح `dist` (`rm -rf dist`)، وخليه يشتغل تلقائياً قبل `build` من غير ما تكتبه.
3. صعب اكتب ملف بـ `import` في مشروع عملته بـ `npm init -y` ، وشوف الـ Error بعينك. حلّه بطريقتين مختلفتين (من غير ما تغيّر الكود).

15

المرحلة 5 · npm بالتفصيل

الفصل الخامس عشر

الـ ^ والـ ~ والـ Lockfile: «بيشتغل عندي ومش عندك»

سارة عملت clone لمشروع زميلها، وكتبت npm install، والمشروع ضرب Error مش موجود عنده. نفس الكود، نفس package.json. الفرق؟ نسخ مختلفة جوه node_modules. الفصل ده بيحل اللغز: نظام أرقام النسخ (SemVer)، ومعنى كل رمز (^ و ~ و x و =>)، والـ Lockfile اللي بيقفل كل نسخة، والفرق بين npm install و npm ci و npm update، و npm outdated بيقرا إيه.

بعد الفصل ده هتكون عارف

- تقرا أي رقم نسخة: MAJOR.MINOR.PATCH ومعنى كل جزء
- تعرف كل Range يقبل إيه: ^ و ~ و x و * و =>
- الاستثناء بتاع 0.x والـ Prerelease
- dist-tags: latest و next و beta
- package-lock.json: بيسجل إيه، وليه يترفع على Git
- الفرق بين npm install و npm ci و npm update
- تقرا جدول npm outdated (Current و Wanted و Latest)

15.1 اللغز

في `package.json` مكتوب `"vite": "^8.3.4"`. يعني إيه بالظبط؟ "Vite 8.3.4"؟ لأ. القالب كاتب كده، والمتسطب عندنا طلع **8.3.5**. فلو زميلك سطّب الشهر اللي فات، أخد 8.3.4، وانت النهارده أخدت 8.3.5. ولو 8.3.5 فيها Bug... "بيشتغل عندي". علشان تفهم ده، محتاج تقرا الرقم الأول.

15.2 SemVer: تلات أرقام وكل واحد ليه معنى



شكل 15.1 8.3.5 = Semantic Versioning: الرقم نفسه بيقولك التحديث آمن ولا لأ.

دي **وعد** من صاحب المكتبة: "لو غيرت حاجة تكسر الكود بتاعك، هزوّد الـ MAJOR". فمن Vite 7 لـ Vite 8 (Rolldown بدل esbuild و Rollup، فصل 18) ممكن تحتاج تعدّل حاجات. لكن من 8.3.4 لـ 8.3.5؟ المفروض صفر تعديل. و`semver` (الـ Package اللي npm نفسه بيستخدمه) بيحسبها:

```
Terminal
user@MacBook ~ % npx semver -i patch 1.2.3
1.2.4
user@MacBook ~ % npx semver -i minor 1.2.3
1.3.0
user@MacBook ~ % npx semver -i major 1.2.3
2.0.0
```

لاحظ: لما الـ MINOR يزيد، الـ PATCH يرجع صفر. ولما الـ MAJOR يزيد، الاتنين يرجعوا صفر.

15.3 الـ Ranges: كل رمز يقبل إيه

بدل ما نحكي، ادياله نفس قايمة النسخ (`1.2.2 1.2.3 1.2.9 1.3.0 1.9.0 2.0.0` ومعاهم `1.3.0-beta.1` و `1.2.4-rc.1`) وسألناه كل Range بيقبل إيه:

```
Terminal
user@MacBook ~ % npx semver -r "^1.2.3" 1.2.2 1.2.3 1.2.9 1.3.0 1.9.0 2.0.0 1.3.0-beta.1
1.2.4-rc.1
1.2.3
1.2.9
1.3.0
1.9.0
```

الـ Range	قبل	بالعربي
`^1.2.3`	1.2.3 · 1.2.9 · 1.3.0 · 1.9.0	أي حاجة من 1.2.3 لحد قبل 2.0.0. **الافتراضي** لما تعمل `npm i` .
`~1.2.3`	1.2.3 · 1.2.9	Patches بس: لحد قبل 1.3.0.
`1.2.3`	1.2.3	بالظبط (اللي بيعمله `-E`).
`>=1.2.3 <1.3.0`	1.2.3 · 1.2.9	نفس `~1.2.3` مكتوبة صريح.
`1.x`	1.2.2 · 1.2.3 · 1.2.9 · 1.3.0 · 1.9.0	أي 1 (حتى الأقدم من 1.2.3).
`*`	كل اللي فوق + 2.0.0	أي حاجة. خطر.

جدول 15.1 النتايج الحقيقية لكل Range على نفس القايمة

ولاحظ إن **ولا Range** قبل `1.3.0-beta.1` أو `1.2.4-rc.1` . دي **Prerelease** (نسخ تجريبية بشَرطة بعد الرقم)، و npm مش بيديهالك إلا لو طلبتها صريح: Range زي `^1.2.4-rc.0` قبل `1.2.4-rc.1` (نفس 1.2.4) بس رفض `1.3.0-beta.1` .

الاستثناء: لما الـ MAJOR يبقى صفر

```
Terminal
user@MacBook ~ % npx semver -r "^0.2.3" 0.2.2 0.2.3 0.2.9 0.3.0 1.0.0
0.2.3
0.2.9
user@MacBook ~ % npx semver -r "^0.0.3" 0.0.3 0.0.4 0.1.0
0.0.3
```

نسخ `0.x` معناها "لسه بنجرّب، أي حاجة ممكن تتكسر". فالـ `^` بتبقى أحرص: `^0.2.3` بتتصرف زي `~`، و`^0.0.3` بتبقى نسخة واحدة بس. القاعدة: `^` مابتغيرش أول رقم مش صفر.

> **من الـ Terminal بتاعك**
>
> عايز تعرف Range معين هيجيب إيه دلوقتي؟ `npm view react@^18 version` بيعرض كل النسخ المطابقة. جرّبناه، وآخر سطور كانت `react@18.3.0 '18.3.0'` و`react@18.3.1 '18.3.1'`. و`npm view vite@~8.2 version` رجّع 8.2.0 و 8.2.1 و 8.2.2 و 8.2.4 (مفيش 8.2.3 منشورة). يعني لو حد كتب `~8.2` هياخد 8.2.4.

15.4 dist-tags: latest مش دايماً الأحدث رقماً

```
Terminal
user@MacBook ~ % npm view vite dist-tags
{
  alpha: '6.0.0-alpha.24',
  beta: '8.3.0-beta.1',
  latest: '8.3.5',
  previous: '6.4.5'
}
user@MacBook ~ % npm view react dist-tags
{
  …
  next: '19.3.0-canary-d5736f09-20260507',
  latest: '19.3.1',
  backport: '19.1.10',
  …
}
```

الـ **dist-tag** اسم بيشاور على نسخة. `npm i vite` = `npm i vite@latest`، و`latest` هي اللي صاحب المكتبة شايفها مستقرة. و`npm i react@next` بيجيب النسخة التجريبية. فـ `npm create vite@latest` معناها "النسخة اللي عليها tag اسمه latest".

15.5 الـ Lockfile: الحل

الـ Range في `package.json` بيقول "أقبل إيه". لكن `package-lock.json` بيسجل "**اخترت** إيه بالظبط"، لكل Package في الشجرة (مش اللي انت طلبته بس):

Dockerfile

1 نسخة صيغة الملف. 3 = صيغة npm 7 وأحدث.	`{`
	`  "name": "np13",`
	`  "version": "1.0.0",`
	`  "lockfileVersion": 3,`
	`  "requires": true,`

2 `""` = مشروعك نفسه، ومعاه نسخة من الـ Ranges.	`  "packages": {`
	`  "": {`
	`  "name": "np13",`
	`  "dependencies": {`
	`  "lodash": "^4.17.20", …`

3 النسخة اللي **اتسطبت فعلاً**.	`  "node_modules/lodash": {` `  "version": "4.17.20",`

4 نزلت منين بالظبط.	`  "resolved":`
	`"https://registry.npmjs.org/`
	`  lodash/-/lodash-4.17.20.tgz",`

5 بصمة الملف (Hash). لو حد غيّر الملف على السيرفر، البصمة مش هتطابق والتسطيب هيفشل. ده حماية (فصل 16).	`  "integrity": "sha512-` `PlhdFcillO…",`
6 والـ License كمان.	`  "license": "MIT"` `  }`

القواعد الثلاثة

- **ارفعه على Git دايماً** (مع `package.json`). و`node_modules` لأ (`.gitignore`، كتاب Git فصل 5).
- **متعدّلوش بإيدك.** npm هو اللي بيكتبه.
- **متمسحوش علشان تحل مشكلة.** مسحه = كل النسخ ممكن تتغير مرة واحدة. ولو حصل Conflict فيه في Git: خد نسخة أي فرع، وشغّل `npm install`، وهو يظبط نفسه.

15.6 install ولا ci ولا update؟

جرّبنا التلاتة على مشروع فيه `"lodash": "^4.17.20"` والـ Lockfile مسجّل 4.17.20، والأحدث المتاح 4.18.1:

Terminal

```
user@MacBook ~ % rm -rf node_modules && npm ci
user@MacBook ~ % npm ls lodash
np13@1.0.0 /tmp/np13
└── lodash@4.17.20
user@MacBook ~ % npm update
changed 1 package, and audited 4 packages in 366ms
found 0 vulnerabilities
user@MacBook ~ % npm ls lodash
np13@1.0.0 /tmp/np13
└── lodash@4.18.1
user@MacBook ~ % npm pkg get dependencies.lodash
"^4.17.20"
```

	npm install	npm ci	npm update
بيقرا	Lockfile + package.json	الـ Lockfile بس	package.json
النسخ	اللي في الـ Lockfile لو لسه بتطابق	الـ Lockfile بالحرف	أحدث نسخة جوه الـ Range
بيغيّر الـ Lockfile؟	لو لزم	أبداً	أيوه
node_modules	يكمّل على الموجود	يمسحه ويبدأ من الصفر	يحدّث
إمتى	بعد clone، أو لما تزوّد Package	في الـ CI والـ Deploy (فصل 25)	لما تقرر تحدّث

جدول 15.2 التلات أوامر

لاحظ إن `npm update` غيّر لـ 4.18.1 (جوه `^4`) بس **ماغيرش** `package.json`: لسه `^4.17.20`، والـ Lockfile هو اللي اتغير لـ 4.18.1. و`npm ci` صارم، وده المطلوب في سيرفر الـ Deploy:

npm ci من غير Lockfile

```
npm error code EUSAGE
npm error The `npm ci` command can only install with an existing package-lock.json or
npm error npm-shrinkwrap.json with lockfileVersion >= 1. Run an install with npm@5 or
npm error later to generate a package-lock.json file, then try again.
```

npm ci بعد ما غيرنا package.json لـ ms@^1.0.0 من غير install

```
npm error `npm ci` can only install packages when your package.json and package-lock.json or
npm-shrinkwrap.json are in sync. Please update your lock file with `npm install` before
continuing.
npm error Invalid: lock file's ms@2.1.3 does not satisfy ms@1.0.0
```

التاني ده بالظبط اللي هيحصل لو عدّلت `package.json` بإيدك ونسيت تعمل `npm install` قبل الـ commit: الـ Deploy هيفشل (فصل 25).

15.7 npm outdated: مين محتاج تحديث

```
Terminal
user@MacBook ~ % npm outdated
Package  Current  Wanted  Latest  Location             Depended by
lodash   4.17.20  4.18.1  4.18.1  node_modules/lodash  np13
ms         2.0.0   2.0.0   2.1.3  node_modules/ms      np13
```

`Current`	المتسطب دلوقتي.
`Wanted`	أحدث نسخة الـ Range بتاعك يسمح بيها. ده اللي `npm update` هيجيبه. شوف `ms` : الـ Range `~2.0.0` ، فالـ Wanted لسه 2.0.0 مع إن فيه 2.1.3.
`Latest`	الـ `latest` tag. علشان توصله لو برة الـ Range: `npm i ms@latest` (وده بيغيّر الـ Range في package.json).
`exit 1`	الأمر بيرجع exit code 1 لو فيه حاجة قديمة. مش Error، دي علشان الـ Scripts تعرف.

التحديث على MAJOR جديد

لما الـ Latest يبقى MAJOR جديد (زي 7 لـ 8)، اقرا "Migration guide" أو الـ Changelog الأول، وحدّث Package واحد في كل commit. وأدوات زي **Dependabot** على GitHub بتعمل Pull Request لكل تحديث لوحدها، والـ CI يجرّبه (كتاب Git فصل 17).

المصدر

GitHub · npm/node-semver (Ranges) · npm Docs · About semantic versioning · Semantic Versioning 2.0.0
npm Docs · dist-tag · npm Docs · npm outdated · npm Docs · npm update · npm Docs · npm ci · npm Docs · package-lock.json
GitHub Docs · Dependabot

ملخص الفصل

- MAJOR.MINOR.PATCH = بيكسر . ميزة . إصلاح.
- `^` = نفس الـ MAJOR (الافتراضي)، و `~` = نفس الـ MINOR، ومن غير رمز = بالظبط.
- مع `0.x` الـ `^` بتبقى أضيق، والـ Prerelease مش بتيجي إلا لو طلبتها.
- الـ Lockfile بيسجل النسخة و resolved و integrity لكل Package. ارفعه على Git ومتمسحوش.
- `npm ci` للـ CI (صارم ويمسح node_modules)، و `npm install` للشغل العادي، و `npm update` للتحديث جوه الـ Range.
- `npm outdated` : Latest / Wanted / Current.

صح ولا غلط؟

1. `^2.4.1` بيقبل 2.9.0. صح / غلط
2. `~2.4.1` بيقبل 2.5.0. صح / غلط
3. `npm update` بيغيّر الـ Range المكتوب في package.json. صح / غلط
4. `npm ci` بيشتغل من غير package-lock.json. صح / غلط

؟ أسئلة مراجعة

1. ارجع للغز بتاع سارة. إيه اللي حصل غالباً؟ وإزاي الـ Lockfile كان هيمنعه؟
2. `^0.4.2` يقبل 0.5.0؟ ليه؟
3. في `npm outdated`، إمتى الـ Wanted يبقى أقل من الـ Latest؟
4. ليه سيرفر الـ Deploy المفروض يستخدم `npm ci` مش `npm install`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — شغّل `npm outdated` على أي مشروع قديم عندك، وفسّر كل صف.
2. متوسط — بـ `npx semver -r` اتأكد من إجاباتك على أسئلة الصح والغلط فوق.
3. صعب — في مشروع تجربة: غيّر Range في package.json بإيدك لنسخة MAJOR تانية، وشغّل `npm ci` وشوف الـ Error، وبعدين صلّحه بالطريقة الصح وقارن الـ Lockfile قبل وبعد بـ `git diff`.

16

المرحلة 5 · npm بالتفصيل

الفصل السادس عشر

pnpm و Yarn و Bun، وأمان الـ Packages

هتفتح repo على GitHub وتلاقي pnpm-lock.yaml بدل package-lock.json، أو كورس بيقول bun install. ده مش لغة جديدة: دول مديري Packages تانيين بيقروا نفس package.json. والنص التاني من الفصل أهم: لما تكتب npm install، انت بتشغّل كود مئات الناس على جهازك. إزاي تحمي نفسك: npm audit، وهجمات حقيقية حصلت في 2025، والتغيير الكبير في npm 12 اللي بقى بيمنع Install Scripts افتراضياً.

بعد الفصل ده هتكون عارف

- تعرف pnpm و Yarn و Bun بيختلفوا في إيه عن npm
- تعرف المشروع بيستخدم أنهي واحد من الـ Lockfile
- تترجم أي أمر بين الأربعة
- تقرا تقرير npm audit وتعرف تصلّح
- تفهم هجمات الـ Supply chain: Typosquatting و postinstall و Account takeover
- allowScripts في npm 12: install-scripts approve و rebuild
- عادات أمان يومية: Lockfile و npm ci و provenance

16.1 ليه فيه أكتر من واحد؟

npm كان بطيء زمان، والـ `node_modules` بيتكرر في كل مشروع. فطلعت بدائل، وكلها بتقرا **نفس** `package.json` ونفس الـ Registry. الفرق في الطريقة اللي بيحطوا بيها الملفات على الجهاز، والسرعة، والـ Lockfile.

16.2 pnpm: نسخة واحدة على الجهاز كله

Terminal

```
user@MacBook ~ % npx pnpm@12.15.0 add -D vite
  Content-addressable store is at: /home/box/.local/share/pnpm/store/v11
  Virtual store is at:             node_modules/.pnpm
Packages: +15
+++++++++++++++
Progress: resolved 15, reused 15, downloaded 0, added 15, done

devDependencies:
+ vite 8.3.5

Done in 90ms using pnpm v12.15.0
user@MacBook ~ % ls -la node_modules
drwxr-xr-x 18 box box 4096 Oct 23 15:47 .pnpm
lrwxrwxrwx  1 box box   34 Oct 23 15:47 vite -> .pnpm/vite@8.3.5/node_modules/vite
```

`Content-addressable store`	مخزن واحد على الجهاز كله، كل ملف فيه متخزن مرة واحدة بس (باسم الـ Hash بتاعه). عشر مشاريع فيها Vite = نسخة واحدة على الديسك.
`reused 15, downloaded 0`	ده كان تاني مشروع نسطب فيه Vite بـ pnpm، فالـ 15 Package كانوا في المخزن، ومفيش ولا Package نزل من النت: 90ms.
`vite -> .pnpm/…`	جوه `node_modules` مفيش غير `vite` (اللي **انت** طلبته) كـ Symlink (اختصار، كتاب الـ Terminal). والباقي مستخبي في `.pnpm`.

> **الفايدة الخفية: مفيش Phantom dependencies**
>
> مع npm، كل الـ 15 Package متسطّحين جنب بعض (Hoisting، فصل 13)، فتقدر تعمل `import { nanoid } from "nanoid"` وهيشتغل، مع إنك عمرك ما طلبته. ولو Vite بطّل يستخدمه في نسخة جاية، مشروعك هيتكسر فجأة. ودي اسمها **Phantom dependency**. مع pnpm مش هتقدر تعمل import لحاجة مش مكتوبة في package.json بتاعك، فالمشكلة بتبان من الأول.

16.3 Bun و Yarn

Terminal

```
user@MacBook ~ % npx bun@1.4.4 add -d vite
Resolving dependencies
Resolved, downloaded and extracted [108]
Saved lockfile

installed vite@8.3.5 with binaries:
 - vite

15 packages installed [353.00ms]
user@MacBook ~ % ls
bun.lock  node_modules  package.json
```

Bun مش مدير Packages بس: ده Runtime كامل زي Node (بيشغّل JS و TS)، ومعاه مدير Packages سريع جداً، و Bundler، و Test runner. والـ `bun.lock` بتاعه ملف نصي. و**Yarn** أقدم بديل (2016، من Facebook). فيه نسختين بتلاقيهم في المشاريع: Yarn 1 (Classic)، آخر نسخة 1.22.22 وفي وضع صيانة) و Yarn 4 (Berry)، 4.18.1 وقت الكتابة). احنا ماجرّبناش Yarn في الكتاب، فالجدول تحت من الدوكيومنتيشن.

عايز	npm	pnpm	Yarn
سطّب الكل	`npm install`	`pnpm install`	`yarn`
زوّد Package	`npm i react`	`pnpm add react`	`yarn add react`
كـ dev	`-D`	`-D`	`-D`
امسح	`npm un x`	`pnpm remove x`	`yarn remove x`

عايز	npm	pnpm	Yarn
شغّل script	`npm run dev`	`pnpm dev`	`yarn dev`
شغّل من غير تسطيب	`npx`	`pnpm dlx`	`yarn dlx`
للـ CI	`npm ci`	`pnpm i --frozen-lockfile`	`yarn install --immutable`
الـ Lockfile	`package-lock.json`	`pnpm-lock.yaml`	`yarn.lock`

جدول 16.1 نفس الشغل بأربع أدوات

أستخدم أنهي واحد؟

- **المشروع موجود؟** استخدم اللي الـ Lockfile بتاعه موجود. متخلطش: لو عملت `npm install` في مشروع pnpm، هيطلعلك `package-lock.json` تاني وكل حاجة هتتلخبط. ومفتاح `packageManager` في package.json بيقولك برضه.
- **بتتعلم؟** npm. كل الكورسات (Jonas و Maximilian) والدوكيومنتيشن بيكتبوه الأول، وجاي مع Node.
- **بعدين؟** pnpm اختيار ممتاز لما يبقى عندك مشاريع كتير. و Bun لو عايز سرعة وبتجرّب.

16.4 انت بتثق في مين بالظبط؟

في مشروع الكتاب النهائي (فصل 26) طلبنا بإيدنا 8 Packages بس، و `npm audit signatures` قال إنه فحص **113** Package. كل واحد منهم كاتبه حد، وأي حد منهم ممكن حسابه يتسرق. وده مش كلام نظري:



شكل 16.1 هجمات الـ Supply chain: بدل ما يهاجموا موقعك، بيهاجموا حاجة موقعك بيعتمد عليها.

أنواع الهجمات

`Account takeover`	سرقة حساب صاحب Package مشهور ونشر نسخة ملغومة (chalk و debug). علشان كده الـ `^` بتاعتك ممكن تجيبها في أي `npm install` لو مفيش Lockfile.
`Typosquatting`	Package اسمه قريب من المشهور (حرف زيادة أو ناقص، أو `-js` في الآخر) فيه كود ضار. بيعتمد إنك تكتب الاسم غلط. اتأكد من الاسم من الدوكيومنتيشن الرسمي، مش من الذاكرة.

`Install scripts`	`preinstall` و `postinstall` في Package بيشتغلوا على جهازك **وقت التسطيب**، قبل ما تشغّل أي كود. وده اللي Shai-Hulud استخدمه.

npm audit 16.5

فاكر `lodash@4.17.20` من فصل 13؟

Terminal

```
user@MacBook ~ % npm audit
npm audit report #

lodash  <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Regular Expression Denial of Service (ReDoS) in lodash - https://github.com/advisories/GHSA-
29mw-wpgm-hmr9
lodash vulnerable to Code Injection via `_.template` imports key names - …
lodash vulnerable to Prototype Pollution via array path bypass in `_.unset` and `_.omit` - …
Lodash has Prototype Pollution Vulnerability in `_.unset` and `_.omit` functions - …
fix available via `npm audit fix`
node_modules/lodash

1 high severity vulnerability

To address all issues, run:
  npm audit fix
```

`lodash <=4.17.23`	الـ Range المصاب: أي نسخة لحد 4.17.23.
`Severity`	critical / high / moderate / low.
`GHSA-…`	رقم الثغرة في قاعدة GitHub Advisory. افتح الرابط تعرف التفاصيل: إمتى بتأثر، وهل انت أصلاً بتستخدم الـ Function المصابة؟
`npm audit fix`	بيحدّث لأقرب نسخة آمنة **جوه الـ Range** (هنا 4.18.1، وبعدها `found 0 vulnerabilities`).
`npm audit fix --force`	مسموحله يطلع برة الـ Range، يعني ممكن MAJOR جديد يكسر مشروعك. شوف إيه اللي هيتغير قبلها.

> ⚠ **audit مش كل حاجة**
>
> `npm audit` بيقارن بثغرات **معروفة ومسجّلة**. كود ملغوم لسه نازل من ساعة مش هيظهر فيه. وفي نفس الوقت، كتير من التنبيهات بتبقى في devDependencies ومش بتأثر على الموقع نفسه. فاقراه بعقل، ومتتجاهلوش ومتخافش منه.

16.6 npm 12: Install Scripts ممنوعة افتراضياً

في 8 يوليو 2026 نزل npm 12 وبقى `latest` على الـ Registry. أهم تغيير: Install Scripts بتاعة الـ Dependencies **مش هتشتغل** إلا لو انت سمحت. ولاحظ إن Node 24 لسه جاي معاه npm 11.19.1، فلو عايز 12: `npm install -g npm@latest`. جرّبناه بـ `npx npm@12` على `esbuild` (اللي محتاج postinstall علشان ينزّل الـ Binary بتاع نظامك):

```
Terminal
user@MacBook ~ % npx npm@12 install esbuild

added 2 packages, and audited 3 packages in 632ms

found 0 vulnerabilities
npm warn install-scripts 1 package had install scripts blocked because they are not covered by
allowScripts:
npm warn install-scripts   esbuild@0.28.2 (postinstall: node install.js)
npm warn install-scripts
npm warn install-scripts Run `npm install-scripts ls` to review, or `npm install-scripts
approve <pkg>` to allow.
user@MacBook ~ % npx npm@12 install-scripts approve esbuild
Approved esbuild:
  added esbuild@0.28.2
user@MacBook ~ % npm pkg get allowScripts
{
  "esbuild@0.28.2": true
}
user@MacBook ~ % npx npm@12 rebuild
rebuilt dependencies successfully
```



شكل 16.2 الطريقة الجديدة في npm 12

وتغييرات تانية في npm 12 تهمك: Dependencies من Git أو من رابط مباشر بقت ممنوعة افتراضياً (`allow-git` و`allow-remote` = `none`)، و`npm init` مابقاش بيكتب License (كان ISC)، و`npm adduser` اتشال (استخدم `npm login`). وكل ده كان موجود ورا تحذيرات من npm 11.16، فلو شفت `npm warn install-scripts` على npm 11، ده التحضير.

أي Package محتاج Scripts؟

اللي فيهم كود Native أو بينزّلوا Binary، زي `esbuild` و`sharp` (الصور). أدوات الكتاب ماطلبتش: جرّبنا `npx npm@12 install -D vite tailwindcss @tailwindcss/vite` وخلص من غير ولا تحذير `install-scripts`، لأن Rolldown و lightningcss بينزّلوا الـ Binary كـ optionalDependency مخصوصة لكل نظام (فصل 13) من غير Scripts. لو حاجة مش شغالة بعد التحديث، أول سؤال: هل فيه Script اتمنع؟ `npm install-scripts ls`.

16.7 عادات أمان يومية

1. **ارفع الـ Lockfile، واستخدم `npm ci` في الـ Deploy.** كده نسخة ملغومة جديدة مش هتدخل لوحدها. والـ `integrity` بيتأكد إن الملف ماتغيرش (فصل 15).
2. **قبل ما تسطب Package جديد:** افتح صفحته على npmjs.com: عدد التنزيلات، وآخر تحديث، ورابط الـ GitHub، والاسم بالحرف.
3. **`npx` بحذر:** بينزّل ويشغّل. اكتب الاسم من الدوكيومنتيشن الرسمي.
4. **Provenance:** على مشروعنا النهائي قال `npm audit signatures` `113 packages have verified registry signatures` و`34 packages have verified attestations`. الـ Attestation (Provenance) = إثبات إن الـ Package اتبنى من كود repo معين على GitHub Actions، مش من جهاز حد. Vite 8.3.5 منهم.
5. **فعّل FA2** على حسابك في npm و GitHub، ومتحطش Tokens في ملفات بترفع (فصل 19).
6. **متستخدمش `sudo npm install`** (فصل 13).

المصدر

Yarn · Getting started — Bun Docs · bun install — pnpm · pnpm add — pnpm · Motivation (store و symlinks)

npm Docs · npm audit — GitHub Changelog · npm v12 install-time security (8 يوليو 2026)

GitHub · npm/cli v12.0.0 release notes — npm Docs · approve-scripts / allowScripts — npm Docs · Provenance

CISA · Shai-Hulud alert (23 سبتمبر 2025) — Aikido · chalk (8 سبتمبر 2025) و debug compromised

ملخص الفصل

- pnpm و Yarn و Bun بيقروا نفس package.json. الفرق في التخزين والسرعة والـ Lockfile.
- pnpm: مخزن واحد + Symlinks، ومفيش Phantom dependencies. Bun: Runtime كامل ومدير Packages سريع.
- استخدم اللي الـ Lockfile بتاعه موجود، ومتخلطش.
- هجمات الـ Supply chain حقيقية: Account takeover و Typosquatting و Install scripts.
- `npm audit` + `audit fix` (من غير `--force`) إلا وانت فاهم.
- npm 12: `install-scripts ls/approve` بعدين `rebuild`، والموافقات في `allowScripts`.

صح ولا غلط؟

1. لو مشروع فيه `pnpm-lock.yaml`، الأحسن تشغّل `npm install` فيه. صح / غلط
2. `npm audit` بيكتشف أي كود ملغوم حتى لو لسه نازل. صح / غلط
3. في npm 12، postinstall بتاع Dependency مش هيشتغل إلا لو موجود في `allowScripts`. صح / غلط

أسئلة مراجعة

1. إيه هي الـ Phantom dependency؟ وليه pnpm بيمنعها؟
2. ترجم `npm ci` و`npx create-vite` لـ pnpm و Bun.
3. ليه الـ Lockfile + `npm ci` بيحموك من نسخة ملغومة نزلت النهارده؟
4. إيه الفرق بين `npm audit fix` و`npm audit fix --force`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `npm audit` و`npm audit signatures` على مشروع من الكورس، واكتب النتيجة.
2. متوسط جرّب `npx pnpm add -D vite` في فولدرين مختلفين، وقارن رقم `downloaded` و`reused` في المرتين.
3. صعب اعمل مشروع تجربة، وشغّل فيه `npx npm@latest install esbuild`. لو اتمنع، اعمل approve و rebuild، واتأكد إن `node -e "console.log(require('esbuild').version)"` شغال.

17

المرحلة 6 · Modules و Vite

الفصل السابع عشر

الـ Modules وليه محتاجين Bundler

في كورس Jonas هتوصل لفصل Modules وتكتب import و export. الفصل ده مش هيعيد شرح الصيغة، هو بيجاوب على أسئلة الإيكوسيستم: المتصفح نفسه بيفهم import إزاي؟ ليه import من npm مش شغال في المتصفح؟ ليه فتح الملف بدبل كليك بيضرب CORS؟ وليه رغم إن المتصفح بيفهم Modules لسه محتاجين Bundler زي Vite؟ كل ده بتجارب في Chrome حقيقي.

بعد الفصل ده هتكون عارف

- تعرف الفرق بين ESM و CommonJS من غير ما تتلخبط
- تشغّل Modules في المتصفح من غير أي أداة
- تفهم الـ Error بتاع Bare specifier وليه بيحصل
- تفهم ليه file:// بيضرب CORS مع Modules
- تعرف المشاكل اللي الـ Bundler بيحلها
- تشوف Tree-shaking و Code splitting بعينك في Build حقيقي
- تعرف تاريخ الأدوات: Browserify و webpack و Rollup و esbuild و Rolldown

17.1 نظامين، وانت هتقابل الاتنين

CommonJS (القديم Node)

```
JavaScript
const express = require('express')
module.exports = { start }
```

Node عمله سنة 2009 قبل ما JavaScript يبقى فيها Modules رسمي. هتلاقيه في كورسات Node قديمة وكتير من الـ Packages.

ES Modules (الرسمي)

```
JavaScript
import express from 'express'
export { start }
```

جزء من اللغة نفسها من 2015 (ES6). المتصفح بيفهمه، و Node بيفهمه، و Vite و React بيستخدموه. ده المستقبل وده اللي هتكتبه.

و Node بيعرف الملف من أنهي نوع إزاي؟ من `"type"` في package.json أو امتداد `.mjs` / `.cjs` (شفنا الـ Error في فصل 14). المتصفح أبسط: `<script type="module">`.

17.2 Modules في المتصفح من غير أي أداة

تلات ملفات، من غير npm ومن غير Vite:

math.js

```
export const add = (a, b) => a + b
export default function square(x) {
  return x * x
}
```

index.html

```
<script type="module" src="./main.js">
</script>
```

main.js

```
import square, { add } from './math.js'
console.log('add:', add(2, 3), 'square:', square(4))
```

شغّلناهم بسيرفر بسيط (`python3 -m http.server` ، أو `npx serve`) وفتحنا الصفحة في Chrome:

Console · Chrome DevTools

```
> add: 5 square: 16
```

شغال. المتصفح شاف `import`، فبعت Request تاني لـ `math.js` (هتشوفه في Network، كتاب 5 فصل 18)، ونفّذ. ولاحظ إن `./math.js` كاملة **بالامتداد**: المتصفح مش بيخمّن.

Error 1: فتح الملف بدبل كليك

file:///.../esm/index.html

```
Access to script at 'file:///workspace/book8/lab/esm/main.js' from origin 'null' has been
blocked by CORS policy: Cross origin requests are only supported for protocol schemes: chrome,
chrome-experimental-site-token-provider, chrome-extension, chrome-untrusted, data, http,
https, isolated-app.
Failed to load resource: net::ERR_FAILED
```

الـ Modules بتتحمّل بقواعد CORS (كتاب 5 فصل 14، وكتاب APIs فصل 13). والصفحة المفتوحة من `file://` الـ Origin بتاعها `null`، فـ Chrome رفض. الحل: أي سيرفر محلي على `http://localhost`. وده واحد من أسباب إن أي مشروع حديث بيبدأ بـ `npm run dev`.

Error 2: import من npm

bare.html

```
<script type="module">import _ from 'lodash'; console.log(_)</script>
```

```
Console · Chrome DevTools
Failed to resolve module specifier "lodash". Relative references must start with either "/", "./", or "../".
```

`'lodash'` من غير `./` اسمه **Bare specifier** (اسم "عريان"). Node بيعرف يدوّر عليه في `node_modules`، بس المتصفح مايعرفش يعني إيه `node_modules` أصلاً: هو بيفهم URLs بس. فيه حل رسمي في المتصفح اسمه **Import map** (`<script type="importmap">` بيقول "lodash = الرابط ده")، بس مع مية Package مش عملي. وده بالظبط شغل الـ Bundler: يحوّل `'lodash'` لملف حقيقي.

17.3 ليه Bundler لو المتصفح بيفهم Modules؟



شكل 17.1 المشاكل اللي الـ Bundler بيحلها. أول اتنين حتى لو كتبت JS عادي.

17.4 Tree-shaking و Code splitting بعينك

خدنا قالب Vite الـ Vanilla وكتبنا ملفين. `math.js` فيه Function مش هنستخدمها، و `chart.js` "تقيل" ومش محتاجينه غير لما المستخدم يدوس زرار:

src/chart.js

```
export function drawChart(el) {
  el.textContent =
    'chart loaded: ' +
'HEAVY_CHART_CODE'
}
```

src/math.js

```
export const add = (a, b) => a + b
export const unusedHelper = () =>
  'TREE_SHAKE_ME'
```

src/main.js

```
import { add } from './math.js'

const app = document.querySelector('#app')
app.innerHTML = `<p>2 + 3 = ${add(2, 3)}</p><button id="b">Show chart</button><div id="c">
</div>`

document.querySelector('#b').addEventListener('click', async () => {
  // dynamic import: load chart.js only when needed
  const { drawChart } = await import('./chart.js')
  drawChart(document.querySelector('#c'))
})
```

Terminal

```
user@MacBook ~ % npx vite build
✓ 7 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                0.38 kB | gzip: 0.26 kB
dist/assets/chart-C7XTFmcn.js  0.08 kB | gzip: 0.10 kB
dist/assets/index-n88G3mra.js  2.52 kB | gzip: 1.22 kB

✓ built in 61ms
user@MacBook ~ % grep -l TREE_SHAKE_ME dist/assets/*.js || echo "unusedHelper: not in bundle"
unusedHelper: not in bundle
user@MacBook ~ % cat dist/assets/chart-C7XTFmcn.js
function e(e){e.textContent=`chart loaded: HEAVY_CHART_CODE`}export{e as drawChart};
```

Tree-shaking	`unusedHelper` مش موجودة في أي ملف. الـ Bundler شاف إن محدش عملها import فشالها، زي ما بتهز الشجرة فالورق الناشف يقع. وده شغال علشان `import` / `export` ثابتين ويتقروا من غير تشغيل الكود، على عكس `require`.
Code splitting	`import()` كـ Function (Dynamic import) خلّى `chart.js` ملف لوحده (`chart-C7XTFmcn.js`) مش بيتحمّل غير لما حد يدوس الزرار. في React بتتعمل بـ `lazy()` لصفحات كاملة.

`Minify`	`drawChart` اتسمّت `e` جوه الملف، والمسافات اتشالت، وحتى `'chart loaded: ' + 'HEAVY_CHART_CODE'` اتجمعوا في نص واحد وقت البناء.
`C7XTFmcn`	Hash من محتوى الملف. لو غيرت `chart.js`، الاسم هيتغير، فالمتصفح هيعرف إنه جديد. ولو ماغيرتوش، يفضل في الـ Cache.

17.5 تاريخ سريع للأدوات

الأداة	إمتى	الفكرة
Browserify	2011	"خلّي `require` بتاع Node يشتغل في المتصفح". أول Bundler مشهور.
webpack	2012	كل حاجة Module (CSS وصور وخطوط) بـ Loaders و Plugins. كان أساس `create-react-app` (اللي اتعمله Deprecate في 2025). قوي جداً وإعداداته معقدة.
Rollup	2015	مبني على ESM من الأول، وأول ما عمل Tree-shaking. Vite لحد نسخة 7 كان بيستخدمه للـ Build.
Parcel	2017	"Zero config": من غير ملف إعدادات.
esbuild	2020	مكتوب بـ Go، أسرع 10-100 مرة. Vite لحد 7 كان بيستخدمه في الـ Dev.
Vite	2020	Dev server بيستخدم ESM بتاع المتصفح مباشرة (من غير Bundle وقت التطوير) + Build سريع. الفصل الجاي.
Turbopack	2022	من Vercel، مكتوب بـ Rust. الافتراضي في Next.js 16 (فصل 22).
Rolldown	2023	Rust، من فريق Vite، متوافق مع Rollup. بقى الـ Bundler الوحيد في Vite 8 (مارس 2026) بدل الاتنين (esbuild و Rollup).

جدول 17.1 هتسمع الأسماء دي في الكورسات والمقالات

الـ Bundler ولا الـ Build tool؟

الـ **Bundler** (Rolldown و webpack) بيجمّع الملفات. والـ **Build tool** (Vite) بيلفّ عليه: Dev server و HMR و env و Plugins. ولما حد يقول "React بـ Vite" يقصد الاتنين. وانت عملياً هتتعامل مع Vite بس، والباقي مستخبي جوه `node_modules` (فاكر `rolldown` في فصل 13؟).

المصدر

MDN · JavaScript modules
MDN · Import maps
MDN · import() (dynamic import)
Node.js Docs · ECMAScript modules
MDN · Tree shaking
Vite · Why Vite
Vite · Announcing Vite 8
React Blog · Sunsetting Create React App

ملخص الفصل

- CommonJS (`require`) قديم بتاع Node، و ESM (`import`) رسمي ومستقبل كل حاجة.
- المتصفح بيفهم ESM بـ `type="module"` ، بشرط سيرفر (مش `file://`) ومسارات كاملة.
- الـ Bare specifier (`'lodash'`) مش بيشتغل في المتصفح، والـ Bundler بيحلّه.
- الـ Bundler بيحل: الأسماء و الـ Waterfall و JSX/TS و CSS و الحجم و الـ Cache.
- Tree-shaking بيشيل اللي مش مستخدم، و `import()` بيقسّم لملفات بتتحمّل وقت الحاجة.

صح ولا غلط؟

1. `import x from 'react'` بيشتغل في المتصفح من غير أي أداة. صح / غلط
2. الـ Tree-shaking بيشتغل أحسن مع `import` / `export` من `require` . صح / غلط
3. Vite 8 بيستخدم Rolldown. صح / غلط

أسئلة مراجعة

1. ليه الصفحة اشتغلت على `http://localhost` وفشلت من `file://` ؟
2. اشرح الـ Waterfall بمثال من 3 ملفات.
3. إيه الفرق بين `import { x } from './a.js'` و `await import('./a.js')` في الـ Build؟
4. ليه اسم الملف في `dist` فيه حروف عشوائية؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل التلات ملفات بتوع أول الفصل، وشغّلهم بـ `npx serve` ، وبص على Network في DevTools: كام Request؟
2. متوسط افتح نفس الصفحة بدبل كليك من Finder، وشوف الـ Error في Console.
3. صعب كرر تجربة Tree-shaking و Code splitting في مشروع Vite، وبعدين غيّر سطر في `chart.js` واعمل build تاني. أنهي Hash اتغير؟

18

المرحلة 6 · Modules و Vite

الفصل التامن عشر

Vite من جوه: dev و build و preview

أول درس في قسم React عند Jonas هيبدأ بـ npm create vite@latest، وبعدها npm run dev، وصفحة بتظهر في ثانية. الفصل ده بيفتح الصندوق: كل ملف في القالب بيعمل إيه، والـ Dev server بيبعت للمتصفح إيه بالظبط، والـ HMR إزاي بيغيّر الألوان من غير ما العداد يتصفّر، و build بيطلّع إيه في dist، و preview فايدته إيه، وإعدادات vite.config.js اللي هتحتاجها.

بعد الفصل ده هتكون عارف

- تعمل مشروع بـ create-vite وتعرف كل Flag
- تشرح كل ملف وفولدر في القالب (public ولا src/assets؟)
- تشوف إيه اللي الـ Dev server بيعدّله في الملفات
- تفهم HMR: إمتى بيحتفظ بالحالة وإمتى بيعمل Reload
- تقرا Output بتاع vite build سطر سطر
- تعرف الفرق بين dev و preview
- تستخدم --host و --port و base في vite.config.js

18.1 create-vite: كل Flag

Terminal

```
user@MacBook ~ % npx create-vite@latest --help
Usage: create-vite [OPTION]... [DIRECTORY]

Create a new Vite project in JavaScript or TypeScript.
When running in TTY, the CLI will start in interactive mode.

Options:
  -t, --template NAME                 use a specific template
  -i, --immediate / --no-immediate    install dependencies and start dev
  --eslint / --no-eslint              use ESLint instead of Oxlint (only for React
templates)
  --overwrite                         remove existing files if target directory is not empty
  --interactive / --no-interactive    force interactive / non-interactive mode
  -h, --help                          display this help message

Available templates:
vanilla-ts          vanilla
vue-ts              vue
react-ts            react
react-compiler-ts   react-compiler
…
svelte-ts           svelte
solid-ts            solid
qwik-ts             qwik
```

`-t` / `--template`	القالب. `react` بـ JavaScript و `react-ts` بـ TypeScript (فصل 21). و `react-compiler` فيه React Compiler جاهز.
`-i` / `--immediate`	يعمل `npm install` ويشغّل `dev` على طول.
`--eslint`	قوالب React بقت بتيجي بـ **Oxlint** (Linter أسرع مكتوب بـ Rust) افتراضياً. لو الكورس بيستخدم ESLint (وده الأغلب)، ضيف `--eslint`. فصل 23.
`--no-interactive`	من غير أسئلة. من غيره (في Terminal عادي) هيسألك على الاسم والـ Framework والـ Variant بالأسهم.
`--overwrite`	يمسح الفولدر لو موجود وفيه ملفات. خطر، اتأكد من الاسم.

> ⚠ **الكورس بيقول create-react-app؟**
>
> لو فيديو قديم بيستخدم `npx create-react-app` : React نفسها أعلنت إنه Deprecated في فبراير 2025 ونصحت بـ Vite أو Framework زي Next.js. ولو الكورس عندك لسه عليه، الفكرة واحدة، والأوامر بتختلف: `npm run dev` بدل `npm start` ، وملفات `VITE_` بدل `REACT_APP_` (فصل 19).

18.2 كل ملف في القالب

Terminal

```
user@MacBook ~ % npm create vite@latest vanilla-app -- --template vanilla --no-interactive
user@MacBook ~ % cd vanilla-app && find . -path ./node_modules -prune -o -type f -print | sort
./.gitignore
./index.html
./package-lock.json
./package.json
./public/favicon.svg
./public/icons.svg
./src/assets/hero.png
./src/assets/javascript.svg
./src/assets/vite.svg
./src/counter.js
./src/main.js
./src/style.css
```

`index.html`	**نقطة البداية**، وفي الجذر مش جوه public (على عكس أدوات قديمة). Vite بيبدأ منه ويتبع كل `import`.
`src/main.js`	أول JS بيشتغل. بيعمل import للـ CSS والصور والـ Counter، ويحط `main.jsx` جوه `#app` في HTML. هتلاقيه React وفيه `createRoot(…).render(<App />)`.
`src/counter.js`	مثال Module: Function بتعمل زرار بيعدّ. موجود علشان تجرّب HMR.

`src/style.css`	الـ CSS، ومتعمله import من JS (`import './style.css'`). ده مش JS عادي، ده شغل Vite.
`src/assets/`	صور بتعملها **import**. Vite بيحسبها: يغيّر اسمها بـ Hash، وأي صورة أصغر من 4 KB بيحطها جوه الـ JS نفسه (javascript.svg 863 بايت فاتحطت كـ `data:image/svg+xml`، و vite.svg 8.7 KB طلعت ملف لوحدها).
`public/`	ملفات بتتنسخ **زي ما هي** بنفس الاسم لجذر `dist`: `favicon.svg` و`robots.txt`. بتشاور عليها بـ `/favicon.svg` من غير import. استخدمه للي محتاج اسم ثابت بس.
`.gitignore`	جاهز وفيه `node_modules` و`dist` و`*.local` (ملفات env السرية، فصل 19) و`.DS_Store` بتاع الماك.
مفيش `vite.config.js`؟	مش محتاجه غير لما تغيّر حاجة. قوالب React فيها واحد علشان Plugin بتاع React. تحت.

18.3 بيحصل إيه: npm run dev

```
Terminal
user@MacBook ~ % npm run dev

  VITE v8.3.5  ready in 135 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

ms135. مفيش Bundle بيتعمل. Vite بيقدّم الملفات واحد واحد لما المتصفح يطلبها، وبيعدّلها في السكة. بصينا بـ `curl` على اللي بيوصل فعلاً:

```
curl localhost:5173/
<!doctype html>
<html lang="en">
  <head>
    <script type="module" src="/@vite/client"></script>

    <meta charset="UTF-8" />
    …
```

curl localhost:5173/src/main.js

```
import "/src/style.css"
import heroImg from "/src/assets/hero.png?import"
import javascriptLogo from "/src/assets/javascript.svg?import"
import viteLogo from "/src/assets/vite.svg?import"
import { setupCounter } from "/src/counter.js"
```

curl localhost:5173/src/style.css (مختصر)

```
import { createHotContext as __vite__createHotContext } from "/@vite/client";…
const __vite__id = "/…/vanilla-app/src/style.css"
const __vite__css = ":root {\n  --text: #6b6375;\n …"
__vite__updateStyle(__vite__id, __vite__css)
import.meta.hot.accept()
```

`/@vite/client`	Script زيادة Vite حطه في الـ HTML: بيفتح اتصال WebSocket مع السيرفر علشان يسمع التغييرات (HMR)، وبيعرض الـ Errors فوق الصفحة.
`"/src/style.css"` بقت `'./style.css'`	المسارات اتحوّلت لمسارات كاملة يفهمها المتصفح. ولو فيه `import 'react'`، هيتحول لمسار جوه `/node_modules/.vite/deps/` (Vite بيجهّز الـ Packages مرة واحدة: ده الـ **Pre-bundling**).
`?import`	علامة إن الصورة متعملها import، فـ Vite بيرجّع JS بيقول "رابط الصورة هو ...".
JS بقت CSS	في الـ Dev، الـ CSS بيوصل كـ JS Module بيحط `<style>` في الصفحة، وبيقول `import.meta.hot.accept()`: "لو اتغيرت، بدّلني من غير Reload".

18.4 HMR: التجربة

فتحنا الصفحة في Chrome، ودوسنا على العداد 3 مرات، وبعدين عدّلنا ملفين وهي مفتوحة:

في المتصفح Console

```
> [vite] connecting...
> [vite] connected.
> before: Count is 3 rgb(8, 6, 13)
> [vite] hot updated: /src/style.css
> after css edit: Count is 3 rgb(29, 99, 237)
> [vite] connecting...
> [vite] connected.
> after js edit: Clicks: 0
```

الـ Terminal اللي فيه npm run dev

```
3:52:09 PM [vite] (client) hmr update /src/style.css
3:52:11 PM [vite] (client) page reload src/counter.js
```



شكل 18.1 HMR = Hot Module Replacement: بدّل الـ Module بس لو هو بيعرف يستقبل التحديث (accept)

الـ CSS بيقول `accept` فبيتبدّل في مكانه. لكن `counter.js` العادي مابيقولش، فـ Vite عمل Reload كامل. وده سبب إن React أحلى في HMR: الـ Plugin بتاعها (`@vitejs/plugin-react`) بيخلّي كل Component يعمل accept، فتعدّل Component والـ State بتاعته تفضل زي ما هي (اسمها **Fast Refresh**).

npm run build 18.5

Terminal

```
user@MacBook ~ % npm run build
vite v8.3.5 building client environment for production...
transforming...
✓ 9 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.46 kB | gzip: 0.29 kB
dist/assets/vite-BF8QNONU.svg    8.70 kB | gzip: 1.60 kB
dist/assets/hero-CLDdwZDr.png   13.05 kB
dist/assets/index-CsUDhMuy.css   4.10 kB | gzip: 1.46 kB
dist/assets/index-CAoPt-vL.js    4.05 kB | gzip: 1.77 kB

✓ built in 65ms
```

`client environment`	يبني الكود اللي هيشتغل في المتصفح. (في SSR فيه Environment تاني للسيرفر، فصل 22.)
`9 modules transformed`	عدد الملفات اللي اتعالجت: `index.html` و main و counter و CSS و 3 صور و...
`-BF8QNONU`	الـ Hash (فصل 17). `public/` مش هتلاقي فيه Hash (`dist/favicon.svg` زي ما هو).

`gzip`	الحجم بعد الضغط اللي السيرفر هيبعته فعلاً (كتاب 5 فصل 9: `Content-Encoding`). الـ PNG مش بيتضغط تاني لأنه مضغوط أصلاً.
`dist/`	ده **الموقع**. ملفات ثابتة ترفعها على أي استضافة (فصل 25). ومش بيترفع على Git.

dist/index.html

```
<head>
  …
  <script type="module" crossorigin src="/assets/index-CAoPt-vL.js"></script>
  <link rel="stylesheet" crossorigin href="/assets/index-CsUDhMuy.css">
</head>
<body>
  <div id="app"></div>
</body>
```

Vite كتب الـ HTML من جديد: `/@vite/client` اختفى، و`/src/main.js` اتبدل بالملف المبني، والـ CSS بقى `<link>` عادي.

18.6 npm run preview

Terminal

```
user@MacBook ~ % npm run preview
  ➜  Local:   http://localhost:4173/
  ➜  Network: use --host to expose
```

`preview` بيقدّم فولدر `dist` زي ما هو، على بورت 4173. ده **مش** للتطوير (مفيش HMR، ولازم build تاني بعد أي تعديل). فايدته: تتأكد إن الـ Build نفسه شغال قبل ما ترفعه، لأن حاجات بتشتغل في dev وتبوظ في build (مسار صورة في `public` غلط، أو `base`). وده اللي هنعمله في المشروع النهائي بدل الـ Deploy الحقيقي.

	`npm run dev`	`npm run preview`
البورت	5173	4173
بيقدّم	ملفات `src` وهو بيحوّلها	`dist` المبني
HMR	أيوه	لأ
إمتى	طول ما انت بتكتب	قبل الـ Deploy

جدول 18.1 dev ولا preview

18.7 Flags و vite.config.js

Terminal

```
user@MacBook ~ % npm run dev -- --host

  VITE v8.3.5  ready in 138 ms

  ➜  Local:   http://localhost:5174/
  ➜  Network: http://172.30.0.2:5174/  enp0s3
```

`--host`	افتح للشبكة المحلية. السطر `Network` فيه IP جهازك، فتفتحه من الموبايل على نفس الـ Wi-Fi وتجرّب الـ Responsive بجد. (لو الـ Firewall بتاع الماك شغال، ممكن يسألك تسمح بالاتصالات.)
`--port 3000`	بورت تاني. ولو 5173 مشغول، Vite بيقولها صريحة ويجرّب اللي بعده: `Port 5173 is in use, trying another one...` (وده حصل معانا فعلاً، ففتح على 5174). اتأكد من الرابط في الـ Terminal.
`--open`	يفتح المتصفح لوحده.
`--mode staging`	يحمّل `.env.staging` (الفصل الجاي).

vite.config.js (المشروع النهائي)

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  base: '/my-links/',
  plugins: [tailwindcss()],
})
```

`defineConfig`	مش بيعمل حاجة غير إن المحرر يقترحلك الخيارات ويكتشف الغلط.
`plugins`	إضافات: `react()` لـ JSX و Fast Refresh، و `tailwindcss()` (فصل 20).
`base`	المسار اللي الموقع هيتنشر عليه. افتراضياً `/`. لكن GitHub Pages بينشر على `username.github.io/my-links/`، فلازم `'/my-links/'`، وإلا كل الـ CSS والـ JS هيطلعوا 404 (فصل 25). وده بيأثر على dev و preview كمان: في تجربة سابقة بـ `base: '/links-card/'`، الـ preview اشتغل على `http://localhost:4173/links-card/` والـ `/` لوحدها رجّعت 302.

`server: { port, open }`	نفس الـ Flags، بس مكتوبة دايماً.

المصدر

Vite · Getting Started · Vite · Features (HMR, CSS, assets) · Vite · Static Asset Handling (public) · Vite · Building for Production · Vite · CLI · Vite · Config · Vite · base · Vite · Dependency Pre-Bundling · Vite · HMR API

ملخص الفصل

- `npm create vite@latest name -- --template react`، و`--eslint` لو الكورس بيستخدم ESLint.
- `index.html` في الجذر هو البداية. `src/assets` بـ import و Hash، و`public` زي ما هو.
- الـ Dev server مش بيعمل Bundle: بيقدّم ملف ملف ويعدّل المسارات ويحقن `/@vite/client`.
- HMR بيبدّل الـ Module لو بيعمل accept (CSS و React Components)، وإلا Reload.
- `build` يطلّع `dist` بأسماء فيها Hash، و`preview` يجرّبه على 4173.
- `--host` للموبايل، و`base` لـ GitHub Pages.

صح ولا غلط؟

1. الملفات اللي في `public/` بتاخد Hash في الاسم. صح / غلط
2. `npm run preview` بيعمل HMR. صح / غلط
3. تعديل ملف CSS في Vite بيحتفظ بحالة الصفحة. صح / غلط

أسئلة مراجعة

1. ليه Vite بيبدأ في ms135 وأدوات قديمة كانت بتاخد ثواني؟
2. صورة اللوجو بتاعك: تحطها في `public` ولا `src/assets`؟ وإمتى التانية؟
3. ليه counter.js عمل Reload و style.css لأ؟
4. ليه ممكن الموقع يشتغل في dev ويطلع صفحة بيضا بعد الـ Deploy؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — اعمل مشروع vanilla، وشغّل dev، وافتح Network: لاقي `/@vite/client` و`main.js` وبص على الـ Response.
2. متوسط — شغّل `npm run dev -- --host` وافتح الموقع من موبايلك.
3. صعب — اعمل build، وبعدين افتح `dist/index.html` بدبل كليك. ليه الصفحة فاضية؟ (افتكر فصل 17). وبعدين جرّب `preview`.

19

المرحلة 6 · Modules و Vite

الفصل التاسع عشر

متغيرات البيئة و env.: الإعدادات والأسرار

رابط الـ API في التطوير غير رابطه في الموقع الحقيقي، واسم الموقع في نسخة التجربة غير الأصلية. فبدل ما تغيّر الكود كل مرة، بتحط الإعدادات دي في ملفات env.. بس فيه فخ كبير: في الـ Front end، أي حاجة بتحطها في الكود بتوصل لأي حد يفتح DevTools. الفصل ده بيوريك الفخ بعينك: مفتاح API سري لقيناه مكتوب جوه ملف الـ JS المبني.

بعد الفصل ده هتكون عارف

- تعرف يعني إيه Environment variable
- تكتب ملفات env. و env.local. و env..[mode] وتعرف مين بيكسب
- تستخدم import.meta.env و VITE_ و MODE و DEV و PROD
- تستخدم %VITE_X% جوه index.html
- تفهم ليه أي VITE_ متغير عام، وتشوف التسريب بعينك
- تعرف مكان الأسرار الصح: السيرفر
- تستخدم process.env و node --env-file في كورس Node

19.1 يعني إيه Environment variable؟

متغير بيتحط في **البيئة** اللي البرنامج شغال فيها، مش في الكود نفسه. في الـ Terminal بتشوفها بـ `echo $HOME` (كتاب الـ Terminal). والفكرة: نفس الكود يتصرف مختلف حسب هو شغال فين، من غير ما تعدّله.



شكل 19.1 نفس الكود، قيم مختلفة. والقيم مش مكتوبة في الكود.

19.2 في Vite: env. و import.meta.env

جرّبنا على قالب Vite الـ Vanilla. تلات ملفات:

.env

```
VITE_APP_TITLE=My Links
VITE_COUNT=5
```

.env.staging

```
VITE_APP_TITLE=Staging Links
```

.env.local

```
VITE_API_KEY=sk-oops-123
SECRET_TOKEN=do-not-ship
```

src/main.js

```
document.querySelector('#app').innerHTML = `
  <h1>${import.meta.env.VITE_APP_TITLE}</h1>
  <p>mode: ${import.meta.env.MODE}, dev: ${import.meta.env.DEV}, prod: ${import.meta.env.PROD}
</p>
`
console.log(import.meta.env.VITE_API_KEY, import.meta.env.SECRET_TOKEN)
console.log(typeof import.meta.env.VITE_COUNT)
```

index.html (سطر واحد اتغير)

```
<title>%VITE_APP_TITLE%</title>
```

وعملنا `npm run build`، وفتحنا ملف الـ JS اللي طلع (آخره):

dist/assets/index-CrYCwyUk.js (آخر الملف)

```
document.querySelector(`#app`).innerHTML=`
  <h1>My Links</h1>
  <p>mode: production, dev: false, prod: true</p>
`,console.log(`sk-oops-123`,void 0),console.log(`string`);
```

`My Links`	`import.meta.env.VITE_APP_TITLE` اتبدلت بالقيمة نفسها **وقت البناء**. مفيش متغير في المتصفح أصلاً، القيمة مكتوبة نص.
`mode: production`	`MODE` = `development` في `npm run dev` و`production` في `build`. و`DEV` و`PROD` Booleans جاهزين. ومعاهم `BASE_URL` (قيمة `base`).
`sk-oops-123`	**المفتاح "السري" مكتوب في الملف**، اللي أي زائر بينزّله. ده التسريب، تحت بالتفصيل.
`void 0`	`undefined` = `SECRET_TOKEN` من غير `VITE_`، فـ Vite **رفض** يحطه. وده الحماية الوحيدة اللي Vite بيديهالك: البادئة.

`"string"`	`VITE_COUNT=5` وصل كنص `"5"` مش رقم. كل القيم نصوص. محتاج رقم؟ `Number(import.meta.env.VITE_COUNT)` . ومحتاج Boolean؟ قارن بـ `=== 'true'` .
`%VITE_APP_TITLE%`	جوه `index.html` بتتكتب كده، وطلعت `<title>My Links</title>` .

19.3 مين بيكسب؟

```
Terminal
user@MacBook ~ % npx vite build --mode staging
✓ built in 41ms
user@MacBook ~ % grep -o '<title>.*</title>' dist/index.html
<title>Staging Links</title>
user@MacBook ~ % VITE_APP_TITLE="From Shell" npx vite build
user@MacBook ~ % grep -o '<title>.*</title>' dist/index.html
<title>From Shell</title>
```

الملف	بيتحمّل إمتى	على Git؟
`.env`	دايماً	أيوه (قيم عادية، مش أسرار)
`.env.local`	دايماً، على جهازك بس	لأ (`*.local` في `.gitignore`)
`.env.[mode]`	في الـ mode ده بس (`.env.production` و `.env.staging` ...)	أيوه
`.env.[mode].local`	الـ mode ده، وعلى جهازك بس	لأ
متغير في الـ Shell	اللي كان موجود قبل ما Vite يشتغل	—

جدول 19.1 ترتيب الملفات (الأقوى تحت)، من دوكيومنتيشن Vite

الملف الخاص بالـ mode أقوى من العام، و `.local` أقوى من غيره، وأي متغير موجود في الـ Shell أقوى من كل الملفات (ودي اللي استخدمتها استضافات زي Vercel و Netlify: بتحط المتغيرات في البيئة وقت الـ Build، فصل 25).

مشكلة 1 **غيرت .env. ومفيش حاجة اتغيرت**

العَرَض: عدّلت قيمة في `.env` والصفحة لسه بالقيمة القديمة.

السبب: Vite بيقرا ملفات `.env` مرة واحدة لما يبدأ.

الحل: وقّف `npm run dev` (`Ctrl+C`) وشغّله تاني. ولو المتغير لسه `undefined` : اتأكد إنه بادئ بـ `VITE_` ، وإن الملف في جذر المشروع جنب `package.json` (مش جوه `src`).

19.4 القاعدة الذهبية: الـ Front end مفيهوش أسرار

> **أي _VITE = عام**
>
> لو المتغير بادئ بـ `VITE_`، اعتبره **منشور على الإنترنت**. أي زائر يقدر يفتح DevTools ← Sources (كتاب 5 فصل 17) أو يعمل `curl` للملف ويلاقيه، زي ما لقينا `sk-oops-123`. البادئة مش "تشفير"، هي بالعكس: قرار منك إن ده **مسموح يتنشر**. و `.env.local` بيحمي المفتاح من Git بس، مش من الـ Bundle.

ينفع (عام أصلاً)	ممنوع (سر)
رابط الـ API بتاعك (`VITE_API_URL`)	مفتاح OpenAI أو Stripe Secret Key
اسم الموقع، و Feature flags	باسورد قاعدة البيانات
Keys معمولة مخصوص للمتصفح (Publishable key بتاع Stripe، أو Key مقيّد بالدومين)	أي Token بيدّي صلاحيات كاملة

جدول 19.2 إيه ينفع يبقى _VITE؟

طب لو محتاج تكلّم API بمفتاح سري؟ اعمل **سيرفر وسيط** (Backend صغير، أو Serverless function على Vercel أو Netlify): المتصفح يكلّم سيرفرك، وسيرفرك هو اللي معاه المفتاح ويكلّم الـ API. وده بالظبط اللي شرحناه في كتاب الـ APIs (فصل 22، Express)، وهتعمله في كورس Maximilian.

19.5 وفي Node: process.env

في السيرفر الوضع مختلف تماماً: الكود مش بيروح لحد، فالأسرار مكانها هنا. والقراية بـ `process.env`، و Node 24 بيقرا ملف `.env` بنفسه من غير أي Package:

Terminal

```
user@MacBook ~ % cat s.cjs
console.log(process.env.SECRET_TOKEN, process.env.VITE_APP_TITLE)
user@MacBook ~ % node s.cjs
undefined undefined
user@MacBook ~ % node --env-file=.env.local s.cjs
do-not-ship undefined
user@MacBook ~ % node --env-file=.env --env-file=.env.local s.cjs
do-not-ship My Links
user@MacBook ~ % node --env-file=missing.env s.cjs
node: missing.env: not found
user@MacBook ~ % node --env-file-if-exists=missing.env s.cjs
missing.env not found. Continuing without it.
undefined undefined
```

`--env-file`	حمّل الملف ده. وتقدر تكرره لأكتر من ملف.
`--env-file-if-exists`	نفسه بس مش هيقف لو الملف مش موجود.

VITE_ مفيش Node في	أي اسم ينفع. الـ VITE_ قاعدة Vite بس، لأن Vite بيحط القيم في كود رايح للمتصفح.
dotenv	في كورسات Node هتلاقي npm i dotenv و require('dotenv').config(). نفس الفكرة قبل ما Node يدعمها، ولسه مستخدمة كتير.

عادات env.

- الأسرار في .env.local (أو .env في مشروع Node بشرط يكون في .gitignore)، ومتعملهاش commit أبداً. ولو حصل: غيّر المفتاح فوراً، لأن الـ Git history بيفضل فاكره (كتاب Git).
- اعمل ملف .env.example فيه الأسامي من غير قيم، وارفعه، علشان زميلك يعرف محتاج إيه.
- على الاستضافة: المتغيرات بتتحط من الـ Dashboard، مش بملف (فصل 25).

المصدر

Vite · Env Variables and Modes · Vite · HTML Constant Replacement · Node.js Docs · --env-file
npm · dotenv · Node.js Docs · process.env

ملخص الفصل

- Environment variables = إعدادات برة الكود، بتختلف حسب البيئة.
- Vite: import.meta.env.VITE_X، و MODE و DEV و PROD، و %VITE_X% في الـ HTML.
- القيم بتتكتب في الـ Bundle وقت البناء، وكلها نصوص.
- الترتيب: .env ثم .local ثم .env.[mode] ثم .env.[mode].local، والـ Shell فوق الكل. ولازم Restart بعد التعديل.
- أي VITE_ عام. الأسرار مكانها السيرفر (process.env و node --env-file).

صح ولا غلط؟

1. حطيت المفتاح في .env.local باسم VITE_SECRET، فهو آمن لأنه مش على Git. صح / غلط
2. import.meta.env.VITE_PORT قيمته رقم لو كتبت VITE_PORT=3000. صح / غلط
3. npm run build --mode staging بيحمّل .env.staging. صح / غلط

أسئلة مراجعة

1. ليه SECRET_TOKEN طلع undefined في المتصفح؟
2. عندك VITE_X في .env و .env.production. أنهي واحدة هتكسب في build؟ وفي dev؟
3. إزاي تستخدم API محتاج مفتاح سري من موقع React؟
4. إيه الفرق بين import.meta.env و process.env؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل `VITE_APP_TITLE` واعرضه في الصفحة وفي `<title>`.
2. متوسط اعمل `.env.staging` وscript اسمه `build:staging` بيبني بيه.
3. صعب كرر تجربة التسريب: حط `VITE_FAKE_KEY`، واعمل build، ولاقي القيمة بـ `grep -r` في `dist`، وبعدين لاقيها من DevTools على `preview`.

20

المرحلة 7 · CSS والـ Rendering و TypeScript

الفصل العشرين

إيكوسيستم الـ CSS: Tailwind و CSS Modules و Sass والمكتبات الجاهزة

في مشروع كبير، الـ CSS العادي بيوجع: اسم title. في ملف بيبوّظ title. في ملف تاني، وملف CSS بيكبر ومحدش بيجرؤ يمسح منه حاجة. الإيكوسيستم طلّع حلول مختلفة: Tailwind (Classes جاهزة في الـ HTML)، و CSS Modules (أسماء محلية لكل ملف)، و Sass (CSS بقدرات زيادة)، ومكتبات Components جاهزة. الفصل ده بيجرّب الأربعة في Vite ويوريك الـ CSS اللي بيطلع فعلاً.

بعد الفصل ده هتكون عارف

- تعرف المشاكل اللي الأدوات دي بتحلها
- تسطّب Tailwind v4 في Vite: @import و @theme وتقرا الـ CSS اللي اتولّد
- تفهم ليه ملف Tailwind صغير مع إن فيه آلاف الـ Classes
- تستخدم CSS Modules وتشوف الأسماء المحلية
- تكتب Sass: متغيرات و Nesting و Mixins
- تعرف مكتبات Components (shadcn/ui و MUI و...) وإمتى تستخدمها
- تختار بينهم

20.1 المشكلة

Global names

كل Class في CSS عام على الصفحة كلها. انت كتبت `.title` للكارت، وزميلك كتب `.title` للقايمة، واللي اتحمّل بعدين كسب (فصل 6: Source order).

كود ميت

Component اتمسح، والـ CSS بتاعه لسه موجود لأن محدش متأكد إن مفيش حاجة تانية بتستخدمه.

تكرار

نفس اللون `#1d63ed` مكتوب 40 مرة. (Custom properties حلّت ده جزئياً، فصل 10.)

التنقّل

علشان تعدّل زرار، بتفتح ملف HTML وملف CSS وتدوّر على الاسم.

20.2 Tailwind CSS v4

الفكرة: بدل ما تكتب CSS وتسمّي Classes، تستخدم Classes جاهزة صغيرة كل واحدة بتعمل حاجة واحدة (**Utility classes**) مباشرة في الـ HTML. التسطيب في Vite (من دوكيومنتيشن Tailwind، وده اللي عملناه في المشروع النهائي):

1. **سطّب:** `npm install tailwindcss @tailwindcss/vite` (عندنا: `added 17 packages`، النسخة 4.3.3).
2. **ضيف الـ Plugin** في `vite.config.js`: `plugins: [tailwindcss()]` (فصل 18).

3 **سطر واحد في الـ CSS:** `@import "tailwindcss";` . مفيش `tailwind.config.js` في v4 (لو فيديو قديم بيعمله، ده v3).

src/style.css (المشروع النهائي)

```
@import 'tailwindcss';

@theme {
  --color-brand: #1d63ed;
}
```

src/main.js (جزء)

```
`<h1 class="text-3xl font-bold text-brand">${title}</h1>`
`<a class="block rounded-xl bg-white p-4 shadow hover:shadow-lg
   focus-visible:outline-2 focus-visible:outline-brand" …>`
```

وده الـ CSS اللي Tailwind ولّده فعلاً في `dist` :

dist/assets/index-NALiIVJV.css (مقتطفات)

```
.text-brand{color:var(--color-brand)}
.p-4{padding:calc(var(--spacing) * 4)}
.rounded-xl{border-radius:var(--radius-xl)}
--spacing:.25rem
--radius-xl:.75rem
--color-brand:#1d63ed
```

`@theme { --color-brand }`	عرّفت لون اسمه `brand` ، فـ Tailwind عمل لوحده `text-brand` و `bg-brand` و `outline-brand` ... كل الـ Design tokens في v4 Custom properties عادية (فصل 10).
`p-4`	`4 × --spacing` = 4 × 0.25rem = 1rem. كل المسافات مضاعفات رقم واحد، فالتصميم يبقى متناسق.
`hover:shadow-lg`	Variant: الـ Class تشتغل في `:hover` بس. ولقيناها في الملف جوه `@media (hover:hover)` ، يعني على الأجهزة اللي فيها ماوس بس (مش اللمس). وفيه `md:` لـ Breakpoints و `dark:` و `focus-visible:` .
`9.53 kB`	ملف الـ CSS كله 9533 بايت، مع إن Tailwind فيه آلاف الـ Classes. السر: Tailwind بيقرا ملفاتك ويولّد **بس** الـ Classes اللي لقاها مكتوبة. دوّرنا على `bg-red-500` (مش مستخدمة): 0 مرة.
`@layer`	الـ CSS جوه `@layer theme, base, components, utilities` (فصل 6)، فالـ Utilities دايماً تكسب.

فخ: أسماء Classes بتتركب

Tailwind بيدوّر على الأسماء **كاملة** كنص في ملفاتك. فلو كتبت `` `text-${color}-500` ``، مش هيلاقي `text-red-500` في أي حتة، ومش هيولّدها، والـ Class هتبقى من غير ستايل. اكتب الاسم كامل: `color === 'red' ? 'text-red-500' : 'text-blue-500'`.

الـ HTML بقى طويل ومزعج؟

ده أكتر انتقاد لـ Tailwind. الرد: في React مش هتكرره، هتعمل Component `<LinkCard>` مرة واحدة. والميزة: الستايل جنب الـ HTML، ولو مسحت Component الـ CSS بتاعه بيختفي من الـ Build لوحده. وامتداد **Tailwind CSS IntelliSense** في VS Code بيكمّلك الأسماء ويوريك الـ CSS لما تقف عليها.

CSS Modules 20.3

أي ملف اسمه `*.module.css`، Vite بيخلّي الـ Classes بتاعته **محلية**. جرّبنا ملفين فيهم نفس الاسم `.title`:

list.module.css

```css
.title {
  color: crimson;
}
```

card.module.css

```css
.title {
  color: #1d63ed;
}
.btn {
  padding: 8px 16px;
}
```

main.js

```js
import card from './card.module.css'
import list from './list.module.css'

document.querySelector('#app').innerHTML = `
  <h2 class="${card.title}">Card title</h2>
  <h2 class="${list.title}">List title</h2>
`
```

build بعد الـ CSS

```
._title_1rm06_1{color:#1d63ed}._btn_1rm06_4{padding:8px 16px}._title_13fkz_1{color:#dc143c}
```

كل `.title` بقى اسم فريد (`_title_1rm06_1` و`_title_13fkz_1`)، و`card` في JS Object فيه `{ title: '_title_1rm06_1', btn: '_btn_1rm06_4' }`. مفيش تصادم ممكن. ولاحظ إن `crimson` اتحولت `#dc143c` (أقصر). في React بتكتبها `className={styles.title}`.

20.4 Sass

Sass = CSS بقدرات زيادة، بتتحول لـ CSS عادي. Vite بيدعمه لو سطبت `sass` : `npm i -D sass` (1.106.1)، وسمّي الملف `.scss`.

src/theme.scss

```scss
$brand: #1d63ed;
$radius: 8px;

@mixin card-box($pad) {
  padding: $pad;
  border-radius: $radius;
}

.card {
  @include card-box(16px);
  border: 1px solid rgba($brand, 0.3);

  .title {
    color: $brand;
  }

  &:hover {
    background: rgba($brand, 0.05);
  }
}
```

الناتج

```css
.card{border:1px solid #1d63ed4d;border-radius:8px;padding:16px}.card
.title{color:#1d63ed}.card:hover{background:#1d63ed0d}
```

`$brand`	متغير وقت البناء (اختفى من الناتج واتبدل بالقيمة). على عكس `--brand` اللي بيفضل في المتصفح ويتغير بـ JS أو Media query.
`@mixin` / `@include`	حتة CSS بتتكرر بـ Parameters.
`Nesting` و `&`	`.title` جوه `.card` = `.card .title`. وده بقى موجود في CSS العادي (فصل 10)، فواحدة من أكبر أسباب Sass راحت.
`rgba($brand, 0.3)`	Function بتاعة Sass طلعت `#1d63ed4d` (لون بشفافية).

ولاحظ حاجة وقت التسطيب: 11 npm طلّع

```
:all-scripts 1 package has install scripts not yet covered by allowScripts: @parcel/watcher@2.6.0
```

. ده تحذير npm 12 بدري (فصل 16): Package بيستخدمه sass عنده Install script.

Sass في 2026؟

هتلاقيه في مشاريع كتير موجودة وفي Bootstrap. لكن مع Custom properties و Nesting و `color-mix()` في الـ CSS نفسه (فصل 10)، مشروع جديد غالباً مش محتاجه. اتعلّم تقراه، ومتبدأش بيه.

20.5 مكتبات Components جاهزة

بدل ما تبني Modal و Dropdown و Date picker من الصفر (وتتعب في الـ Keyboard والـ ARIA، فصل 5)، فيه مكتبات جاهزة. أغلبها لـ React:

النوع	أمثلة	الفكرة
Headless (من غير ستايل)	Radix UI و React Aria و Headless UI	السلوك والـ Accessibility جاهزين، والشكل عليك.
Copy-paste	shadcn/ui	مش Package: أمر بينسخ كود الـ Component في مشروعك (مبني على Radix و Tailwind)، فتعدّل فيه براحتك.
Styled كاملة	MUI (Material) و Chakra UI و Ant Design و Mantine	شكل جاهز ومتناسق وسريع، بس تغيير الشكل أصعب.
Classes فوق Tailwind	daisyUI	`btn` و `card` جاهزين.
من غير Framework	Bootstrap	CSS + JS. الأقدم، وفيه Templates كتير.

جدول 20.1 أشهر الأنواع

20.6 تختار إيه؟

	CSS عادي	CSS Modules	Tailwind	Sass
تصادم الأسماء	ممكن	مستحيل	مفيش أسماء	ممكن
تسطيب	مفيش	مفيش (Vite)	Plugin + Package	Package
هتلاقيه في	كل حتة	React و Next.js	أغلب المشاريع الجديدة و shadcn/ui	مشاريع أقدم و Bootstrap

جدول 20.2 مقارنة سريعة

نصيحتي ليك: اتعلّم CSS العادي كويس الأول (الفصول 6-10)، لأن كل الأدوات دي بتطلّع CSS في الآخر، ولما حاجة تبوظ هتفتح DevTools وتلاقي CSS عادي. وبعدها جرّب Tailwind في مشروع React، لأنه هيقابلك في أغلب الـ Tutorials والشغل الحديث.

المصدر

Tailwind CSS · Installing with Vite
Tailwind CSS · Theme variables (@theme)
Tailwind CSS · Detecting classes in source files
Tailwind CSS · Hover, focus and other states
Vite · CSS Pre-processors
Vite · CSS Modules
Sass · Basics
shadcn/ui · Docs
Radix Primitives
MUI · Getting started

ملخص الفصل

- CSS عام وبيتراكم، والأدوات بتحل ده بطرق مختلفة.
- Tailwind v4: `@import "tailwindcss"` + `@theme` + Plugin، وبيولّد اللي انت استخدمته بس. واكتب أسماء الـ Classes كاملة.
- CSS Modules: `*.module.css` = أسماء فريدة لكل ملف.
- Sass: متغيرات و Mixins و Nesting، وبيطلع CSS عادي. أغلبه بقى في الـ CSS نفسه.
- مكتبات الـ Components: Headless و shadcn/ui و Styled كاملة.

صح ولا غلط؟

1. Tailwind v4 محتاج `tailwind.config.js` . صح / غلط
2. في CSS Modules، `.title` في ملفين مختلفين بيطلعوا نفس الاسم. صح / غلط
3. المتغير `$brand` بتاع Sass ممكن تغيّره بـ JS في المتصفح. صح / غلط

أسئلة مراجعة

1. ليه ملف Tailwind النهائي صغير؟ وإيه الفخ اللي ممكن يخلّي Class متتولّدش؟
2. إيه الفرق بين متغير Sass و Custom property؟
3. إيه الفرق بين shadcn/ui و MUI؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل سطّب Tailwind في مشروع Vite بالتلات خطوات، واعمل كارت بـ `p-4 rounded-xl shadow` .
2. متوسط ضيف لون `brand` في `@theme` واستخدمه في `bg-brand` و `hover:bg-brand/80` . دوّر عليهم في الـ CSS بعد build.
3. صعب اعمل نفس الكارت 3 مرات: CSS Module و Tailwind و Sass، وقارن حجم الـ CSS في `dist` في كل حالة.

21

المرحلة 7 · CSS والـ Rendering و TypeScript

الفصل الحادي والعشرين

TypeScript: نظرة من فوق

هتلاقي في create-vite نسخة ts- من كل قالب، وأغلب إعلانات الشغل بتطلب TypeScript. الفصل ده مش كورس TypeScript، هو بيجاوب على: هي إيه، وبتحل إيه، وبتشتغل فين في الـ Pipeline (مين بيشيك على الأنواع ومين بيشيلها)، وكل ملف tsconfig بيعمل إيه، و TypeScript 7 الجديدة المكتوبة بـ Go. وامتى تبدأ تتعلمها في رحلتك.

بعد الفصل ده هتكون عارف

- تعرف TypeScript بتحل مشكلة إيه بمثال حقيقي
- تقرا Error بتاع tsc وتفهمه
- تفهم إن Vite بيشيل الأنواع ومابيشيكش عليها، وليه build فيه tsc &&
- تقرا tsconfig.json بتاع Vite سطر سطر
- تعرف ملفات قالب react-ts
- تشغّل ts. في Node 24 مباشرة وتعرف حدودها
- تعرف TypeScript 7 وامتى تتعلم TS

21.1 المشكلة

في JavaScript ده كود "سليم":

```javascript
function formatPrice(p) {
  return `${p.name}: ${p.price.toFixed(2)} EGP`
}
formatPrice({ name: 'Book', price: '120' })   // TypeError at runtime
```

هيضرب `p.price.toFixed is not a function`، بس **لما المستخدم يوصل للسطر ده**. ممكن بعد أسبوع في الإنتاج. لأن `'120'` نص مش رقم (والقيمة جاية غالباً من `input.value` أو JSON، كتاب الـ APIs). JavaScript + **أنواع** = TypeScript، وأداة بتقرا الكود وتلاقي الغلط ده **قبل ما يشتغل**.

21.2 التجربة

قالب `vanilla-ts`، وملف جديد:

```typescript
// src/price.ts
type Product = {
  name: string
  price: number
  onSale?: boolean
}
```

```ts
export function formatPrice(p: Product): string {
  return `${p.name}: ${p.price.toFixed(2)} EGP`
}

const input = document.querySelector<HTMLInputElement>('#qty')
console.log(input?.value)

formatPrice({ name: 'Book', price: '120' })
```

`type Product = {…}`	شكل الـ Object: `name` نص، و`price` رقم، و`onSale?` اختياري (الـ `?`).
`(p: Product): string`	الـ Parameter لازم يبقى Product، والـ Function بترجّع نص.
`<HTMLInputElement>`	"العنصر ده input"، فالمحرر يعرف إن فيه `.value`. و`input?.value` لأن `querySelector` ممكن يرجّع `null`، و TS بيجبرك تفكر في الحالة دي.

Terminal

```
user@MacBook ~ % npm run build

> ts-app@0.0.0 build
> tsc && vite build

src/price.ts(14,29): error TS2322: Type 'string' is not assignable to type 'number'.
user@MacBook ~ % echo $?
2
```

تشريح الأمر

```
src/price.ts(14,29): error TS2322: Type 'string' is not assignable to type 'number'.
```

1. `src/price.ts` — الملف.
2. `(14,29)` — السطر 14، الحرف 29: عند `price: '120'` بالظبط.
3. `TS2322` — رقم الـ Error. دوّر بيه لو الرسالة مش واضحة.
4. `Type 'string'…` — "اديتني نص في مكان المفروض فيه رقم".

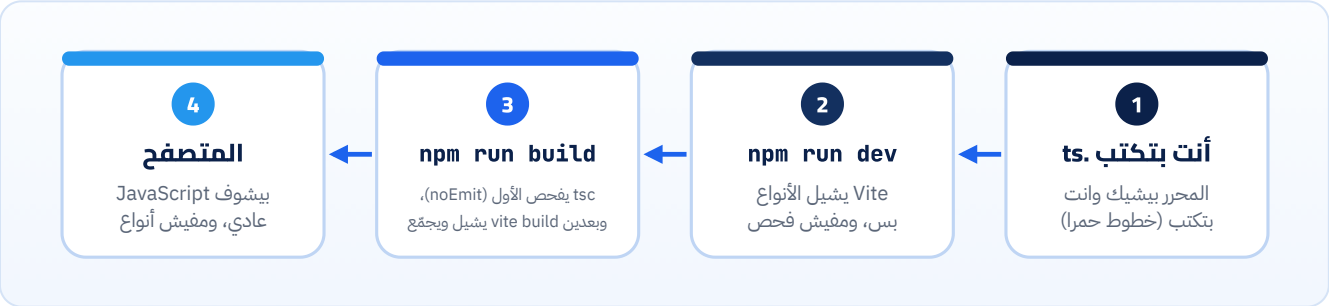
و`tsc` رجع exit code 2، فالـ `&&` (كتاب الـ Terminal) وقّفت `vite build`، والـ Deploy مش هيكمل بكود فيه غلط. وفي VS Code هتشوف خط أحمر تحت `'120'` وانت بتكتب، قبل أي build.

21.3 مين بيشيك ومين بيشيل؟

جرّبنا نفس الكود بـ `vite build` لوحده من غير `tsc`:

```
Terminal
user@MacBook ~ % npx vite build
✓ built in 79ms
```

نجح، والغلط جوه الـ Bundle. Vite (عن طريق Oxc في Vite 8) بيعمل حاجة واحدة مع TS: **يشيل الأنواع** ويطلّع JS. ودوّرنا على كلمة `Product` في الـ JS الناتج: 0 مرة. مابيشيكش على حاجة، علشان يفضل سريع.



شكل 21.1 TypeScript مش بتشتغل في المتصفح أبداً.

21.4 سطر سطر tsconfig.json

Dockerfile

1 أنهي مميزات JS والمتصفح موجودة. `DOM` هي اللي بتعرّف `document` و `HTMLInputElement`.	`"target": "es2023",` `"lib": ["ES2023", "DOM"],`
2 أنواع Vite: علشان `import.meta.env` و `import logo from './x.svg'` مايطلعوش Errors.	`"types": ["vite/client"],`
3 "فيه Bundler هو اللي هيحل الـ imports"، فمسموح `import './price.ts'` بالامتداد.	`"moduleResolution": "bundler",` `"allowImportingTsExtensions": true,`
4 لو بتعمل import لنوع بس، اكتب `import type`، علشان الأداة تعرف تشيله.	`"verbatimModuleSyntax": true,`
5 `tsc` يفحص بس ومايكتبش ملفات. Vite هو اللي بيطلّع الـ JS.	`"noEmit": true,`
6 متغير أو Parameter مش مستخدم = Error (شبه ESLint، فصل 23).	`"noUnusedLocals": true,` `"noUnusedParameters": true,`
7 ممنوع أي TS بيعمل كود حقيقي (زي `enum`). المسموح بس الحاجات اللي "تتمسح" فيفضل JS سليم. تحت هتعرف ليه.	`"erasableSyntaxOnly": true,`

قالب react-ts

```
Terminal
user@MacBook ~ % npm create vite@latest rts -- --template react-ts --no-interactive
user@MacBook ~ % ls -a rts
.gitignore  .oxlintrc.json  README.md  index.html  package.json  public  src
tsconfig.app.json  tsconfig.json  tsconfig.node.json  vite.config.ts
```

`tsconfig.json`	فاضي تقريباً: `"references"` لملفين.
`tsconfig.app.json`	لكود `src` (المتصفح، فيه DOM).
`tsconfig.node.json`	لـ `vite.config.ts` (بيشتغل في Node، مفيهوش DOM).
`"build": "tsc -b && vite build"`	`-b` = Build mode: يفحص المشروعين اللي في الـ references.
`@types/react` و `@types/react-dom` و `@types/node`	React نفسها مكتوبة JS، فالأنواع بتاعتها في Packages منفصلة `@types/…` (مشروع اسمه DefinitelyTyped). مكتبات تانية (زي Vite) بتيجي بأنواعها جواها.
`.tsx`	JSX + TypeScript. `App.tsx` و `main.tsx`.

21.5 TypeScript في Node 24 مباشرة

```
Terminal
user@MacBook ~ % cat t.ts
type User = { name: string }
const u: User = { name: "Sara" }
console.log("hi", u.name)
user@MacBook ~ % node t.ts
hi Sara
user@MacBook ~ % node e.ts
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript enum is not supported in strip-
only mode
```

Node 24 بيشغّل `.ts` على طول بإنه **يمسح** الأنواع (Type stripping)، من غير فحص. وعلشان كده `enum` فشل: مش "نوع" ينفع يتمسح، ده كود. وده سبب `erasableSyntaxOnly` في tsconfig: يخلّي كودك يشتغل في أي أداة بتمسح بس.

21.6 TypeScript 7

في 8 يوليو 2026 نزلت **TypeScript 7.0**: نفس اللغة، بس الـ Compiler اتكتب من جديد بلغة **Go** بدل JavaScript، وفريق TypeScript بيقول إنه أسرع حوالي 10 مرات في المشاريع الكبيرة. جرّبناها على نفس المشروع:

```
Terminal
user@MacBook ~ % npx -p typescript@7.0.2 tsc --version
Version 7.0.2
user@MacBook ~ % npx -p typescript@7.0.2 tsc --noEmit -p .
src/price.ts(14,29): error TS2322: Type 'string' is not assignable to type 'number'.
```

نفس الـ Error بالحرف. بس لاحظ إن قوالب create-vite وقت الكتابة لسه كاتبة `"typescript": "~6.0.3"` (الـ `~` مش هتجيب 7 أبداً، فصل 15)، وإن أدوات بتعتمد على API بتاع TypeScript (زي typescript-eslint) محتاجة وقت تلحق. الكلام ده للمعرفة: هتشتغل بالنسخة اللي القالب جايبها.

أتعلمها إمتى؟

مش دلوقتي. خلّص JavaScript مع Jonas، وبعدين Node، وبعدين React. لما تبقى مرتاح في React، ابدأ مشروع بـ `react-ts`. هتلاقي إن 80% من الشغل: `type` للـ Props، وأنواع الـ `useState`، و`?.`. وكل اللي اتعلمته في JS بيفضل زي ما هو، لأن TS = JS + أنواع.

المصدر

Vite · TypeScript (transpile only) · TypeScript · TSConfig Reference · TypeScript · TS for the New Programmer
React · Using TypeScript · TypeScript Blog (TypeScript 7) · Node.js Docs · Modules: TypeScript (type stripping)

ملخص الفصل

- TypeScript = JS + أنواع، بتلاقي الغلط قبل التشغيل.
- المحرر و`tsc` بيفحصوا، و Vite و Node بيشيلوا الأنواع بس من غير فحص.
- علشان كده `"build": "tsc && vite build"`.
- `noEmit` و`moduleResolution: bundler` و`erasableSyntaxOnly` أهم سطور الـ tsconfig.
- TypeScript 7 (Go) أسرع، والقوالب لسه على 6.0. اتعلم TS بعد React.

صح ولا غلط؟

1. المتصفح بيفهم TypeScript مباشرة. صح / غلط
2. `npm run dev` في مشروع Vite بيوقف لو فيه Type error. صح / غلط
3. Node 24 بيشغّل ملف `.ts` فيه `enum` عادي. صح / غلط

أسئلة مراجعة

1. ليه `vite build` لوحده نجح والكود فيه غلط؟
2. اقرا الـ Error: `src/App.tsx(8,5): error TS2322`. هتروح فين؟
3. ليه React محتاجة `@types/react` و Vite لأ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل مشروع `vanilla-ts` وغيّر حاجة في `counter.ts` لنوع غلط. شوف الخط الأحمر في VS Code.
2. متوسط كرر تجربة `price.ts` وشغّل `npm run build` و `npx vite build` وقارن.
3. صعب شغّل `npx tsc --noEmit` بنسخة القالب وبـ `npx -p typescript@7 tsc --noEmit`، وقارن الوقت بـ `time`.

22

المرحلة 7 · CSS والـ TypeScript و Rendering

الفصل الثاني والعشرين

CSR و SSR و SSG و ISR و Hydration: الصفحة بتتبني فين؟

افتح View Source لأي موقع React معمول بـ Vite: هتلاقي div فاضي. وافتح View Source لموقع Next.js: هتلاقي المحتوى كله. الفرق ده هو أهم قرار معماري في الـ Front end: الـ HTML بيتبني فين وإمتى؟ في المتصفح، ولا على السيرفر مع كل زيارة، ولا مرة واحدة وقت البناء؟ الفصل ده بيشرح الأربع طرق والـ Hydration، وبيبني مشروع Next.js 16 حقيقي ويقرا جدول الـ Routes بتاعه.

بعد الفصل ده هتكون عارف

- تفرق بين CSR و SSR و SSG و ISR بمثال
- تعرف مميزات وعيوب كل واحدة (السرعة و SEO و التكلفة)
- تفهم Hydration وليه ممكن يطلع Error
- تعرف Next.js بيضيف إيه فوق React
- تقرا جدول next build: ○ و ◐ والـ Revalidate
- تفهم Partial Prerendering و Suspense بالتجربة
- تعرف إمتى React + Vite كفاية وإمتى Next.js

22.1 CSR: كل حاجة في المتصفح

ده اللي عملناه طول الكتاب بـ Vite. الـ `dist/index.html` بتاع مشروع React:

react-app/dist/index.html (الـ body)

```
<body>
  <div id="root"></div>
</body>
```

ده كل اللي السيرفر بيبعته. والمحتوى؟ JS بينزل (222 kB في مشروع React، فصل 2)، وبيشتغل، وبيبني الصفحة جوه `#root`. ده **Client-Side Rendering**.



شكل 22.1 CSR: المستخدم مستني لحد آخر خطوة

كويس لـ

Dashboards، وأي حاجة ورا Login، وتطبيقات تفاعلية جداً. الاستضافة رخيصة (ملفات ثابتة، فصل 25)، ومفيش سيرفر.

وحش لـ

أول ظهور على موبايل ونت بطيء (LCP، فصل 24). وأي حاجة بتعتمد على محركات البحث أو معاينة الروابط في واتساب، لأن اللي بيقرا الـ HTML من غير JS بيلاقي صفحة فاضية.

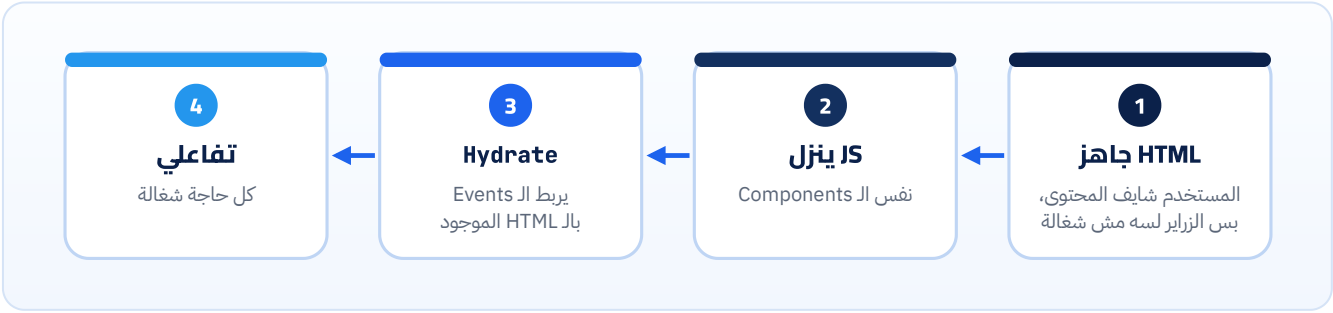
22.2 SSR و SSG و ISR

	بيتبني فين	إمتى	مثال
Client-Side Rendering CSR	المتصفح	كل زيارة	Dashboard
Server-Side Rendering SSR	السيرفر	كل Request	صفحة فيها بيانات المستخدم أو أسعار لحظية
Static Site Generation SSG	جهاز البناء	مرة واحدة وقت `build`	Landing page، و Docs، و Blog
Incremental Static Regeneration ISR	السيرفر	زي SSG، بس بيتجدد كل فترة أو عند الطلب	صفحة منتج، أخبار

جدول 22.1 الأربع طرق: الـ HTML بيتبني فين وإمتى؟

وفي الـ SSR والـ SSG، المستخدم بيشوف المحتوى من أول HTML، والـ JS بييجي بعدين يخلّي الصفحة تفاعلية. الخطوة دي اسمها **Hydration**.

22.3 Hydration: "ترطيب" الصفحة



شكل 22.2 الصفحة «ناشفة» (HTML بس) وبعدين بتترطّب بالـ JS

مشكلة 1 **text content does not match / Hydration failed**

العَرَض: Error في Console بيقول إن الـ HTML اللي جه من السيرفر مش زي اللي React عمله في المتصفح.

السبب: الـ Component طلّع نتيجة مختلفة في المكانين: `new Date()` أو `Math.random()` (السيرفر في وقت أو منطقة زمنية تانية)، أو `window` و`localStorage` (مش موجودين على السيرفر)، أو Extension في المتصفح عدّل الـ HTML.

الحل: أي حاجة خاصة بالمتصفح تتعمل في `useEffect` (بيشتغل بعد الـ Hydration في المتصفح بس). وجرّب في نافذة Incognito من غير Extensions.

22.4 Next.js: React + سيرفر + Router

React لوحدها مكتبة UI (فصل 2). **Next.js** Framework فوقها (Meta-framework) بيضيف: Routing بالفولدرات، و SSR و SSG و ISR، و Server Components، و API routes، و تحسين الصور والخطوط. ومن Vercel، فالـ Deploy عليها أسهل حاجة (فصل 25). عملنا مشروع حقيقي:

Terminal

```
user@MacBook ~ % npx create-next-app@16.4.2 nx --js --no-tailwind --no-eslint --app \
    --no-src-dir --use-npm --disable-git --no-agent-feedback \
    --no-react-compiler --import-alias "@/*" --yes
…
Installing dependencies:
- next
- react
- react-dom

added 22 packages, and audited 23 packages in 6s
…
Success! Created nx at /tmp/nx
user@MacBook ~ % cat nx/package.json
```

```
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "16.4.2",
    "react": "19.3.0",
    "react-dom": "19.3.0"
  }
```

لاحظ: `start` هنا بيشغّل **سيرفر Node** (مش ملفات ثابتة زي Vite)، والنسخ مكتوبة من غير `^`. وفي `next.config.mjs` لقينا `cacheComponents: true` و`partialPrefetching: true`: من Next 16.4 المشاريع الجديدة بتيجي بـ **Cache Components** شغالة. وكمان القالب حط ملف `AGENTS.md` بيقول لأدوات الـ AI تقرا الدوكيومنتيشن اللي جوه `node_modules/next/dist/docs/` لأن الـ APIs اتغيرت. وده اللي عملناه احنا كمان قبل ما نكتب الكود تحت.

تلات صفحات، تلات طرق

الـ Routing بالفولدرات: `app/now/page.js` = الصفحة `/now`، و`[id]` = جزء متغير.

app/posts/[id]/page.js

```
export async function generateStaticParams() {
  return [{ id: '1' }, { id: '2' }]
}

export default async function Post({ params }) {
  const { id } = await params
  return <h1>Post {id} (built at build time)</h1>
}
```

app/now/page.js

```
import { Suspense } from 'react'
import { connection } from 'next/server'

async function ServerTime() {
  await connection() // wait for a real request: render per request
  return <p>Rendered on the server at {new Date().toISOString()}</p>
}

export default function NowPage() {
  return (
    <main>
      <h1>Now</h1>
      <Suspense fallback={<p>Loading time...</p>}>
        <ServerTime />
```

```
      </Suspense>
    </main>
  )
}
```

app/news/page.js

```
import { cacheLife } from 'next/cache'

export default async function News() {
  'use cache'
  cacheLife('minutes') // rebuild this page in the background every few minutes
  return <h1>News generated at {new Date().toISOString()}</h1>
}
```

Terminal

```
user@MacBook ~ % npm run build
▲ Next.js 16.4.2 (Turbopack)
- Cache Components enabled
- Partial Prefetching enabled

  Creating an optimized production build ...
✓ Compiled successfully in 2.0s
…
✓ Generating static pages using 7 workers (9/9) in 427ms

Route (app)        Revalidate  Expire
┌ ○ /
├ ○ /_not-found
├ ○ /news                  1m      1h
├ ◐ /now
└   /posts/[id]
  ├ ◐ /posts/[id]
  ├ ○ /posts/1
  └ ○ /posts/2

○  (Static)             prerendered as static content
◐  (Partial Prerender)  prerendered as static HTML with dynamic server-streamed content
```

○ /posts/1 و /posts/2	**SSG**: `generateStaticParams` قال "ابني 1 و 2"، فاتبنوا HTML جاهز وقت البناء.
◐ /posts/[id]	أي id تاني (زي 3) هيتبني أول ما حد يطلبه ويتخزن. طلبنا `/posts/3` بعد البناء ورجع `Post 3 (built at build time)` (النص ثابت في الكود، بس الصفحة نفسها اتعملت عند الطلب).

○ /news · 1m · 1h	**ISR**: Static، بس `cacheLife('minutes')` قال: بعد دقيقة (Revalidate) أي زيارة تخلّي السيرفر يبني نسخة جديدة في الخلفية، وبعد ساعة (Expire) النسخة القديمة متتعرضش خالص.
◐ /now	**Partial Prerender**: الجزء الثابت (`<h1>Now</h1>` والـ Fallback) اتبنى وقت البناء، والجزء اللي جوه `Suspense` بيتعمل على السيرفر مع كل Request ويتبعت Stream.

التجربة على السيرفر

شغّلنا `npx next start -p 3055` (`✓ Ready in 112ms`) وطلبنا الصفحات بـ `curl`:

```
Terminal
user@MacBook ~ % curl -s localhost:3055/news | grep -o '<h1>.*</h1>'
<h1>News generated at <!-- -->2026-10-23T12:58:37.754Z</h1>
user@MacBook ~ % curl -sI localhost:3055/posts/1 | grep -i -E "cache-control|x-nextjs"
x-nextjs-cache: HIT
x-nextjs-prerender: 1
x-nextjs-stale-time: 300
Cache-Control: s-maxage=31536000
```

`12:58:37.754Z`	الـ `Z` = UTC، يعني 3:58 العصر بتوقيت القاهرة. ده وقت **البناء** مش وقت الطلب: الصفحة متخزنة.
`x-nextjs-cache: HIT`	اتقدّمت من الـ Cache من غير ما تتبني تاني.
`s-maxage=31536000`	"أي CDN يخزّنها سنة" (كتاب 5 فصل 15). صفحة SSG = ملف ثابت عملياً.
`<!-- -->`	علامات React بتحطها بين النصوص علشان الـ Hydration يعرف كل نص يبدأ وبيخلص فين.

وطلبنا `/now` مرتين: الـ HTML فيه `<h1>Now</h1>` و`Loading time...` على طول، وتحت في نفس الـ Response وصل الجزء الديناميكي: `Rendered on the server at 2026-10-23T12:58:48.556Z` في المرة الأولى و`…12:58:48.578Z` في التانية. وقت مختلف لكل Request = SSR. وده كله Response واحد بيوصل على مراحل (Streaming).

> **Server Components**
>
> في الـ App Router كل Component افتراضياً **Server Component**: بيشتغل على السيرفر بس، والكود بتاعه مش بيوصل للمتصفح أصلاً (يقدر يقرا قاعدة بيانات مباشرة). ولو محتاج `useState` أو `onClick`، بتكتب `'use client'` أول الملف. هتتعلمه بالتفصيل لما توصل Next.js بعد React.

22.5 React + Vite ولا Next.js؟

	(SPA) React + Vite	Next.js
الـ Rendering	CSR	SSR و SSG و ISR و PPR و CSR كمان
الاستضافة	ملفات ثابتة: GitHub Pages و Netlify و Vercel	محتاج سيرفر Node (أو Vercel وأمثالها)، أو `output: 'export'` لموقع ثابت بدون مميزات السيرفر
SEO ومعاينة الروابط	ضعيف	ممتاز
التعقيد	أقل: React بس	أكتر: سيرفر و Cache و Client/Server components
مناسب لـ	تتعلم React، و Dashboards، وتطبيقات ورا Login	مواقع عامة، و E-commerce، و Blogs، ومشاريع Full-stack

جدول 22.2 الاختيار

وفيه Meta-frameworks تانية بنفس الفكرة: **React Router** v7 (Framework mode)، وده اللي Remix اندمج فيه)، و**Nuxt** لـ Vue، و**SvelteKit** لـ Svelte، و**Astro** للمواقع اللي أغلبها محتوى. وترتيبك: React مع Vite الأول (كورس Jonas)، وبعدين Next.js.

المصدر `Next.js · Installation` `Next.js · Caching و Prerendering` `Next.js · generateStaticParams` `Next.js · cacheLife` `Next.js · connection` `Next.js · cacheComponents` `Next.js Blog · Next.js 16` `web.dev · Rendering on the Web` `React · hydrateRoot`

ملخص الفصل

- CSR: HTML فاضي و JS بيبني كل حاجة. سهل ورخيص، بس أبطأ في الظهور وضعيف في SEO.
- SSR: مع كل Request. SSG: مرة وقت البناء. ISR: Static وبيتجدد.
- Hydration = ربط JS بـ HTML جاهز. والـ Mismatch بييجي من حاجات بتختلف بين السيرفر والمتصفح.
- Next.js 16: Routing بالفولدرات و Cache Components و `'use cache'` و Suspense و Streaming.
- جدول `next build`: ○ Static و ◐ Partial Prerender و Revalidate/Expire.

صح ولا غلط؟

1. في CSR، محرك البحث اللي مابيشغّلش JS بيشوف المحتوى كامل. صح / غلط
2. صفحة SSG بتتبني من جديد مع كل زيارة. صح / غلط
3. `new Date()` جوه Component بيتعمله SSR ممكن يعمل Hydration mismatch. صح / غلط

؟ أسئلة مراجعة

1. موقع مطعم فيه المنيو (بتتغير كل شهر) وصفحة طلبات المستخدم. تختار إيه لكل صفحة؟
2. ليه الوقت في `/news` كان قبل وقت الطلب؟
3. إيه الفرق بين ○ و ◐ في جدول Next.js؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل افتح View Source (`⌘⌥U`) في Chrome على الماك) لموقع React بـ Vite ولموقع Next.js. قارن الـ body.
2. متوسط اعمل مشروع Next.js، وزوّد صفحة بـ `generateStaticParams`، وشوف علامتها في `npm run build`.
3. صعب كرر صفحة `/now` و `/news`، وشغّل `next start`، واطلب كل واحدة كذا مرة بـ `curl` وقارن الأوقات. استنى أكتر من دقيقة واطلب `/news` مرتين: إمتى الوقت اتغير؟

23

المرحلة 8 · الجودة والأداء والنشر

الفصل التالت والعشرين

ESLint و Prettier: مصحح ومنسّق أوتوماتيك

كتبت consol.log بدل console.log، ومحدش لاحظ لحد ما الصفحة ضربت في الإنتاج. وزميلك بيحب Double quotes وانت Single، وكل Pull request نصه تغييرات مسافات. أداتين بيحلوا المشكلتين: ESLint بيدوّر على الأخطاء والكود المشكوك فيه، و Prettier بيوحّد الشكل. الفصل ده بيعمل الاتنين في مشروع Vite ويشرح كل سطر في ملفات الإعدادات وكل Output.

بعد الفصل ده هتكون عارف

- تفرق بين Linter و Formatter
- تسطّب ESLint 10 بالـ Flat config وتشرح كل سطر
- تقرا Output بتاع ESLint: السطر والعمود والـ Rule
- تستخدم --fix وتعرف إمتى بيصلّح
- تسطّب Prettier بـ --save-exact وتعرف .prettierrc و .prettierignore.
- تخلّي ESLint و Prettier مايتخانقوش (eslint-config-prettier)
- تربطهم بـ VS Code وبالـ scripts وبالـ CI

23.1 اتنين مختلفين

ESLint = مصحح

بيقرا الكود ويدوّر على **مشاكل**: متغير مش معرّف، متغير مش مستخدم، كود مش هيتنفذ أبداً، Hooks متكتبة غلط في React. بيقولك "ده غالباً Bug".

Prettier = منسّق

مابيهتمش الكود صح ولا غلط. بياخده ويعيد كتابته **بشكل واحد ثابت**: مسافات، و Quotes، وطول السطر. مفيش نقاش في الفريق تاني.

23.2 ESLint في المشروع النهائي

الطريقة الرسمية لمشروع جديد: `npm init @eslint/config@latest` (يعني `npx @eslint/create-config`، فصل 13)، وهيسألك أسئلة ويكتب الملف. ومحتاج Node `^20.19` أو `^22.13` أو `>=24`. احنا سطبناها بإيدنا علشان نفهم كل Package:

Terminal

```
user@MacBook ~ % npm install -D eslint @eslint/js globals eslint-config-prettier
added 80 packages, and audited 113 packages in 1s
```

`eslint`	الأداة نفسها (10.13.0).
`@eslint/js`	الـ Rules الأساسية بتاعة JavaScript، ومنها `recommended`.

`globals`	لستة المتغيرات الموجودة جاهزة: `document` و `window` في المتصفح، أو `process` في Node. من غيرها ESLint هيقول `'document' is not defined`.
`eslint-config-prettier`	بيقفل أي Rule ليها علاقة بالشكل، علشان Prettier هو المسؤول عنه.

Dockerfile

1 الملف ES Module عادي (`eslint.config.js`). ده الـ **Flat config**، الوحيد في ESLint 10. لو شفت `.eslintrc.json` في فيديو قديم، دي الطريقة القديمة.	`import js from '@eslint/js'`
	`import globals from 'globals'`
	`import { defineConfig, globalIgnores }`
	`  from 'eslint/config'`
	`import prettier from`
	`  'eslint-config-prettier/flat'`

2 متفحصش `dist` (كود مبني ومضغوط).	`export default defineConfig([` `  globalIgnores(['dist']),`
3 الإعدادات دي لكل ملفات `.js`.	`  {` `  files: ['**/*.js'],`
4 ابدأ بالـ Rules الموصى بيها، وبعدين اقفل بتوع الشكل. `prettier` لازم **آخر واحد**.	`  extends: [js.configs.recommended,` `  prettier],`
5 الكود ده بيشتغل في المتصفح.	`  languageOptions:` `  { globals: globals.browser },`

6 غيّرنا Rule واحدة: المتغير اللي مش مستخدم `warn` (أصفر) بدل `error`. القيم: `"off"` و `"warn"` و `"error"`.	`  rules: {`
	`  'no-unused-vars': 'warn',`
	`  },`
	`  },`
	`])`

التجربة

زوّدنا آخر `src/main.js` تلات سطور فيهم غلطتين، وفعّلنا مؤقتاً Rule زيادة `'prefer-const': 'error'`:

JavaScript

```
let count = 0
const unused = 42
consol.log("count is", count)
```

```
Terminal
user@MacBook ~ % npx eslint .

/tmp/lint/src/main.js
  24:5  error    'count' is never reassigned. Use 'const' instead  prefer-const
  25:7  warning  'unused' is assigned a value but never used       no-unused-vars
  26:1  error    'consol' is not defined                           no-undef

✖ 3 problems (2 errors, 1 warning)
  1 error and 0 warnings potentially fixable with the `--fix` option.
```

تشريح الأمر

```
1    2
26:1 error 'consol' is not defined    no-undef
                                  4
```

1. `26:1` السطر 26، العمود 1.
2. `error` الخطورة. أي error يخلّي الأمر يرجع exit 1 (فيوقف الـ CI).
3. `'consol'…` الرسالة: الاسم ده مش معرّف في أي حتة. غلطة إملائية كانت هتبقى `ReferenceError` في المتصفح.
4. `no-undef` اسم الـ Rule. دوّر عليه في `eslint.org/docs/latest/rules/no-undef` تعرف ليه وأمثلة.

```
Terminal
user@MacBook ~ % npx eslint . --fix

/tmp/lint/src/main.js
  25:7  warning  'unused' is assigned a value but never used  no-unused-vars
  26:1  error    'consol' is not defined                      no-undef

✖ 2 problems (1 error, 1 warning)
user@MacBook ~ % tail -3 src/main.js
const count = 0
const unused = 42
consol.log("count is", count)
```

`--fix` غيّر `let` لـ `const` لوحده (الحل واضح ومضمون)، بس **ماقدرش** يصلّح `consol` ولا يمسح `unused`: دول محتاجين قرار منك. وفي React هتلاقي Rules مهمة جداً من `eslint-plugin-react-hooks` (القالب بيجيبها مع `--eslint`، فصل 18).

وقوالب React الجديدة فيها Oxlint

create-vite بقى بيدّي React **Oxlint** افتراضياً: Linter مكتوب بـ Rust، أسرع بكتير، وبيدعم أغلب Rules بتاعة ESLint، وإعداداته في `.oxlintrc.json` والأمر `"lint": "oxlint"`. الفكرة نفسها والـ Output شبهه. لو الكورس بيستخدم ESLint، اعمل المشروع بـ `--eslint`.

Prettier 23.3

```
Terminal
user@MacBook ~ % npm install -D --save-exact prettier
added 1 package, and audited 114 packages in 397ms
```

ليه `--save-exact` ؟ دوكيومنتيشن Prettier بيطلبها صريح: أي نسخة جديدة (حتى Patch) ممكن تغيّر شكل التنسيق شوية، فلو كل واحد في الفريق عنده نسخة، كل واحد هيعيد تنسيق الملفات بشكل مختلف. فـ `"prettier": "3.9.10"` بالظبط (فصل 15).

.prettierrc

```
{
  "singleQuote": true,
  "semi": false
}
```

Single quotes ومن غير `;` (زي ستايل كتير من الكورسات).
الدوكيومنتيشن بيقترح تبدأ بـ `{}` وتسيب الافتراضي.

.prettierignore

```
dist
package-lock.json
```

`node_modules` بيتجاهل Prettier) ملفات ماتنسقهاش.
لوحده، وبيقرا `.gitignore` كمان.)

قبل

```
const user={name:"Sara",age:25,skills:["html","css","js"]}
function greet(u){return "Hi "+u.name+"!";}
if(user.age>18){console.log(greet(user))}
```

بعد npx prettier

```
const user = { name: 'Sara', age: 25, skills: ['html', 'css', 'js'] }
function greet(u) {
  return 'Hi ' + u.name + '!'
}
if (user.age > 18) {
  console.log(greet(user))
}
```

check ولا write؟

```
Terminal
user@MacBook ~ % npm run format:check
Checking formatting...
[warn] src/main.js
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
user@MacBook ~ % echo $?
1
```

```
user@MacBook ~ % npm run format
.github/workflows/deploy.yml 14ms (unchanged)
…
src/main.js 8ms
src/style.css 12ms (unchanged)
user@MacBook ~ % diff before.js src/main.js
< consol.log("count is", count)
---
> consol.log('count is', count)
```

`"format": "prettier . --write"`	نسّق كل الملفات واكتبها. السطر اللي مفيهوش `(unchanged)` هو اللي اتغير.
`"format:check": "prettier . --check"`	افحص بس ومتغيرش. بيرجع exit 1 لو فيه ملف مش متنسق. ده اللي يتحط في الـ CI.
`"lint": "eslint ."`	النقطة = الفولدر الحالي كله.

> ⚠ **npx prettier من غير تسطيب**
>
> دوكيومنتيشن Prettier بيحذّر: لو كتبت `npx prettier` في مشروع مش متسطب فيه، npx هينزّل **أحدث نسخة** (فصل 13)، اللي ممكن تنسّق غير نسخة الفريق. سطّبه في المشروع الأول.

23.4 في VS Code وفي الـ CI

1. **الامتدادات:** "ESLint" (من Microsoft) بيحط خط تحت المشاكل وانت بتكتب، و"Code formatter - Prettier" (من Prettier).
2. **Format on save:** من Settings (`⌘,`) دوّر على `Format On Save` وفعّله، واختار Prettier في `Default Formatter`. كل `⌘S` = ملف متنسق.
3. **الـ CI:** في GitHub Actions بتاع المشروع النهائي فيه خطوة `npm run lint` قبل `npm run build` (فصل 25). أي error يوقف الـ Deploy.

المصدر

ESLint · Rules Reference · ESLint · Configuration Files (flat config) · ESLint · Getting Started · Prettier · Integrating with Linters · Prettier · Options · Prettier · Install · ESLint · CLI (--fix) · Oxc · Oxlint · GitHub · eslint-config-prettier

ملخص الفصل

- ESLint بيدوّر على الأخطاء، و Prettier بيوحّد الشكل.
- Flat config في `eslint.config.js` : `recommended` + `globals` + `prettier` في الآخر.
- Output = سطر:عمود + الخطورة + الرسالة + اسم الـ Rule. و `--fix` بيصلّح المضمون بس.
- Prettier بـ `--save-exact`، و `--write` للتنسيق و `--check` للـ CI.
- Format on save في VS Code، و lint في الـ CI.

صح ولا غلط؟

1. Prettier هيقولك إن `consol` مش معرّف. صح / غلط
2. `eslint --fix` بيصلّح كل المشاكل. صح / غلط
3. `eslint-config-prettier` لازم يبقى آخر حاجة في `extends` . صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `warn` و `error` عملياً في الـ CI؟
2. ليه ESLint محتاج Package اسمه `globals` ؟
3. ليه Prettier يتسطب بنسخة بالظبط؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل فعّل Format on save، واكتب كود "وحش" واعمل `⌘S` .
2. متوسط زوّد ESLint و Prettier لمشروع من كورس Jonas، وصلّح كل اللي يطلع.
3. صعب زوّد Rule `eqeqeq` بقيمة `error` ، ودوّر على كل `==` في مشروعك. ليه دي Rule مفيدة؟

24

المرحلة 8 · الجودة والأداء والنشر

الفصل الرابع والعشرين

Lighthouse و Core Web Vitals وفحص الـ Accessibility

الموقع شغال عندك على الماك وواي فاي البيت سريع. بس المستخدم الحقيقي على موبايل متوسط ونت G4 ضعيف، وفيه اللي بيستخدم Screen reader. إزاي تعرف تجربتهم؟ الفصل ده بيشرح Core Web Vitals (المقاييس اللي جوجل بتقيس بيها تجربة المستخدم)، وبيشغّل Lighthouse على مشروعنا وعلى صفحة سيئة عن قصد، ويقرا كل رقم، ويوريك حدود الفحص الأوتوماتيكي.

بعد الفصل ده هتكون عارف

- تعرف LCP و INP و CLS والحدود بتاعتهم
- تفرق بين Lab data و Field data
- تشغّل Lighthouse من DevTools ومن الـ CLI
- تقرا التقرير: الـ Scores والـ Metrics والـ Insights
- تصلّح أشهر المشاكل
- تفحص الـ Accessibility أوتوماتيك وتعرف اللي الأداة مش هتلاقيه

24.1 Core Web Vitals: تلات أسئلة

المقياس	بيسأل	جيد	سيئ
Largest Contentful Paint LCP	أكبر حاجة في الشاشة (صورة أو عنوان) ظهرت إمتى؟	≤ 2.5 ثانية	> 4 ثواني
Interaction to Next Paint INP	لما بدوس، الصفحة بترد بسرعة؟	≤ 200ms	> 500ms
Cumulative Layout Shift CLS	الحاجات بتتنطط وهي بتحمّل؟	≤ 0.1	> 0.25

جدول 24.1 الحدود الرسمية (web.dev). «جيد» لازم يتحقق لـ 75% من الزيارات (p75)

INP حل محل FID (First Input Delay) في مارس 2024، فلو مقال قديم بيتكلم عن FID، ده المقياس القديم. وكل واحد ليه أسباب مشهورة:

بطيء `LCP`	صورة كبيرة مش متضغطة، أو CSR (الصفحة فاضية لحد ما JS يخلص، فصل 22)، أو سيرفر بطيء، أو CSS و JS بيعطّلوا الرسم (Render-blocking).
عالي `INP`	JS كتير شغال على الـ Main thread، فالمتصفح مشغول ومش قادر يرد على الضغطة.
عالي `CLS`	صورة من غير `width` و `height` (فالمتصفح مايعرفش يحجزلها مكان)، أو إعلان أو Banner بيظهر فوق المحتوى، أو خط بيتغير.

24.2 Lab ولا Field؟

Lab data

اختبار على جهاز واحد في ظروف ثابتة: Lighthouse. ممتاز علشان تلاقي المشاكل وتجرّب الحلول. بس مش بيقيس INP (مفيش مستخدم بيدوس)، فبيقيس **TBT** (Total Blocking Time) كبديل.

Field data

أرقام من زوار حقيقيين على Chrome (اسمها CrUX). بتظهر في **PageSpeed Insights** لو موقعك عليه زيارات كفاية. ده اللي جوجل بتعتمد عليه فعلاً.

24.3 نشغّل Lighthouse

1. **من Chrome:** DevTools (`⌥⌘I`) ← تاب **Lighthouse** ← اختار Mobile والـ Categories ← Analyze page load. افتحه في نافذة Incognito علشان الـ Extensions ماتأثرش.
2. **من الـ Terminal:** `npx lighthouse <url>` (احنا استخدمنا 13.5.0 مع Output بصيغة JSON).
3. **على الـ Build مش الـ Dev:** شغّل `npm run build` و `npm run preview` واختبر الـ preview. الـ Dev server مش مضغوط وفيه HMR، فالأرقام هتطلع أوحش من الحقيقة.

شغّلناه على `preview` بتاع مشروع links-card (نسخة أولى من المشروع النهائي: Tailwind + Vite)، والإعدادات الافتراضية: `mobile` و `simulate` (بيحاكي موبايل متوسط ونت G4 بطيء):

Category	Score	Metric	القيمة
Performance	100	FCP	1.0 s
Accessibility	100	LCP	1.0 s
Best Practices	100	TBT	0 ms
SEO	90	CLS	0
Agentic Browsing	100	Speed Index	1.0 s

جدول 24.2 النتيجة الحقيقية (Lighthouse 13.5.0)

`FCP 1.0 s`	First Contentful Paint: أول حاجة اترسمت. والـ LCP نفس الوقت لأن الصفحة صغيرة.
`TBT 0 ms`	مفيش JS تقيل بيعطّل (1.29 kB JS بس، فصل 26). قارن بـ 222 kB بتوع React (فصل 2).
`SEO 90`	نقص بسبب `meta-description` : "Document does not have a meta description". الحل سطر: `<meta name="description" content="…">`.

`Agentic Browsing`	Category جديدة في Lighthouse بتقيس: هل أدوات الـ AI اللي بتتصفح المواقع تقدر تفهم صفحتك وتتعامل معاها. (وصفها التفصيلي في دوكيومنتيشن Lighthouse؛ احنا بننقل الـ Score بس.)
`Insights`	طلع اتنين: `render-blocking-insight` (ملف الـ CSS بيوقف الرسم لحد ما ينزل) و `network-dependency-tree-insight` (سلسلة Requests ورا بعض). نصايح مش أخطاء، وأثرها هنا صغير جداً.

> **متعبدش الـ 100**
>
> الـ Score بيتغير من تشغيلة للتانية حسب الجهاز. وبيقيس صفحة واحدة في ظروف معينة. استخدمه علشان تلاقي مشاكل وتقارن قبل وبعد، مش كهدف في حد ذاته.

24.4 الـ Accessibility: صفحة سيئة عن قصد

فاكر فصل 5؟ كتبنا صفحة فيها كل الأخطاء المشهورة:

badpage/index.html

```html
<html>
  <head><meta charset="UTF-8" /><title>bad</title></head>
  <body>
    <div style="font-size:24px" onclick="go()">Welcome</div>
    <img src="…" width="200" height="100">
    <p style="color:#bbb;background:#fff">Light grey text that is hard to read</p>
    <input type="email" placeholder="Email">
    <button><svg width="16" height="16"></svg></button>
    <a href="/more">click here</a>
  </body>
</html>
```

Lighthouse (Accessibility بس) إداها **53**، والمشاكل اللي لقاها:

Audit	الرسالة	الحل
`html-has-lang`	element does not have a `<html>` attribute `[lang]`	`<html lang="ar" dir="rtl">`
`image-alt`	Image elements do not have `[alt]` attributes	`alt` بيوصف الصورة، أو `alt=""` لو زينة
`color-contrast`	do not have a sufficient contrast ratio…	#bbb على أبيض نسبته أقل من 4.5:1. غمّق اللون
`button-name`	Buttons do not have an accessible name	`aria-label="Search"` أو نص مخفي
`landmark-one-main`	Document does not have a main landmark.	`<main>` حوالين المحتوى

جدول 24.3 اللي Lighthouse لقاه

واللي ماشافوش

الأداة الأوتوماتيك بتلاقي جزء بس

Lighthouse **مالقاش** اتنين من أسوأ المشاكل في الصفحة:

- `<div onclick>` : مش بيوصله Tab ولا بيشتغل بـ Enter (فصل 5). الأداة مش عارفة إن الـ div ده المفروض "زرار".
- `<input placeholder="Email">` من غير `<label>` : الـ Placeholder اتحسب كاسم فعدّى، مع إنه بيختفي وانت بتكتب (فصل 4).
- و"click here" كنص رابط مش بيقول الرابط رايح فين.

علشان كده لازم كمان تجرّب بنفسك: امشي في الصفحة بـ **Tab** بس، وشغّل **VoiceOver** على الماك (`⌘F5`)، وبص على الـ Accessibility tree في DevTools (فصل 3).

وفيه أدوات زيادة مبنية على نفس المحرك (axe-core) اللي Lighthouse بيستخدمه: امتداد **axe DevTools**، وفي الـ Tests مكتبات زي `@axe-core/playwright` . وفي React، Plugin `eslint-plugin-jsx-a11y` بيمسك جزء من المشاكل وانت بتكتب.

24.5 أشهر الإصلاحات

1. **الصور:** صيغة حديثة (WebP أو AVIF)، ومقاس مناسب، و`width` / `height` دايماً (CLS)، و`loading="lazy"` للي تحت الشاشة بس (مش صورة الـ LCP).
2. **JS أقل:** Code splitting بـ `import()` (فصل 17)، وراجع الـ Packages الكبيرة قبل ما تسطّبها.
3. **الخطوط:** `font-display: swap` ، وخطوط أقل.
4. **Cache:** الأسماء اللي فيها Hash بتتخزن (فصل 18)، والاستضافات الحديثة بتعمل ده لوحدها.
5. **Semantic HTML:** أغلب مشاكل الـ a11y بتتحل بإنك تستخدم العنصر الصح (الفصول 3-5).

المصدر web.dev · Web Vitals · web.dev · LCP · web.dev · INP · web.dev · CLS

PageSpeed Insights · Chrome for Developers · Lighthouse · web.dev · INP becomes a Core Web Vital (مارس 2024)

Chrome UX Report (CrUX) · GitHub · axe-core · Apple · VoiceOver User Guide

ملخص الفصل

- LCP ≤ 2.5s، و INP ≤ 200ms، و CLS ≤ 0.1، على 75% من الزيارات.
- Lab (Lighthouse) للتشخيص، و Field (CrUX و PageSpeed Insights) للحقيقة.
- اختبر الـ preview مش الـ dev، وفي Incognito.
- Lighthouse لقى 5 مشاكل a11y، بس مالقاش الـ div اللي بيتداس ولا الـ input من غير label. جرّب بالكيبورد و VoiceOver.

صح ولا غلط؟

1. Lighthouse بيقيس INP مباشرة. صح / غلط
2. صورة من غير `width` و `height` ممكن تزوّد CLS. صح / غلط
3. 100 Accessibility في Lighthouse يعني الموقع Accessible تماماً. صح / غلط

أسئلة مراجعة

1. ليه نختبر الـ preview مش الـ dev server؟
2. موقع CSR بـ React صفحته الرئيسية LCP 5 ثواني. اقترح 3 حلول.
3. اذكر مشكلتين a11y الأدوات الأوتوماتيك مش هتلاقيهم.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل Lighthouse من DevTools على مشروع من الكورس، واكتب الـ 4 Scores.
2. متوسط اعمل الصفحة السيئة، وصلّحها لحد ما Accessibility تبقى 100، وبعدين صلّح الحاجتين اللي Lighthouse مالقاهمش.
3. صعب جرّب موقعك على PageSpeed Insights بعد ما تنشره. فيه Field data؟ لو لأ، ليه؟

25

المرحلة 8 · الجودة والأداء والنشر

الفصل الخامس والعشرين

النشر: GitHub Pages و Netlify و Vercel

الموقع شغال على localhost، ومحدش في الدنيا شايفه غيرك. النشر (Deploy) يعني ترفع فولدر dist على سيرفر ليه رابط عام. الفصل ده بيشرح إيه اللي بيحصل في أي Deploy، وتلات استضافات مجانية بخطواتهم من الدوكيومنتيشن الرسمي، و Workflow بتاع GitHub Actions سطر سطر، ومتغيرات البيئة على الاستضافة، وأشهر مشكلتين: صفحة بيضا و 404 لما تعمل Refresh.

بعد الفصل ده هتكون عارف

- تفهم الـ Deploy: build ثم ارفع dist
- تعرف الفرق بين Static hosting وسيرفر
- تنشر على GitHub Pages بـ Actions وتشرح الـ Workflow
- تعرف خطوات Netlify و Vercel بالـ Git وبالـ CLI
- تفهم Preview deployments و Production
- تحط Environment variables على الاستضافة
- تحل الصفحة البيضا (base) و 404 بتاع الـ SPA

قبل ما نبدأ: ماعملناش Deploy حقيقي

النشر على الإنترنت حاجة **عامة** بحسابك، فالكتاب ماعملهاش نيابة عنك. كل اللي في الفصل ده: الخطوات من الدوكيومنتيشن الرسمي (Vite و GitHub و Netlify و Vercel)، والملفات اللي جهّزناها وجرّبناها محلياً (build و preview و lint). الـ Output بتاع النشر نفسه هتشوفه انت لما تعمله.

25.1 الـ Deploy بيعمل إيه؟



شكل 25.1 كل استضافة حديثة بتعمل نفس الخطوات أوتوماتيك مع كل push

موقع React + Vite (SPA) = ملفات ثابتة: HTML و CSS و JS وصور. مفيش سيرفر بيشغّل كود. فأي **Static hosting** ينفع، وأغلبهم مجاني للمشاريع الشخصية. أما Next.js بـ SSR (فصل 22) فمحتاج استضافة بتشغّل Node، زي Vercel.

	GitHub Pages	Netlify	Vercel
الرابط	`username.github.io/repo/`	`name.netlify.app`	`name.vercel.app`
الإعداد	Workflow في الـ repo + `base`	Import من GitHub	Import من GitHub (بيكتشف Vite لوحده)
Preview لكل PR	لأ	أيوه	أيوه
SSR / Serverless functions	لأ	أيوه	أيوه، والأفضل لـ Next.js
CLI	`gh` (كتاب Git فصل 17)	`netlify-cli`	`vercel`

جدول 25.1 التلاتة

25.2 GitHub Pages بـ GitHub Actions

كتاب Git (فصل 17) شرح Pages و Actions. هنا التطبيق على Vite، من صفحة "Deploying a Static Site" في دوكيومنتيشن Vite:

1. **اضبط `base` في `vite.config.js`:** `base: '/my-links/'` (اسم الـ repo بين شرطتين). لو الـ repo اسمه `username.github.io` أو عندك دومين خاص، سيبه `/`.
2. **اعمل ملف الـ Workflow:** `.github/workflows/deploy.yml` (تحت).
3. **فعّل Pages:** في الـ repo ← Settings ← Pages ← Source: **GitHub Actions**.
4. **push على main:** تاب Actions هيوريك الـ Workflow شغال، ولما يخلص الرابط هيظهر.

.github/workflows/deploy.yml (المشروع النهائي)

```yaml
name: Deploy to GitHub Pages

on:
  push:
    branches: ['main']
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: 'pages'
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
```

```yaml
    url: ${{ steps.deployment.outputs.page_url }}
  runs-on: ubuntu-latest
  steps:
    - name: Checkout
      uses: actions/checkout@v7
    - name: Set up Node
      uses: actions/setup-node@v7
      with:
        node-version: lts/*
        cache: 'npm'
    - name: Install dependencies
      run: npm ci
    - name: Lint
      run: npm run lint
    - name: Build
      run: npm run build
    - name: Setup Pages
      uses: actions/configure-pages@v6
    - name: Upload artifact
      uses: actions/upload-pages-artifact@v5
      with:
        path: './dist'
    - name: Deploy to GitHub Pages
      id: deployment
      uses: actions/deploy-pages@v5
```

`on: push / workflow_dispatch`	يشتغل مع كل push على main، أو بزرار "Run workflow" من تاب Actions.
`permissions`	أقل صلاحيات لازمة: يقرا الكود، ويكتب على Pages، و `id-token` علشان يثبت هويته لـ Pages من غير Token تحطه انت.
`concurrency`	لو عملت push مرتين ورا بعض، الأقدم يتلغي.
`runs-on: ubuntu-latest`	جهاز Linux نضيف عند GitHub. علشان كده `node_modules` بتاع الماك مايترفعش أصلاً (فصل 13).
`node-version: lts/*` و `cache: 'npm'`	آخر LTS، وخزّن كاش npm بين التشغيلات (بيعتمد على `package-lock.json`).
`npm ci` ثم `lint` ثم `build`	نفس اللي بتعمله على جهازك بالظبط. لو ESLint لقى error، الـ Workflow يقف ومفيش نشر (فصل 23). الـ Lint خطوة احنا زودناها على مثال Vite.
`upload-pages-artifact: ./dist`	ارفع `dist` بس، و `deploy-pages` ينشره.
`@v7` / `@v5`	نسخ الـ Actions. مثال Vite في الدوكيومنتيشن بيثبّتهم بـ **SHA** كامل (أأمن: الـ Tag ممكن يتنقل، الـ SHA لأ). احنا استخدمنا الـ Tags للقراية. لما تعمله، انسخ اللي في الدوكيومنتيشن وقتها.

25.3 Netlify

بالـ Git (الأسهل)

1. ارفع الكود على GitHub.
2. في Netlify: Import project واختار الـ repo.
3. اختار الـ Branch، و Build command `npm run build`، و Publish directory `dist`، والـ Environment variables.
4. Deploy.

بالـ CLI

```
Terminal
user@MacBook ~ % npm install -g
netlify-cli
user@MacBook ~ % netlify init
user@MacBook ~ % netlify deploy
preview URL to check first #
user@MacBook ~ % netlify deploy --prod
```

وبعد الربط: أي push على Branch تاني أو Pull request = **Preview deployment** برابط مختلف تجرّب عليه، وأي push على main = **Production**. ومع Netlify `base` يفضل `/` لأن الموقع على جذر الدومين.

25.4 Vercel

بالـ Git

1. ارفع على GitHub.
2. Import في Vercel.
3. Vercel بيكتشف إن المشروع Vite ويظبط الإعدادات لوحده.
4. نفس فكرة Preview و Production.

بالـ CLI

```
Terminal
user@MacBook ~ % npm i -g vercel
user@MacBook ~ % vercel
:or without global install #
user@MacBook ~ % npx vercel
```

و Vercel هي الشركة اللي بتعمل Next.js، فـ SSR و ISR و Cache Components (فصل 22) شغالين عليها من غير إعداد.

25.5 Environment variables على الاستضافة

`.env.local` مش بيترفع على Git (فصل 19)، فالاستضافة مش هتشوفه. الحل: في Netlify أو Vercel، Project settings ← Environment variables ← زوّد `VITE_API_URL` بقيمته. الاستضافة بتحطها في البيئة وقت `npm run build`، و Vite بيقراها (وهي أقوى من ملفات `.env`، فصل 19). وفي GitHub Actions: Settings ← Secrets and variables ← Actions، وبعدين `env:` في خطوة الـ Build. وافتكر: أي `VITE_` هيبقى في الـ JS العام مهما كان مكانه.

25.6 أشهر مشكلتين

مشكلة 1 صفحة بيضا على GitHub Pages و Console فيه 404

العَرَض: الرابط بيفتح فاضي، و Network فيه `404` على `/assets/index-….js` .

السبب: الموقع على `/my-links/` بس الـ HTML بيطلب `/assets/…` من جذر الدومين، لأن `base` مش متظبط.

الحل: `base: '/my-links/'` بنفس اسم الـ repo بالظبط (Case-sensitive). واتأكد قبل الرفع: في المشروع النهائي كل مسارات `dist/index.html` بدأت بـ `/my-links/` .

مشكلة 2 Refresh على /about بيدّي 404

العَرَض: الموقع شغال من الصفحة الرئيسية، والتنقل جوه شغال، بس Refresh على `/about` أو فتح الرابط مباشرة = 404.

السبب: في SPA مع React Router، مفيش ملف اسمه `about.html` . الـ Routing بيحصل بـ JS جوه `index.html` . فالسيرفر بيدوّر على الملف ومش لاقيه.

الحل: قول للاستضافة "أي مسار مش موجود، رجّع `index.html` ". في Netlify: ملف `public/_redirects` فيه `/* /index.html 200` . في Vercel: `rewrites` في `vercel.json` (Vercel بتشرحها في صفحة Vite). وفي GitHub Pages مفيش طريقة رسمية، وده سبب إن ناس كتير بتستخدم Netlify أو Vercel لتطبيقات React.

المصدر

GitHub Docs · Pages publishing source · Vite · Deploying a Static Site · Netlify Docs · Redirects · Netlify Docs · Vite · GitHub Docs · Security hardening (pin actions to SHA) · Vercel Docs · Environments (Preview / Production) · Vercel Docs · CLI · Vercel Docs · Vite · Netlify Docs · CLI

ملخص الفصل

- Deploy = `npm ci` ثم `build` ثم ارفع `dist` . أي SPA = ملفات ثابتة.
- GitHub Pages: `base: '/repo/'` + Workflow :Source + GitHub Actions.
- Netlify و Vercel: Import من GitHub، و Preview لكل PR، و Production من main. وفيه CLI.
- متغيرات البيئة تتحط في الاستضافة، و `VITE_` لسه عامة.
- صفحة بيضا = `base` . و 404 مع Refresh = محتاج Rewrite لـ `index.html` .

صح ولا غلط؟

1. موقع React + Vite محتاج سيرفر Node علشان يشتغل. صح / غلط
2. على Netlify، الـ `base` لازم يبقى اسم الـ repo. صح / غلط
3. الـ Workflow بيستخدم `npm ci` علشان يلتزم بالـ Lockfile. صح / غلط

؟ أسئلة مراجعة

1. رتّب خطوات أي Deploy.
2. إيه فايدة Preview deployment؟
3. الموقع بيفتح على GitHub Pages بس صفحة بيضا. هتعمل إيه خطوة بخطوة؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل build و preview للمشروع بتاعك، واتأكد إن كل حاجة شغالة.
2. متوسط انشر مشروع من كورس Jonas على Netlify بال Git (لما تكون جاهز إنه يبقى عام).
3. صعب انشر المشروع النهائي (الفصل الجاي) على GitHub Pages بال Workflow، وبعدين على Vercel، وقارن.

26

المرحلة 8 · الجودة والأداء والنشر

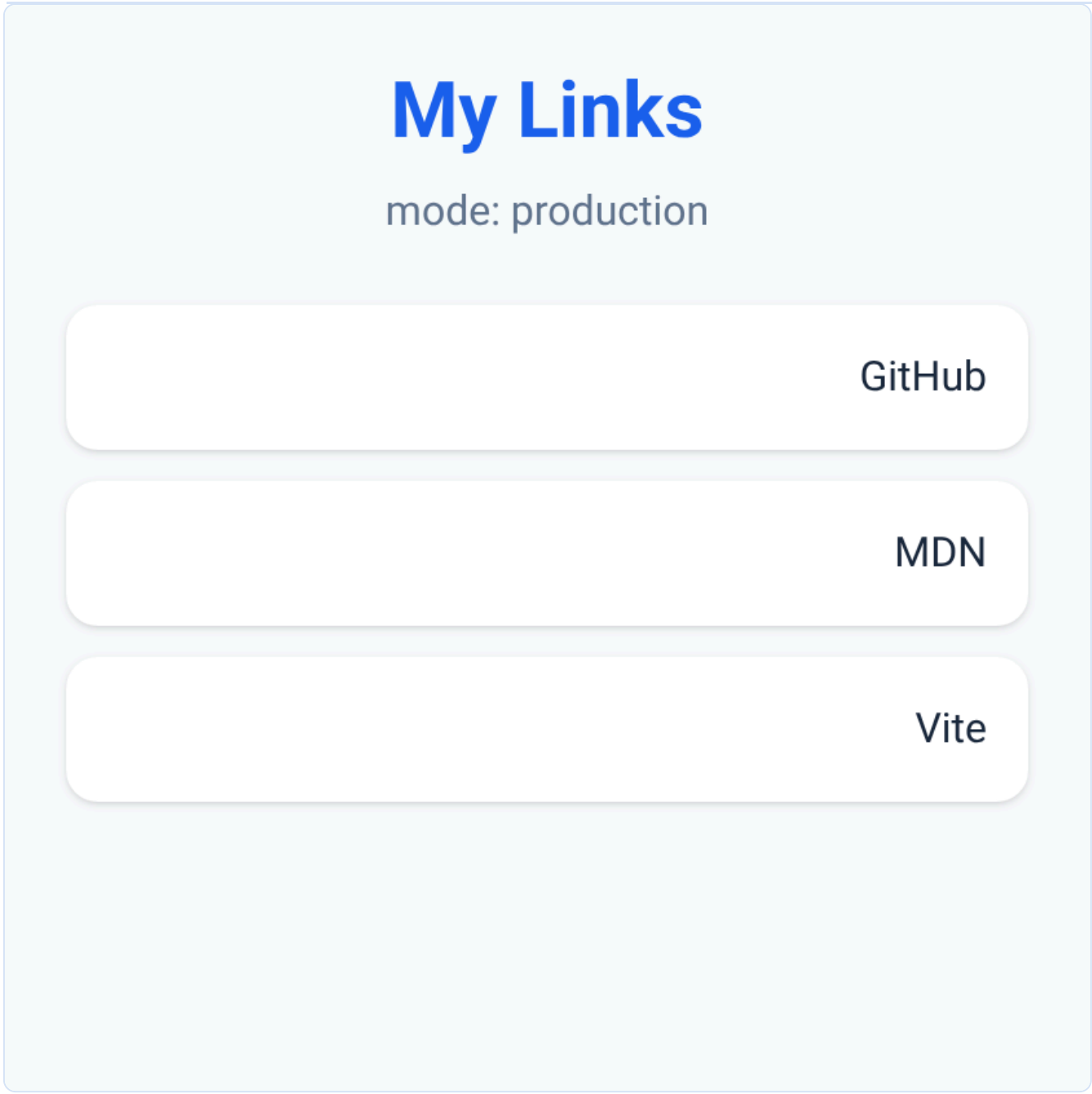
الفصل السادس والعشرين

المشروع النهائي: صفحة روابط من الصفر للـ Deploy

كل اللي في الكتاب في مشروع واحد صغير: صفحة «My Links» (زي Linktree) بـ Vite و Tailwind، و ESLint و Prettier، ومتغير بيئة، و Git، و Workflow جاهز لـ GitHub Pages. كل أمر بالـ Output الحقيقي بتاعه، وكل ملف اتعمل أو اتعدّل مشروح ومربوط بالفصل بتاعه. نفّذه على الماك خطوة خطوة.

بعد الفصل ده هتكون عارف

- تعمل مشروع Vite من الصفر بـ create-vite
- تزوّد Tailwind v4 وتعرّف لون في @theme
- تكتب الصفحة بـ HTML سليم و RTL ومتغير بيئة
- تزوّد ESLint و Prettier والـ scripts
- تعمل build و preview وتتأكد من base
- تعمل commit بالملفات الصح بس
- تجهّز Workflow للنشر



شكل 26.1 النتيجة: لقطة حقيقية من `vite preview` على `http://localhost:4190/my-links/`. الكروت على اليمين لأن الصفحة `dir="rtl"`.

26.1 · 1 اعمل المشروع

```
Terminal
user@MacBook ~ % npm create vite@latest my-links -- --template vanilla --no-interactive
create-vite v9.3.1
|
◇  Scaffolding project in /…/my-links...
|
└  Done. Now run:
```

```
  cd my-links
  npm install
  npm run dev
user@MacBook ~ % cd my-links
user@MacBook ~ % npm install

added 15 packages, and audited 16 packages in 2s

8 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

فصل 13 و 18: `create-vite` عمل القالب، و `npm install` نزّل Vite والـ 14 Package بتوعه. امسح ملفات المثال اللي مش هنحتاجها (`src/counter.js` والصور في `src/assets` و`public/icons.svg`)، وسيب `public/favicon.svg`.

> **«looking for funding»**
>
> سطر معلومات مش تحذير: بعض الـ Packages عندها صفحة تبرعات. `npm fund` يعرضهم.

26.2 Tailwind · 2

Terminal

```
user@MacBook ~ % npm install tailwindcss @tailwindcss/vite

added 17 packages, and audited 33 packages in 2s
```

vite.config.js (ملف جديد)

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  base: '/my-links/',
  plugins: [tailwindcss()],
})
```

src/style.css (بدّل المحتوى كله)

```
@import 'tailwindcss';

@theme {
  --color-brand: #1d63ed;
}
```

`plugins: [tailwindcss()]`	Tailwind بيشتغل جوه Vite (فصل 20).
`base: '/my-links/'`	علشان GitHub Pages (فصل 25). لو هتنشر على Netlify أو Vercel، امسحه.
`--color-brand`	لون الكتاب، وبيولّد `text-brand` و `outline-brand`.

26.3 · 3 الصفحة

index.html

```
<!doctype html>
<html lang="ar" dir="rtl">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>%VITE_APP_TITLE%</title>
  </head>
  <body class="min-h-svh bg-slate-50 text-slate-800">
    <main id="app" class="mx-auto max-w-md p-6"></main>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

`lang="ar" dir="rtl"`	لغة الصفحة واتجاهها (فصل 3 و 5، و Lighthouse بيدوّر على `lang`).
`%VITE_APP_TITLE%`	العنوان من `.env` (فصل 19).
`<main>`	بدل `div` : Landmark (فصل 3).
`min-h-svh` / `mx-auto max-w-md p-6`	ارتفاع الشاشة على الأقل، وعمود في النص أقصاه 28rem، و Padding.

.env

```
VITE_APP_TITLE=My Links
```

.env.local (مش بيترفع)

```
SECRET_TOKEN=do-not-ship
```

src/links.js

```
export const links = [
  { label: 'GitHub', url:
'https://github.com/' },
  { label: 'MDN', url:
'https://developer.mozilla.org/' },
  { label: 'Vite', url:
'https://vite.dev/' },
]
```

src/main.js

```
import './style.css'
import { links } from './links.js'

const app = document.querySelector('#app')
const title = import.meta.env.VITE_APP_TITLE

app.innerHTML = `
  <header class="mb-6 text-center">
    <h1 class="text-3xl font-bold text-brand">${title}</h1>
    <p class="mt-2 text-slate-500">mode: ${import.meta.env.MODE}</p>
  </header>
  <nav aria-label="links">
    <ul class="flex flex-col gap-3">
      ${links
        .map(
          (l) =>
            `<li><a class="block rounded-xl bg-white p-4 shadow hover:shadow-lg focus-visible:outline-2 focus-visible:outline-brand" href="${l.url}">${l.label}</a></li>`,
        )
        .join('')}
    </ul>
  </nav>
`
```

`import './style.css'`	Vite بيحوّله لـ `<link>` في الـ build (فصل 18).
`import { links }`	ES Module (فصل 17). البيانات منفصلة عن العرض.
`<nav aria-label>` و `<ul>`	Screen reader هيقول "links, navigation, list 3 items" (فصل 3 و 5).
`focus-visible:outline-brand`	إطار واضح للي بيستخدم Tab بس (فصل 5).
`.map().join('')`	JS عادي من كورس Jonas. ونفس الشكل ده في React هيبقى `{links.map(l => <li key=…>)}`. ولاحظ إن `innerHTML` مع بيانات من المستخدم خطر (XSS)، هنا البيانات بتاعتنا فمقبول.

شغّل `npm run dev` وافتح `http://localhost:5173/my-links/` (فيه `/my-links/` بسبب `base`).

26.4 · 4 ESLint و Prettier

```
Terminal
user@MacBook ~ % npm install -D eslint @eslint/js globals eslint-config-prettier

added 80 packages, and audited 113 packages in 1s
user@MacBook ~ % npm install -D --save-exact prettier

added 1 package, and audited 114 packages in 397ms
user@MacBook ~ % npm pkg set scripts.lint="eslint ."
user@MacBook ~ % npm pkg set scripts.format="prettier . --write"
user@MacBook ~ % npm pkg set scripts.format:check="prettier . --check"
```

وبعدين الملفات اللي شرحناها في فصل 23: `eslint.config.js` و`.prettierrc` (`{"singleQuote": true, "semi": false}`) و`.prettierignore` (`dist` و`package-lock.json`). وشغّلنا `npm run format` مرة (غيّر `@import "tailwindcss"` لـ Single quotes)، وبعدها:

```
Terminal
user@MacBook ~ % npm run lint

> my-links@0.0.0 lint
> eslint .

user@MacBook ~ % npm run format:check
Checking formatting...
All matched files use Prettier code style!
```

ESLint من غير Output = مفيش مشاكل.

26.5 · 5 package.json النهائي

```
package.json
{
  "name": "my-links",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "eslint .",
    "format": "prettier . --write",
    "format:check": "prettier . --check"
  },
  "devDependencies": {
```

```
    "@eslint/js": "^10.0.1",
    "eslint": "^10.13.0",
    "eslint-config-prettier": "^10.1.8",
    "globals": "^17.13.0",
    "prettier": "3.9.10",
    "vite": "^8.3.4"
  },
  "dependencies": {
    "@tailwindcss/vite": "^4.3.3",
    "tailwindcss": "^4.3.3"
  }
}
```

لاحظ: `prettier` من غير `^` (`-E`)، و npm حط Tailwind في `dependencies` لأننا ماكتبناش `-D` (زي ما دوكيومنتيشن Tailwind بيقول). في موقع Vite مش هتفرق (فصل 14)، وحطه في devDependencies لو تحب.

26.6 · 6 build و preview

Terminal

```
user@MacBook ~ % npm run build

…
✓ 6 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.57 kB │ gzip: 0.35 kB
dist/assets/index-NALiIVJV.css   9.53 kB │ gzip: 2.56 kB
dist/assets/index-Ba828Msq.js    1.29 kB │ gzip: 0.73 kB

✓ built in 98ms
```

dist/index.html (الـ head)

```
<link rel="icon" type="image/svg+xml" href="/my-links/favicon.svg" />
<title>My Links</title>
<script type="module" crossorigin src="/my-links/assets/index-Ba828Msq.js"></script>
<link rel="stylesheet" crossorigin href="/my-links/assets/index-NALiIVJV.css">
```

كل المسارات فيها الـ `base`. ده اللي هيمنع الصفحة البيضا على GitHub Pages (فصل 25).	`/my-links/…`
`%VITE_APP_TITLE%` اتبدل.	`<title>My Links</title>`
مفيش Framework. و TBT 0 في Lighthouse (فصل 24).	`1.29 kB JS`

```
Terminal
user@MacBook ~ % npx vite preview --port 4190
  ➜  Local:   http://localhost:4190/my-links/
  ➜  Network: use --host to expose
```

(احنا اخترنا بورت 4190 بـ `--port` وقت التجربة. `npm run preview` عادي هيفتح على 4173.) افتح الرابط واتأكد إن الشكل زي dev، وجرّب Tab على الروابط، وشغّل Lighthouse (فصل 24).

26.7 Git · 7

```
Terminal
user@MacBook ~ % git init -b main
user@MacBook ~ % git add .
user@MacBook ~ % git commit -m "my-links: vite + tailwind + eslint + prettier"
user@MacBook ~ % git show --stat HEAD
 .env                        |    1 +
 .github/workflows/deploy.yml |   45 +
 .gitignore                  |   24 +
 .prettierignore             |    2 +
 .prettierrc                 |    4 +
 eslint.config.js            |   16 +
 index.html                  |   13 +
 package-lock.json           | 2450 ++++++++++++++++++++++++++++++++++++++++++
 package.json                |   26 +
 public/favicon.svg          |    1 +
 src/links.js                |    5 +
 src/main.js                 |   22 +
 src/style.css               |    5 +
 vite.config.js              |    8 +
 14 files changed, 2622 insertions(+)
user@MacBook ~ % git check-ignore -v .env.local
.gitignore:13:*.local   .env.local
```

راجع اللستة: `package-lock.json` موجود (فصل 15)، و`node_modules` و`dist` و`.env.local` **مش موجودين**، و`check-ignore` بيأكد إن `.env.local` متجاهل بسطر `*.local` في `.gitignore` بتاع القالب (كتاب Git فصل 5). و`.env` اترفع لأنه مفيهوش أسرار.

26.8 النشر (عليك) · 8

الـ Workflow (`.github/workflows/deploy.yml`) جاهز في الـ commit، ومشروح سطر سطر في فصل 25. الباقي على حسابك:

1. اعمل repo على GitHub اسمه `my-links` (لازم نفس اسم الـ `base`).
2. `git remote add origin …` و`git push -u origin main` (كتاب Git فصل 9).
3. Settings ← Pages ← Source: GitHub Actions.

4 تاب Actions: استنى الـ ✓، والرابط `https://USERNAME.github.io/my-links/` .

خطوات تكمّل بيها

- زوّد `<meta name="description">` (الـ SEO هيبقى 100).
- اعمل نفس المشروع بـ `--template react` : Component اسمه `LinkCard` و `links.map` في JSX.
- جرّبه بـ `react-ts` واعمل `type Link = { label: string; url: string }` .
- انشره على Netlify أو Vercel (امسح `base`).

المصدر

Vite · Getting Started — Tailwind CSS · Installing with Vite — ESLint · Getting Started — Prettier · Install

Vite · Deploy to GitHub Pages

ملخص الفصل

- create-vite ثم install ثم Tailwind ثم الصفحة ثم ESLint و Prettier ثم build و preview ثم Git ثم Workflow.
- 14 ملف في الـ commit، من غير `node_modules` و `dist` والأسرار.
- الناتج: HTML 0.57 kB و CSS 9.53 kB و JS 1.29 kB، وكل المسارات تحت `/my-links/` .

أسئلة مراجعة

1. لكل ملف في `git show --stat` : اتعمل في أنهي خطوة؟ وأنهي فصل بيشرحه؟
2. لو هتنشر على Vercel، إيه اللي هتغيره؟
3. ليه `.env` اترفع و `.env.local` لأ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل نفّذ المشروع على الماك بتاعك خطوة خطوة، وقارن أرقامك بأرقام الكتاب.
2. متوسط زوّد Dark mode بـ `dark:` في Tailwind.
3. صعب انشره على GitHub Pages، وشغّل Lighthouse على الرابط الحقيقي و PageSpeed Insights.

27

المرحلة 8 · الجودة والأداء والنشر

الفصل السابع والعشرين

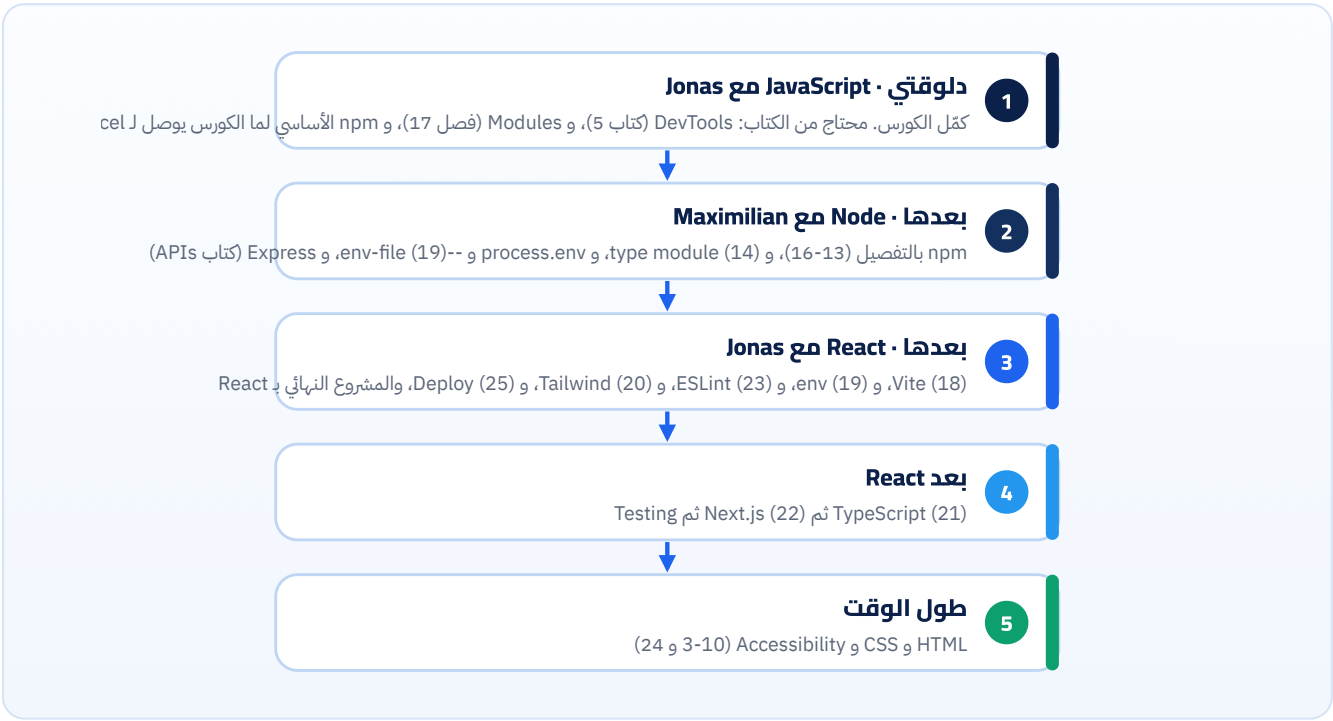
الخريطة: تتعلم إيه وإمتى

بعد 26 فصل، الأسماء كتير: npm و Vite و Tailwind و TypeScript و Next.js و ESLint... السؤال الطبيعي: أتعلمهم كلهم دلوقتي؟ لأ. الفصل ده بيرتّب الرحلة على الكورسات اللي انت ماشي فيها، وبيقولك إيه اللي محتاجه في كل مرحلة وإيه اللي يستنى، وإزاي تفضل متابع من غير ما تغرق.

بعد الفصل ده هتكون عارف

- تعرف ترتيب التعلم على رحلة JS ثم Node ثم React
- تعرف إيه من الكتاب هتحتاجه في كل مرحلة
- تعرف إيه اللي يستنى لبعدين
- تعرف مصادر رسمية تتابع منها
- تعمل خطة مشاريع صغيرة

27.1 الرحلة



شكل 27.1 الكتاب ده مرجع ترجعله في كل مرحلة، مش حاجة تخلّصها مرة واحدة

27.2 إيه اللي محتاجه دلوقتي وإيه اللي يستنى

اتعلمه دلوقتي	بعد React	لما تحتاجه
Semantic HTML و Forms (3-4)	(21) TypeScript	(20) Sass

اتعلمه دلوقتي	بعد React	لما تحتاجه
Cascade و Flexbox و Grid و Responsive (6-9)	Next.js و SSR (22)	pnpm و Bun (16)
npm: install و scripts و ^ و Lockfile (13-15)	Testing: Vitest و Playwright	Vue أو Svelte أو Angular (12)
Vite و env (18-19)	مكتبة Components (20)	CSS Modules (20)
Git و Deploy بسيط (25)	Core Web Vitals بعمق (24)	Workspaces و Monorepos

جدول 27.1 أولويات

> **القاعدة**
>
> اتعلّم الأداة لما **تحس بالمشكلة** اللي بتحلها. Tailwind هيبقى منطقي لما الـ CSS بتاعك يكبر، و TypeScript لما تقضي ساعة تدوّر على `undefined`، و Next.js لما تحتاج SEO. لو اتعلمتهم قبل المشكلة، هتحفظ أوامر من غير ما تفهمها.

27.3 مشاريع صغيرة على الطريق

1. **مع JS:** المشروع النهائي (فصل 26) بـ Vanilla، وبعدين زوّد Form يضيف روابط ويحفظها في `localStorage`.
2. **مع Node:** API صغير بـ Express يرجّع الروابط JSON (كتاب APIs)، وخلّي الصفحة تعمل `fetch` منه، والمفتاح السري في `.env` على السيرفر.
3. **مع React:** نفس الصفحة بـ `--template react` و Tailwind، و ESLint، ونشر على Netlify أو Vercel.
4. **بعد React:** نفس المشروع بـ `react-ts`، وبعدين Next.js بصفحة لكل رابط بـ `generateStaticParams`.

27.4 تتابع إزاي من غير ما تغرق

المصادر الرسمية	MDN و web.dev للمنصة، و `vite.dev/blog` و `react.dev/blog` و `nextjs.org/blog` و GitHub Changelog لـ npm. كل لينك في ملحق الروابط آخر الكتاب.
`Baseline`	قبل ما تستخدم ميزة CSS أو JS جديدة: هل هي "Widely available"؟ (فصل 10).
`npm outdated`	مرة كل فترة على مشاريعك (فصل 15)، واقرا الـ Changelog قبل أي MAJOR.
التاريخ على المقال	الإيكوسيستم بيتغير بسرعة: `create-react-app` و `tailwind.config.js` و FID و `.eslintrc` كلهم اتغيروا في سنتين. أي Tutorial أقدم من سنة، قارنه بالدوكيومنتيشن. والكتاب ده نفسه صورة من أكتوبر 2026.

الأساس مابيتغيرش

الأدوات بتتغير كل سنة، بس HTML و CSS و JavaScript و HTTP و Git ثابتين من سنين. اللي فاهم الـ Cascade هيفهم أي أداة CSS، واللي فاهم Modules والـ Bundler هيفهم أي Build tool جديد. استثمر في الأساس، والأدوات هتيجي.

المصدر `MDN · Learn web development` `web.dev · Learn` `React · Learn` `Vite · Blog` `Next.js · Learn` `roadmap.sh · Frontend` `Vitest · Guide` `Playwright · Intro`

ملخص الفصل

- JS ثم Node ثم React ثم TypeScript و Next.js، و HTML و CSS و a11y طول الوقت.
- اتعلم الأداة لما تحس بمشكلتها.
- مشروع صغير في كل مرحلة، وتابع من المصادر الرسمية وبص على التاريخ.

أسئلة مراجعة

1. إيه أهم 3 فصول من الكتاب هترجعلها في كورس React؟
2. لقيت Tutorial بيستخدم `create-react-app` و `tailwind.config.js`. إيه اللي هتراجعه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اكتب خطتك للشهرين الجايين بالمشاريع الصغيرة.
2. متوسط افتح ملحق الروابط، واعمل Bookmarks folder لكل مرحلة.
3. صعب بعد ما تخلّص React: ارجع للكتاب كله واعمل التحديات الصعبة اللي فاتتك.

ملحق أ

Cheat Sheet وجدول الـ Flags وجدول الرموز

ورقة مراجعة لأهم أوامر npm و npx و Vite و Tailwind وأدوات الجودة والنشر، مقسّمة حسب الموضوع، وبعدها جدول الـ Flags وجدول لرموز الـ SemVer والـ CSS وغيرها.

npm	التسطيب
`npm init -y`	package.json جديد
`npm i`	سطّب كل اللي في package.json
`npm i react`	dependencies
`npm i -D vite`	devDependencies
`npm i -E prettier`	نسخة بالظبط (من غير ^)
`npm i ms@~2.0.0 / x@latest`	Tag أو Range
`npm un x`	امسح
`npm ci`	تسطيب نظيف من الـ Lockfile (CI)
`npm update`	أحدث نسخة جوه الـ Range
`npm outdated`	Latest / Wanted / Current

npm	الفحص والإدارة
`npm ls / npm ls --all`	الشجرة
`npm explain x`	مين جاب x
`npm view x versions / dist-tags`	اسأل الـ Registry
`npm audit / audit fix`	الثغرات المعروفة
`npm audit signatures`	Signatures و Provenance
`npm pkg get x / set a.b=c`	اقرا وعدّل package.json
`npm prefix -g`	مكان الـ Global
`npm install-scripts ls / approve x`	Install scripts :npm 12
`npm rebuild`	شغّل الـ Scripts المسموحة

npm run	npx و Scripts
`npm run`	لستة الـ Scripts
`npm run dev / build / preview`	Vite Scripts بتاعة
`npm start / npm test`	من غير run
`npm run lint -- --fix`	-- تعدّي Flags للأمر
`preX / postX`	بيشتغلوا قبل/بعد X لوحدهم
`npx x`	شغّل من bin. أو نزّل مؤقتاً
`npm create vite@latest app -- --template react`	npx create-vite@latest =

Vite	Vite
`npm run dev -- --host`	افتح للشبكة (موبايل)
`vite --port 3000 --open`	بورت ويفتح المتصفح
`vite build --mode staging`	يحمّل .env.staging

`vite preview`	يقدّم dist على 4173
`base: '/repo/'`	GitHub Pages
`import.meta.env.VITE_X`	متغير عام (نص)
`MODE / DEV / PROD / BASE_URL`	متغيرات جاهزة
`%VITE_X%`	جوه index.html
`import('./x.js')`	Code splitting
`*.module.css`	CSS Modules

Tailwind v4 — CSS	
`npm i tailwindcss @tailwindcss/vite`	تسطيب
`plugins: [tailwindcss()]`	vite.config.js
`@import "tailwindcss";`	سطر الـ CSS
`@theme { --color-brand: #1d63ed; }`	لون ← text-brand و bg-brand

`p-4 = calc(var(--spacing) * 4)`	rem1
`hover: / focus-visible: / md: / dark:`	Variants

Quality & Deploy — الجودة والنشر	
`npx eslint . [--fix]`	Lint
`npx prettier . --check / --write`	Format
`npx tsc --noEmit`	فحص الأنواع
`node --env-file=.env app.js`	Node في env
`npx lighthouse URL`	تقرير Lighthouse
`netlify deploy [--prod]`	Netlify CLI
`vercel / npx vercel`	Vercel CLI
`Settings ← Pages ← GitHub Actions`	GitHub Pages

1.أ جدول الـ Flags

نفس الحرف ممكن يبقى ليه معنى مختلف من أداة للتانية. ولما تشك: `npm help install` أو `npx vite --help` أو `npx create-vite@latest --help`.

الـ Flag	في	المعنى
`-y`	npx / npm init	وافق على كل حاجة
`-D`	npm i	--save-dev
`-E`	npm i	--save-exact
`-g`	ls / npm i	Global
`--prefix`	npm	مكان بديل
`--all`	npm ls	الشجرة كلها
`--omit=dev`	ci / npm i	من غير devDependencies
`--force`	npm audit fix	يسمح بـ MAJOR

الـ Flag	في	المعنى
`--legacy-peer-deps`	npm i	تجاهل الـ peers (بحذر)
`--`	create / npm run	اللي بعدي للأمر
`-t / --template`	create-vite	القالب
`--eslint`	create-vite	ESLint بدل Oxlint (React)
`--no-interactive`	create-vite	من غير أسئلة
`-i`	create-vite	dev + install
`--host`	vite	افتح للشبكة
`--port`	vite	البورت
`--mode`	vite	ملف .env..[mode]
`--fix`	eslint	صلّح المضمون
`--check / --write`	prettier	افحص / اكتب
`--noEmit`	tsc	افحص بس
`-b`	tsc	Build mode لـ references
`--frozen-lockfile`	bun / pnpm	زي npm ci
`--env-file`	node	حمّل .env.

جدول أ.1 الـ Flags ومعانيها حسب الأداة

2.أ جدول الرموز

الرمز	المعنى	مثال
`^1.2.3`	أي 1.x من 1.2.3 (الافتراضي)	`>=1.2.3 <2.0.0`
`~1.2.3`	Patches بس	`>=1.2.3 <1.3.0`
`1.2.3`	بالظبط	`-E`
`^0.2.3`	مع 0.x أضيق	`>=0.2.3 <0.3.0`
`1.x / 1.*`	أي 1	`1.0.0 … 1.9.9`
`*`	أي نسخة	خطر
`>= <`	حدود صريحة	`>=1.2.3 <1.3.0`
`||`	أو	`^5.2.0 || ^6`

الرمز	المعنى	مثال
`1.3.0-beta.1`	Prerelease	`^ مش بتيجي مع`
`@scope/name`	Package تحت Scope	`@tailwindcss/vite`
`x@latest / x@next`	dist-tag	`vite@latest`
`--`	نهاية Flags بتاعة npm	`npm run dev -- --host`
`./ ../ /`	مسار نسبي/مطلق (مش Bare)	`./math.js`
`#id .class`	Specificity (0,1,0) / (1,0,0)	`#buy / .btn`
`:where() / :is()`	Specificity صفر / أعلى واحد	`:where(h1,p)`
`:has()`	الأب حسب الولاد	`li:has(a)`
`&`	Nesting (CSS و Sass)	`&:hover`
`$x`	متغير Sass (وقت البناء)	`$brand`
`--x / var(--x)`	Custom property (وقت التشغيل)	`var(--color-brand)`
`@layer`	طبقات الـ Cascade	`@layer base, components`
`%VITE_X%`	env في HTML	`<title>`
○ ◐	Partial Prerender / Static	`next build`

جدول أ.2 رموز SemVer و npm و CSS في الكتاب

ملحق ب

قاموس المصطلحات

كل مصطلح مهم في الكتاب في سطر واحد، مترتب أبجدياً. ارجعه كل ما تقابل كلمة ومش فاكر معناها.

#

`^` / `~`

Range: نفس الـ MAJOR / نفس الـ MINOR.

A

Accessibility (a11y)

إن أي حد يقدر يستخدم الموقع: بالكيبورد، أو Screen reader، أو تكبير، أو نظر ضعيف.

Accessibility tree

شجرة بيبنيها المتصفح من الـ HTML فيها الدور والاسم والحالة لكل عنصر.

ARIA

Attributes بتوصف الدور والحالة للتقنيات المساعدة. أول قاعدة: متستخدمهاش لو فيه عنصر HTML بيعمل نفس الحاجة.

B

Bare specifier

import بالاسم بس (`'react'`). المتصفح مايفهمهوش، والـ Bundler بيحلّه.

Baseline

مؤشر إن ميزة الويب مدعومة في المتصفحات الرئيسية (Widely / Newly available).

Bundler

أداة بتجمّع الـ Modules والـ Packages في ملفات قليلة مصغّرة (webpack و Rolldown).

C

Cascade

القواعد اللي بتحدد أنهي CSS يكسب: الأهمية، ثم inline، ثم Layers، ثم Specificity، ثم الترتيب.

CLS

Cumulative Layout Shift: الحاجات بتتنطط قد إيه. جيد ≤ 0.1.

Code splitting

تقسيم الكود لملفات بتتحمّل وقت الحاجة بـ `import()`.

CommonJS

نظام Modules القديم بتاع Node: `require` و `module.exports`.

Container query

`@container`: ستايل حسب عرض العلبة مش الشاشة.

Core Web Vitals

تلات مقاييس لتجربة المستخدم: LCP و INP و CLS.

CSR

Client-Side Rendering: HTML فاضي و JS يبني الصفحة في المتصفح.

CSS Modules

ملفات `*.module.css` أسماء الـ Classes فيها محلية وفريدة.

Custom property

متغير CSS (`--x`) بيعيش في المتصفح ويتغير وقت التشغيل.

D

Dependency

Package مشروعك محتاجه. dev = أدوات، peer = «سطّبه انت»، optional = «لو فشل كمّل».

Deploy

نشر الموقع: build ثم رفع dist على استضافة.

Derived state

قيمة بتتحسب من الـ State بدل ما تتخزن (زي إجمالي السلة).

dist

فولدر الناتج بتاع `npm run build`. ده الموقع اللي بيترفع.

dist-tag

اسم بيشاور على نسخة: latest و next و beta.

E

ES Modules (ESM)

نظام الـ Modules الرسمي: `import` و `export`.

ESLint

Linter: بيدوّر على الأخطاء والكود المشكوك فيه.

F

Flat config

ملف `eslint.config.js`، طريقة الإعداد الوحيدة في ESLint 10.

Framework

كود بينادي كودك في أماكن محددة ويحدد الهيكل (Angular و Next.js).

H

Hash (في اسم الملف)

حروف من محتوى الملف (`index-Ba828Msq.js`) علشان الـ Cache.

HMR

Hot Module Replacement: تبديل الملف المتعدّل في الصفحة من غير Reload.

Hoisting

npm بيحط الـ Packages متسطّحة في أول مستوى من node_modules.

Hydration

ربط JS بـ HTML جاي جاهز من السيرفر علشان يبقى تفاعلي.

I

INP

Interaction to Next Paint: سرعة رد الصفحة على الضغط. جيد ≤ 200ms. بدل FID من مارس 2024.

ISR

Incremental Static Regeneration: صفحة Static بتتجدد كل فترة.

J

JSX

HTML جوه JavaScript في React، بيتحول لـ JS.

L

LCP

Largest Contentful Paint: أكبر عنصر ظهر إمتى. جيد ≤ 2.5 ثانية.

Library

كود انت اللي بتناديه وقت ما تحب (React بتوصف نفسها كده).

Lighthouse

أداة من Chrome بتقيس الأداء والـ Accessibility والـ SEO (Lab data).

Lockfile

`package-lock.json`: النسخ اللي اتسطبت بالظبط مع resolved و integrity.

M

Meta-framework

Framework فوق مكتبة UI بيضيف Routing و SSR (Next.js و Nuxt و SvelteKit).

N

Node.js

Runtime بيشغّل JavaScript برة المتصفح. كل أدوات الفرونت شغالة بيه.

node_modules

فولدر الـ Packages المتسطبة. مش بيترفع على Git.

npm

مدير Packages Node: Registry + CLI + موقع.

npx

يشغّل أمر من node_modules/.bin أو ينزّله مؤقتاً. = npm exec.

O

overrides

مفتاح في package.json بيجبر نسخة Package جوه الشجرة.

P

Package

أي فولدر فيه package.json.

Partial Prerendering (PPR)

جزء ثابت اتبنى وقت البناء وجزء ديناميكي بيتبعت Stream (◐ في Next.js).

Phantom dependency

Package بتعمله import وهو مش في package.json بتاعك، موجود بالصدفة.

pnpm

مدير Packages بمخزن واحد على الجهاز و Symlinks.

Prettier

Formatter: بيوحّد شكل الكود.

Provenance

إثبات إن الـ Package اتبنى من repo معين على CI (Attestation).

R

Registry

السيرفر اللي فيه كل الـ Packages: registry.npmjs.org.

S

Semantic HTML

استخدام العنصر اللي معناه مناسب (button و nav و main) مش div لكل حاجة.

SemVer

MAJOR.MINOR.PATCH: بيكسر . ميزة . إصلاح.

Server Component

Component في Next.js بيشتغل على السيرفر بس وكوده مش بيوصل للمتصفح.

Signals

طريقة Reactivity: كل قيمة عارفة مين بيستخدمها (Vue و Svelte و Angular).

SPA

Single Page Application: صفحة واحدة و JS بيغيّر المحتوى.

Specificity

«دقة» الـ Selector: (IDs, Classes, Elements).

SSG

Static Site Generation: الـ HTML بيتبني مرة واحدة وقت build.

SSR

Server-Side Rendering: الـ HTML بيتبني على السيرفر مع كل Request.

Supply chain attack

هجوم على Package موقعك بيعتمد عليه بدل موقعك نفسه.

T

Tailwind CSS

Utility classes في الـ HTML، وبيولّد اللي انت استخدمته بس.

Transitive dependency

Package جه لأن Package تاني محتاجه.

Tree-shaking

شيل الكود اللي مش متعمله import من الـ Bundle.

TypeScript

JavaScript + أنواع، بيلاقي الأخطاء قبل التشغيل.

Typosquatting

Package باسم قريب من مشهور فيه كود ضار.

V

Virtual DOM

نسخة في الذاكرة من الواجهة بيقارنها React علشان يحدّث اللي اتغير.

Vite

Build tool: Dev server سريع بـ ESM و HMR و Build بـ Rolldown.

VITE_

بادئة المتغيرات اللي Vite بيحطها في كود المتصفح. يعني عامة.

W

WCAG

معايير الـ Accessibility الدولية (2.2).

ملحق ج

ملحق الروابط الرسمية

كل صفحة رسمية أو Spec اعتمد عليها الكتاب، مجمّعة حسب المصدر، وجنب كل لينك الفصول اللي استخدمته. اللينكات دي اتفتحت كلها واتأكدنا إنها شغالة يوم 23 أكتوبر 2026. الأدوات بتتحدث بسرعة، فلما تلاقي تعارض بين الكتاب والصفحة، الصفحة هي الصح.

ج.1 MDN

MDN · How the web works · فصل 1
https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works

MDN · SPA (Single-page application) · فصل 1
https://developer.mozilla.org/en-US/docs/Glossary/SPA

MDN · AJAX · فصل 1
https://developer.mozilla.org/en-US/docs/Glossary/AJAX

MDN · Introduction to client-side frameworks · فصل 2، 11
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction

MDN · Semantics · فصل 3
https://developer.mozilla.org/en-US/docs/Glossary/Semantics

MDN · HTML elements reference · فصل 3
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements

MDN · HTML: A good basis for accessibility · فصل 3
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Accessibility/HTML

MDN · Your first form · فصل 4
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Your_first_form

MDN · Client-side form validation · فصل 4
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

MDN · autocomplete · فصل 4
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/autocomplete

MDN · FormData · فصل 4
https://developer.mozilla.org/en-US/docs/Web/API/FormData

MDN · :user-invalid · فصل 4
https://developer.mozilla.org/en-US/docs/Web/CSS/:user-invalid

MDN · Accessibility · فصل 5
https://developer.mozilla.org/en-US/docs/Web/Accessibility

MDN · Introduction to the CSS cascade · فصل 6
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_cascade/Cascade

MDN · Specificity · فصل 6
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_cascade/Specificity

MDN · @layer · فصل 6
https://developer.mozilla.org/en-US/docs/Web/CSS/@layer

MDN · Inheritance · فصل 6
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_cascade/Inheritance

MDN · The box model · فصل 6
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Box_model

MDN · Flexbox · فصل 7
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Flexbox

MDN · Basic concepts of flexbox · فصل 7
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox

MDN · CSS grid layout · فصل 8
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Grids

MDN · grid-template-areas · فصل 8
https://developer.mozilla.org/en-US/docs/Web/CSS/grid-template-areas

MDN · repeat() · فصل 8
https://developer.mozilla.org/en-US/docs/Web/CSS/repeat

MDN · Subgrid · فصل 8
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Subgrid

MDN · Responsive design · فصل 9
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

MDN · Viewport meta element · فصل 9
https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Viewport_meta_element

MDN · Using media queries · فصل 9
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries

MDN · Container queries · فصل 9
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_containment/Container_queries

MDN · Responsive images · فصل 9
https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images

MDN · Using CSS custom properties · فصل 10
https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

MDN · Using CSS nesting · فصل 10
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_nesting/Using_CSS_nesting

MDN · :has() · فصل 10
https://developer.mozilla.org/en-US/docs/Web/CSS/:has

MDN · Logical properties and values · فصل 10
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_logical_properties_and_values

MDN · oklch() · فصل 10
https://developer.mozilla.org/en-US/docs/Web/CSS/color_value/oklch

MDN · color-mix() · فصل 10
https://developer.mozilla.org/en-US/docs/Web/CSS/color_value/color-mix

MDN · Cross-site scripting (XSS) · فصل 11
https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS

MDN · JavaScript modules · فصل 17
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

MDN · Import maps · فصل 17
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/type/importmap

MDN · import() (dynamic import) · فصل 17
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import

MDN · Tree shaking · فصل 17
https://developer.mozilla.org/en-US/docs/Glossary/Tree_shaking

MDN · Learn web development · فصل 27
https://developer.mozilla.org/en-US/docs/Learn_web_development

ج.2 Chrome و web.dev

Chrome DevTools · Accessibility features reference · فصل 3
https://developer.chrome.com/docs/devtools/accessibility/reference

web.dev · Learn Accessibility · فصل 5
https://web.dev/learn/accessibility

web.dev · Learn CSS: The cascade · فصل 6
https://web.dev/learn/css/the-cascade

web.dev · Learn CSS: Flexbox · فصل 7
https://web.dev/learn/css/flexbox

Chrome DevTools · Inspect and debug CSS flexbox · فصل 7
https://developer.chrome.com/docs/devtools/css/flexbox

web.dev · Learn CSS: Grid · فصل 8
https://web.dev/learn/css/grid

Chrome DevTools · Inspect CSS grid · فصل 8
https://developer.chrome.com/docs/devtools/css/grid

web.dev · Learn Responsive Design · فصل 9
https://web.dev/learn/design

web.dev · Baseline · فصل 10
https://web.dev/baseline

web.dev · Rendering on the Web · فصل 22
https://web.dev/articles/rendering-on-the-web

web.dev · Web Vitals · فصل 24
https://web.dev/articles/vitals

web.dev · LCP · فصل 24
https://web.dev/articles/lcp

web.dev · INP · فصل 24
https://web.dev/articles/inp

CLS · web.dev · فصل 24
https://web.dev/articles/cls

INP becomes a Core Web Vital (مارس 2024) · web.dev · فصل 24
https://web.dev/blog/inp-cwv-launch

Lighthouse · Chrome for Developers · فصل 24
https://developer.chrome.com/docs/lighthouse/overview

PageSpeed Insights · فصل 24
https://pagespeed.web.dev/

Chrome UX Report (CrUX) · فصل 24
https://developer.chrome.com/docs/crux

Learn · web.dev · فصل 27
https://web.dev/learn

ج.3 W3C و WHATWG و WebAIM

Introduction to Web Accessibility · W3C WAI · فصل 5
https://www.w3.org/WAI/fundamentals/accessibility-intro/

WCAG 2.2 · W3C · فصل 5
https://www.w3.org/TR/WCAG22/

Using ARIA (the first rule) · W3C · فصل 5
https://www.w3.org/TR/using-aria/

The WebAIM Million · WebAIM · فصل 5
https://webaim.org/projects/million/

ج.4 Node و npm ومديري الـ Packages

Introduction · Node.js · فصل 2
https://nodejs.org/en/learn/getting-started/introduction-to-nodejs

About npm · npm Docs · فصل 13
https://docs.npmjs.com/about-npm

npm install · npm Docs · فصل 13
https://docs.npmjs.com/cli/v11/commands/npm-install

npm exec (npx) · npm Docs · فصل 13
https://docs.npmjs.com/cli/v11/commands/npm-exec

npm init / create · npm Docs · فصل 13
https://docs.npmjs.com/cli/v11/commands/npm-init

Folders (prefix, node_modules, .bin) · npm Docs · فصل 13
https://docs.npmjs.com/cli/v11/configuring-npm/folders

Resolving EACCES · npm Docs · فصل 13
https://docs.npmjs.com/resolving-eacces-permissions-errors-when-installing-packages-globally

Download · Node.js · فصل 13
https://nodejs.org/en/download

Node.js · Releases (LTS) · فصل 13
https://nodejs.org/en/about/previous-releases

npm Docs · package.json · فصل 14
https://docs.npmjs.com/cli/v11/configuring-npm/package-json

npm Docs · scripts · فصل 14
https://docs.npmjs.com/cli/v11/using-npm/scripts

npm Docs · npm run · فصل 14
https://docs.npmjs.com/cli/v11/commands/npm-run

npm Docs · npm pkg · فصل 14
https://docs.npmjs.com/cli/v11/commands/npm-pkg

Node.js Docs · Modules: Packages (type, exports) · فصل 14
https://nodejs.org/api/packages.html

Semantic Versioning 2.0.0 · فصل 15
https://semver.org/

npm Docs · About semantic versioning · فصل 15
https://docs.npmjs.com/about-semantic-versioning

GitHub · npm/node-semver (Ranges) · فصل 15
https://github.com/npm/node-semver

npm Docs · package-lock.json · فصل 15
https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json

npm Docs · npm ci · فصل 15
https://docs.npmjs.com/cli/v11/commands/npm-ci

npm Docs · npm update · فصل 15
https://docs.npmjs.com/cli/v11/commands/npm-update

npm Docs · npm outdated · فصل 15
https://docs.npmjs.com/cli/v11/commands/npm-outdated

npm Docs · dist-tag · فصل 15
https://docs.npmjs.com/cli/v11/commands/npm-dist-tag

pnpm · Motivation (store g symlinks) · فصل 16
https://pnpm.io/motivation

pnpm · pnpm add · فصل 16
https://pnpm.io/cli/add

Bun Docs · bun install · فصل 16
https://bun.sh/docs/cli/install

Yarn · Getting started · فصل 16
https://yarnpkg.com/getting-started

npm Docs · npm audit · فصل 16
https://docs.npmjs.com/cli/v11/commands/npm-audit

GitHub · npm/cli v12.0.0 release notes · فصل 16
https://github.com/npm/cli/releases/tag/v12.0.0

npm Docs · approve-scripts / allowScripts · فصل 16
https://docs.npmjs.com/cli/v12/commands/npm-approve-scripts/

npm Docs · Provenance · فصل 16
https://docs.npmjs.com/generating-provenance-statements

Node.js Docs · ECMAScript modules · فصل 17
https://nodejs.org/api/esm.html

Node.js Docs · --env-file · فصل 19
https://nodejs.org/api/cli.html#--env-filefile

Node.js Docs · process.env · فصل 19
https://nodejs.org/api/process.html#processenv

npm · dotenv · فصل 19
https://www.npmjs.com/package/dotenv

Node.js Docs · Modules: TypeScript (type stripping) · فصل 21
https://nodejs.org/api/typescript.html

ج.5 Vite والأدوات حواليه

Vite · Getting Started · فصل 2، 18، 26
https://vite.dev/guide/

Vite · Why Vite · فصل 17
https://vite.dev/guide/why

Vite · Announcing Vite 8 · فصل 17
https://vite.dev/blog/announcing-vite8

Vite · Features (HMR, CSS, assets) · فصل 18
https://vite.dev/guide/features

Vite · Static Asset Handling (public) · فصل 18
https://vite.dev/guide/assets

Vite · Building for Production · فصل 18
https://vite.dev/guide/build

Vite · CLI · فصل 18
https://vite.dev/guide/cli

Vite · Config · فصل 18
https://vite.dev/config/

Vite · base · فصل 18
https://vite.dev/config/shared-options#base

Vite · Dependency Pre-Bundling · فصل 18
https://vite.dev/guide/dep-pre-bundling

Vite · HMR API · فصل 18
https://vite.dev/guide/api-hmr

Vite · Env Variables and Modes · فصل 19
https://vite.dev/guide/env-and-mode

Vite · HTML Constant Replacement · فصل 19
https://vite.dev/guide/env-and-mode#html-constant-replacement

Vite · CSS Modules · فصل 20
https://vite.dev/guide/features#css-modules

Vite · CSS Pre-processors · فصل 20
https://vite.dev/guide/features#css-pre-processors

Vite · TypeScript (transpile only) · فصل 21
https://vite.dev/guide/features#typescript

Oxc · Oxlint · فصل 23
https://oxc.rs/docs/guide/usage/linter

Vite · Deploying a Static Site · فصل 25
https://vite.dev/guide/static-deploy

Vite · Deploy to GitHub Pages · فصل 26
https://vite.dev/guide/static-deploy#github-pages

Vite · Blog · فصل 27
https://vite.dev/blog

Vitest · Guide · فصل 27
https://vitest.dev/guide/

ج.6 React و Frameworks و TypeScript

React · react.dev · فصل 2
https://react.dev/

React · Thinking in React · فصل 11
https://react.dev/learn/thinking-in-react

React · Reacting to Input with State · فصل 11
https://react.dev/learn/reacting-to-input-with-state

React · Quick Start · فصل 12، 27
https://react.dev/learn

Vue · Introduction · فصل 12
https://vuejs.org/guide/introduction.html

Svelte · Overview · فصل 12
https://svelte.dev/docs/svelte/overview

Svelte · $state · فصل 12
https://svelte.dev/docs/svelte/$state

Angular · Overview · فصل 12
https://angular.dev/overview

Angular · Signals · فصل 12
https://angular.dev/guide/signals

React Blog · Sunsetting Create React App · فصل 17
https://react.dev/blog/2025/02/14/sunsetting-create-react-app

TypeScript · TS for the New Programmer · فصل 21
https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html

TypeScript · TSConfig Reference · فصل 21
https://www.typescriptlang.org/tsconfig/

TypeScript Blog (TypeScript 7) · فصل 21
https://devblogs.microsoft.com/typescript/

React · Using TypeScript · فصل 21
https://react.dev/learn/typescript

React · hydrateRoot · فصل 22
https://react.dev/reference/react-dom/client/hydrateRoot

ج.7 Next.js

Next.js · Installation · فصل 22
https://nextjs.org/docs/app/getting-started/installation

Next.js · Caching g Prerendering · فصل 22
https://nextjs.org/docs/app/getting-started/caching

Next.js · generateStaticParams · فصل 22
https://nextjs.org/docs/app/api-reference/functions/generate-static-params

Next.js · cacheLife · فصل 22
https://nextjs.org/docs/app/api-reference/functions/cacheLife

Next.js · connection · فصل 22
https://nextjs.org/docs/app/api-reference/functions/connection

Next.js · cacheComponents · فصل 22
https://nextjs.org/docs/app/api-reference/config/next-config-js/cacheComponents

Next.js Blog · Next.js 16 · فصل 22
https://nextjs.org/blog/next-16

Next.js · Learn · فصل 27
https://nextjs.org/learn

ج.8 Tailwind و CSS

Tailwind CSS · Installing with Vite · فصل 20، 26
https://tailwindcss.com/docs/installation/using-vite

Tailwind CSS · Theme variables (@theme) · فصل 20
https://tailwindcss.com/docs/theme

Tailwind CSS · Detecting classes in source files · فصل 20
https://tailwindcss.com/docs/detecting-classes-in-source-files

Tailwind CSS · Hover, focus and other states · فصل 20
https://tailwindcss.com/docs/hover-focus-and-other-states

Sass · Basics · فصل 20
https://sass-lang.com/guide/

shadcn/ui · Docs · فصل 20
https://ui.shadcn.com/docs

Radix Primitives · فصل 20
https://www.radix-ui.com/primitives

MUI · Getting started · فصل 20
https://mui.com/material-ui/getting-started/

ج.9 ESLint و Prettier

ESLint · Getting Started · فصل 23، 26
https://eslint.org/docs/latest/use/getting-started

ESLint · Configuration Files (flat config) · فصل 23
https://eslint.org/docs/latest/use/configure/configuration-files

ESLint · Rules Reference · فصل 23
https://eslint.org/docs/latest/rules/

ESLint · CLI (--fix) · فصل 23
https://eslint.org/docs/latest/use/command-line-interface

Prettier · Install · فصل 23، 26
https://prettier.io/docs/install

Prettier · Options · فصل 23
https://prettier.io/docs/options

Prettier · Integrating with Linters · فصل 23
https://prettier.io/docs/integrating-with-linters

ج.10 GitHub و Netlify و Vercel

GitHub Docs · Dependabot · فصل 15
https://docs.github.com/en/code-security/dependabot

GitHub Changelog · npm v12 install-time security (8 يوليو 2026) · فصل 16
https://github.blog/changelog/2026-07-08-npm-install-time-security-and-gat-bypass2fa-deprecation/

GitHub · eslint-config-prettier · فصل 23
https://github.com/prettier/eslint-config-prettier

GitHub · axe-core · فصل 24
https://github.com/dequelabs/axe-core

GitHub Docs · Pages publishing source · فصل 25
https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

GitHub Docs · Security hardening (pin actions to SHA) · فصل 25
https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions

Netlify Docs · Vite · فصل 25
https://docs.netlify.com/frameworks/vite/

Netlify Docs · Redirects · فصل 25
https://docs.netlify.com/manage/routing/redirects/overview/

Netlify Docs · CLI · فصل 25
https://docs.netlify.com/cli/get-started/

Vercel Docs · Vite · فصل 25
https://vercel.com/docs/frameworks/frontend/vite

Vercel Docs · CLI · فصل 25
https://vercel.com/docs/cli

Vercel Docs · Environments (Preview / Production) · فصل 25
https://vercel.com/docs/deployments/environments

ج.11 مراجع تانية

CERN · The birth of the web · فصل 1
https://info.cern.ch/

Flexbox Froggy · فصل 7
https://flexboxfroggy.com/

CISA · Shai-Hulud alert (23 سبتمبر 2025) · فصل 16
https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem

Aikido · chalk g debug compromised (8 سبتمبر 2025) · فصل 16
https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised

Apple · VoiceOver User Guide · فصل 24
https://support.apple.com/guide/voiceover/welcome/mac

roadmap.sh · Frontend · فصل 27
https://roadmap.sh/frontend

Playwright · Intro · فصل 27
https://playwright.dev/docs/intro

ملحق د

المصادر ومنهجية التحقق

إزاي اتجمعت المعلومات في الكتاب ده، وإيه اللي اتجرب فعلاً وإيه اللي متجربش، علشان تعرف تثق في إيه وتراجع إيه بنفسك.

د.1 تاريخ اللقطة

كل المعلومات اتاخدت من الدوكيومنتيشن الرسمي (MDN و web.dev و npm و Node و Vite و Tailwind و ESLint و Prettier و TypeScript و Next.js و GitHub و Netlify و Vercel) يوم **23 أكتوبر 2026**، وكل اللينكات اللي في ملحق الروابط اتفتحت في نفس اليوم. الإيكوسيستم ده من أسرع الحاجات تغيّراً، فلو أمر أو رسالة عندك اختلفت عن الكتاب، الدوكيومنتيشن والنسخة اللي عندك هما الصح.

د.2 النسخ اللي اتجربت عليها

- Node.js 24.22.1 (LTS) و npm 11.19.1 (اللي جاي معاه)، و npm 12.2.1 بـ `npx npm@12` لتجارب فصل 16.
- create-vite 9.3.1 و Vite 8.3.5، و Tailwind CSS 4.3.3، و sass 1.106.1.
- ESLint 10.13.0 و @eslint/js 10.0.1 و eslint-config-prettier 10.1.8 و Prettier 3.9.10.
- TypeScript 6.0.3 (اللي في قالب create-vite) و TypeScript 7.0.2 للمقارنة.
- Next.js 16.4.2 (create-next-app 16.4.2)، و React 19.3.1 في قوالب Vite (و 19.3.0 في قالب Next).
- pnpm 12.15.0 و Bun 1.4.4 (بـ npx)، و Lighthouse 13.5.0، و Google Chrome على Linux لتجارب المتصفح.
- Vue 3.5 و Svelte 5 و Angular 22 لمقارنة فصل 12.

د.3 إيه اللي اتجرب وإزاي

- **كل أمر في الكتاب ليه Output اتشغّل فعلاً**، والخرج منقول منه (أحياناً مختصر بـ `…`). المسارات اللي فيها `/workspace` أو `/tmp` من جهاز التجربة.
- تجارب المتصفح (Accessibility tree، و Forms، و الـ Cascade، و HMR، و ESM و CORS) اتعملت في Chrome حقيقي بـ Puppeteer، والقيم المحسوبة منقولة منه.
- أحجام الـ Builds من Output بتاع Vite نفسه. حجم Angular في فصل 12 تقدير من Output الـ CLI، مش نفس طريقة القياس بالظبط.
- **البيئة Linux مش macOS.** أوامر الماك (Homebrew، و `/opt/homebrew`، و `⌘`، و VoiceOver، و Packages زي `darwin-arm64` و fsevents) مكتوبة من الدوكيومنتيشن ومن معرفة سلوك npm، ومتوقع فروق صغيرة: اسم الـ Package الخاص بالنظام في `node_modules`، ورسالة zsh بدل bash، ومكان `npm prefix -g`.
- **ماعملناش Deploy حقيقي.** النشر حاجة عامة بحسابك. خطوات GitHub Pages و Netlify و Vercel من الدوكيومنتيشن الرسمي، والـ Workflow اتكتب واتجرّب محلياً (lint و build و preview) بس ماتشغّلش على GitHub.

د.4 حاجات جديدة أو بتتغير

- npm 12 (يوليو 2026) هو `latest` على الـ Registry، بس Node 24 لسه جاي معاه npm 11. خطوات `allowScripts` ممكن تتغير في نسخ x.12 الجاية (الدوكيومنتيشن بيذكر `approve-scripts` كمان).
- TypeScript 7 جديدة، والقوالب لسه على 6.0، وأدوات بتعتمد على API بتاع TS محتاجة وقت.
- Cache Components في Next.js 16.4 شغالة افتراضياً في المشاريع الجديدة، والـ APIs بتاعتها (`use cache` و `cacheLife`) لسه بتتطور.
- قوالب React في create-vite بقت Oxlint افتراضياً، و Lighthouse فيه Category جديدة اسمها Agentic Browsing.

د.5 ملاحظات لقيناها وإحنا بنجرّب

- `npm init -y` في npm 11 بيكتب `"type": "commonjs"`، فـ `import` بيضرب Error في Node.
- `npm run x --flag` من غير `--`: npm بياكل الـ Flag.
- متغير `VITE_` في `.env.local` بيوصل للـ Bundle عادي. `.local` بيحمي من Git بس.
- Lighthouse ماشافش `div onclick` ولا `input` بـ placeholder من غير label.
- `vite build` لوحده بيعدّي Type errors، و `tsc` هو اللي بيوقفها.

ملحق ه

الإجابات والحلول

جاوب بنفسك الأول، وبعدين قارن. لكل فصل: إجابات أسئلة المراجعة، وبعدها صح ولا غلط، وبعدها مفاتيح التحديات. أغلب التحديات تجارب عملية، فلو وصلت لنفس النتيجة بطريقة تانية فهي صح.

الفصل 1: حكاية الفرونت إند

1. مثلاً بنك: تطبيق الموبايل والشاشات اللي بتشوفها = فرونت، والسيرفر اللي بيحسب الرصيد ويحفظ التحويلات = باك.
2. AJAX حل مشكلة «كل ضغطة = صفحة جديدة» (تجيب بيانات من غير Reload). jQuery حل مشكلة «كل متصفح بيتكلم DOM بطريقة مختلفة» (API واحد سهل).
3. الصفحة فاضية لحد ما JS ينزل (بطء أول ظهور)، و SEO ومعاينة الروابط ضعيفة، وBundle كبير.
4. لأن كل الأدوات (npm و Vite و ESLint و TypeScript) برامج JS بتشتغل على جهازك بـ Node.

صح ولا غلط: (1) غلط: أي كود في المتصفح مقروء. · (2) غلط: ده Server-rendered. الـ SPA بيغيّر الصفحة بـ JS. · (3) صح.

التحديات:

1. عدد قليل جداً من الـ Tags (title و h1 و a و dl و p...).
2. طلبات Fetch/XHR بتطلع والصفحة ماعملتش Reload.
3. View Source فيه محتوى = Server-rendered أو Meta-framework (Next.js = next/_/)، و div فاضي = SPA.

الفصل 2: خريطة الإيكوسيستم

1. المنصة (HTML و Web APIs)، ولغات فوق اللغة (TypeScript و JSX)، والأدوات على جهازك (npm و Vite)، والمكتبات (React و Tailwind)، والـ Meta-framework (Next.js و Nuxt)، والاستضافة (Vercel و Netlify).
2. انت بتنادي الـ Library وقت ما تحب. الـ Framework هو اللي بينادي كودك في أماكن محددة (Inversion of control).
3. لأن node_modules فيه أدوات التطوير كلها (Vite و ESLint ونسخ لكل نظام)، والموقع بياخد بس الكود اللي اتعمله import بعد التصغير.
4. Routing و SSR/SSG و Data fetching على السيرفر وتحسين الصور والخطوط.

صح ولا غلط: (1) غلط: لازم يتحول لـ JS. · (2) غلط: أداة تطوير، بس الناتج بتاعها اللي بيتبعت. · (3) صح.

التحديات:

1. مثلاً React في UI، و Next.js في Meta-framework، و Jest في الجودة...
2. غالباً نسخ أحدث ونفس الـ scripts. قارن بنفسك.
3. /_/next/static = Next.js، و /assets/index-xxxx.js = غالباً Vite.

الفصل 3: Semantic HTML

1. شجرة بيبنيها المتصفح من الـ HTML فيها الدور والاسم والحالة لكل عنصر. بتستخدمها Screen readers وأدوات الأتمتة.
2. article: حاجة مستقلة ليها معنى لوحدها (بوست، كارت منتج). section: جزء ليه عنوان من حاجة أكبر. div: للتنسيق بس من غير معنى.
3. مش بيوصله Tab، ومش بيشتغل بـ Enter/Space، ومالوش Role «button» في الـ Accessibility tree.
4. المتصفح يدخل Quirks mode وحسابات CSS زي الـ Box model ممكن تختلف.

صح ولا غلط: (1) غلط: العناوين للهيكل، والحجم بالـ CSS. · (2) غلط: alt="" = «صورة زينة، تجاهلها». من غير alt بيقرا اسم الملف. · (3) صح: الافتراضي type="submit".

التحديات:

1. قارن بالحل في الفصل.
2. كل Landmark ظاهر بدوره (banner و navigation و main و complementary و contentinfo).
3. مثلاً: div بيتداس، وصور من غير alt، وخانات من غير label، وتباين ضعيف، و Focus مش باين.

الفصل 4: Forms

1. id للربط جوه الصفحة (label for و CSS و JS). name هو المفتاح اللي بيتبعت مع البيانات.
2. submit بيلقط Enter وزرار الإرسال وأي طريقة إرسال، وبيدّيك الـ Validation بتاع المتصفح.
3. invalid: من أول ما الصفحة تفتح. :user-invalid بعد ما المستخدم يتعامل مع الخانة.
4. new-password في التسجيل وتغيير الباسورد (علشان يقترح باسورد قوي)، و current-password في تسجيل الدخول.

صح ولا غلط: (1) غلط: من غير name مش بيتبعت. · (2) صح: pattern مابيتشيكش على خانة فاضية. · (3) غلط: أي حد يقدر يبعت بـ curl، فالسيرفر لازم يشيك.

التحديات:

1. "username"=autocomplete و "current-password".
2. ?q=...&sort=... في الـ URL، وبعدين preventDefault و fetch بـ FormData.
3. a)setCustomValidity(b ? '' : 'Passwords do not match').

الفصل 5: Accessibility

1. حد في مواصلات من غير سماعات، أو في مكان دوشة، بيقرا الـ Captions.
2. لأنه بيظهر للي بيستخدم الكيبورد بس، ومن غيره مستخدم الكيبورد مش عارف هو فين.
3. «متستخدمش ARIA لو فيه عنصر HTML بيعمل نفس الحاجة». الغلط بيوعد Screen reader بسلوك مش موجود (WebAIM: صفحات فيها ARIA فيها أخطاء أكتر).
4. لرسالة بتتغير من غير ما الـ Focus يروحلها: نتيجة إرسال، عدد أخطاء، «اتضاف للسلة».

صح ولا غلط: (1) غلط: لازم تكتب الـ Keyboard handlers بنفسك. · (2) غلط: بيخليه يستقبل Focus بالـ JS بس. · (3) غلط: بتمسك جزء بس (فصل 24).

التحديات:

1. أول عنصر في الـ body، ويظهر مع الـ Focus.
2. aria-expanded true/false و hidden على القايمة.
3. الأخطاء بتتقري مع كل خانة، والملخص بيتقري بعد الإرسال.

الفصل 6: الـ Cascade

1. الأهمية والمصدر، ثم الـ inline style، ثم الـ Layers، ثم الـ Specificity، ثم الترتيب في الملف.
2. #menu = 1 (1,2,3)، و :active و :hover = 2، و ul و li و a = 3.
3. لأن المتصفح نفسه عامل قاعدة لون للـ a، وأي قاعدة على العنصر نفسه بتكسب الوراثة.
4. content-box: 300 + 20 + 4 = 324px. border-box: 300px.

صح ولا غلط: (1) غلط: أي ID بيكسب أي عدد Classes. · (2) صح: زي ما شفنا في #buy. · (3) غلط: margin مش بيتورث.

التحديات:

1. اقرا القاعدة الكسبانة فوق في Styles.
2. @layer reset, base, components; والـ btn. بيكسب.
3. :where(h1, h2, h3, p) { margin: 0 } (Specificity صفر).

الفصل 7: Flexbox

1. space-between: مفيش مسافة على الأطراف. space-evenly: نفس المسافة بين الكل وعلى الأطراف.
2. flex: 1 = 1 1 0% (يبدأ من صفر ويتقسّم بالتساوي). flex: auto = 1 1 auto (يبدأ من حجم المحتوى).
3. لأنه بيغيّر الترتيب اللي بتشوفه بس، والـ Tab والـ Screen reader بيمشوا بترتيب الـ HTML.
4. body :{display:flex; flex-direction:column; min-height:100svh} و main { flex: 1 }.

صح ولا غلط: (1) صح. · (2) غلط: الولاد المباشرين بس. · (3) صح.

التحديات:

1. justify-content: space-between، والاتجاه بيقلب لوحده.
2. آخر كارت بيتمط على السطر كله، و Grid بيمنع ده.
3. —

الفصل 8: Grid

1. Flex بيوزّع كل سطر لوحده، فآخر سطر بيكبّر عناصره. Grid أعمدة ثابتة لكل الصفوف.
2. «العمود عمره ما يقل عن 120px، ولو فيه مساحة ياخد نصيبه منها».

3. auto-fill بيسيب أعمدة فاضية لو العناصر قليلة. auto-fit بيقفلها فالعناصر تتمط.
4. "header header header" "left main right" "footer footer footer".

صح ولا غلط: (1) غلط: Grid بس. · (2) غلط: Responsive لوحدها. · (3) صح.

التحديات:

1. عدد الأعمدة بيتغير مع العرض.
2. Media query واحدة بتغيّر grid-template-areas لعمود واحد.
3. dense بيملا الفراغات بعناصر بعدين.

الفصل 9: Responsive

1. الموبايل كان بيعرض الصفحة كأنها 980px ويصغّرها (من أيام iPhone الأول). السطر بيقول «العرض = عرض الجهاز».
2. لأن rem بيحترم حجم الخط اللي المستخدم اختاره في الإعدادات.
3. (min-width: 640px) and (max-width: 1023.98px) و (640px <= width < 1024px).
4. لما الكومبوننت بيظهر في أماكن بعروض مختلفة في نفس الشاشة (سايدبار ومحتوى).

صح ولا غلط: (1) غلط: الصفحة بتتعرض كأنها 980px. · (2) غلط: بيسأل عن عرض الـ Container. · (3) غلط: تأخّر الـ LCP. Lazy للي تحت الشاشة بس.

التحديات:

1. من غير السطر الكلام صغير جداً.
2. Mobile-first بـ min-width.
3. كل نسخة حسب عرض علبتها.

الفصل 10: CSS الحديث

1. el.style.setProperty('--brand','red'). متغيرات Sass بتختفي وقت البناء فمفيش حاجة تتغير.
2. li:has(a).
3. border-inline-end: 1px solid; padding-inline-start: 1rem; text-align: start;
4. Newly: كل المتصفحات الرئيسية دعمتها مؤخراً. Widely: بقالها 30 شهر على الأقل.

صح ولا غلط: (1) غلط: بتفضل في المتصفح. · (2) صح. · (3) غلط: بيتجاهل السطر ده بس.

التحديات:

1. .danger { --brand: ... } والزرار نفسه يستخدم var(--brand).
2. [data-theme="dark"] { --bg: ...; --text: ... }.
3. form:has(:user-invalid) button { ... }.

الفصل 11: ليه Frameworks

1. Imperative: «امشي يمين، اطلع الدور التالت...». Declarative: «عايز أوصل الشقة دي».
2. لأن innerHTML بيمسح الخانة ويعمل واحدة جديدة مع كل حرف، فالـ Focus بيروح لـ BODY.
3. لأن الرقم المتخزن ممكن ينسى يتحدّث ويبقى غلط. احسبه من الـ State كل مرة.
4. Virtual DOM: يعيد تشغيل الكومبوننت ويقارن. Signals: كل قيمة عارفة مين بيستخدمها وتحدّثه هو بس.

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط.

التحديات:

1. الخانة بتفقد الـ Focus.
2. حدّث h1 و ul بس.
3. النسخة التانية أقصر وأسهل تتصلّح.

الفصل 12: مقارنة الـ Frameworks

1. React: useState، و Vue: ref، و Svelte: $state، و Angular: signal.
2. JSX = JavaScript فيها HTML (تكتب map و ternary). Templates = HTML فيها إضافات (v-if و {#if}).
3. لأن التطبيق الحقيقي كوده أكبر من الـ Framework، والفريق والإيكوسيستم والوظايف بيفرقوا أكتر.
4. Components و Props، و State و Derived state، و Events، و Conditional/List rendering.

صح ولا غلط: (1) غلط: الـ Compiler بيحوّلها. · (2) صح. · (3) غلط: الحجم الثابت حاجة، والأداء بيعتمد على التطبيق.

التحديات:

1. شغال على 5173.

2. React: react و react-dom في dependencies، والباقي devDependencies.
3. —

الفصل 13: Node و npm

1. لأن vite بيعتمد على Packages وهما بيعتمدوا على غيرهم: Transitive dependencies.
2. انت طلبت نسخة بالظبط، و npm كتب ^4.17.20 (أي x.4 من 4.17.20 وطالع).
3. علشان كل مشروع ياخد نسخته المكتوبة في package.json، والفريق ياخد نفس النسخة.
4. بتقول لـ npm «اللي بعدي مش ليك»، فـ template-- يوصل لـ create-vite.

صح ولا غلط: (1) غلط: devDependencies. · (2) غلط: فيه Packages Native لكل نظام. · (3) صح.

التحديات:

1. v24.x و x.11 أو x.12، و /opt/homebrew أو فولدر nvm، و registry.npmjs.org.
2. Apple Silicon: darwin-arm64، و Intel: darwin-x64.
3. من غير Lockfile ممكن تاخد نسخ أحدث جوه الـ Range.

الفصل 14: package.json

1. علشان npm يرفض ينشره بالغلط على الـ Registry العام.
2. prebuild ثم build. لو prebuild فشل (exit مش صفر)، build مش هيشتغل.
3. dependency: الـ Package بيسطّبه لنفسه. peer: «لازم انت تسطّبه»، علشان الكل يستخدم نسخة واحدة (react).
4. لما Package جوه الشجرة نسخته فيها مشكلة وصاحبه مش هيحدّث. الخطر: الـ Package الأب ماتجرّبش مع النسخة دي.

صح ولا غلط: (1) غلط: Cannot use import statement outside a module. · (2) غلط: npm بياكله، لازم (3) · .-- صح.

التحديات:

1. React و react-dom تطبيق، و vite و eslint أدوات.
2. "prebuild": "rm -rf dist".
3. "type": "module" أو غيّر الامتداد لـ mjs..

الفصل 15: SemVer و Lockfile

1. مفيش Lockfile متوحد، فكل واحد خد نسخة مختلفة جوه الـ ^. الـ Lockfile بيقفل النسخة لكل الناس.
2. لأ: مع x.0 الـ ^ مابيغيرش الـ MINOR، فيقبل x.0.4 بس.
3. لما الـ Latest برة الـ Range (MAJOR جديد، أو ~ ومحتاج MINOR جديد).
4. لأنه بيلتزم بالـ Lockfile بالحرف، وبيفشل لو مش متوافق مع package.json، وبيبدأ من node_modules نضيف.

صح ولا غلط: (1) صح. · (2) غلط: ~ = Patches بس. · (3) غلط: بيغيّر الـ Lockfile بس. · (4) غلط: EUSAGE.

التحديات:

1. Current/Wanted/Latest لكل Package.
2. npx semver -r "^2.4.1" 2.9.0 بيرجّع 2.9.0، و ~2.4.1 مع 2.5.0 مايرجّعش حاجة.
3. npm ci يفشل بـ EUSAGE، والحل npm install والـ diff يبين النسخة الجديدة.

الفصل 16: بدائل npm والأمان

1. import لـ Package مش في package.json بتاعك بس موجود بالصدفة لأن حد تاني جابه. pnpm مابيحطش غير اللي طلبته في node_modules.
2. pnpm i --frozen-lockfile و bun install --frozen-lockfile، و pnpm dlx create-vite و bunx create-vite.
3. لأن الـ Lockfile فيه النسخة القديمة بالـ integrity، و npm ci مش هيجيب نسخة جديدة.
4. fix: جوه الـ Range. force--: ممكن MAJOR جديد يكسر المشروع.

صح ولا غلط: (1) غلط: استخدم pnpm. · (2) غلط: الثغرات المعروفة بس. · (3) صح.

التحديات:

1. عدد الثغرات وعدد الـ signatures والـ attestations.
2. المرة التانية reused أكتر و downloaded أقل.
3. install-scripts approve esbuild ثم npm rebuild، وبعدها النسخة بتطبع.

الفصل 17: Modules و Bundlers

1. الـ Modules بتتحمّل بقواعد CORS، و file:// الـ Origin بتاعه null.

2. main.js يستورد app.js، و app.js يستورد button.js: المتصفح مايعرفش button غير بعد ما app ينزل.
3. الأول جوه نفس الملف. التاني (Dynamic) ملف لوحده بيتحمّل وقت الحاجة.
4. Hash من المحتوى علشان الـ Cache: الاسم بيتغير لما المحتوى يتغير بس.

صح ولا غلط: (1) غلط: Bare specifier. · (2) صح. · (3) صح.

التحديات:

1. index و main و math (وغالباً favicon).
2. CORS error من 'null' origin.
3. Hash بتاع chart اتغير، وغالباً index كمان لأنه بيشاور على اسم chart.

الفصل 18: Vite

1. لأنه مش بيعمل Bundle في الـ Dev، بيقدّم ESM للمتصفح ملف ملف.
2. src/assets (بـ import)، و Hash ويتحذف لو مش مستخدم). public لملف محتاج اسم ثابت (favicon و robots.txt).
3. CSS بيعمل accept للـ HMR، و counter.js لأ، فـ Vite عمل Reload.
4. base غلط (مسارات assets/)، أو مسار ملف في public غلط، أو env مش موجود وقت البناء.

صح ولا غلط: (1) غلط. · (2) غلط. · (3) صح.

التحديات:

1. /@vite/client بيفتح WebSocket.
2. رابط Network من سطر الـ Terminal.
3. file:// و CORS = Modules. preview بيحلها.

الفصل 19: env و .env

1. لأنه مش بادئ بـ VITE_، فـ Vite رفض يحطه في كود المتصفح.
2. build: env.production. (الـ mode أقوى). dev: env. لأن الـ mode development.
3. سيرفر وسيط (Express أو Serverless function) معاه المفتاح، والمتصفح يكلّم السيرفر.
4. import.meta.env في كود Vite وقيمه بتتكتب وقت البناء. process.env في Node وقت التشغيل على السيرفر.

صح ولا غلط: (1) غلط: VITE_ = في الـ Bundle. · (2) غلط: نص. · (3) غلط: لازم npm run build -- --mode staging.

التحديات:

1. %VITE_APP_TITLE% في HTML.
2. "build:staging": "vite build --mode staging".
3. grep هيلاقيها في dist/assets.

الفصل 20: CSS Ecosystem

1. Tailwind بيولّد الـ Classes اللي لقاها مكتوبة بس. الفخ: أسماء متركبة بـ template string.
2. Sass وقت البناء ويختفي. Custom property في المتصفح وتتغير بـ JS و Media queries.
3. shadcn/ui كود بيتنسخ في مشروعك وتعدّله. MUI Package بشكل جاهز.

صح ولا غلط: (1) غلط: @import و @theme. · (2) غلط: Hash مختلف لكل ملف. · (3) غلط: بيختفي وقت البناء.

التحديات:

1. remp-4 = 1.
2. bg-brand. و hover بنسبة شفافية.
3. الأرقام بتختلف، و Tailwind بيضيف Reset و Theme فبيبقى أكبر في كارت واحد، والفرق بيقل مع كبر المشروع.

الفصل 21: TypeScript

1. لأن Vite بيشيل الأنواع ومابيفحصش.
2. src/App.tsx السطر 8 العمود 5، و TS2322 = نوع مش متوافق.
3. React مكتوبة JS وأنواعها في DefinitelyTyped. Vite بيجيب أنواعه معاه.

صح ولا غلط: (1) غلط. · (2) غلط: بيكمّل ويعرض الصفحة. · (3) غلط: ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX.

التحديات:

1. خط أحمر تحت القيمة.
2. npm run build يفشل بـ exit 2، و vite build ينجح.
3. 7 غالباً أسرع، والفرق بيبان في المشاريع الكبيرة.

الفصل 22: Rendering و Next.js

1. المنيو SSG أو ISR (تتغير نادراً و SEO مهم). صفحة الطلبات SSR أو CSR ورا Login.
2. لأنها اتبنت وقت البناء ومتخزنة (cacheLife minutes)، فالوقت = وقت آخر بناء.
3. ○ Static كاملة وقت البناء. ◐ جزء ثابت وجزء ديناميكي بيتبعت Stream.

صح ولا غلط: (1) غلط. · (2) غلط. · (3) صح.

التحديات:

1. Vite: div فاضي. Next.js: المحتوى.
2. ○ للمسارات اللي في generateStaticParams.
3. now/ بيتغير كل طلب، و news/ بعد الدقيقة: أول طلب بيرجّع القديم ويبني الجديد في الخلفية، والطلب اللي بعده بيجيب الجديد.

الفصل 23: ESLint و Prettier

1. error بيرجّع exit 1 ويوقف الـ CI، و warn لأ.
2. علشان يعرف المتغيرات الجاهزة في المتصفح (document و window) أو Node.
3. لأن أي نسخة ممكن تغيّر التنسيق، فكل الفريق لازم نفس النسخة.

صح ولا غلط: (1) غلط: ده شغل ESLint. · (2) غلط: المضمون بس. · (3) صح.

التحديات:

1. الملف بيتنسق مع الحفظ.
2. —
3. == بيعمل تحويل أنواع (0 == '' = true)، و === بيقارن من غير تحويل.

الفصل 24: الأداء والـ a11y

1. لأن الـ Dev مش مضغوط وفيه HMR وملفات كتير، فالأرقام مش حقيقية.
2. SSR/SSG (Next.js)، و Code splitting، وصورة أصغر بصيغة حديثة، و Preload للصورة الأساسية.
3. div بيتداس، وinput بـ placeholder بس، ونص لينك «click here»، وترتيب Focus غلط.

صح ولا غلط: (1) غلط: TBT بدله. · (2) صح. · (3) غلط.

التحديات:

1. Performance و Accessibility و Best Practices و SEO.
2. lang و alt و تباين و aria-label و main، وبعدين button و label.
3. Field data محتاجة زيارات كفاية من Chrome.

الفصل 25: النشر

1. push ثم npm ci ثم build ثم رفع dist ثم رابط.
2. تجرّب كل تغيير على رابط لوحده قبل ما يوصل الـ Production.
3. افتح Network، ولو 404 /assets ظبط base لاسم الـ repo واعمل push.

صح ولا غلط: (1) غلط: ملفات ثابتة. · (2) غلط: / لأنه على جذر الدومين. · (3) صح.

التحديات:

1. preview زي dev.
2. —
3. Pages: /my-links/ و Vercel: /.

الفصل 26: المشروع النهائي

1. create-vite: index.html و gitignore. و favicon، و Tailwind: vite.config.js و style.css، والصفحة: main.js و links.js و env.، و Lint: eslint.config.js و prettierrc. و prettierignore.، و package.json و Lockfile مع كل install، و deploy.yml للنشر.
2. امسح base (أو خليه /)، و Import في Vercel، وزوّد VITE_APP_TITLE لو مش هترفع env..
3. env. فيه قيم عامة، و env.local. للأسرار والـ gitignore. فيه local*..

التحديات:

1. الأرقام قريبة، والـ Hashes ممكن تختلف.
2. dark:bg-slate-900 و dark:text-slate-100.
3. Lighthouse على الرابط الحقيقي.

الفصل 27: الـ Roadmap

1. غالباً Vite (18) و env (19) و Deploy (25).
2. Vite بدل create-react-app (فصل 18)، و Tailwind v4 بـ @import و @theme (فصل 20).

دلوقتي الإيكوسيستم مش غابة

بقيت عارف كل ملف في مشروع Vite بيعمل إيه، وكل سطر في package.json، والـ ^ والـ Lockfile، وليه node_modules تقيل وإزاي تأمّنه، والـ Modules والـ Bundler والـ HMR، والـ env وإيه اللي مينفعش يبقى فيها، و Tailwind و TypeScript و Next.js بيحلوا إيه، وإزاي تفحص وتنسّق وتقيس وتنشر. ادخل كورس React وانت فاهم الأرض اللي واقف عليها.

```
$ npm run build → ✓ built in 98ms · dist/assets/index-Ba828Msq.js 1.29 kB
```